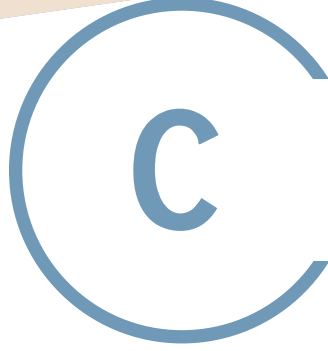


C Programlama Dili ile YAPISAL PROGRAMLAMA

Kavramlar ve Algoritma Mantığı



Elektronik Yük. Müh.
İlhan ÖZKAN

Dördüncü Sürüm, Mart 2026, Ankara

C Programlama Dili ile YAPISAL PROGRAMLAMA

Yazar: Elektronik Yük. Müh. İlhan ÖZKAN

© Mart 2026, İlhan ÖZKAN. Tüm hakları saklıdır.

Bu kitap kâr amacı güdülmeden hazırlanmıştır ve **referans gösterilerek** her yerde kullanılabilir. C programlama dilinin eğitimini veren ve alan tüm kişiler için serbestçe kullanılabilir.

Bu kitabın bir kısmı veya tamamı, **yazarın önceden yazılı izni alınarak** elektronik, mekanik, fotokopi veya herhangi bir kayıt sistemi ile çoğaltılabilir, yayımlanabilir veya depolanabilir.

ISBN: 978-625-00-6034-6

Baskı Tarihi: Mart 2026

Baskı Yeri: Ankara, Türkiye

Önsöz

Ülkemizde her alanda olduğu gibi yazılım alanında da bir terminoloji karmaşasının yaşandığı yazılan kitaplardan anlaşılmaktadır. Son dönemde yazılan kitaplar, adeta çeviri yapan programların bir gecede ürettiği çıktılar birleştirilerek hazırlanmaktadır. Bu kitaplar incelendiğinde, bilindiği varsayılan kavramların bile tanımlanırken birbirinin içerisine girdiği görülmektedir. 1989 yılından beri birçok programlama dili kullanarak yazılım geliştiriyor olmama rağmen, bu kitapların içeriğini anlama konusunda oldukça zorlanıyorum.

Yapısal programlamanın en çok uygulandığı Pascal dilidir. Pascal diline yakın zamanda ortaya çıkan C dili, kısa sürede sistem dili olarak tarihte yerini almıştır. Günümüzde sistem programlama ve yüksek hız gerektiren gömülü sistemlerde vazgeçilmez bir dil olmuştur.

Daha sonra C dilinden etkilenecek olan C++, Java, C# gibi birçok nesne yönelimli programlama dili ve yapısal programlama üzerine kurulmuş dillerdir. Birçok modern programlama diline esin kaynağı olan bu dilin özellikle bilgisayar mühendisleri tarafından bilinmesi gerekir.

Kitapta verilen konular çok sade olarak hazırlanmıştır. İşlenen kavramlar her bölümde üst üste konularak ilerlediğinden bir bölüm özümsemeden bir sonrakine geçilmemesi önerilir.

Yapısal programlama mantığını oluşturan kavramların programcıların kafasında aynı ışığı yakması için bu kitap hazırlanmıştır. Bu dilin eğitimini verecek tüm kişiler için serbestçe kullanılabilir.

Bu kitabın güncel sürümüne <https://github.com/HoydaBre/clanguage> üzerinden erişilebilir. Amacım Türkçe düşünen ve konuşan öğrencilerimize kavramları doğru bir şekilde aktarmaktır. Katkılarınızı bekler iyi çalışmalar dilerim.

Elektronik Yüksek Mühendisi, İlhan ÖZKAN

ilhanozkan[at]outlook.com

[linkedin.com/in/ilhanozkan](https://www.linkedin.com/in/ilhanozkan)

Sürümler

İkinci Sürüm

İlk sürümden farklı olarak aşağıdaki değişiklikler yapılmıştır.

- ◊ Genel yazım hataları düzeltildi.
- ◊ Her bölüme “Düşün ve Çöz” başlığı eklendi

Üçüncü Sürüm

İkinci sürümden farklı olarak aşağıdaki değişiklikler yapılmıştır.

- ◊ Genel yazım hataları düzeltildi.
- ◊ İlk Bölüme onaltılı sayılar eklendi.

Dördüncü Sürüm

Üçüncü sürümden farklı olarak aşağıdaki değişiklikler yapılmıştır.

- ◊ Genel yazım hataları düzeltildi.
- ◊ Önemli Vurgular çerçeve içine alındı.
- ◊ Kaynak kodlar renklendirildi
- ◊ Değerle çağırma ve referansla çağırma başlığı yenilendi.
- ◊ Jenerik gösterici eklendi
- ◊ Ön İşlemci yönergeleri yenilendi.

İçindekiler

Önsöz	ii
Sürümler	iii
1 Temel Bilgisayar Kavramları	1
1.1 Mühendis ve Teknisyen	1
1.2 Bilgisayara Niçin İhtiyaç Duyulur?	1
1.3 Veri ve Bilgi	1
1.4 En Basit Bilgisayar	1
1.5 Emri Alma, Çözme ve İcra Çevrimi	3
1.6 İşlemci Tasarlama Stratejileri	4
1.7 Onaltılı Sayı Sisteminin Kullanılması	5
1.8 Program ve Programlama	5
1.9 Genel Amaçlı Bilgisayarlar ve İşletim Sistemleri	6
1.10 Düşün ve Çöz	7
2 Temel Yazılım Kavramları	8
2.1 Düşük Düzey Programlama	8
2.2 Genel Amaçlı Yüksek Düzey Diller	9
2.3 Arapsaçı Kod	9
2.4 Değişken	11
2.5 Fonksiyon	12
2.6 Yapısal Programlama	13
2.7 Yapısal Programlama Genel Çerçevesi	13
2.8 Düşün ve Çöz	14
3 Derleyiciler ve Çalıştırma Ortamı	15
3.1 Derleyici	15
3.2 C derleyicileri	15
3.3 Entegre Geliştirme Ortamı	16
3.4 Çevrimiçi Derleyiciler	17
3.5 Derleme Zamanı	17
3.6 Hatalar	18
3.7 C Programlama Kullanım Alanları	18
3.8 C Programlama Dilinin Üstünlükleri	19
3.9 C Dilinin Zayıf Yönleri	19
3.10 C Sürümlerinin Tarihçesi	19
3.11 Düşün ve Çöz	20
4 C Programlama Diline Giriş	21
4.1 En Basit C Programı	21
4.2 Söz Dizim Kuralları	21
4.2.1 Kaynak Kodumuzu Oluşturacak Karakterler	21
4.2.2 Talimatlar ve Anahtar Kelimeler	21
4.2.3 Açıklamalar	23
4.3 Değişken Tanımlama	23
4.4 Değişken Kimliklendirme Kuralları	24
4.5 Sabitler	25
4.6 Yazdığımız Kod Nasıl İcra Edilir?	27
4.7 İşleçler	28
4.7.1 Aritmetik İşleçler	28
4.7.2 Tekli İşleçler	29
4.7.3 İlişkisel İşleçler	30
4.7.4 Bit Düzeyi İşleçler	30
4.7.5 Mantıksal İşleçler	31

4.7.6	Atama işleçleri	31
4.7.7	İşleç Öncelikleri	32
4.8	Kayan Noktalı Sayı Hesapları	32
4.9	Düşün ve Çöz	33
5	Giriş Çıkış İşlemleri	34
5.1	Modüler Programlama	34
5.2	Konsola Biçimlendirilmiş Veri Yazma	34
5.3	Klavyeden Biçimlendirilmiş Veri Okuma	37
5.4	Düşün ve Çöz	38
6	Kontrol İşlemleri	39
6.1	Ardışık İşlem ve Kontrol İşlemleri	39
6.2	Akış Diyagramları	39
6.3	Duruma Göre Seçimler	40
6.3.1	If Talimatı	40
6.3.2	If-Else Talimatı	42
6.3.3	Sarkan Else	45
6.3.4	Switch Talimatı	45
6.3.5	Üçlü Koşul İşleci	47
6.4	İlişkisel Döngüler	48
6.4.1	Sayaç Kontrollü Döngüler	48
6.4.2	While Talimatı	49
6.4.3	Do-While Talimatı	50
6.4.4	For Talimatı	51
6.4.5	İç İç Döngü Kodu	53
6.4.6	Gözcü Kontrollü Döngüler	54
6.5	Dallanmalar	55
6.5.1	Continue ve Break Talimatları	55
6.5.2	Return Talimatı	57
6.5.3	Goto Talimatı	58
6.6	Düşün ve Çöz	58
7	Fonksiyonlar	59
7.1	Fonksiyon Nedir?	59
7.2	Hazır Fonksiyonlar	59
7.2.1	Math.h Başlık Dosyası	59
7.2.2	Stdlib.h Başlık Dosyası	60
7.3	Kullanıcı Tanımlı Fonksiyonlar	61
7.3.1	Fonksiyon Tanımı Nasıl Yapılır?	62
7.3.2	Fonksiyon Bildirimi Nasıl Yapılır?	64
7.4	Çağrı Kuralı	67
7.5	Depolama Sınıfları	67
7.5.1	auto Depolama Sınıfı	68
7.5.2	static Depolama Sınıfı	68
7.5.3	extern Depolama Sınıfı	69
7.5.4	register Depolama Sınıfı	69
7.6	Evrensel ve Yerel Değişken	70
7.7	Değerle Çağırma	71
7.8	Özyinelemeli Fonksiyonlar	72
7.9	Satır İç Fonksiyonlar	74
7.10	Düşün ve Çöz	74
8	Diziler	75
8.1	Dizi Nedir	75
8.2	Tek boyutlu Diziler	75
8.3	İki Boyutlu Diziler	79
8.4	Çok Boyutlu Diziler	81
8.5	Parametre Olarak Diziler	83
8.6	Dizi Elemanlarını Sıralama	84
8.6.1	Kabarcık Sıralama	84

8.6.2	Seçme Sıralama	85
8.6.3	Araya Sokma Sıralaması	87
8.6.4	Shell Sıralaması	87
8.6.5	Çabuk Sıralama	88
8.6.6	Hazır Sıralama Fonksiyonları	89
8.7	Düşün ve Çöz	89
9	Göstericiler	90
9.1	Gösterici nedir? Niçin İhtiyaç Duyarız?	90
9.2	Gösterici Tanımlama	90
9.3	Gösterici Aritmetiği	93
9.4	Gösterici Dizileri	94
9.5	Referansla Çağırma	95
9.6	Göstericilerin Dizileri İşaret Etmesi	95
9.7	Fonksiyonlardan Bir Diziyi Geri Döndürme	97
9.8	Fonksiyon Göstericisi	98
9.9	Düşün ve Çöz	98
10	Tip Dönüşümleri	99
10.1	Üstü Kapalı Tip Dönüşümleri	99
10.2	Bilinçli Tip Dönüşümü	99
10.3	Üstünlük ve zayıflıklar	100
10.4	Düşün ve Çöz	100
11	Numaralandırma ve Takma İsimler	102
11.1	Numaralandırma	102
11.2	Takma İsimler	102
11.3	Düşün ve Çöz	103
12	Dizgiler	104
12.1	Dizgi Tanımlama	104
12.2	Konsola Metin Yazma	104
12.3	Klavyeden Metin Okuma	105
12.4	Çok Kullanılan Dizgi Fonksiyonları	106
12.4.1	strlen() fonksiyonu	106
12.4.2	strcpy() fonksiyonu	106
12.4.3	strcat() fonksiyonu	107
12.4.4	strchr() fonksiyonu	107
12.4.5	strcmp() fonksiyonu	107
12.5	Karakter Fonksiyonları	108
12.6	Metinden İlkel Veri Tiplerine Dönüşüm	109
12.7	Arama Tabloları	110
12.8	Düşün ve Çöz	110
13	Yapılar ve Birlikler	112
13.1	Yapılar	112
13.2	Yapı Elemanlarına Erişim	112
13.3	Yapı Göstericileri	113
13.4	Yapılar ve Fonksiyonlar	113
13.5	Anonim Yapılar	114
13.6	Öz Referanslı Yapılar	115
13.7	Yapı Dolgusu	116
13.8	Birlikler	117
13.9	Düşün ve Çöz	118
14	Dinamik Bellek Yönetimi	119
14.1	Bellek Tahsisi ve Serbest Bırakma	119
14.1.1	malloc() fonksiyonu	119
14.1.2	calloc() fonksiyonu	119
14.1.3	realloc() fonksiyonu	119
14.1.4	free() fonksiyonu	120
14.2	Dinamik Bellek Kullanım Örnekleri	120

14.3	Dinamik Veri Yapıları	121
14.3.1	Bağlı Liste	121
14.3.2	Yığın	122
14.3.3	Kuyruk	123
14.3.4	İkili Ağaç	124
14.3.5	Sözlük	125
14.4	Düşün ve Çöz	126
15	Değişken Sayıda Argüman Alan Fonksiyonlar	128
15.1	Değişken Sayıda Parametre Tanımı	128
15.2	Değişken Sayıda Argüman Alan Ana Fonksiyon	129
16	Dosyalar	131
16.1	Dosya Nedir?	131
16.2	Standart Dosyalar	131
16.2.1	Karakterler İçin Giriş Çıkış Fonksiyonları	131
16.2.2	Biçimlendirilmemiş Metin İçin Giriş Çıkış Fonksiyonları	131
16.2.3	Biçimlendirilmiş Metin İçin Giriş Çıkış Fonksiyonları	132
16.3	Standart Olmayan Dosyalar	132
16.3.1	Dosyayı Açma ve Kapatma	132
16.3.2	İkili Dosyalar	134
16.4	Düşün ve Çöz	137
17	Ön İşlemci Yönergeleri	138
17.1	Ön İşlemci Yönergesi Nasıl Çalışır?	138
17.2	Ön İşlemci Makroları	138
17.3	Parametrelili Ön İşlemci Makroları	139
17.4	Ön Tanımlı Ön İşlemci Makroları	139
17.5	Kullanıcı Tanımlı Başlık Dosyası	140
17.6	Derleme Sürecinin Detayı	141
17.7	Düşün ve Çöz	141
Dizin		143

Şekiller Listesi

1.1	Motorola 6802, Intel 8086 İşlemciler ile 2732 EPROM Belleği	2
1.2	Frekans 1 olan Kare Dalga İşareti	3
1.3	Basit Bir İşlemcinin Yapısı	3
1.4	İşlemcinin Emri Alma, Çözme ve İcra Çevrimi	4
6.1	Akış Diyagramları Genel Şekilleri	39
6.2	Dik Üçgenin Alana Hesabını Gösteren Akış Diyagramı	40
6.3	If Talimatı İcra Akışı	40
6.4	Bir Sayının İşaretini Bulan Akış Diyagramı	42
6.5	If-Else Talimatı İcra Akışı	43
6.6	Bir Sayının İşaretini Bulan If-Else Akış Diyagramı	44
6.7	Switch Talimatı İcra Akışı	46
6.8	Sayaç Kontrollü Döngü İcra Akışı	49
6.9	While Talimatı İcra Akışı	50
6.10	Do-While Talimatı İcra Akışı	50
6.11	For Talimatı İcra Akışı	51
6.12	While ve Do-While Döngülerinde Break ve Continue Talimatları İcra Akışı	56
6.13	For Döngüsünde Break ve Continue Talimatları İcra Akışı	57
7.1	Fonksiyon Tanımı Örneği	63
7.2	Fonksiyon Bildirimi Örneği	64
7.3	Fonksiyon Çağırma Sürecinde Yığın Bellek	72
8.1	Tek Boyutlu Dizi Tanımlama	75
8.2	İki Boyutlu Dizi Tanımlama	79
17.1	Derleme Süreci	141

Tablolar Listesi

1.1	Bazı İşlemcilerin Adres ve Veri Yolu Genişlikleri	2
1.2	Onaltılı Sayı Sistemi	5
1.3	6800 Emir Seti Örneği	6
2.1	Tam Sayılar ve Sayı Sınırları	11
2.2	Kayan Noktalı Sayılar ve Sınırları	12
2.3	İki İle Çarpma Fonksiyonu	12
4.1	Anahtar Kelimeler	22
4.2	Aritmetik İşleçler	28
4.3	Tekli İşleçler	29
4.4	İlişkisel İşleçler	30
4.5	VE, VEYA, ÖZEL VEYA ve DEĞİL İşlemlerinin Doğruluk Tabloları	30
4.6	Bit Düzeyi İşleçler	31
4.7	Mantıksal İşleçler	31
4.8	Atama İşleçleri	32
4.9	İşleç Öncelikleri	32
5.1	En Çok Kullanılan Başlık Dosyaları	34
5.2	Biçim Belirleyiciler	35
5.3	Kaçış Dizileri	36
7.1	Math.h Başlık Dosyası Fonksiyonları	60
7.2	Stdlib.h Başlık Dosyasının Bazı Fonksiyonları	60
7.3	Depolama Sınıfları	70
12.1	Çok Kullanılan Dizgi Fonksiyonları	106
12.2	Karakter Fonksiyonları	108
12.3	Metinden İlkel Veri Tiplerine Dönüşüm Fonksiyonları	109
15.1	Stdarg.h Fonksiyonları	128
16.1	Standart Giriş-Çıkış Dosyaları	131
16.2	Dosya Açma Modları	133
16.3	İkili Dosya Açma Modları	135
17.1	Ön İşlemci Yönergeleri	138
17.2	Standart Ön İşlemci Makroları	139

1. Bölüm

Temel Bilgisayar Kavramları

1.1 Mühendis ve Teknisyen

Mühendis (**engineer**), ihtiyaç duyulan ürünü; ekonomik (**cost**), güvenli (**safe**) ve kullanışlı (**functional**) olarak zamanında (**on time**) ortaya koyan kişilerdir. Bunu yapmak için; keşif (**invent**), tasarım (**design**), analiz (**analysis**), uygulama (**build**) ve sınama (**test**) yaparlar ¹.

Teknisyenler (**technician**) ise genellikle laboratuvarlarda teknik ekipmanlarla ilgilenen veya bu ekipmanlar ile pratik çalışmalar yapan kişilerdir. Genel olarak amacı ekipmanın bakım ve kullanımına ilişkindir.

Konumuz yazılım olduğundan, yazılım mühendisleri istenilen bilgisayar yazılımını ekonomik, güvenli ve kullanışlı olarak zamanında ortaya koyan kişilerdir.

Programcılar ise teknisyenlere karşılık gelir. Yani geliştirme aşamasında bazı yazılım kısımlarının kodlanmasını veya geliştirilen yazılımın çalışır tutulmasını sağlarlar.

1.2 Bilgisayara Niçin İhtiyaç Duyulur?

Bir çiftçi niçin traktöre ihtiyaç duyar? Bir berber niçin elektrikli tıraş makinesi kullanır? Benzer şekilde bir mühendis niçin bilgisayara ihtiyaç duyar? İşte bütün bu soruların cevabı yapılacak işleri daha kısa sürede yapmaktır. Yani bütün araç gereç ve makinalara olan ihtiyacımız işlerimizi **daha hızlı** (**speed**) yapabilmek içindir. Bütün makineler gibi bilgisayar da veriden hızlıca bilgi elde etmemizi ve tekrarlanan hesaplamaları hatasız yapmamızı hızlıca sağlayan makinelerdir.

Veri sorumlusu, veri hazırlama kontrol işletmeni gibi unvanlar bu sebeple ortaya çıkmıştır. Bu unvanlara ihtiyaç duyulmasının sebebi verinin belli bir şekle uygun olarak bilgisayara girilmesi ve işlenen verinin saklanması ve raporlanmasının bilgisayar dünyasında önemli bir yer tutmasıdır. Bilgisayarlar ilk zamanlarda daha çok cebirsel ifadeleri hatasız ve hızlı bir şekilde yapmak için kullanılmışlardır. Bunu daha sonradan ALGOL (**ALGO**ritmic **L**anguage) adını alacak IAL (**I**nternational **A**lgebraic **L**anguage) ya da FORTRAN (**FOR**mula **TRAN**slation) dilinden de anlayabiliriz.

1.3 Veri ve Bilgi

Veriler (**data**), genellikle bir ölçüm ya da sayım sonucu elde edilen ve hakkında az şey bilinen varlıklardır. Örneğin ölçülen hava sıcaklığı, derse giren öğrenci sayısı, tartılan sebzenin ağırlığı gibi şeyler.

Bilgi (**information**) ise verinin işlenerek ortaya çıkan anlamlı verilerdir. Örneğin ortalama hava sıcaklığı, ölçülen yerde bir ölçüm yapmaya gerek kalmadan sıcaklığı yaklaşık olarak bilmeye yarayan bir bilgidir. Bir sınıftaki not ortalaması, o sınıftan rastgele seçilen bir öğrencinin notunu yaklaşık belirleyen bir bilgidir. Kısacası bilgi, işlenmiş veridir

1.4 En Basit Bilgisayar

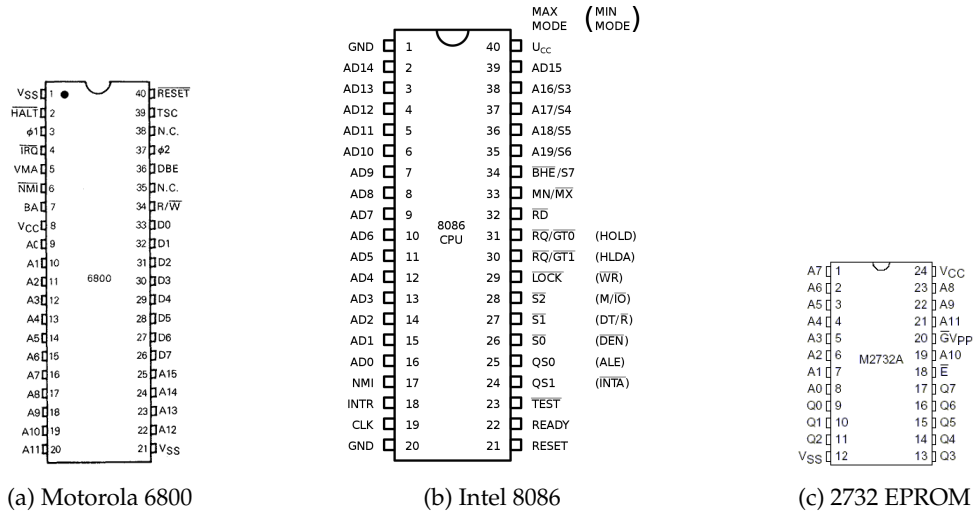
En basit bilgisayar üç bileşenden oluşur;

1. Merkezi İşlem Birimi CPU (**C**entral **P**rocessing **U**nit): Kısaca **işlemci** (**processor**) olarak adlandırılır.
2. Bellek (**memory**): İşlemcinin işleyeceği **emirleri** (**instruction**) ve verileri barındırır.
3. Yol (**bus**): İşlemci ile belleği birbirine bağlayan elektrik yollarıdır.

Hem belleğin hem de işlemcinin adres (**address**) veri (**data**) bacakları bulunur. Bu bacakları karşılıklı olarak yollar (**bus**) birbirine bağlar. Bunların yanında hem işlemcinin hem de belleğin okuma (**read**), yazma (**write**) bacakları bulunur. Ayrıca işlemciye özel kesme (**interrupt**) veya kristal bağlanan bacakları gibi kontrol (**control**) bacakları da bulunur. İşlemci ve bellek, entegre devre (**integrated circuit**) kısaca **yonga** şeklinde üretilirler.

Şekil 1.1'de görüldüğü üzere 6800 işlemcisinin 15 adet adres bacağı, 8 adet veri bacağı bulunmaktadır. 8086 işlemcisinin ise 20 adet adres bacağı bulunmakta olup bu bacakların 16 adedi aynı zamanda veri bacağı olarak kullanılmaktadır. İşlemciye karşılık olarak kullanılacak 2732 belleğin ise 12 adet adres bacağı, Q ile gösterilen 8 adet veri bacağı bulunmaktadır.

¹www.bls.gov/ooh/architecture-and-engineering



Şekil 1.1: Motorola 6802, Intel 8086 İşlemciler ile 2732 EPROM Belleği

Görüldüğü üzere işlemci ile bellek arasında bağlantı sağlayan elektrik **yolları (bus)** adres veya veri taşıır. Bu yolların veri taşıyanlarına **veri yolu (data bus)**, adres taşıyanına ise **adres yolu (address bus)** adı verilir. Veri yolu; hem işlemci tarafından belleğe veri yazmak veya bellekten işlemciye taşınacak veriler için kullanıldığından çift yönlüdür. Adres yolu ise belleğin hangi bölgesindeki veriye ulaşılacağını belirleyen hatlar olup tek yönlüdür. 8086 işlemcisinde AD ile işaretli veri ve adres için kullanılan ortak yolların ne zaman adres ne zaman veri göstereceği S0 bacağından belli olur.

Bilgisayarlar elektrikle çalışan aletlerdir. İşlemci ile bellek arasında bir hatta elektrik varsa **1** yoksa **0** olarak anlamlandırılır. Bunlar **ikili (binary)** sayı sisteminin rakamlarıdır ve kısaca bit (**B**inary **D**igi**T**) olarak adlandırılır. Dolayısıyla; bir elektrik hattının taşıyacağı veri, bir haneli ikili sayı ile, iki elektrik hattının taşıyacağı veri, iki haneli ikili sayı ile ... ifade edilebilir.

İşlemcinin veri yolunun genişliği **kelime uzunluğudur (word length)**. ve işlemcinin aynı anda işlem yaptığı bit sayısını da verir. Sekiz (8) bitlik veriye **bayt (byte)** adı verilir ve bellek miktarı genellikle bayt olarak ölçülür.

İşlemcinin veri yolu, 8 bit ise BYTE işlemci, 16 bit ise WORD işlemci, 32 bit ise DWORD (Double WORD) işlemci, 64 bit ise QWORD (Quad WORD) işlemci olarak adlandırılır.

İşlemcinin adreslenebilir bellek miktarı ise işlemcinin adres hatlarının sayısı yani adres yolu ile belirlenir. **Adres yoluna konulan her bir adres ile 1 bellek baytı adreslenir.** Adres yolu ne kadar geniş ise adreslenebilir bellek miktarı da o kadar artar. Örneğin 64 bit adres bacağına sahip işlemci $2^{64}=4.294.967.296$ kadar bellek baytını adresler.

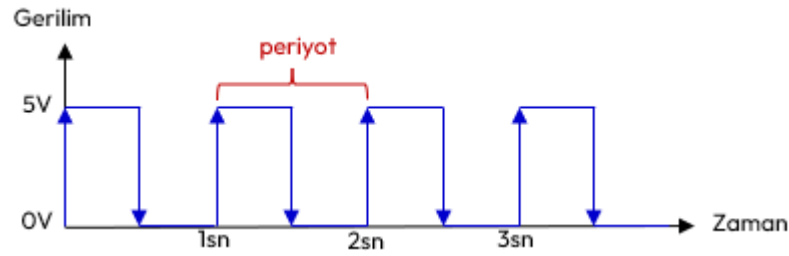
İşlemci	Veri Yolu	Adres Yolu	Adreslenebilir Bellek	Kilo bayt	Mega bayt	Giga bayt
Motorola 6802	8 Bit	16 Bit	$2^{16}=65.536$	64		
Intel 8086	16 Bit	20 Bit	$2^{20}=1.048.576$	1024	1	
Intel 80286	16 Bit	24 Bit	$2^{24}=16.777.216$		16	
Intel Pentium	32 Bit	32 Bit	$2^{32}=4.294.967.296$		4096	4
Intel i7	64 Bit	36 Bit	$2^{36}=68.719.476.736$			64

Tablo 1.1: Bazı İşlemcilerin Adres ve Veri Yolu Genişlikleri

Bilgisayarlarda kablodaki gerilimin değeri genelde 5V, 3.3V, 2.8V, 2V, 1.5V veya 1V gibi değerler alır. **Programcılar için gerilimin ne olduğu değil varlığı önemlidir.** Gerilim değerin yüksek olması fazla akım çekme ve ısınma anlamına gelir. Bu nedenle günümüzde daha düşük gerilimle çalışan işlemciler tasarlanmaktadır.

İşlemciler kare dalga olarak oluşturulmuş elektriksel bir işareti saat (**clock**) olarak kullanır. Şekil 1.2'de gösterilen bu elektrik işareti işlemciye ϕ ya da CLK bacaklarından uygulanır. Saniyede 1 kez bir dalga üreten işaretin frekansı $1/1\text{sn} = 1 \text{ Hertz} = 1 \text{ Hz}$. olarak hesaplanır. Bir mikro saniyede bir kez dalga üreten işaretin frekansı; $1/(1\mu\text{s}) = 1/(10^{-6}\text{s}) = 10^6 \text{ Hertz} = 1.000.000 \text{ Hertz} = 1 \text{ MHz}$ olarak hesaplanır.

İşlemciye uygulanan saatin frekansı ne kadar yüksek olursa işlemci o kadar hızlanır. Ancak işlemcilerin de tasarımından ötürü karşılayabileceği belli bir frekans değeri bulunmaktadır. Günümüzde 5 GHz frekansını karşılayan işlemciler bulunmaktadır.

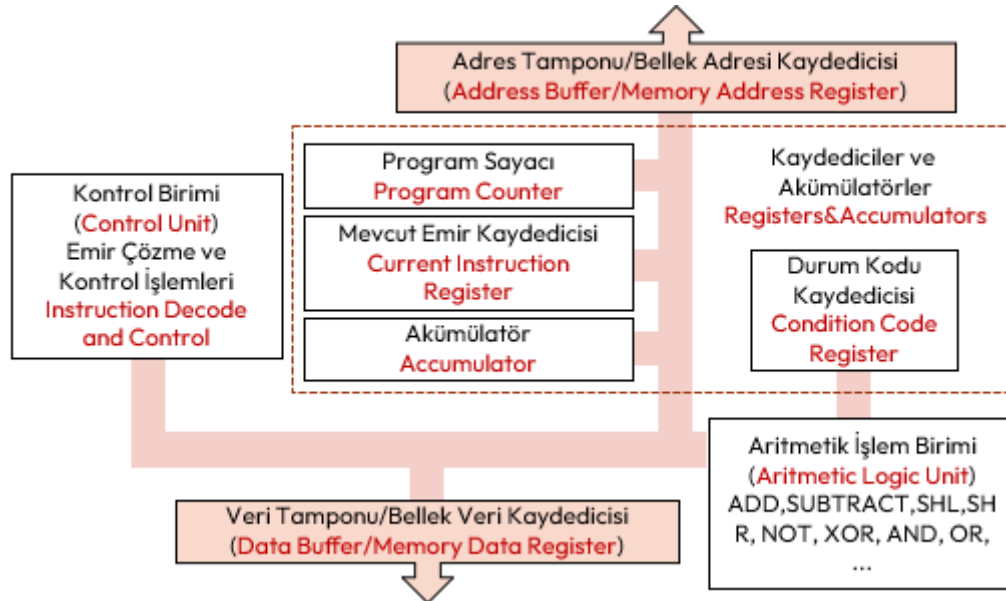


Şekil 1.2: Frekansı 1 olan Kare Dalga İşareti

1.5 Emri Alma, Çözme ve İcra Çevrimi

Basit bir işlemciadaki bileşenleri aşağıdaki şekilde açıklayabiliriz;

- ◊ **Kaydediciler (register)**, işlemci içerisinde **çok hızlı e geçici adres veya veri saklayan** alanlardır.
- ◊ **Akümülatörler (accumulator)** ise bellekten alınan verileri saklamak veya işlenen sonuçları saklamak için en sık kullanılan kaydedicilerdir.
- ◊ **Program sayacı PC (program counter)**, işlemcinin icra edeceği **emri (instruction)** takip için kullanılır. Getirilecek bir sonraki emrin bellek adresini içerir.
- ◊ **Mevcut Emir kaydedicisi CIR (current instruction register)** işlemcinin o an icra ettiği emirdir.
- ◊ **Koşul kodu kaydedicisi (condition code register)**, herhangi bir işlemin durumunu gösteren farklı bayraklar (**flag**) içeren kaydedicidir.
- ◊ **Aritmetik İşlem Birimi ALU (arithmetic logic unit)**, işlemcinin toplama çıkarma ve karşılaştırma yapan birimidir.
- ◊ **Kontrol Birimi CU (control unit)** Orkestra şefi gibidir, emirleri çözer ve yorumlar ve gerekli bileşenlere sinyal gönderir.
- ◊ **Veri Yolları (bus,,)**, verinin bileşenler arasında taşındığı birden çok elektrik hattıdır.



Şekil 1.3: Basit Bir İşlemcinin Yapısı

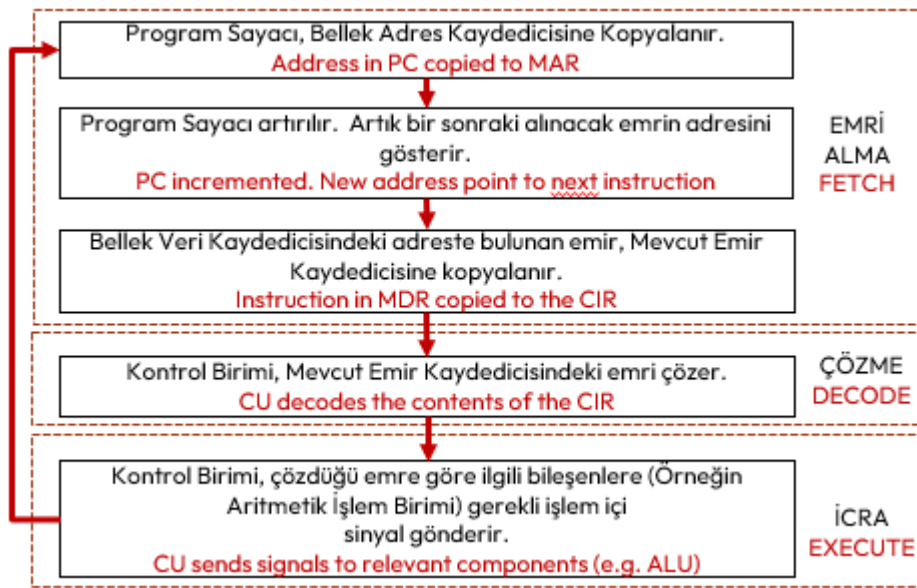
Bu işlemci tasarımı aynı zamanda **depolanmış program (stored program)** kavramı olarak adlandırılır ve **Von Neumann Mimarisi** olarak da bilinir. *John Von Neumann* tarafından 1946 ila 1949 yılları arasında tasarlanan ve Şekil 1.3'de verilen bu mimari günümüzde modern bilgisayarlarda hala kullanılmaktadır.

Basit bir bilgisayara elektrik verildiğinde, işlemci aşağıdaki üç adımlık **emir alma (fetch)**, **çözme (decode)** ve **icra (execute)** çevrimini elektrik kesilene kadar sürekli tekrarlanır². Bu çevrime **emir çevrimi (instruction cycle)** adı da verilir.

1. **Emri Alma (fetch)**: Bu aşamada program sayacının (**program counter**) gösterdiği adresteki emir (**instruction**) bellekten okunur:
 - (a) Program sayacı, bir sonraki emrin adresini tutar.
 - (b) Bu adres **bellek adres kaydedicisine (memory address register)** kop-

²<https://www.youtube.com/watch?v=ByllwN8q2ss>

- yalanır.
- (c) Talimat bellekten çağrılır ve **bellek veri kaydedicisine** (**memory data register**), oradan da **mevcut emir kaydedicisine** (**current instruction register**) aktarılır.
 - (d) Program Sayacı, bir sonraki emri işaret etmek için bir artırılır.
2. **Çözme** (**decode**): Kontrol birimi (control unit) tarafından emrin ne anlama geldiği ve buna bağlı nelerin yapılacağını (örneğin ulaşılacak bellek adresi değiştirilir) belirler:
- (a) Mevcut emir kodu (**current instruction code**), ne yapılacağını belirten kısım yani işlem kodu (**opcode**) ve hangi veriyle yapılacağını belirten işlenen (**operand**) kısımdan oluşur.
 - (b) Kontrol birimi, varsa emrin işaret ettiği veriyle işlemin gerçekleşmesi için gerekli donanım parçalarını hazırlar.
3. **İcra** (**execute**): Çözülen emir koduna bağlı olarak işlemcinin bazı bileşenleri çalıştırılır:
- (a) Eğer bir matematiksel işlem veya mantıksal karşılaştırma gerekiyorsa, bu görev aritmetik işlem birimini ALU (**Aritmetic Logic Unit**) tarafından yapılır. İşlem sonuçları ortaya çıkan değerler ise akümülatörlere (**accumulator**) veya bellek veri kaydedicisi (**memory data register**) üzerinden belleğe yazılır.
 - (b) Bazı durumlarda ise kaydedici (**register**) içerikleri değiştirilir.



Şekil 1.4: İşlemcinin Emri Alma, Çözme ve İcra Çevrimi

Burada çözülecek emirlerin (**instruction**) her biri bir ikili sayıya karşılık gelir. Bu sayılara **emir kodu** (**instruction code**), **makine kodu** (**machine code**) veya **işlem kodu** (**opcode**) olarak da bilinirler. İşlemciler tüm sayıları emir kodu olarak anlamaz. Yani sınırlı sayıda sayı emri tanırlar. Tablo 1.1’de verilen Motorola 6802 işlemcisi 72 farklı emri, Intel 8086 işlemcisi ise 116 farklı emri tanır. Çözülebilecek tüm emirlerin oluşturduğu kümeye **emir seti** (**instruction set**) adı verilir.

1.6 İşlemci Tasarlama Stratejileri

İşlemciler, aşağıda verilen üç strateji ile tasarlanırlar;

1. **İndirgenmiş Emir Setli İşlemciler** olan RISC (**Reduced Instruction Set Computing**) İşlemci: Az sayıda emir alan yüksek hızlı işlemci tasarımı. RISC işlemciler az sayıda emre sahip olduğundan emrin çözülmesi ve icrası da kısa zaman almaktadır. Bu da çalıştırılacak kodu büyütür.
2. **Karmaşık Emir Setli İşlemciler** olan CISC (**Complex Instruction Set Computing**) İşlemci: Çok fazla emri anlayan karmaşık nispeten yavaş işlemci tasarımı. CISC işlemciler, çok sayıda emir tanıyabildiğinden emrin çözülmesi ve icra edilmesi nispeten daha uzun sürer. Ancak yüksek düzey dillerin daha kolay makine koduna çevrilmesini kolaylaştırır.
3. **Özel İşlemci**: Bunlar grafik kartları ve yapay zekâ gibi belli amaçlar için tasarlanmış işlemcilerdir.

Genel olarak işlemcilerde emir çevrimi (**instruction cycle**) 1 saat periyodunda tamamlanmaz. RISC işlemciler için bu çevrim, birçok emir için 1 saat periyodunda tamamlanır. CISC işlemciler için ise birden fazla saat periyodu gerekir. Çünkü CISC işlemcilerin emir seti daha geniş olduğundan emrin çözülmesi daha fazla vakit alır. Bu nedenle RISC işlemciler CISC işlemcilere göreli olarak daha hızlıdır.

Günümüzde ticari olarak satılan işlemcilerin çoğu ortak emir seti kullanmaya başlamışlardır;

- ◇ Masaüstü bilgisayarların birçoğu Intel ve AMD Marka işlemciler kullanır ve bunlar IA-16, IA-32, x86-16, x86-64, AMD64, ... emir setleri ve eklentilerini desteklemektedir.
- ◇ Bilinen birçok telefon işlemcisi Acorn Risc Machine-ARM tabanlı işlemciler kullanır. Bu işlemcilerde ARMv7, ARMv8, ... emir setleri ve eklentileri kullanılmaktadır.

1.7 Onaltılı Sayı Sisteminin Kullanılması

İşlemci ile bellek arasında verilerin elektrik yolları (bus) ile taşındığı anlatılmıştı. Veri taşıyan elektrik hattı sayısı işlemci ile bellek arasında taşınabilecek verinin sınırlarını da belirler. Bir elektrik hattının taşıyacağı veri bir bit ile (0 veya 1) gösterilebilir. İki elektrik hattının taşıyacağı veri ise iki bit ile (00, 01, 10 veya 11) 2^2 kadar farklı gösterilebilir. Benzer şekilde n adet elektrik hattının taşıyacağı veri 2^n farklı alternatifte veri olacaktır.

Bit sayısı arttıkça ikili sayının okunup yazılması da çok zor olacaktır. Günümüzde 64 bitlik hatta 1024 bitlik veri yoluna sahip bilgisayarlar bulunmaktadır. Bunun üstesinden gelmek için (hexadecimal) sayı sistemi kullanılır. Çünkü her 4 elektrik hattının verisi onaltılık sayı sisteminin bir rakamına karşılık gelir.

Onaltılı Rakam	İkili Karşılığı	Onlu Karşılığı
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
A	1010	10
B	1011	11
C	1100	12
D	1101	13
E	1110	14
F	1111	15

Tablo 1.2: Onaltılı Sayı Sistemi

Kod içerisinde onaltılık sayıların rakamlarının diğer sayı sistemindeki rakamlarla karışmaması için sonuna **h** konularak **113Ah** şeklinde veya çoğunlukla başına **0x** konularak **0x1133** şeklinde kullanılır.

Örnek olarak ikili tabanda verilen 64 bitlik 0001 1010 1011 1101 0101 0100 1001 1010 0011 0010 0100 0110 1001 1101 1100 1111 sayısının onaltılı tabanda karşılığı oldukça kısa (16 hane) olarak 1ABD 549A 3246 9DCF şeklindedir.

Onaltılı sayının A,B,C,D,E ve F rakamları büyük harf olacağı gibi küçük harf de olabilir. Anlam olarak bir şey değişmez.

Bilgisayar dünyasında ikili sayıları onaltılı sayı sistemiyle ifade etmek çoğunlukla zorunluluktur. Zamanla standart hale gelmiştir.

1.8 Program ve Programlama

Belli bir işlemi gerçekleştirmek üzere, işlemciye verilen bir dizi emre (instruction), **bilgisayar programı** (computer program) denir. Bu bir dizi emrin, işlemcinin çalıştıracağı şekilde hazırlanması işlemine **programlama** (programming) denir. Programlama **emir kodları** (instruction code) ile yapıldığından bu programın dili, **makine dili** (machine language) olarak adlandırılır³.

Her işlemci üreticisi kullandığı emir seti farklı olduğundan yani her bir emre farklı makine kodu verdiğinden programlama oldukça zorlu ve teknik bilgi gerektirmekteydi. Burada daha önce yazılmış bir programın ne yaptığını anlamak için her defasında bu listelere defalarca bakmak gerekiyordu. Programın okunurluğunu artırmak için her bir emre, hatırlatıcı **sembolik isimler** (mnemonic) verilmeye başlandı. Tablo1.2'de 6802 işlemcisinin birkaç sembolik ismi verilmiştir.

³ISO/IEC 2382:2015. ISO. 2020-09-03. [Software includes] all or part of the programs, procedures, rules, and associated documentation of an information processing system

Sembolik İsim (Mnemonic)	Emir Kodu (Instruction Code)	Açıklama
ABA	1B	ADD A to B → A
INCA	4C	INCREMENT A
INCB	5C	INCREMENT B
SBA	10	SUBTRACT B from A → A
LDA	96	LOAD M to A, M: Memory
LDB	D6	LOAD M to B, M: Memory

Tablo 1.3: 6800 Emir Seti Örneği

Bu sembolik isimlerle yazılan program, **montaj kodu** (**assembly code**) olarak adlandırılır. Montaj kodları bir metin dosyasına yazılarak bir dosyaya saklanır. Bu dosyalar montaj dosyaları olarak adlandırılır. Montaj dosyalarını makine diline çeviren programlar yazılmıştır. Bu programlar ilk **derleyicilerdir** (**compiler**). Bu derleyiciler, sembolik isimlerle yazılmış program kodlarını makine koduna çeviren programlardır.

Aşağıda x86 emir seti kullanan Linux işletim sisteminde örnek montaj kodu örneği verilmiştir; Örnekte, genel amaçlı CL kaydedicisine koyulan sayıya kadar 2 ve 3'e bölünebilen sayıların toplamını bulup yine genel amaçlı DX kaydedicisine konulmaktadır⁴.

```
section .text ; programı oluşturan emir kodları buradan başlar
global _start

_start:
    mov dx, 0 ; dx 0 olsun. Toplamı tutacaktır.
    mov cl, 99 ; 99 kez tekrarlanacak.
    sum:
    mov ax, cx ; cx, ax e kopyalanır. Bölme işlemi için.
    mov bl, 3 ; bl 3 olsun.
    div bl ; ax/bl.
    cmp ah, 0 ; Kalan olup olmadığı kontrol ediliyor.
    jne cont ; Kalan yoksa, 'cont' etiketine git.
    mov bl, 2 ; Sonra bl 2 olsun.
    div bl ; ax/bl
    cmp ah, 0 ; Kalan olup olmadığı kontrol ediliyor.
    jne cont ; Kalan yoksa, 'cont' etiketine git.
    add dx, c ; Değilse, cx deki sayı 2 ve 3'e bölünür. Toplama ekle.
cont:
    loop sum ; Bir sonraki sayı için 'sum' etiketine dön.
exit:
    mov eax, 1 ; Artık programdan çıkıyoruz.
    mov ebx, 0 ; Return 0.
    int 80h ; linux çekirdeğini çağır. (soft interrupt 80h!).
```

1.9 Genel Amaçlı Bilgisayarlar ve İşletim Sistemleri

Genel amaçlı yani herkesin kendi amacına göre kullanacağı bilgisayarlar, veri girişini sağlayan klavye, verileri görüntüleneceği monitör, işlenen verilerin elektrik kesildiğinde saklanacağı disk ve benzeri donanımlara sahip olmalıdır.

Genel amaçlı bilgisayarın kullanıcı isteklerine cevap verebilmesi için;

1. Çeşitli programlara sahiptirler. İşte bu programlar bütününe işletim sistemi (**operating system**) adı verilir. DOS, Windows, Unix, Linux, MacOS, ChromeOS, OS/2 bunların en çok bilinenleridir.
2. Elektrik verildiğinde klavye, monitör ve disk olup olmadığı kontrol eden **BIOS** (**Basic Input Output System**) programı çalıştırılır. BIOS gerekli kontrollerden sonra bilgisayara işletim sistemini yükler. İşletim sistemi yüklendikten sonra, işletim sistemi kullanıcının vereceği komutları bekler.
3. Kullanıcı işletim sistemine vereceği komutlarla yeni bir program derleyebilir, derlenmiş bir programı çalıştırabilir.
4. Kullanıcının vereceği komutlar grafik olmayan işletim sistemlerinde (DOS, Unix, Linux gibi) komut yazılarak verilir. Komut yazılan bu ekrana konsol, terminal ya da komut satırı adı verilir. Sunucu işletim sistemlerinde hala tercih edilmektedir.
5. Grafik ara yüzlerine sahip işletim sistemlerinde (Windows ve MacOS İşletim Sistemleri ile KDE, Mate, GNome, Cinnamon, LXQt, XFce ve Deepin gibi grafik ara yüzleri içeren Linux İşletim Sistemleri) fare

⁴<https://github.com/armut/x86-Linux-Assembly-Examples>

tıklamasıyla çoğu komut verilebilmektedir.

İşletim sistemlerinin yaygınlaşması ile kullanıcının bilgisayarı donanım bilmeden yapabilmesi sağlanmıştır.

Günümüzdeki işletim sistemlerinin birkaçı hariç tamamında temel programlama dili olarak C dili kullanılmaktadır.

1.10 Düşün ve Çöz

- ◇ **Düşün:** Bir mühendis ile teknisyen arasındaki temel farkı, bir yazılım projesindeki görev dağılımı üzerinden örneklendiriniz.
- ◇ **Düşün:** CPU içindeki kaydedicilerin (**register**) hız ve kapasite açısından bellekten farkı nedir? Neden tüm verilerimizi kaydedicilerde tutmuyoruz?
- ◇ **Düşün:** *Von Neumann* mimarisinde veri ve komutların aynı yolu (bus) kullanmasının **darboğaz** (**bottle-neck**) yaratma riskini tartışınız.
- ◇ **Çöz:** 32-bit veri yoluna ve 24-bit adres yoluna sahip bir işlemcinin adresleyebileceği maksimum bellek miktarını MB cinsinden hesaplayınız.
- ◇ **Çöz:** 16-bit adres yoluna sahip bir mikrodenetleyicinin adresleyebileceği maksimum bellek miktarını KB cinsinden hesaplayınız.
- ◇ **Çöz:** Bir bilgisayarın 8 GB RAM'i tam olarak adresleyebilmesi için adres yolunun (**address bus**) en az kaç bit olması gerektiğini bulunuz.

2. Bölüm

Temel Yazılım Kavramları

2.1 Düşük Düzey Programlama

1642 Yılında *Blaise Pascal* tarafından icat edilen Pascalline Hesap Makinesi, eldeli toplama, ödünç almalı çıkarma yapıyordu.

1671 Yılında *Gottfried Wilhelm Leibniz* tarafından icat edilen Leibniz Çarkı, toplama, ödünç almalı çıkarmanın yanında bölme, çarpma ve karekök işlemlerini yapıyordu.

1801 Yılında *Joseph Maria Jacquard* dokuma tezgahlarında desenleri oluşturmak için **delikli kartlar (punch card)** kullanmayı icat etmiştir. Sonradan bilgisayara veri girdisi sağlamak ve sonuçların çıktısını göstermek için kullanılacaktır.

1830 Yılında *Charles Babbage* fark makinesi olan analitik makineyi icat etmiştir. Buhar gücü kullanan bu makinenin mantıksal işlem birimi, veri depolama birimi, giriş çıkış üniteleri bulunuyordu. *Augusta Ada Byron*, Babbage ile beraber çalışmıştır. Byron, analitik makinenin bir dizi matematiksel işlemi gerçekleştirebildiğini fark etti ve bu makineyi Bernoulli sayılarını üretmek için kullandı. Bu sayıların hesaplanması için yazdığı algoritma tarihteki ilk bilgisayar programı olarak kabul edilir.

Augusta Ada Byron, **ilk programcı** olarak kabul edilir.

1854 Yılında *George Boole* tarafından ikili sayı sistemini geliştirmiş bugün bilgisayarlarımızın kullandığı **bool cebiri** üzerine çalışmalar yapmıştır.

1890 Yılında *Herman Hollerith* tarafından delikli kartlarla bilgilerin yüklenebildiği ve bu bilgiler üzerinde toplama işlemlerinin yapılabilirdiği bir elektro mekanik araç geliştirdi. ABD'nin 1890 nüfus sayımında başarılı biçimde kullanıldı. Hollerith başarılı olunca bir şirket kurdu ve daha sonra üç firma birleşerek 1924 yılında adını **IBM** olarak değiştirdi.

1941 Yılında *Konrad Zuze* Z3 isimli elektrik motorları ile çalıştırılan mekanik bir bilgisayar yaptı. Bu (Z1, Z2, Z3 ve Z4 serisi) program kontrollü ilk bilgisayardır.

1944 Yılında *Howard Aitken*, IBM ile iş birliği yaparak **MARK-I**'i yaptı. Bilgiler, delikli kartlarla veriliyor ve sonuçlar yine delikli kartlarla alınıyordu. Saniyede 5 işlem yapabiliyordu. Boyutları, 18 m uzunluğunda ve 2,5 m yüksekliğinde idi. İnsan müdahalesi olmadan sürekli olarak, hazırlanan programı yürüten ilk bilgisayar idi ve tam olarak elektronik bir bilgisayar değildi.

1943-1945 Yılları arasında *Konrad Zuze*, ilk yüksek düzey programlama dili olan Plankalkül'ü, Z1 bilgisayarı için geliştirmiştir.

Emir kodlarını (**instruction code**) ve sembolik isimleri (**mnemonic**) içermeyen programlama dilleri, **yüksek düzey programlama dili (high level programming language)** olarak adlandırılır. Yüksek düzey programlama dilleri çoğunlukla İngilizce sözcükleri kullanır.

1946 Yılında *John Von Neumann* önderliğinde Pensilvanya Üniversitesinde **ENIAC** (Elektronik Sayısal Hesaplayıcı ve Doğrulayıcı) isimli sayısal elektronik bilgisayar tamamlandı. Askeri amaçla üretildi ve top mermilerinin menzillerini hesaplamak için kullanıldı. Neumann tarafından geliştirilmiş bu mimari (**Stored-program computer and Universal Turing machine** § **Stored-program computer**), günümüzdeki bilgisayarlarda hala kullanılmaktadır. 18,000 adet elektronik tüp kullanılan ENIAC; 150 kW gücünde idi ve 50 ton ağırlığıyla 167 m² yer kaplıyordu. Saniyede 5.000 toplama işlemi yapabiliyordu. MARK-I bilgisayarından bin kat daha hızlıydı.

Yukarıda anlatılan bilgisayarlar, bir önceki bölümde anlatılan emir kodu (**instruction code**) veya sembolik isimlerle (**mnemonic**) yazılan programlar ile programlanmaktadır. Emir kodu ve sembolik isimlerle yapılan ve bilgisayar mimarisini bilmeyi gerektiren programlamaya **düşük düzey programlama (low level programming)** olarak adlandırılır. Daha çok elektronik ve bilgisayar mühendisleri ya da sistemi tasarlayan matematikçiler bu programlama türünü kullanır.

Düşük düzey programlamanın yapılabilmesi için hem işlemcinin hem de hafıza ve işlemciye bağlanan diğer bileşenlerin nasıl çalıştığı ve tasarlandığını çok iyi bilinmesi gerekir.

2.2 Genel Amaçlı Yüksek Düzey Diller

Sembolik isimler veya makine kodu yerine bildiğimiz konuşma diline yakın talimatlarla metin olarak yazılan programlama dilleri, **yüksek düzey programlama dilleri** (**high level programming language**) olarak adlandırılır. Yani emir kodu (instruction code) ve sembolik isim (mnemonic) kullanmayan programlama dilleri yüksek düzey programlama dilleridir.

Yüksek düzey dillerde **talimatlar** (**statement**) olarak yazılan programlar yine kendine has derleyiciler tarafından makine diline çevrilir. Her talimat, genellikle birden fazla emirden (**instruction**) oluşur.

Yüksek seviyeli bir dilde talimatlar, yapılması gerekeni kısa ve net olarak belirten **mantıksal satırlardır** (**logical sequence**).

1954 Yılında **FORTRAN** (**FORmula TRANslation**), *John Backus* liderliğinde IBM 704 için geliştirildi. FORTRAN, **READ**, **WRITE**, **IF** gibi 32 farklı talimata (**statement**) sahiptir. İlk genel amaçlı, yüksek düzey bir programlama dilidir. Aşağıda üç kenarı teyp ünitesinden alınan bir üçgenin alanını hesaplayan FORTRAN-II programı verilmiştir¹. Görüldüğü üzere talimatlar kısa öz ve anlaşılırdır.

```
C AREA OF A TRIANGLE - HERON'S FORMULA
C INPUT - CARD READER UNIT 5, INTEGER INPUT
C OUTPUT -
C INTEGER VARIABLES START WITH I,J,K,L,M OR N
READ(5,501) IA,IB,IC
501 FORMAT(3I5)
IF (IA) 701, 777, 701
701 IF (IB) 702, 777, 702
702 IF (IC) 703, 777, 703
777 STOP 1
703 S = (IA + IB + IC) / 2.0
AREA = SQRT( S * (S - IA) * (S - IB) * (S - IC) )
WRITE(6,801) IA,IB,IC,AREA
801 FORMAT(4H A= ,I5,5H B= ,I5,5H C= ,I5,8H AREA= ,F10.2,
$13H SQUARE UNITS)
STOP
END
```

1956 Yılında ticari amaçlarla kullanılabilen ve seri halde üretimi yapılan ilk bilgisayar **UNIVAC-I** oldu. UNIVAC, giriş-çıkış birimleri manyetik bant idi ve bir yazıcıya sahipti. UNIVAC 1951-1959 arasında üretilen bilgisayarlarda vakum tüpleri kullanıldı. Bu tüpler bir ampul büyüklüğünde, çok fazla enerji harcamakta ve çok fazla ısı yaymakta idiler. Veri ve programlar manyetik teyp ve tambur gibi bilgi saklama araçlarıyla saklandı. Veriler ve programlar bilgisayara delgi kartları ile yükleniyordu.

1959 yılı sonrasında üretilen bilgisayarlarda transistörler (yaklaşık 10 bin adet) kullanıldı. Bu yıl ve sonrasında transistör içeren **ikinci kuşak bilgisayarlar** hayatımıza girmiştir.

2.3 Arapsaçı Kod

1955-1959 Yılları arasında **FLOW-MATIC B0** (**Business Language Version 0**) Dili, *Grace Hopper* tarafından UNIVAC-I için geliştirildi. İngilizceye benzeyen ilk yüksek düzey programlama dilidir. Aşağıda B0 programlama dilinde örnek bir program verilmiştir².

```
(0) INPUT INVENTORY FILE-A PRICE FILE-B ; OUTPUT PRICED-INV FILE-C UNPRICED-INV
FILE-D ; HSP D .
(1) COMPARE PRODUCT-NO (A) WITH PRODUCT-NO (B) ; IF GREATER GO TO OPERATION 10 ;
IF EQUAL GO TO OPERATION 5 ; OTHERWISE GO TO OPERATION 2 .
(2) TRANSFER A TO D .
(3) WRITE-ITEM D .
(4) JUMP TO OPERATION 8 .
```

¹https://github.com/scivision/fortran-II-examples/blob/main/herons_formula.f

²<https://gist.github.com/marcgg/f9f2da31a973ba319809>

```

(5)  TRANSFER A TO C .
(6)  MOVE UNIT-PRICE (B) TO UNIT-PRICE (C) .
(7)  WRITE-ITEM C .
(8)  READ-ITEM A ; IF END OF DATA GO TO OPERATION 14 .
(9)  JUMP TO OPERATION 1 .
(10) READ-ITEM B ; IF END OF DATA GO TO OPERATION 12 .
(11) JUMP TO OPERATION 1 .
(12) SET OPERATION 9 TO GO TO OPERATION 2 .
(13) JUMP TO OPERATION 2 .
(14) TEST PRODUCT-NO (B) AGAINST ZZZZZZZZZZ ; IF EQUAL GO TO OPERATION 16 ;
      OTHERWISE GO TO OPERATION 15 .
(15) REWIND B .
(16) CLOSE-OUT FILES C ; D .
(17) STOP . (END)

```

Görüldüğü üzere bu tarihlere kadar yazılan programlarda bir sürü **GO TO** ya da **JUMP TO** talimatı (**statement**) bulunmaktadır. Bu durumda programın ne yaptığı konusunda fikir yürütmeye çalıştığımızda **okunaklılık oldukça azdır**. İşte okunaklılığın az olduğu bu program kodlarına **arapsaçı kod (spaghetti code)** adı verilir. Günümüzde ne yaptığı kolay anlaşılamayan kodlar için bu kavram çokça kullanılmaktadır.

İşin doğası gereği **makine diline çevrilecek tüm kaynak kodlar gözle okunmalıdır!**

1958 Yılında **LISP** programlama dili, *John McCarthy* önderliğinde *Steve Russell* tarafından geliştirilmiş ikinci yüksek düzey programlama dilidir;

```

(format t "Adinizi Giriniz: ")
(setq name (read))
(princ "Merhaba ")
(write name)

```

1959 Yılında **COBOL** (**Common Business-Oriented Language**) programlama dili, CODASYL komitesi tarafından İngilizceye benzer ikinci yüksek düzey bir dil olarak geliştirilmiştir;

```

IDENTIFICATION DIVISION.
PROGRAM-ID. GREET.

```

```

DATA DIVISION.
WORKING-STORAGE SECTION.
01 USER-NAME PIC A(30).

```

```

PROCEDURE DIVISION.
DISPLAY "Enter your name: ".
ACCEPT USER-NAME.
DISPLAY "Hello, " USER-NAME "!".
STOP RUN.

```

1958 Yılında daha sonra **ALGOL** (**ALGOritmic Language**) adını alan **IAL** (**International Algebraic Language**) programlama dili geliştirilmiştir. Bu dil daha sonra ortaya çıkan birçok dile önderlik etmiştir. Bu dil ile birlikte kod blokları (**code block**), iç içe fonksiyon (**nested function**), değişken kapsamı (**scope**) ve dizi (**array**) kavramları programcının hayatına girmiştir. ALGOL dili, barındırdığı özellikler sebebiyle, kendisinden sonra ortaya çıkan bütün programlama dilleri için bir esin kaynağı olmuştur;

```

BEGIN
FILE F (KIND=REMOTE);
EBCDIC ARRAY E [0:11];
REPLACE E BY "HELLO WORLD!";
WRITE (F, *, E);
END.

```

1958 Yılındaki FORTRAN diline diline çıktı üretmeyen fonksiyonlar için **SUBROUTINE**, değer üreten fonksiyonlar için **FUNCTION**, **CALL** ve **RETURN** özellikleri eklenmiş ve **FORTRAN-II** adını almıştır.

Benzer şekilde 1958 ile 1969 yılları arasında ALGOL diline çıktı üretmeyen fonksiyonlar için **PROCEDURE** ve **BEGIN-END** kod bloğu özellikleri eklenmiştir.

1966 Yılındaki FORTRAN-IV diline **INTEGER**, **REAL**, **DOUBLE PRECISION**, **COMPLEX** ve **LOGICAL veri tipleri (data type)** ile Mantıksal **IF**, aritmetik (üç yollu) **IF** ve **DO** döngüsü talimatları (**statement**) eklenmiştir.

1964 yılında Dartmouth College'da John G. Kemeny ve Thomas E. Kurtz tarafından geliştirilen **BASIC** (**B**eginner's **A**ll-purpose **S**ymbolic **I**nstruction **C**ode) öğrenilmesi ve kullanılması kolay olacak şekilde tasarlanmış bir programlama dilidir. Bilimsel olmayan alanlardaki öğrencilerin ve yeni başlayanların bilgisayar kullanabilmesini sağlamak amacıyla İngilizce benzeri komutlarla tasarlanmıştır. Karmaşık donanım detaylarını gizleyerek programcının mantığa odaklanmasına izin verir. Kodlar genellikle satır satır gerçek zamanlı olarak çalıştırılır, bu da hataların anında bulunmasını kolaylaştırır. 1980'li yılların başında kadar ev bilgisayarlarının (Apple II, Commodore 64, IBM PC gibi) varsayılan dili haline gelmiştir;

```
10 REM A simple program to greet the user
20 CLS ' Clears the screen (command might vary by implementation)
30 PRINT "What is your name? ";
40 INPUT NAME$ ' The '$' indicates a string variable
50 PRINT "Hello, "; NAME$; "!"
60 GOTO 30 ' Jumps back to line 30 to run the program again
70 END ' Terminates the program
```

2.4 Değişken

Değişkenler (**variable**) aslında cebirde kullandığımız değişkenlerdir. Cebir (**algebra**) işlemlerinde doğru-
dan problemde verilen sayıları değil, onları temsil eden değişkenleri kullanırız.

$$3x^2 + 2x + 10$$

$$c^2 = a^2 + b^2$$

$$c = 2\pi r$$

Yukarıdaki cebirsel ifadelerin ilkinde bir polinom, ikincisinde ise bir dik üçgenin kenar uzunlukları arasındaki ilişki verilmiştir. Bu ifadelerde x , a , b ve r **bağımsız değişken**, c ise **bağımlı değişkendir**. Örneklerdeki 3,2,10,2 ve π ise **sabit** (**constant**) sayılardır.

Değişkenler, bilgisayarlarda bir miktar bellek bölgesini işgal eder ve bu bellek bölgesinde çeşitli biçimlerde tutulurlar.

Tam sayılar, ikili (**binary**) sayı olarak bellekte tutulur. Tamsayıyı ifade eden ikili sayının en anlamlı biti, işaret (**sign**) biti olarak adlandırılır ve bu bitin değeri 1 ise sayı negatif, 0 ise sayı pozitifdir. Tablo 2.1'de en çok kullanılan tam sayılara ilişkin bellek miktarı ve sayı sınırları verilmiştir.

Bit Sayısı	Bellek Mik-tarı	Sayı Sınırları
8 Bit	1 bayt	-127 ile +128 arasında veya 0 ile 255 arasında
16 bit	2 bayt	-32.767 ile +32.768 arasında veya 0 ile 65.535 arasında
32 bit	4 bayt	-2.147.483.648 ile +2.147.483.647 arasında veya 0 ile 4.294.967.295 arasında
64 bit	8 bayt	-9.223.372.036.854.775.808 (-2^{63}) ile 9.223.372.036.854.775.807 ($2^{63} - 1$) arasında veya 0 ile 18.446.744.073.709.551.615 ($2^{64} - 1$) arasında

Tablo 2.1: Tam Sayılar ve Sayı Sınırları

Gerçek sayılar, kayan noktalı sayı (**floating point number**) olarak biçimlendirilir. Kayan noktalı sayı kavramı ilk olarak 1914 yılında Leonardo Torres Quevedo tarafından Charles Babbage'ın analitik makinesine kullanılmak üzere yayınlanmıştır.

$$12345 = 1234,5 \times 10^1 = 123,45 \times 10^2 = 12,345 \times 10^3 = 1,2345 \times 10^4 = 0,12345 \times 10^5$$

Yukarıda ondalık tabanda verilen gerçek bir sayıya ilişkin kesir noktasının kaydırılması ile aynı sayı ifade edildiği bir örnek verilmiştir. Kesir noktası sola kaydırıldığında sayı, 10 ile çarpılmış, sağa kaydırıldığında ise sayı 10'a bölünmüş olur. Kayan noktalı sayının üs kısmı dışında kalan kısmı taban veya **kesir** (**fraction**) olarak adlandırılır. Tabanın kesir noktası sağında kalan hanelerine ise **mantis** (**mantissa**) adı verilir. Tablo 2.2'de en çok kullanılan kayan noktalı sayılara ilişkin bellek miktarı ve sayı sınırları verilmiştir³.

1938 yılında Konrad Zuse, ilk ikili (**binary**) programlanabilir mekanik bilgisayar olan Z1'i yapmıştı. Bu bilgisayar, 7 bitlik işaretli üs, 17 bitlik anlamlı değer ve bir işaret biti içeren 24 bitlik ikili tabanda kayan noktalı sayı kullanmıştır.

³IEEE Computer Society (2019-07-22). IEEE Standard for Floating-Point Arithmetic. IEEE STD 754-2019. IEEE. pp. 1-84.

Bit Sayısı	Bellek Mik-tarı	Sayı Sınırları
32 bit	4 bayt	$1,2 \times 10^{-38}$ ile $3,4 \times 10^{+38}$ arasında tek hassasiyetli gerçek sayılar; en soldaki 1 bit işaret biti, sonraki 8 bit üs ve geri kalan 23 bit ise kesir olarak kullanılan ikilik tabanda sayılardır.
64 bit	8 bayt	$2,3 \times 10^{-308}$ ila $1,7 \times 10^{+308}$ arasında çift hassasiyetli gerçek sayılar; en soldaki 1 bit işaret biti, sonraki 11 bit üs ve geri kalan 52 bit ise kesir olarak kullanılan ikilik tabanda sayılardır.

Tablo 2.2: Kayan Noktalı Sayılar ve Sınırları

Tam sayılarda olduğu gibi **kayan noktalı sayılar da bilgisayarlarda ikili (binary) sayı tabanında tutulur.**

Örneğin ondalık tabanda 50,25 sayısı ikili tabanda 1.1001001×2^5 olarak ifade edilir.

Program yazılırken tanımlanacak değişkenlere ilişkin bellek ihtiyacı programcı tarafından belirlenir. Programcı değişkene tahsis edilecek bellek miktarına ve tutacağı içeriğe göre **veri tipi (data type)** belirler. Veri tiplerine verilen adlar dilden dile değişir; Örneğin PASCAL dilinde 8 bitlik tam sayıya **bayt** denilirken, C dilinde **char** adı verilir. Programcı tarafından kullanılacağı amaca göre belirleyeceği veri tipinde istediği kadar değişken tanımlayabilir.

2.5 Fonksiyon

Matematikte **fonksiyon (function)**, bir kümenin bir veya daha fazla kümenin elemanını tek bir kümenin elemanına eşleyen **ilişkidir (relationship)**.

Girdi	İlişki	Çıktı
1	x2	2
2	x2	4
3	x2	6
4	x2	8
...		

Tablo 2.3: İki İle Çarpma Fonksiyonu

Girdi olan bağımsız değişkenler ile çıktı olan bağımlı değişken arasındaki bu ilişki, matematikte aşağıdaki şekilde **ifade (expression)** edilir.

$$f(x) = 2x$$

Burada f, fonksiyon anlamında x ise girdi anlamında olup **parametre** olarak adlandırılır. Böylece tabloda verilen fonksiyon tablosuna her girdiyi yazmak zorunda kalmayız. Bazen fonksiyona aşağıdaki örneği verilen şekilde bir ad verilmez;

$$y = 2x$$

Bazı durumlarda ise girdi daha detaylı tanımlanır;

$$x \in R, f(x) = 2x + 33$$

Bu fonksiyon, reel sayı kümesindeki her x elemanını, hesaplanan f(x) değerine eşleyen bir ifadedir. Yani x=1.0 argümanını f(1.0)=5.0 reel sayısına eşler. Kısaca fonksiyon 5.0 reel sayısını hesaplayıp **döndürür (return)**.

$$g(a, b) = 2a + b$$

Bu fonksiyon ise reel sayı kümesindeki her a elemanı ile b elemanını, hesaplanan g(a,b) değerine eşleyen bir ifadedir. Yani a=1.0 ve b=1.0 argümanlarını g(1.0,1.0) fonksiyonu yine reel sayı kümesinden 3.0 reel sayısına eşler. Kısaca bu fonksiyon, 3.0 reel sayısını hesaplayıp döndürür (**return**).

Örneklerdeki x, a ve b parametre olarak adlandırılır. f(1.0) ifadesindeki 1.0 ise parametrenin o an andığı değeri belirtir ve **gerçek parametre (actual parameter)** ya da **argüman (argument)** olarak adlandırılır.

$$f(3.0) + g(3.0, 2.0) + f(2.0) + g(1.0, 3.0)$$

Fonksiyonlar ifade olarak tanımlandıktan sonra yukarıdaki gibi tekrar tekrar **çağrılabilirler (call)**.

2.6 Yapısal Programlama

1970'li yıllara kadar her ne kadar çıktı üretmeyen fonksiyonlar için **SUBROUTINE**, birden fazla çağrı noktası olabilen **COROUTINE** ve **FUNCTION** kullanılıyor olsa da hala **GO TO** ve **JUMP TO** ifadeleri kullanılmakta ve kafa karışıklığına devam etmekteydi.

1968 Yılında *Edsger W. Dijkstra* önderliğinde **GO TO** ve **JUMP TO** ifadeleri **zararlı olarak ilan edilmiştir**.

1970 Yılında *Niklaus Wirth* tarafından **Pascal** dili geliştirildi. Bu dil ilk geliştirilen yapısal programlama dilidir. **Yapısal programlamada (structural programming)** verileri taşıyan veri yapıları (değişkenler ve diziler) ile bunu işleyen kontrol yapıları (**if**, **for**, **do**, **repeat**) birbirinden ayrılmıştır. Pascal, kendi kendini çağıran **özyinelemeli (recursive)** fonksiyonlar ile değişkenlerin adreslerini tutan **göstericileri (pointer)** desteklemektedir⁴. Aşağıda iki sayıyı kullanıcıdan alan ve konsola toplamı yazdıran bir program örneği verilmiştir;

```
program ToplamaIslemi;
var
sayi1, sayi2, toplam: integer; { Değişken tanımlama bölümü }
begin
{ Kullanıcıdan veri alma }
writeln('Birinci sayiyi giriniz:');
readln(sayi1);
writeln('İkinci sayiyi giriniz:');
readln(sayi2);
{ Hesaplama }
toplam := sayi1 + sayi2;
{ Sonucu ekrana yazdırma }
writeln('Girdiginiz sayilarin toplami: ', toplam);
{ Programın hemen kapanmaması için bir tuş beklemesi (isteğe bağlı) }
readln;
end.
```

Yapısal programlamada programlama, fonksiyonların birbirini çağırmasıyla yapılır. Yapısal programlamada veri ile veriyi işleyen yapılar birbirinden ayrıldığından arapsaçı programlamanın aksine okunaklılık çok yüksektir.

2.7 Yapısal Programlama Genel Çerçevesi

1. Programın başladığı yeri belirten bir **ana fonksiyon (main function)** tanımlanır. Kodlaması yapılacak işi birbirini çağıran fonksiyonlar olarak bu ana fonksiyonda yazarız.
2. Her fonksiyonda ilk önce **veri yapıları (data structure)** tanımlanır. Veri yapısı;
 - (a) En basit veri yapısı **char**, **integer**, **float** ve **double** gibi **ilkel veri tiplerinden (primitive data type)** olabilen **değişken (variable)**,
 - (b) İlkel veri tiplerinden meydana gelen **dizi (array)**,
 - (c) İlkel veri tiplerinin birkaçından veya dizilerinden bir araya gelmesiyle oluşan **yapılar (structure)**,
 - (d) Ya da dinamik olarak yani çalışma anında birbirine bağlanmış verilerden oluşan **bağlı liste (linked list)** olabilir.
3. Her fonksiyonda tanımlanan veri yapılarının ardından bu yapıları işleyecek **kontrol yapıları (control structure)** tanımlanır. Kontrol yapıları bir veya daha fazla veya iç içe **if**, **if-else**, **switch**, **while**, **do-while**, **for**, **continue**, **break**, **goto** ve **return** **talimatlarından (statement)** oluşur.

Yapısal programlamanın en çok uygulandığı Pascal dilinde ana fonksiyon nokta ile biten **BEGIN-END** bloğudur. C dilinde ana fonksiyon, **main()** fonksiyonudur.

C Programlama Dili, *Dennis M. Ritchie* tarafından Bell Laboratuvarlarında UNIX işletim sistemini geliştirmek için geliştirilen genel amaçlı, üst düzey bir dildir. C, ilk olarak 1972'de DEC PDP-11 bilgisayarında çalıştırılmıştır. *Ken Thompson* tarafından geliştirilen B adlı daha eski bir dilden gelişmiştir. 1978 yılında *Brian Kernighan ve Dennis Ritchie*, günümüzde K&R standardı olarak bilinen C'nin ilk kamuya açık kılavuzunu hazırlamışlardır.

C programlama dili; Öğrenmesi kolaydır, yapısal programlama dilidir, hızlı ve verimli programlar üretir, assembly dili desteği ile **düşük düzey programlamayı (low level programming)** da destekler ve standart

⁴Wirth, Niklaus (1976). Algorithms + Data Structures = Programs. Prentice-Hall. p. 126

başlık dosyaları sayesinde çeşitli bilgisayar platformlarında derlenebilir. Bu özellikleri nedeniyle neredeyse tüm işletim sistemlerinin çekirdeği C dilinde yazılmıştır ve sistem dili olarak da adlandırılmaktadır.

2.8 Düşün ve Çöz

- ◇ **Düşün:** Arapsaçı kod (**spaghetti code**) kavramının yapısal programlama ile nasıl çözüldüğünü ve fonksiyonların bu süreçteki rolünü tartışınız.
- ◇ **Düşün:** Algoritma tasarımında yalancı kod (**pseudocode**) kullanmanın, doğrudan kod yazmaya başlama göre hata payını nasıl azalttığını açıklayınız.
- ◇ **Düşün:** Modüler programlama yaklaşımı, büyük bir yazılım projesinde farklı mühendislerin aynı anda çalışmasını nasıl kolaylaştırır?
- ◇ **Çöz:** Bir öğrencinin vize ve final notlarını temsil eden iki değişken tanımlayınız. Bu verilerin işlenerek ortalama bilgisine dönüşme sürecini basit bir algoritma ile gösteriniz.
- ◇ **Çöz:** Bir sayının asal olup olmadığını kontrol eden bir algoritmanın **akış diyagramını (flowchart)** çiziniz.
- ◇ **Çöz:** Kullanıcının girdiği üç sayıdan en büyüğünü bulan algoritmanın adımlarını mantıksal bir sıra ile listeleyiniz.

3. Bölüm

Derleyiciler ve Çalıştırma Ortamı

3.1 Derleyici

Genel amaçlı bilgisayarların çok kullanılmasıyla birlikte metin editörleri de (Windows Not Defteri, Mac TextEdit, Notepad++, Epsilon, EMACS, nano, vim veya vi) işletim sistemlerinin bir parçası olmuştur. Metin editörlerinde yazılan programlar ise **derleyiciler (compiler)** tarafından makine koduna çevrilerek **icra edilebilir (executable)** dosya olarak kaydedilebilmektedir.

C dilinde programlama öğrenmeye başlamak için ilk adım, programı yazabileceğiniz bir metin editörü kullanmak ve yazdığınız programa ilişkin kaynak kodları `.c` uzantısı ile dosya olarak saklamaktır. İkinci adım ise **işletim sisteminizde (operating system)** çalışabilen bir icra edilebilir dosya oluşturan bir derleyici kurmaktır. Yani bilgisayarınızda iki yazılım aracına ihtiyacınız vardır; birincisi C derleyicisi, ikincisi ise basit bir metin editörüdür.

Derleyici için girdi, kaynak kodu dosyasıdır. Kaynak dosyasında yazılan kaynak kodu, içerisinde C **talimatları (statement)** olan, insan tarafından okunabilen metin dosyasıdır. Kaynak kodda verilen talimatlara göre işlemcinizin programı çalıştırabilmesi için, bunun makine diline **derlenmesi (compile)** gerekir.

Derleme, bir başka programlama dilinden **sembolik isim (mnemonic)** ya da **makine koduna (instruction code)** dönüştürme işlemidir.

Bunu yapan programlara **derleyici (compiler)** adı verilir.

3.2 C derleyicileri

Günümüzde birçok C derleyicisi mevcuttur. En çok kullanılanları aşağıda verilmiştir;

- ♦ **GNU Derleyici Koleksiyonu (GCC)**: Popüler bir açık kaynaklı C derleyicisidir¹. Windows, macOS ve Linux dahil olmak üzere çok çeşitli platformlarda kullanılabilir. GCC, çok çeşitli özellikleri ve çeşitli C standartlarına desteğiyle bilinir.
- ♦ **Clang**: LLVM projesinin bir parçası olan açık kaynaklı bir C derleyicisidir². Windows, macOS ve Linux dahil olmak üzere çok çeşitli platformlarda kullanılabilir. Clang, hızı ve optimizasyon yetenekleriyle bilinir.
- ♦ **Microsoft Visual C** : Microsoft tarafından geliştirilen tescilli bir C derleyicisidir³. Yalnızca Windows için kullanılabilir. Visual C, Microsoft Visual Studio geliştirme ortamıyla entegrasyonu ile bilinir.

Bilgisayarınızın 64 bitlik Windows olduğu varsayılarak GCC derleyicisi aşağıdaki şekilde kurulur;

- ♦ MINGW sürümü ilgili web sayfasından indirilerek kurulur⁴.
- ♦ Kurulumu tamamlandıktan sonra `PATH` değişkenine GCC derleyicisinin klasörü eklenir.

Böylece kurulum tamamlanmış olur. **Komut istemi (command prompt veya terminal)** açıldığında `gcc -v` komutu yazılarak derleyicinin düzgün kurulup kurulmadığı kontrol edilebilir.

```
ozkan@ILHANOZKAN:~$ gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/libexec/gcc/x86_64-linux-gnu/13/lto-wrapper
OFFLOAD_TARGET_NAMES=nvptx-none:amdgc-ndhsa
OFFLOAD_TARGET_DEFAULT=1
Target: x86_64-linux-gnu
Configured with: ../src/configure -v
--with-pkgversion='Ubuntu 13.3.0-6ubuntu2~24.04.1'
--with-bugurl=file:///usr/share/doc/gcc-13/README.Bugs
--enable-languages=c,ada,c++,go,d,fortran,objc,obj-c++,m2 --prefix=/usr
--with-gcc-major-version-only --program-suffix=-13
```

¹<https://gcc.gnu.org/>

²<https://clang.llvm.org/>

³<https://visualstudio.microsoft.com/tr/vs/features/cplusplus/>

⁴<https://www.mingw-w64.org/downloads/>

```
--program-prefix=x86_64-linux-gnu- --enable-shared
--enable-linker-build-id --libexecdir=/usr/libexec
--without-included-gettext --enable-threads=posix --libdir=/usr/lib
--enable-nls --enable-bootstrap --enable-clocale=gnu
--enable-libstdcxx-debug --enable-libstdcxx-time=yes
--with-default-libstdcxx-abi=new --enable-libstdcxx-backtrace
--enable-gnu-unique-object --disable-vtable-verify --enable-plugin
--enable-default-pie --with-system-zlib
--enable-libphobos-checking=release --with-target-system-zlib=auto
--enable-objc-gc=auto --enable-multiarch --disable-werror --enable-cet
--with-arch=32=i686 --with-abi=m64 --with-multilib-list=m32,m64,mx32
--enable-multilib --with-tune=generic
--enable-offload-targets=nvptx-none=/build/gcc-13-EldibY/
gcc-13-13.3.0/debian/tmp-nvptx/usr,amdgc-n-amdhsa=/build/
gcc-13-EldibY/gcc-13-13.3.0/debian/tmp-gcn/usr
--enable-offload-defaulted --without-cuda-driver
--enable-checking=release --build=x86_64-linux-gnu
--host=x86_64-linux-gnu --target=x86_64-linux-gnu
--with-build-config=bootstrap-lto-lean --enable-link-serialization=2
Thread model: posix
Supported LTO compression algorithms: zlib zstd
gcc version 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
ozkan@ILHANOZKAN:~$
```

Kaynak kodumuzu yazmak için bir metin editörü gereklidir. **Windows** işletim sisteminde **notepad**, **Linux** ve **macOs** işletim sistemleri için **nano** programları metin yazmak için idealdir. İlk kaynak kodumuzu oluşturmak için Windows komut isteminde **notepad hello.c**, Linux ve macOS işletim sistemlerinde ise **nano hello.c** komutu yazılır. Editör uygulaması açılır. Eğer böyle bir dosya yoksa metin editörü bu dosyayı oluşturmak için onay ister. Açılan editörü penceresine ilk C programı yazılır ve **hello.c** adlı dosyaya kaydedilir.

```
#include <stdio.h>
```

```
int main() {
    /* İlk C Programı */
    printf("Hello, World! \n");
    return 0;
}
```

Kaydedilen dosyayı komut isteminde **gcc hello.c -o hello.exe** komutu ile derlenir. Böylece hazırlanan kaynak kodumuz derleyici tarafından derlenir (**compile**). Böylece kaynak kodumuz icra edilebilir (**executable**) **hello.exe** dosyasına dönüştürülmüş olur. Derleme ve çalıştırma sürecini **Windows** komut isteminde yapmak için aşağıdaki komutlar yazılır;

```
C:\Users\ILHANOZKAN>notepad hello.c
C:\Users\ILHANOZKAN>gcc hello.c -o hello.exe
C:\Users\ILHANOZKAN>hello.exe
Hello, World!
C:\Users\ILHANOZKAN>
```

Benzer şekilde derleme ve çalıştırma sürecini **Linux** veya **macOs** komut isteminde yapmak için aşağıdaki komutlar yazılır;

```
ozkan@ILHANOZKAN:~$ nano hello.c
ozkan@ILHANOZKAN:~$ gcc hello.c -o hello.exe
ozkan@ILHANOZKAN:~$ ./hello.exe
Hello, World!
ozkan@ILHANOZKAN:~$
```

3.3 Entegre Geliştirme Ortamı

Şimdiye kadar anlatılan kod yazma (**implementation**), derleme (**compile**), icra (**execute**) ya da koşma (**run**) ve izleme (**trace**) gibi süreçlerin tamamını tek bir çatı altında yürütmemizi sağlayan programlar, entegre geliştirme ortamı kısaca **IDE** (**Integrated Development Environment**) olarak adlandırılır.

IDE'ler kodumuzu renklendirir (**source code coloring** veya **syntax highlighting**) ve kod yazarken daha az hata yapmamızı sağlarlar. Ancak kodu dosyaya yine sade metin olarak kaydederler. Ayrıca kod yazarken

talimatları doğru tamamlamak (**intellisense**) için programcıya yardımcı da olurlar. Bunun gibi kolaylaştırıcı birçok özellikleri bulunur. Örneğin yukarıda yazdığımız düz metin IDE içinde aşağıdaki gibi renkli görünür;

```
#include <stdio.h>
int main() {
    /* İlk C Programı */
    printf("Hello, World! \n");
    return 0;
}
```

C programlama dili için en çok kullanılan entegre geliştirme ortamları aşağıda verilmiştir;

- ♦ **Code::Blocks:** Açık kaynaklı bir C/C++ geliştirme ortamı olup bir grafik kullanıcı ara yüzüne **GUI (Graphic User Interface)** sahiptir. Windows, macOS ve Linux işletim sistemleri üzerine kurulabilmektedir.
- ♦ **Visual Studio:** Microsoft tarafından geliştirilen, C dilinde yazılmış, güçlü, yüksek performanslı uygulamalar oluşturmak için kullanılabilen bir geliştirme ortamıdır. Visual Studio, **kod tamamlama (intellisense)**, kullanıcı ara yüzü **UI (user interface)** hazırlama desteği ve **hata ayıklama (debugger)** ve birçok **eklentisi (plug-in)** olan muazzam özelliklere sahiptir.
- ♦ **Visual Studio Code:** Microsoft tarafından geliştirilen açık kaynaklı bir geliştirme ortamıdır. Windows, macOS ve Linux gibi işletim sistemlerinde çalışır. Git **sürüm kontrol (version control)** sistemleriyle çok iyi çalışır. Ayrıca, akıllı kod tamamlamanın dikkate değer özellikleriyle birlikte gelir.
- ♦ **CLion:** JetBrains tarafından geliştirilmiştir ve C++ programcıları için en çok önerilen çapraz platformlu (CMake derleme sistemiyle entegre macOS, Linux ve Windows altında çalışır) geliştirme ortamıdır. Ayrıca, yerel (**local**) ve uzaktan (**remote**) desteğe sahip birkaç IDE'den biri olarak kabul edilir. Bu da yerel bir makinede kod yazmanıza ancak uzak sunucularda derlemenize olanak tanır. Kaynak kodlarımız yönetmemizi sağlayan **CVS (Concurrent Versions System)** ve **TFS (Team Foundation Server)** ile entegre edilebilir.
- ♦ **Eclipse:** Java'da yazılmış ve IBM tarafından geliştirilmiş ücretsiz ve açık kaynaklı bir geliştirme ortamıdır. Yaklaşık otuz programlama dilini desteklediği için geniş topluluk desteğiyle iyi bilinir. C++ için Eclipse IDE, kod tamamlama, otomatik kaydetme, derleme ve hata ayıklama desteği, uzak sistem gezgini, statik kod analizi, **profilleme (profiling)** ve **yeniden düzenleme (refactoring)** gibi beklenebilecek tüm özelliklere sahiptir. Ayrıca çeşitli harici eklentileri entegre ederek işlevselliğini genişletebilirsiniz. Windows, Linux ve macOS'ta çalışabilir.
- ♦ **NetBeans:** Apache Yazılım Vakfı – Oracle Corporation tarafından geliştirilen ücretsiz ve açık kaynaklı bir geliştirme ortamıdır. Öğrenciler veya başlangıç seviyesindeki C/C++ geliştiricileri için şiddetle tavsiye edilmesinin sebebi, Eclipse'e benzer şekilde daha iyi sürükle ve bırak işlevlerine sahip olmasıdır. Windows, Linux, MacOSX ve Solaris gibi birden fazla platformda çalışır.
- ♦ **Xcode:** MacOSX işletim sisteminde yazılım geliştirmek için kullanılan bir geliştirme ortamıdır.

3.4 Çevrimiçi Derleyiciler

Bütün bu entegre geliştirme ortamlarının yanında online olarak da derleme yapılan web uygulamaları da bulunmaktadır. Bunlardan birkaçı aşağıda verilmiştir;

- ♦ https://www.tutorialspoint.com/compile_c_online.php
- ♦ https://www.onlinegdb.com/online_c_compiler
- ♦ <https://www.programiz.com/c-programming/online-compiler/>
- ♦ <https://onecompiler.com/c>
- ♦ https://www.online-ide.com/online_c_compiler
- ♦ <https://onlinecompilers.com/online-c-compiler>

3.5 Derleme Zamanı

Derleyici programının derleme işlemine başlayıp bitirildiği sürece **derleme zamanı (compile time)** denir. Bu süreç başarısızlıkla da sonuçlanabilir ve eğer derleme işleminde hata meydana gelirse programcı hata mesajları ile uyarılır. Bir derleyici program, kaynak dosyayı makine diline çevirme çabasında, kaynak dosyanın C dilinin **sözdizimi (syntax)** kurallarına uygunluğunu da denetler.

Kod yazımı sırasında dilin kurallarına uyulmamışsa, derleyici bu durumu bildiren bir **derleme hatası (compile error)** alınarak derleme işlemi sonlandırılacaktır;

```
int main()
{
    return
}
```

Örneğindeki gibi hatalı yazılmış bir kaynak kod aşağıdaki hatayı verecektir;

```
ozkan@ILHANOZKAN:~$ gcc hello.c -o hello.exe
hello.c: In function 'main':
hello.c:4:1: error: expected expression before '}' token
4 | }
  | ^
ozkan@ILHANOZKAN:~$
```

Kod yazımı sırasında dilin kurallarına uyulmuş ise en fazla **derleme uyarısı** (**compile warning**) alınacak, ancak derleme tamamlanacaktır;

```
int main()
{
    return;
}
```

Örneğindeki gibi yazılmış bir kaynak kod aşağıdaki uyarıyı verecektir;

```
ozkan@ILHANOZKAN:~$ gcc hello.c -o hello.exe
hello.c: In function 'main':
hello.c:3:9: warning: 'return' with no value, in function returning non-void
3 |     return;
  |     ^~~~~~
hello.c:2:5: note: declared here
2 | int main() {
  |     ^~~~
ozkan@ILHANOZKAN:~$
```

Kaynak kod içerisindeki metinlerde Türkçe karakter kullanılması halinde **Windows** altında derlenen programı çalıştırdığınızda aynı metni göremeyebilirsiniz. Bunu düzeltmek için komut isteminde **Chcp 65001** komutu verilerek komut satırının UTF-16 karakter kümesi ile işlem yapmasını sağlayabiliriz.

3.6 Hatalar

Programcı tarafından kodlama yapılırken genellikle aşağıdaki üç tip hata yapılır;

1. **Derleme zamanı hataları** (**compile time error**): Bu tip hatalar genelde kullanılan dilin yazım kurallarına (**syntax**) uyulmadığından, talimatların (**statement**) yanlış yazılmasından ya da programcının kod metninde uygun olmayan karakterlerin kullanılmasından kaynaklanır.
2. **Koşma zamanı hataları** (**run time error**): Kaynak kod, kurallara uygun olarak yazılmıştır ve herhangi bir yazım hatası bulunmaz. Bu tip hatalara en iyi örneklerden birisi sıfıra bölme hatasıdır. Taşma hataları da bu hatalardandır. Daha sonra değinilecektir.
3. **Mantıksal hatalar** (**logical error**): Programcının çözüm için gerekli adımların oluşturulmasında, çözüm yönteminin yanlış olmasından ya da yanlış işlemler (**operator**) kullanılmasından kaynaklanır. Örneğin bir büyüktür (>) işleci yerine küçüktür (<) işleci kullanıldığında ne bir yazım hatası ne de bir çalışma zamanı hatası ortaya çıkar. Fakat program kendisinden istenilen davranışları yerine getirmez ve uygun çıktıları üretmez. Faktöriyel hesaplanırken **0!=1** yerine **0!=0** kabul etmek buna örnek verilebilir.

3.7 C Programlama Kullanım Alanları

C dili, başlangıçta sistem geliştirme çalışmaları için, özellikle işletim sistemini oluşturan programlar için kullanıldı. **Montaj** (**assembly**) dilinde yazılan kod kadar hızlı çalışan kod ürettiği için sistem geliştirme dili olarak benimsendi. Bilgisayar mühendislerinin bilmesi gereken bu dilin birçok kullanım alanı mevcuttur;

- ◊ İşletim Sistemleri
- ◊ Dil Derleyicileri
- ◊ Assembler Derleyicileri
- ◊ Metin Düzenleyicileri
- ◊ Yazdırma Biriktiricileri
- ◊ Ağ Sürücüler
- ◊ Gömülü Programlar
- ◊ Veri tabanları

3.8 C Programlama Dilinin Üstünlükleri

Verimlilik (efficiency) ve **hız (speed)**: C, yüksek performanslı ve verimli olmasıyla bilinir. Düşük seviyede bellekle çalışmanıza olanak tanır ve donanımına doğrudan erişim sağlar, bu da onu hız ve ekonomik kaynak kullanımı gerektiren uygulamalar için ideal hale getirir.

Taşınabilir (portable): C programları, minimum veya hiç değişiklik yapılmadan farklı platformlarda derlenebilir ve yürütülebilir. Bu taşınabilirlik, dilin standartlaştırılmış olması ve derleyicilerin küresel olarak çeşitli işletim sistemlerinde kullanılabilmesinden kaynaklanmaktadır.

Donanım Yakınlık: C, göstericiler ve düşük seviyeli işlemler kullanılarak donanımın doğrudan manipülasyon yapılmasına olanak tanır. Bu, onu sistem programlama ve donanım kaynakları üzerinde ayrıntılı denetim gerektiren uygulamalar geliştirmek için uygun hale getirir.

Standart Kütüphaneler: Giriş/çıkış işlemleri, metin işleme ve matematiksel hesaplamalar gibi yaygın görevler için C, geliştiricilerin önceden oluşturulmuş işlevlerden yararlanarak daha verimli kod yazmalarına yardımcı olan büyük bir standart kütüphaneyle birlikte gelir.

Yapısal Programlama: C, kodu modüler ve anlaşılması kolay veri ve kontrol yapılarına sahiptir. Fonksiyonlar, döngüler ve koşullarla geliştiriciler, bakımı kolay net kod üretebilirler.

Emreden Paradigma: C dilinde programlama fonksiyonların birbirini çağırması ile yapılır. Bu programlama bakış açısı **yordamlı programlama (procedural programming)** olarak adlandırılır. **Prosedür (procedure)**, bir değer üretmeyen fonksiyonlardır. Fonksiyonlar içinde ise bilgisayarın gerçekleştireceği süreç, basit ve anlaşılır **talimatlar (statement)** halinde art arda yazılır. Talimatların art arda verilmesi **emreden programlama (imperative programming)** olarak da bilinir.

3.9 C Dilinin Zayıf Yönleri

Manuel Bellek Yönetimi: C dili, bir geliştiricinin belleği açıkça ayırma ve ayırmayı kaldırmasıyla ilgilenmesi gereken manuel bellek yönetimine ihtiyaç duyar. Programcı bunu kodu yazarken yapar.

Nesne Yönelimli Olmaması: Günümüzde, programlama dillerinin çoğu **nesne yönelimli programlamayı (object oriented programming)** özelliklerini destekler. Ancak C dili bunu desteklemez.

Çöp Toplama Olmaması: C dili, bellekte yer ayrılan ve kullanılmayan bileşenleri bellekten otomatik kaldırmaz. Yani **çöp toplama (garbage collection)** kavramını desteklemez. Programcının belleği manuel olarak ayırması ve kaldırması gerekir ve bu hataya açık olabilir ve bellek sızıntılarına veya verimsiz bellek kullanımına yol açabilir.

İstisna İşleme Olmaması: C dili, istisnaları (**exception**) ya da hata ayıklama (**error handling**) işlemek için herhangi bir kütüphane sağlamaz. Programcının her türlü beklentiyi işlemek için kod yazması gerekir.

3.10 C Sürümlerinin Tarihçesi

1971 yılında *Dennis Ritchie*, C üzerinde çalışmaya başladı ve Bell Labs'daki diğer geliştiricilerle birlikte C'yi geliştirmeye devam etti. Geleneksel C'den sonraki C sürümlerinin tarihi aşağıda verilmiştir.

K&R C, *Dennis Ritchie*, *Brian Kernighan* ile birlikte "The C Programming Language" adlı kitaplarının ilk baskısını 1978 yılında yayınladı. Halk arasında K&R olarak bilinen kitap, uzun yıllar boyunca dilin gayri resmi bir kılavuzu olarak hizmet etti. Açıkladığı C sürümüne genellikle "K&R C" denir ve ayrıca C78 olarak da adlandırılır. K&R C'de tanımlanan C dilinin birçok özelliği, 2018'de onaylanan dilin bir parçasıdır. C'nin erken sürümlerinde, yalnızca **int** dışındaki tipleri döndüren fonksiyonlar, fonksiyon tanımı öncesinde kullanılıyorsa bildirilmelidir; önceden bildirilmeden kullanılan fonksiyonların **int** tipini döndürdüğü varsayılırdı.

AT&T C ve diğer satıcılar tarafından üretilen C derleyicileri, K&R C diline eklenen çeşitli özellikleri destekledi. C popülerlik kazanmaya başlasa da farklılıklar problem yaşatmaya başladı. Bu nedenle, dil özelliklerinin standartlaştırılması gerektiği düşünüldü.

ANSI C, 1980'lerde, Amerikan Ulusal Standartlar Enstitüsü (ANSI), C dili için resmi bir standart üzerinde çalışmaya başladı. Bu, 1989'da standartlaştırılan ANSI C'nin geliştirilmesine yol açtı. ANSI C, birkaç yeni özellik tanıttı ve dilin önceki sürümlerinde bulunan belirsizlikleri giderdi.

C89/C90, ANSI C standardı uluslararası olarak kabul edildi ve C89 (veya onaylanma yılına bağlı olarak C90) olarak tanındı. Uzun yıllar boyunca derleyiciler ve geliştirme araçları için temel teşkil etti.

C99, 1999'da ISO/IEC, C99 olarak bilinen C standardının güncellenmiş bir sürümünü onayladı. C standardı 1990'ların sonlarında daha da revize edildi. C99, **satır içi fonksiyonlar (inline function)**, karmaşık sayıları temsil eden karmaşık bir tür gibi çeşitli yeni veri türleri ve değişken uzunluklu diziler vb. gibi yeni özellikler sundu. Ayrıca **//** ile başlayan C++ tarzı tek satırlık yorumlar için destek ekledi.

C11, 2011'de yayınlanan C11, C standardının bir başka önemli revizyonudur. C11 standardı, C'ye ve kütüphaneye yeni özellikler ekler ve çoklu iş parçacığı (multi threading) desteği, anonim yapılar (anonym structure) ve birleşimler ve geliştirilmiş Unicode desteği gibi özellikler sunar. Tür genel makroları, anonim yapılar, geliştirilmiş Unicode desteği, atomik işlemler, çoklu iş parçacığı ve sınır denetimli işlevler içerir. C++ ile geliştirilmiş bir uyumluluğu vardır.

C17, Bu standart Haziran 2018'de yayınlanmıştır. C17, C programlama dili için geçerli standarttır. Bu standart revizyonuyla yeni özellikler getirilmemiştir. Sadece belirli teknik düzeltmeler ve C11'deki kusurlara yönelik açıklamalar yapar.

C18, C standardının en son sürümü olan C18, 2018'de yayınlandı. C11'e kıyasla küçük revizyonlar ve hata düzeltmeleri içeriyor.

C23, C23, 2024'te yayınlanan son C dili standardı revizyonunun adıdır. Bu revizyonda 14 yeni anahtar kelime dile eklenmiştir.

C programlama dili, basitliği, verimliliği ve çok yönlülüğü nedeniyle zamanla popülerliğini korudu. İşletim sistemleri, gömülü sistemler, uygulamalar ve oyunlar dahil olmak üzere çeşitli yazılımlar oluşturmak için kullanıldı. C'nin sözdizimi ve semantiği, C++, Java ve Python gibi farklı modern programlama dillerini de etkilemiştir.

3.11 Düşün ve Çöz

- ◊ **Düşün:** "Derleme Zamanı Hatası" ile "Çalışma Zamanı Hatası" arasındaki farkı açıklayınız. Hangisinin bir mühendis için tespit edilmesi daha zordur? Neden?
- ◊ **Düşün:** Yorumlayıcı (**interpreter**) diller ile Derleyici (**compiler**) diller arasındaki performans farkının temel sebebi donanım seviyesinde nasıl açıklanır?
- ◊ **Düşün:** Statik bağlama (**static linking**) ve Dinamik bağlama (**dynamic linking**) arasındaki farkları araştırıp, programın dosya boyutu üzerindeki etkisini tartışınız.
- ◊ **Çöz:** Yazdığınız bir C kodunun makine diline dönüşene kadar geçtiği aşamaları (Önişlemci → Derleyici → Bağlayıcı) sırasıyla listeleyiniz.
- ◊ **Çöz:** Bir C programının derleme aşamasında oluşan ara dosyaları (**.i**, **.s**, **.o**) ve bu dosyaların içeriklerini gösteren bir şema hazırlayınız.
- ◊ **Çöz:** Sadece bir metin editörü (Notepad vb.) ve bir derleyici (GCC gibi) kullanarak "Merhaba Dünya" programını komut satırından derleyip çalıştırınız.

4. Bölüm

C Programlama Diline Giriş

4.1 En Basit C Programı

C programlama dili, her programcının ihtiyacını görecekt genel amaçlı, emreden paradigmatlı , yapısal programlama dilidir. Kısa ve öz talimatlar birbiri ardına verilerek yapılan programlamaya emreden paradigma (**imperative programming**) adı verilir. Yapısal programlamanın çerçevesi 2.7 başlığı altında anlatılmıştır.

Yapısal programlamada, programın başladığı yeri belirten bir **ana fonksiyon** (**main function**) tanımlanır. C dilinde bu fonksiyon **main()** fonksiyonudur. Bu fonksiyonun olmadığı hiçbir c programı çalışmaz. Aşağıda en basit C programı verilmiştir.

```
1 int main()
2 {
3     return 0;
4 }
```

Kodu şu şekilde açıklayabiliriz;

- ◊ Birinci satırda, yapısal programlamanın ana fonksiyonu olan (**main function**) tanımlanmıştır. Başındaki **int**, fonksiyonun bir tamsayı (**integer**) geri döndüreceğini belirtir. Program çalışmaya ana fonksiyondan başlar ve bittiğinde işletim sistemine geri döner.
- ◊ C dilinde her fonksiyon ikinci satırdaki gibi süslü parantez karakteri ile başlayan ve dördüncü satırdaki gibi biten bir kod bloğuna (**code block**) sahiptir. Her fonksiyonda olan bu blok, fonksiyon bloğu (**function block**) olarak da adlandırılır.
- ◊ Ana fonksiyon içinde üçüncü satırda yazılan **return 0;** talimatı (**statement**) ana fonksiyondan sıfır geri döndürerek çıktıldığını söyleyen talimattır. Sıfır dışında bir değerin işletim sistemine gönderilmesi, programın hata ile sonlandığı olarak kabul edilir. Eğer programdan çıkışta işletim sistemine 11 göndermek istersek bu talimatı **return 11;** olarak yazmalıyız.

4.2 Söz Dizim Kuralları

C dili için oluşturulacak kaynak kod programcı tarafından metin dosyasına yazılırken belli **söz dizim kurallarına** (**syntax**) uyar. Bunlar aşağıda başlıklar halinde verilmiştir.

§4.2.1 Kaynak Kodumuzu Oluşturacak Karakterler

1. İngiliz alfabesindeki büyük harfler:
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
2. İngiliz Alfabesindeki küçük harfler:
a b c d e f g h i j k l m n o p q r s t u v w x y z
3. Rakamlar:
0 1 2 3 4 5 6 7 8 9
4. Özel Karakterler:
< > . , _ () ; \$: % [] # ? ' & { } " ^ ! * / | - ~ +
5. Metinde görünmeyen ancak metni biçimlendiren **beyaz boşluk** (**white space**) karakterleri;
 - (a) Boşluk (**space**)
 - (b) Yeni satır (**new line**)
 - (c) Satır başı (**carriage return**)
 - (d) Geri (**backspace**)
 - (e) Sekme (**tab**)

§4.2.2 Talimatlar ve Anahtar Kelimeler

Yüksek seviyeli bir dilde yazılmış bir **talimat** (**statement**), işlemciye belirtilen bir eylemi gerçekleştirmesini söyleyen cümledir. Yüksek seviyeli bir dildeki tek bir talimat, birkaç emir kodunu (**instruction code**) temsil edebilir. Talimatlar, yapılması gerekeni kısa ve net olarak belirten mantıksal satırlardır (**logical sequence**).

Anahtar kelimeler (**keywords**), programlamada kullanılan ve C derleyicisi tarafından özel anlama sahip, önceden tanımlanmış, **ayrılmış kelimeler** (**reserved words**) olup programı oluşturan talimatlar (**statement**) Tablo 4.1'de verilen bu kelimeler kullanılarak yazılırlar.

auto	else	long	switch
break	enum	register	typedef
case	extern	return	union
char	float	short	unsigned
const	for	signed	void
continue	goto	sizeof	volatile
default	if	static	while
do	int	struct	double

Tablo 4.1: Anahtar Kelimeler

Anahtar kelimeler, küçük-büyük harf ayrımı yapıldığından **her zaman küçük harflerle yazılırlar**.

Talimatlar BASIC ve FORTRAN gibi dillerde genellikle satır sonunda biter. FORTRAN dilinde yedinci sütundan yazılmaya başlanır. BASIC dilinde ise her satırın başına satır numarası yazılır. Pascal ve C dillerinde böyle bir kural söz konusu değildir. Talimat herhangi bir yerde başlar ve noktalı virgül karakteri ile biter. Aradaki beyaz boşluklar ne kadar çok olursa olsun tek bir tane varmış gibi kabul edilir. Bu nedenle kod **gelişigüzel** (**free format**) yazılır.

Her talimat (**statement**), anahtar kelimeler içerecek şekilde yazılır ve **noktalı virgül karakteri ile biter**.

Aşağıda gelişigüzel yazılan programların hepsi derlenebilir ve derleme sonucu aynı çalıştırılabilir dosyayı üretir. Beyaz boşlukları alınmış en kısa kod aşağıda verilmiştir:

```
int main() {return 0;}
```

Kod içerisinde beyaz boşluk karakterleri anahtar kelimeleri bölmediği sürece istenildiği kadar kullanılabilir:

```
int          main(          ) {
    return 0; }
```

Fonksiyon parantezleri içine veya fonksiyon blokları içine istenildiği kadar beyaz boşluk konulabilir:

```
int main (
{ return 0; }
```

Kodun güzel görünmesi şart değildir:

```
int main() {
    return 0;
}
```

Anahtar kelime argümanları arasına da beyaz boşluklar gelebilir:

```
int main() {
return 0;
}
```

Parantezler ve fonksiyon blokları hizalı olmayabilir:

```
int main(
) {
    return 0;
}
```

Ancak kodun bir başkası tarafından bakımı yapılacağı göz önüne alınarak okunaklı yazılması çok önemlidir. **En doğrusu programı aşağıdaki sitilde yazmaktır**:

- ◊ Fonksiyonların gövdesi ve bazı kontrol talimatları kod bloklarından oluşur. Kod blokları için süslü parantez kullanılır.
- ◊ Süslü parantez içindeki kodlar, okunabilirliği artırmak için genellikle **4 boşluk** veya **1 sekme (tab)** içeriden başlar. Kodu bu şekilde yazmaya **girintili** (**indented**) yazma adı verilir.
- ◊ Ayrıca değişkenler ve anahtar kelimeler arasında boşluk karakteri veya sekme karakteri kullanılmalıdır:

```
int main() {
    //Ana fonksiyon bloğuna ait girinti
    int sayac=0;
```



```

    if (sayac==0) { //iç blok
        //iç bloğa ait girinti...
        int sonuc=1;
        return sonuc;
    }
    if (sayac==1) // Blok başlangıcı böyle de yazılabilir
    { //iç blok
        //iç bloğa ait girinti...
        int sonuc=1;
        return sonuc;
    }
    return 0;
}

```

§4.2.3 Açıklamalar

Talimatlar (statement), yapılması gerekeni kısa ve net olarak belirten **mantıksal satırlardır (logical sequence)**. Programcı kodu yazarken **açıklama (comment)** yapma gereği duyabilir. Bunu iki şekilde yapar;

1. Kod metninde bulunulan yerden sonra satırın sonuna kadar olan bir açıklama yapılmak istendiğinde, açıklama öncesinde iki adet bölü (//) karakteri kullanılır.
2. Eğer birden çok satırı içeren bir açıklama yapılacak ise açıklama (/*) ile (*/) karakterleri arasına alınır.

Aşağıda açıklamalar eklenmiş ana fonksiyon örneği yer almaktadır;

```

/*
Bu program, açıklama (comment) içeren basit bir ana fonksiyonu olan
C programıdır. İlhan ÖZKAN, Aralık 2024
*/
int main() // Yapısal Programlamadaki "Ana Fonksiyon"
{ //fonksiyon bloğu başlangıcı
    /* Ana Fonksiyon, programın başladığı yerdir. Ana fonksiyonu olmayan bir program
    ↪ çalışmaz. */

    return 0; /*fonksiyondan dönüş talimatı noktalı virgül karakteri ile bitmiştir.*/
} //fonksiyon bloğu bitişi

```

İstedikimiz kadar açıklama yapabiliriz. Derleyici açısından bunun önemi yoktur. Çünkü derlemenin ilk aşamasında **bütün bu açıklamalar ve beyaz boşluk karakterleri metinden çıkartılır.**

4.3 Değişken Tanımlama

Her yapısal programlama dilinde olduğu gibi C dilinde de **veri yapıları (data structure)** ve **kontrol yapıları (control structure)** birbirinden ayrıdır. Dolayısıyla veriyi işleyecek kontrol yapısı kodlanmadan önce veriyi taşıyan veri yapıları tanımlanmalıdır.

Değişkenler (variable) veri yapılarının temel öğeleridir. Bu başlıkta anlatılan tamsayı ve kayan noktalı sayı değişkenleri, belleğin belli bir bölgesini işgal eder ve fiziki olarak bellek bitlerden oluştuğundan ikili sayı (**binary**) olarak veri tutarlar.

C dilinde ise 2.4 başlığında anlatılan tamsayı ve kayan noktalı sayılar, **ilkel veri tipleri (primitive data type)** olarak adlandırılır. Bu veri tipleri için kullanılan anahtar kelimeler aşağıda verilmiştir;

1. Tamsayı olarak kullanılacak veri tipleri (**integer data type**);
 - (a) **char**: bellekte 1 bayt yani 8 bit yer kaplayan işaretli tamsayıları tutan veri tipidir.
 - (b) **short**: bellekte 2 bayt yani 16 bit yer kaplayan işaretli tam sayıları tutan veri tipidir.
 - (c) **int**: bellekte 2/4 bayt yani 16/32 bit yer kaplayan işaretli tam sayıları tutan veri tipidir.
 - (d) **long**: bellekte 4 bayt yani 32 bit yer kaplayan işaretli tam sayıları tutan veri tipidir.
2. Gerçek sayı veri tipleri (**floating point data type**);
 - (a) **float**: bellekte 4 bayt yani 32 bit yer kaplayan **tek hassasiyetli (single precision)** kayan noktalı gerçek sayıları tutan veri tipidir.
 - (b) **double**: bellekte 8 bayt yani 64 bit yer kaplayan **çift hassasiyetli (double precision)** kayan noktalı gerçek sayıları tutan veri tipidir.

C programlama dilinde bir de **hiçbir değeri olmayan ve bellekte yer kaplamayan void** veri tipi vardır. Bu veri tipini hiçbir değer döndürmeyen fonksiyon tanımlamalarında ve dinamik bellek kullanımlarında ilerde çokça göreceğiz.

Değişkenlerin veri tipine (**data type**) bağlı olarak benzersiz bir **kimlik (identifier)** verilerek **tanımlanması (definition)** gerekir.

```
/* Bu program değişken tanımlamak için oluşturulmuştur.*/
int main() {
    char yas; /* veri tipi "char" ve kimliği "yas" olan değişken tanımlandı. -128 ila +127
    → arasında değer alabilir. */
    short kat; /* veri tipi "short" ve kimliği "kat" olan değişken tanımlandı. -32768 ila
    → +32768 arasında değer alabilir. */
    float kilo; /* veri tipi "float" ve kimliği "kilo" olan değişken tanımlandı. */
    return 0;
}
```

Yukarıda tanımlaması yapılan **yas**, **kat** ve **kilo** değişkenleri ana fonksiyon bloğu içinde tanımlandığından, tanımlanan değişkenler sadece bu blok içinde geçerlidir!

Aynı blok içinde bir değişkene verilen kimlik, bir başka değişkene verilemez!

Bildirim (declaration) derleyiciye böyle bir değişken, fonksiyon veya nesne var demektir. **Tanımlama, (definition)** derlemenin sorunsuz olabilecek şekilde eksiksiz yapılan bir işlemdir.

İşaretsiz bir tamsayı kullanmak yerine en anlamlı işaret bitini de sayıya dahil ederek tam sayıları işaretsiz olarak (**unsigned**) kullanabiliriz. Tamsayı veri tipleri olan **char**, **short**, **int** ve **long** veri tipleri (**data type**) aslında **işaretsiz (signed)** tamsayılardır. Yani önlerinde **signed** sıfatı olduğu varsayılır. Tam sayıları işaretsiz olarak kullanmak istememiz halinde **unsigned** sıfatını tamsayı veri tipinin önünde kullanırız.

```
/* Bu program işaretsiz tamsayı değişken tanımlamak için oluşturulmuştur.*/
int main() {
    unsigned yas; /* veri tipi "unsigned char" ve kimliği "yas" değişkeni
    tanımlandı. yas değişkeni 0 ile 255 arasında bir değer alabilir. */
    unsigned short kat; /* veri tipi "unsigned short" ve kimliği "kat" olan değişken
    → tanımlandı.kat değişkeni 0 ila 65535 arasında değer alabilen bir değişkendir. */
    unsigned int pozitifTamsayi; /* pozitifTamsayi değişkeni 0 ile 4.294.967.295 arasında
    → değer alabilen bir değişkendir. */
    return 0;
}
```

Değişkenin kapsamı (variable scope); Değişkenin bildiriminin (**variable declaration**) yapılabileceği, değişkene erişilebileceği (**access**), değişkenin üzerinde çalışılabileceği program kodu olarak tanımlanır. Değişkenin kapsamı, tanımlandığı kod bloğu (ve varsa bu blok içindeki iç bloklar) ile sınırlıdır.

4.4 Değişken Kimliklendirme Kuralları

C programlama dilinde **değişkenlerin kimliklendirilmesinde (identifier definition)** dört temel kural vardır. Bunlar;

1. Kimliklendirmede anahtar kelimeler kullanılamaz!
2. Kimliklendirme asla rakamla başlayamaz!
3. Kimlikler en fazla 32 karakter olmalıdır! Daha fazlası derleyici tarafından dikkate alınmaz!
4. Kimliklendirmede ancak İngiliz alfabesindeki Büyük ve Küçük harfler, rakamlar ile alt çizgi karakteri kullanılabilir.

Aşağıda kimliklendirme kurallarına uygun tanımlama yapılmıştır;

```
/* Bu program değişken tanımlama örneklerini içerir. */
int main() {
    /* Tamsayı değişkenler: */
    char yas;
    int tam_sayi;
    long uzun_tam_sayi;
```

```

/* Gerçek Sayı Değişkenler:*/
float kilo;
double reel_sayi;
/*
Aynı veri tipinde birden fazla değişkene aralarına virgül koyarak kimlik verilebilir.
*/
int kat1,kat2,kat3;
float olcek1,olcek2;
return 0;
}

```

Bütün bu kuralların yanında değişkenlere kimlik verilirken yazılan kodun okunaklılığını artırmak için **deve yazımı** (**camel case**) stili kimlik verilir. Bu stilde ilk sözcük tamamıyla küçük, izleyen sözcüklerin ise baş harfi büyük yazılır. Bu kural zorunlu değildir ama koda bakım yapacak sonraki programcılarının işini kolaylaştırmak için ahlaki olarak tercih edilir;

```

/* Bu program camelCase değişken kimliklendirme örneklerini içerir.*/
int main()
{
    int sayac;
    int ogrenciBoyu;
    int asansorunOlduguKat;
    float asansorAgirligi;
    int ogrenciYasi;
    char ilkHarf;
    char ogrenciAdi[50];
    char ogrenciSoyadi[50];
    float ortalamaNot;
    return 0;
}

```

4.5 Sabitler

Programcı, kodlama yaparken bazı **sabitleri** (**constant**) ister istemez kullanır. Sabitler iki çeşittir. Bunların biri **değişmez** (**literal**), diğeri ise değeri değiştirilemeyen sabit değişkenlerdir (**const variable**). Derleme öncesinde de sonrasında da kaynak koddakinin kelimesi kelimesine aynısı olan **değişmezler** (**literal**) kullanılır. Değişkenlere verilen **ilk değerler** (**initial value**) ile **katsayılar** (**coefficient**) en çok kullanılan değişmezlerdir.

Değişkenler tanımlanırken sahip olacağı ilk değerleri değişmezler ile verme işlemine **başlatma** (**initialization**) adı verilir.

Aşağıda çeşitli değişkenleri başlatma örneği verilmiştir;

```

1  /* Bu program değişkenlere ilk değer (initial value) vermede kullanılan için değişmez (literal)
   → örneklerini içerir. */
2  int main() {
3      int i,j=0;
4      char c=65;
5      float f=2.5;
6      char harf='A'; //char harf=65;
7      return 0;
8  }

```

Yukarıda verilen kod şu şekilde açıklanabilir;

- ♦ **int** veri tipindeki **i** değişkeni tanımlandı ve yine **int** veri tipindeki **j** kimlikli değişkene **0** tamsayı değişmezi (**integer literal**) ile ilk değer verildi. Burada sadece **j** değişkeni **0** ile **başlatılmıştır** (**initialize**) (3. Satır).
- ♦ **char** veri tipindeki **c** kimlikli deki değişkene **65** tamsayı değişmezi ile ilk değer verildi (4. Satır).
- ♦ **float** veri tipindeki **f** kimlikli değişkene Türkçe yazılışı 2,5 olan kayan noktalı değişmezi ile ilk değer verildi. Kod İngilizce yazıldığından, kodun içerisindeki gerçek sayı değişmezleri için **ondalık ayracı nokta olarak yazılır** (5. Satır).
- ♦ **char** veri tipindeki **harf** kimlikli değişkene **'A'** karakter değişmezi (**character literal**) ile ilk değer verildi (6. Satır). **Karakter değişmezler tek tırnak içinde yazılır**. Klavyeden basılan her bir harf bellekte sayı olarak tutulur. **'A'** karakterinin kodu **65**, **'B'** karakterinin kodu **66**, ... Bu **ANSI** (**American National**

Standards Institute) standardıdır. Bu nedenle bu talimat `char harf=65;` şekilde de yazılabilir .

Kodu 0 olan karakter ise **boş karakter (NULL Character)** olarak adlandırılır. Günümüzde ANSI standardı yerine birden fazla karakterden oluşan UTF-8 ve UTF-16 karakter kodları kullanılmaktadır.

Aşağıdaki kod örneğinde programcı, sırasıyla 3, 3.14, 2.0 ve 3.14 olmak üzere dört farklı değişmez (**literal**) kullanılmıştır. Bu değişmezlerin her biri derleyici tarafından farklı olarak değerlendirilir ve çalıştırma anında (**run time**) değiştirilemez.

```
/* Bu program, değişmez örneğidir. */
int main() {
    int yariCap=3;
    float daireninAlani=3.14*yariCap*yariCap;
    float daireninCevresi=2.0*3.14*yariCap;
    return 0;
}
```

Değişmezlerin (literal) çokça kullanılması derleme sonrası üretilen icra edilebilir makine kodunu da büyütür. Bunun yerine çok kullanılan değişmezler bellekte bir kez yer kaplasın diye **const sıfatıyla (const qualifier) sabit değişkenler (const variable)** tanımlanabilir. Sabit değişkenler etik olarak büyük harfle kimliklendirilir.

```
/* Bu program, const sıfatı örneğidir. */
int main() {
    int yariCap=3;
    const float PI=3.14; // Sabit değişken tanımı!
    float daireninAlani=PI*yariCap*yariCap;
    float daireninCevresi=2.0*PI*yariCap;
    return 0;
}
```

Böylece aynı sabit **PI** sabit değişkeni birden çok yerde kullanılır ve her seferinde aynı bellek bölgesine erişildiğinden bellekten tasarruf edilmiş olunur. Tanımlama (**definition**) sırasında **const** sıfatı kullanılan değişkene, ilk değer (**initial value**) değişmez (**literal**) olarak verilmelidir. **Daha sonra bu değişkene bir değer ataması olamaz!**

Bir değişkeni sabit tanımlamanın üstünlükleri aşağıda sıralanmıştır;

- ◊ **Gelişmiş Kod Okunabilirliği:** Bir değişkene **const** sıfatı vermek, diğer programcılara değerinin değiştirilmemesi gerektiğini belirtir; bu da kodun anlaşılmasını ve sürdürülmesini kolaylaştırır.
- ◊ **Gelişmiş Veri Tipi Güvenliği:** **const** kullanılması, değerlerin yanlışlıkla değiştirilmemesini sağlayabilir ve kodumuzda hata ve eksiklik olma olasılığını azaltabilir.
- ◊ **Gelişmiş Optimizasyon:** Derleyiciler, değerlerinin program yürütme sırasında değişmeyeceğini bildikleri için **const** değişkenlerini daha etkili bir şekilde optimize edebilirler. Bu, daha hızlı ve daha verimli kodla sonuçlanabilir.
- ◊ **Daha İyi Bellek Kullanımı:** Değişkenlere **const** sıfatı vermek, değerlerinin bir kopyasını oluşturmayı engeller; bu da bellek kullanımını azaltabilir ve performansı artırabilir.
- ◊ **Gelişmiş Uyumluluk:** Değişkenlere **const** sıfatı vermek, kodunuzu **const** değişkenleri kullanan diğer başlıklarla daha uyumlu hale getirebilir.
- ◊ **Gelişmiş Güvenilirlik:** **const** kullanarak, değerlerin beklenmedik şekilde değiştirilmemesini sağlayarak kodunuzu daha güvenilir hale getirebilir ve kodunuzdaki hata ve eksiklik riskini azaltabilirsiniz.

Kod içerisinde değişmezleri (**literal**) tekrar tekrar yazmamanın bir yolu da **#define ön işlemci yönergesi (preprocessor directive)** ile **makro tanımlamaktır**. Ön işlemciler, kaynak kodları derlemeden önce derleyiciyi yönlendirerek kaynak kodlar üzerinde değişiklik yapmayı sağlar. Aşağıdaki tanımlanan **PI** makrosuyla derleyici, derlemeye geçmeden **PI** görülen yerleri 3.14 olarak değiştirmesi söylenir. Derleme bu ön işlemci işleminden sonra gerçekleşir. Makro kimlikleri sabit değişkenler gibi **büyük harfle kimliklendirilir**.

Talimatlar noktalı virgül karakteriyle biter Ancak, ön işlemci makrolar sonunda noktalı virgül karakteri ile bitmez!

```
/* Bu program, #define ön işlemci yönergesi örneğidir. */

#define PI 3.14
/* Bu makro, derleme öncesinde PI görülen yerlere 3.14 yazılmasını sağlar. */
```

```
int main() {
    int yariCap=3;
    float daireninAlani=PI*yariCap*yariCap;
    float daireninCevresi=2.0*PI*yariCap;
    return 0;
}
```

Ön işlemci yönergelerini yerine getiren derleyici aynı kodu aşağıdaki şekle çevirerek derleme işlemine geçer. Ayrıca ön işlemci yönergeleri yerine getirilmeden önce bütün açıklamalar ve **beyaz boşluklar (white space)** kaynak koddan kaldırılır. Bu durumda ön işlemci yönergeleri sonrasında bir üstte verilen kaynak kodumuz aşağıdaki hale döner;

```
int main(){int yariCap=3;float daireninAlani=3.14*yariCap*yariCap;float
↪ daireninCevresi=2.0*3.14*yariCap;return 0;}
```

#define ön işlemci yönergeleri ile çok tekrarlanan değişmezler için makro tanımlanabildiği gibi aşağıdaki örnek verildiği şekilde kod tasnifi için de makro kullanılabilir;

```
/* Bu program, #define ön işlemci yönergelerinin bir başka kullanım örneğidir. */
#define BEGIN int main() {
#define END }
#define PI 3.14
BEGIN
const char ILKHARF='A';
const float AVAGADROSAYISI=6.022e23;
float yarimRadyan=PI/2;
return 0;
END
```

4.6 Yazdığımız Kod Nasıl İcra Edilir?

Derleyici tarafından işlemcinin anlayacağı makine diline çevrilen programımız aslında yazdığımız talimatları (**statement**) sırasıyla çalıştırır. C dilinde her talimat noktalı virgül ile bittiği 4.2.2 başlığında anlatılmıştı. Aşağıdaki örnek programı göz önüne alalım;

```
1 int main() {
2     float boy=1.80;
3     float kilo=100.0;
4     float bki;
5     float boyKare;
6     boyKare=boy*boy;
7     bki=kilo/boyKare;
8     boy=1.50;
9     return 0;
10 }
```

Yapısal programlamada program, ana fonksiyondan başlar. Bu nedenle icra, **main()** fonksiyonu içindeki ilk talimatla başlar ve aşağıdaki sırayla icra edilir;

- İcra edilecek ilk talimat, **float boy=1.80;** talimatıdır. (2. Satır): **boy** Değişkenine bellek tahsis edilerek değişkene **1.80** değeri atanır. Yani **boy** değişkeni **1.80** değeriyle **başlatılır (initialization)**.
- Sonra **float kilo=100.0** talimatı icra edilir (3. Satır): **kilo** Değişkeni **100.0** ile başlatılır.
- Daha sonra **float bki;** talimatı icra edilir (4. Satır): tanımlanan **bki** değişkenine sadece bellek tahsis edilir ve değeri belli değildir. Bazı derleyiciler değerleri sıfır olarak varsayar.
- Daha sonra **float boyKare;** talimatı icra edilir (5. Satır): Tanımlanan **boyKare** değişkenine sadece bellek tahsis edilir ve değeri belli değildir.
- Daha sonra **boyKare=boy*boy;** icra edilir (6. Satır): Önceden tanımlanmış **boyKare** değişkenine **boy** değişkeni kendisiyle çarpılarak işlem sonucu atanır. **boyKare** değişkeni öncesinde tanımlanmamış olsaydı derleme hatası alacaktık.
- Daha sonra **bki=kilo/boyKare;** talimatı icra edilir (7. Satır): **kilo** Değişkenini **boyKare** değişkenine böler ve çıkan sonucu **bki** değişkenine atar.
- Daha sonra **boy=1.50** talimatı icra edilir (8. Satır): **boy** Değişkenine **1.50** atanarak değeri değiştirilir.
- Son olarak **return 0;** talimatı icra edilir (9. Satır): **return** Talimatı içinde bulunan fonksiyondan çıkış talimatıdır. Ana fonksiyondan çıkıldığı için işletim sistemine dönlür.

4.7 İşleçler

İşleçler (**operator**) matematikten bildiğimiz işleçlerdir.

$$y = 3 + 2$$

Yukarıda verilen cebirsel ifadede toplama işleci (+) iki tarafına bulunan argümanları toplar. Bu argümanlara **işlenen** (**operand**) adı verilir. Yüksek düzey dillerde de aritmetik işleçlerin yanında birçok işleç de bulunur.

En çok kullanılan işleç, atama (=) işlecidir.
Atama işleci, sağdaki işleneni soldakine atar.
 Bu nedenle kodlama yapılırken $2+2=x$ veya $a+b=x$ yazılamaz!

§4.7.1 Aritmetik İşleçler

C programlama dilinde aritmetik işleçlerin çalışma şekli **işlenenlerin** (**operand**) veri tipine göre değişir. Aritmetik işlemler ve atama işleci için kurallar;

1. **İşlenenlerden birinin bellekte kapladığı yer küçük ise ikisi aynı olacak şekilde bellekte daha fazla yer kaplayanına dönüştürülür** ve işlem sonra yapılır ve sonuç dönüştürülen veri tipinde üretilir. Örneğin **char** veri tipindeki değişken ile **long** veri tipindeki bir değişken toplanmaya çalışıldığı zaman **char** veri tipindeki değişkenin değeri **long** veri tipine otomatik dönüştürülür. Sonuç **long** veri tipinde üretilir.
2. **İşlenenlerden biri tamsayı diğeri kayan noktalı sayı ise işlem yapılmadan önce işlenenler kayan noktalı sayıya dönüştürülür.** Örneğin **int** veri tipindeki değişken ile **float** veri tipindeki bir değişken toplanmaya çalışıldığı zaman **int** veri tipindeki değişkenin değeri **float** veri tipine otomatik dönüştürülür. Sonuç **float** veri tipinde üretilir.

İşleç	İşlenenler üzerinde yapılan işlem
+	Sol ve sağındaki yer alan işlenenleri toplar.
-	Soldaki işlenenden sağdakini çıkarır.
*	Sol ve sağındaki yer alan işlenenleri çarpar.
/	Soldaki işleneni sağdakine böler. Eğer işlenenlerin her iki de tamsayı ise sadece kalan kısmı atılır. Bu durumda son uç yine tamsayı olarak üretilir.
%	Kalan (modulus) işleci, sadece tamsayıları işlenen olarak kabul eder. Soldaki işlenenin sağdakine bölümünden kalanı verir. Kalan yine tamsayıdır.

Tablo 4.2: Aritmetik İşleçler

Aritmetik işleçlere (**arithmetic operator**) ilişkin örneği aşağıdaki görebilirsiniz;

```
/* Bu program, aritmetik işleç örneğidir. */
int main() {
    int a = 9, b = 4, res;
    float fa=6.6, fb= 2.2, fres;
    res = a + b; // işlem sonucu res=13
    res = a - b; // işlem sonucu res=5
    res = a * b; // işlem sonucu res=36
    res = a / b; // işlem sonucu res=9/4=2 tam 1/4=2
    /* Bölme işlecinin işlenenleri tamsayı olduğunda bölme sonucundaki tamsayı olarak
       ↪ hesaplanır. Bölme işleminin tam kısmını içeren sonuç döndürülür. */
    res = a / 4.3 ;
    /* işlem sonucu res tamsayı olduğundan;
       res=9/4.3=9.0/4.3=2.0930= 2 tam 0.0930=2 */

    res = a % b; // işlem sonucu res=9%4= 2 tam 1/4=1
    fres= fa / fb; // işlem sonucu res=6.6/2.2=3.00
    return 0;
}
```

C ve C++ programlama dilinde tamsayı bölme işlemi işlemcinin en basit yetenekleriyle çözülür. Bu nedenle diğer programlama dillerinden ayrılır. Aşağıda buna ilişkin örnek verilmiştir.

```
/* Bu program, aritmetik işleç BÖLME örneğidir. */
int main () {
    int a = 19 / 2 ; // a = 2*9+1 = 9
```



```

int b = 18 / 2 ; // b = 9
int c = 255 / 4; // c = 4*63+3 = 63
int d = 45 / 4 ; // d = 4*11+1 = 11

double aa = 19 / 2.0 ; // aa = 9.5
double bb = 18.0 / 2 ; // bb = 9.0
double cc = 255 / 2.0; // cc = 127.5
double dd = 45.0 / 4 ; // dd = 11.25

double ee = 45 / 4 ;
/* ee = 11 çünkü bölme tamsayılar arasında yapılmıştır */
return 0;
}

```

§4.7.2 Tekli İşleçler

Tekli işleçler (unary operator) sadece bir işlenen üzerinde işlem yapar. Tekli demek tek bir işlenen (**operand**) ile işlem yapması anlamındadır.

İşleç	İşlenen üzerinde yapılan işlem
-	Kendisinden sonra gelen işlenenin işaretini değiştirir. İşlenen pozitif ise negatif yapar, negatif ise pozitif yapar.
+	Kendisinden sonra gelen işlenenin işaretini pozitif yapar.
++	Artırım (increment) işleci, işlenenin sağında veya solunda kullanılabilir.
--	Eksiltme (decrement) işleci, işlenenin sağında veya solunda kullanılabilir.
!	Mantıksal DEĞİL işleci işlenenden önce kullanılır. İşleneninin mantıksal durumunu tersine çevirmek için kullanılır.
sizeof()	Bu işleç, kendisinden sonra gelen işlenenin bellek boyutunu bayt cinsinden döndürür.
&	Adres işleci (address operator), kendisinden sonra gelen işlenenin bellek adresini döndürür.

Tablo 4.3: Tekli İşleçler

Aşağıda tekli işleçlerin kullanımına ilişkin örnek program verilmiştir;

```

/* Bu program, tekli işleç örneğidir. */
int main() {
    int a = 5, b = 5, res;
    res = -a; // tekli eksi (unary minus): res=-5;
    res = ++a+2;
    /* ++a işlemi önce-artırım (pre-increment) olarak adlandırılır. a değişkeni işleme
    → girmeden önce artırılır.
    İşlem sonucu res=8, a=6 */
    res = 2+b++;
    /* b++ işlemi sonra-artırım (post-Increment) olarak adlandırılır. b değişkeni işleme
    → girdikten sonra artırılır.
    İşlem sonucu res=7, b=6 */
    a=5; b=5;
    /* iki talimat(statement) yan yana yazılmıştır.Daha fazla da yazılabilir. */
    res = --a+2;
    /* Pre-Decrement: işlemden sonra res=6, a=4 */
    res = 2+b--;
    /* Post-Decrement: işlemden sonra res=7, b=4 */
    a=1; b=0;
    res=!a; // işlem sonucu res=0
    res=!b; // işlem sonucu res=1
    res=sizeof(char);
    /* char veri tipinin bellek miktarını öğrenme:
    işlem sonucu res=1 */
    res=sizeof a;
    /* a değişkeninin bellek miktarını öğrenme:
    işlem sonucu res=4 */
    return 0;
}

```



```
}

```

§4.7.3 İlişkisel İşleçler

İlişkisel işleçler (**relational operator**), işlenenlerin arasındaki ilişkiyi verirler. C dilinde, doğru (**true**) ya da yanlış (**false**) gibi değer tutabilen mantıksal veri tipi (**data type**) bulunmaz.

Programcı kodlama yaparken tam sayıları mantıksal veri tipi olarak kullanır. Tam sayının değeri 0 ise **YANLIŞ**, sıfırdan farklı ise **DOĞRU** olarak anlamlandırılır.

İşleç	İşlenenler üzerinde yapılan işlem
==	Eşit mi? İşleci: İki işlenenin değeri birbirine eşit ise 1, değilse 0 verir
!=	Farklı mı? İşleci: İki işlenenin değeri birbirinden farklı ise 1, değilse 0 verir
>	Büyük mü? İşleci: soldaki işlenenin değeri soldakinden fazla ise 1, değilse 0 verir
<	Küçük mü? İşleci: soldaki işlenenin değeri soldakinden az ise 1, değilse 0 verir
>=	Büyük veya eşit mi? İşleci: soldaki işlenenin değeri soldakinden fazla ya da iki işlenen eşit ise 1, değilse 0 verir
<=	Küçük veya eşit mi? İşleci: soldaki işlenenin değeri soldakinden az ya da iki işlenen eşit ise 1, değilse 0 verir

Tablo 4.4: İlişkisel İşleçler

Aşağıda verilen ilişkisel işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
/* Bu program, ilişkisel işleç örneğidir. */
int main() {
    int a = 5, b = 6, res;

    res = a<b; // işlem sonucu res=0
    res = a<=b; // işlem sonucu res=1
    res = a>b; // işlem sonucu res=0
    res = a>=b; // işlem sonucu res=1
    res = a==b; // işlem sonucu res=0
    res = a!=b; // işlem sonucu res=1
    return 0;
}
```

§4.7.4 Bit Düzeyi İşleçler

Bit düzeyi işleçler (**bitwise operator**) özellikle ikilik tabandaki tam sayılarda karşılıklı bitler üzerinde yapılan işlemlerdir. Aslında bu işlemler çarpma, toplama, bölme ve çıkarma işlemleri için kullanılır. A ve B iki tam sayının karşılıklı bitleri olmak üzere bit düzeyi VE, VEYA, ÖZEL VEYA ve DEĞİL işlemlerinin doğruluk tabloları aşağıda verilmiştir;

A	B	A & B	A B	A ^ B	~ A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Tablo 4.5: VE, VEYA, ÖZEL VEYA ve DEĞİL İşlemlerinin Doğruluk Tabloları

Aşağıda tam sayılar üzerinde yapılan bit düzeyi işlemler gösterilmiştir;

```
/* Bu program, bit düzeyi işleç örneğidir. */
int main() {
    char a = 00000110; // 6
    char b = 00000010; // 2
    char bitwise_and = a & b; // 00000010
    char bitwise_or = a | b; // 00000110
    char bitwise_xor = a ^ b; // 00000100
    char bitwise_not = ~b; // 00001101
    char left_shift = b << 2; // 00001000
}
```

İşleç	İşlenenler üzerinde yapılan işlem
&	Bit düzeyi VE (AND): Tamsayının karşılıklı bitleri VE işlemine tabi tutulur.
	Bit düzeyi VEYA (OR): Tamsayının karşılıklı bitleri VEYA işlemine tabi tutulur.
^	Bit düzeyi ÖZEL VEYA (XOR): Tamsayının karşılıklı bitleri ÖZEL VEYA işlemine tabi tutulur.
~	Bit düzeyi DEĞİL (NOT): Tekli işleç olup sonrasında gelecek sayının bitlerini 1 ise 0, 0 ise 1 olacak şekilde işlem yapar.
<<	Bit düzeyi sola kaydırma: Tekli işleç olup sonrasında gelecek sayının bitlerini bir bit sola kaydırır. En anlamlı bit taşar. Taşan bit işlemcinin durum kaydedicisinde taşma bayrağına gider.
>>	Bit düzeyi sağa kaydırma: Tekli işleç olup sonrasında gelecek sayının bitlerini bir bit sağa kaydırır. En anlamsız bit taşar. Taşan bit işlemcinin durum kaydedicisinde taşma bayrağına gider.

Tablo 4.6: Bit Düzeyi İşleçler

```
char right_shift = b >> 1; // 00000001
return 0;
}
```

§4.7.5 Mantıksal İşleçler

Mantıksal ifadeler doğru (**true**) ve yanlış (**false**) olabilen ifadelerdir. C dilinde mantıksal bir veri tipi yoktur ve int veri tipi bunun için kullanılır. Sıfırdan farklı bir tamsayı ifade doğru, sıfır olan tamsayı yanlış olarak değerlendirilir. C Dilinde VE(AND), VEYA(OR), ÖZEL VEYA(XOR) ve DEĞİL (NOT) gibi mantıksal ifadelerin işlenmesi için aşağıdaki işleçler kullanılır. Mantıksal işleçlere (**logical operator**) işlenen olarak giren ifadeler öncelikle **int** veri tipine dönüştürülür. Yani bu işleçler için beklenen işlenen (**operand**) bir tamsayı ifadedir.

Aşağıda verilen ilişkisel ve mantıksal işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
/* Bu program, mantıksal işleç örneğidir. */
int main() {
    int a = 5, b = 6;
    int res;
    res = a&&b; // işlem sonucu res=1
    res = a||b; // işlem sonucu res=1
    res = !a;   // işlem sonucu res=0
    return 0;
}
```

İşleç	İşlenenler üzerinde yapılan işlem
&&	Şartlı Mantıksal VE: Eğer iki işlenen sıfırdan farklı ise 1 verir. Soldaki işlenen yanlış ya da sıfır ise sağdakine bakılmaz.
	Şartlı Mantıksal VEYA: Eğer iki işlenenden biri sıfırdan farklı ise 1 verir. Soldaki işlenen doğru ya da sıfırdan farklı ise sağdakine bakılmaz.
!	Mantıksal DEĞİL: İşlenenden önce kullanılır. Tekli işleçlerde verilmiştir. İşlenenin mantıksal durumunu tersine çevirmek için kullanılır. Eğer işlenen sıfırdan farklı ise 0, değilse 1 verir.

Tablo 4.7: Mantıksal İşleçler

§4.7.6 Atama işleçleri

Atama işleçleri (assignment operator) en çok kullanılan işleçlerdir. **Atama işleçleri her zaman sağdaki değeri sola atar!** Dolayısıyla soldaki işlenen, atama yapılacak uygun veri tipinde bir değişken (**variable**) olmak zorundadır.

Aşağıda verilen atama işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
/* Bu program, atama işleçleri örneğidir. */
int main() {
    int a = 10; // a değişkenine 10 değışmezi = işleciyle atanıyor.
    a += 10;    // a=a+10; ile eşdeğer: a değişkenine 10 ekleniyor.
    a -= 10;    // a=a-10; ile eşdeğer: a değişkeninden 10 çıkarılıyor.
    a *= 10;    // a=a*10; ile eşdeğer: a değişkeni 10 kat yapılıyor.
    a /= 10;    // a=a/10; ile eşdeğer: a değişkeni 1/10 kat yapılıyor.
    a %= 10;
}
```

İşleç	İşlenenler üzerinde yapılan işlem
=	Sağdaki işleneni sola atar.
+=	Soldaki işleneni sağdaki işlenen üzerine toplar ve sonucu sola atar.
-=	Soldaki işleneni sağdaki işlenenden çıkarır ve sonucu sola atar.
*=	Soldaki işleneni sağdaki işlenene çarpar ve sonucu sola atar.
/=	Soldaki işleneni sağdaki işlenene böler ve sonucu sola atar. Eğer işlenenler tamsayı ise kesirli kısım göz ardı edilir. Bu durumda sonuç tamsayıdır.
%=	Bu işleç tamsayılarla çalışır. Soldaki işleneni sağdaki işlenene böler ve kalanı sola atar.

Tablo 4.8: Atama İşleçleri

```

/* a=a%10; ile eşdeğer: a değişkeninin 10 a kalanı bulunup ona eşitleniyor. */
return 0;
}

```

§4.7.7 İşleç Öncelikleri

Cebirsel ifadelerde nasıl ki önce çarpma ve bölme sonra toplama işlemleri yapılıyor. C programlama dilinde yazılan ifadelerde de işleç öncelikleri (**operator precedence**) vardır. İfadelerde bir işlemi öncelikli hale getirmek için parantez içerisine alınır. En içteki parantez en öncelikli olarak gerçekleştirilir.

Öncelik	İşleç Grubu	İşleçler	Gruptaki Öncelik
1	Sonek (postfix)	() ++ --	→
2	Tekli (unary)	+ - ! ++ -- sizeof()	←
3	Çarpım	* / %	→
4	Toplam	+ -	→
5	Bit Düzeyi Kaydırma	<< >>	→
6	İlişkisel (relational)	< <= > >=	→
7	Eşitlik (equality)	== !=	→
8	Bit düzeyi VE	&	→
9	Bit düzeyi ÖZEL VEYA	^	→
10	Bit Düzeyi VEYA		→
11	Mantıksal VE	&&	→
12	Mantıksal VEYA		→
13	Atama (assignment)	= += -= *= /= %= <<= >>= &= ^= =	←

Tablo 4.9: İşleç Öncelikleri

Başka işleçler de bulunmaktadır bunlar ilerleyen bölümlerde yeri geldikçe anlatılacaktır. Aşağıda işleçlerin önceliklerine ilişkin örnek programı lütfen inceleyiniz.

```

/* Bu program, işleç öncelikleri örneğidir. */
int main() {
    int a=10, b=5;
    a= a-b+2; // Öncelik Sırası +, -, = İşlem sonunda a=7
    a= a/b+2; // Öncelik Sırası /, +, = İşlem Sonunda a=7/5+2=3
    a= 2*10-20/2+3+(12-10); // Öncelik Sırası: (-) * / - + +
    /* İfadenin çözüm sırası:
       > 2*10-20/2+3+2
       > 20-20/2+3+2
       > 20-10+3+2
       > 10+3+2
       > 13+2
       > 15 */
    return 0;
}

```

4.8 Kayan Noktalı Sayı Hesapları

Kayan noktalı sayıların IEEE-754 standardında ikili tabanda tutulduğu 2.4 başlığında anlatılmıştı. Kodlarken onluk tabanda değerler verdiğimiz kayan noktalı sayılar, aslında arka planda ikilik tabanda tutulur. Bu durum yazdığımız kodlarda garip davranışların ortaya çıkmasına sebep olur. Buna örnek olarak; hemen hemen her programcının yaptığı ilk hata, aşağıdaki kodun amaçlandığı gibi çalışacağını varsaymaktır;

```
float total = 0;
for(float a = 0; a != 2; a += 0.01f) {
    total += a;
}
```

Acemi programcı bunun 0, 0.01, 0.02, 0.03, gibi artarak, 1.97, 1.98, 1.99 aralığındaki her bir sayıyı toplayacağını ve 199 sonucunu, yani matematiksel olarak doğru cevabı vereceğini varsayar. Bu kodda doğru yürümeyen iki şey olur: Birincisi yazıldığı haliyle program asla sonuçlanmaz. *a* asla 2'ye eşit olmaz ve döngü asla sonlanmaz. İkincisi ise döngü mantığını *a < 2* şartını kontrol edecek şekilde yeniden yazarsak, döngü sonlanır, ancak toplam 199'dan farklı bir şey olur. IEEE-754 uyumlu işlemlerde, genellikle bunun yerine yaklaşık 201'e ulaşır. Bunun olmasının nedeni, kayan noktalı sayıların onluk tabanda kendilerine atanan değerlerin, ikilik tabanda yaklaşık değerlerini temsil etmesidir. Bu duruma aşağıdaki klasik örnek verilir;

```
#include <stdio.h>
int main()
{
    double a = 0.1;
    double b = 0.2;
    double c = 0.3;
    if(a + b == c)
        printf("Bu satır İcra Edilmez!\n"); //IEEE754 standardında işlem yaparsak.
    else
        printf("Kayan Noktalı Sayılar İkili Sayı Sisteminde"
            " Tutulduğundan Yaklaşık Olarak Hesaplanır." );
    return 0;
}
```

Programcı olarak gördüğümüz şey onluk tabanda yazılmış üç sayı olsa da derleyicinin (ve altta yatan donanımın) gördüğü şey ikili sayılardır. 0.1, 0.2 ve 0.3, ona mükemmel bölmeyi gerektirdiğinden (ki bu onluk sayı sisteminde oldukça kolaydır), ancak ikili sayı sisteminde imkansızdır (bu sayılar, onluk sayı sistemindeki 1/3 sayısı gibi 0.333333333333333... şeklinde kesin olmayan biçimde depolanması gerektiğindendir);

```
#include <stdio.h>
int main()
{
    /*64 bitlik kayan noktalı sayılar, tam sayı kısmı dahil olmak üzere
    53 basamaklı hassasiyete sahiptir */
    double a = 0011111110111001100110011001100110011001100110011001100110011010;
    //onluk sistemdeki 0.1 ikili sistemde kusurlu olarak saklanır
    double b = 0011111111001001100110011001100110011001100110011001100110011010;
    //onluk sistemdeki 0.2 ikili sistemde kusurlu olarak saklanır
    double c = 0011111111010011001100110011001100110011001100110011001100110011;
    //onluk sistemdeki 0.3 ikili sistemde kusurlu olarak saklanır
    double a + b = 0011111111010011001100110011001100110011001100110011001100110100;
    /*c değişkeni ile a+b nin onluk sistemde 0,3'e tam olarak eşit olmadığını unutmayın! */
    return 0;
}
```

4.9 Düşün ve Çöz

- ♦ **Düşün:** C dilindeki anahtar kelimelerin (**keywords**) niçin değişken ismi olarak kullanılamayacağını dilin söz dizimi kuralları açısından değerlendiriniz.
- ♦ **Çöz:** *a = 5, b = 10, c = 15* değerleri için *a + b * c / a - b % 3* ifadesinin sonucunu işlem öncelik sırasına göre adım adım hesaplayan bir program parçacığı yazınız.
- ♦ **Düşün:** *x = x + 1* ile *x++* ifadeleri arasında performans açısından (assembly seviyesinde) bir fark var mıdır? Araştırınız.
- ♦ **Çöz:** Bit düzeyi (**bitwise**) işlemleri kullanarak, bir sayının 2'nin tam kuvveti olup olmadığını bulan bir C ifadesi yazınız.
- ♦ **Düşün:** Mantıksal VE (**&&**) operatöründe kısa devre (**short-circuit**) değerlendirmesi nedir? İlk koşul yanlış olduğunda ikincisine bakılmaması neden önemlidir?
- ♦ **Çöz:** Virgül operatörünün (*a = (b=3, b+2)*) çalışma mantığını analiz ediniz ve *a* ile *b*'nin son değerlerini tahmin ediniz.

5. Bölüm

Giriş Çıkış İşlemleri

5.1 Modüler Programlama

C programlama dili, yapısal olmasının yanında aynı zamanda modüler bir programlama dilidir. **Modüler programlamada** (**modular programming**) yazılan fonksiyonlar gruplar halinde başka kod dosyalarına konulur ve bu bileşenler gerektiğinde projemize **#include** ön işlemci yönergeleriyle (**preprocessor directive**) dahil edilir.

Modüllere ayırmanın **soyutlama** (**abstraction**), **değişim yönetimi** (**change management**) ve **yeniden kullanım** (**reusing**) gibi üstünlükleri vardır. Modüllere ayrılan bir kodun bakımı da daha kolaydır. Ancak daha fazla fonksiyonun çağrılması, işlemciyi yorar. Çünkü her fonksiyon çağrısında işlemcinin mevcut durumu ve kaydediciler belleğe itilir ve fonksiyondan geri döndüğünde eski duruma dönmek için bu veriler tekrar geri çekilir. İşlemciye yüklenen bu **çağırma yükü** (**calling overhead**) günümüz bilgisayarlarında oldukça ihmal edilebilir seviyededir. Ancak mili saniyeler bazında yapılması gereken zaman kritik işler gerçekleştirmek için bu durum göz önüne alınmalıdır.

C programlama dilinde en çok kullanacağımız modüllerden biri de standart giriş-çıkış yani **IO** (**input-output**) işlemlerinin tanımlı olduğu **stdio.h** başlık (**header**) dosyasıdır. Bu modülü kodumuza **dahil etmek** (**include**) için kaynak kodun başına **#include** ön işlemci yönergelerini (**preprocessor directive**) aşağıdaki şekilde ekleriz.

```
#include <stdio.h>
```

Böylece **konsola** (konsol kelimesi UNIX/Linux işletim sistemindeki terminal, Windows işletim sisteminde ise komut satırını ifade eder) bir şey yazmak için **printf** veya klavyeden bir şey okumak ve bir değişkene atamak için ise **scanf** fonksiyonlarını kullanabilir hale geliriz.

C programlama dilinde her işletim sisteminde de çalışan standart birçok başlık dosyası bulunmaktadır. Ancak bunların dışında, işletim sistemine özel olan başlık dosyaları da bulunmaktadır;

Başlık Dosyası	Açıklama
stdio.h	Standart giriş-çıkış komutları
math.h	Matematiksel fonksiyonlar
stdlib.h	Dönüşüm, sıralama, arama vb. komutları
string.h	Alfa sayısal ve bazı bellek yönetim komutları
curses.h	Curses, metin terminali ekranında imleç (cursor) denetimini destekleyen eski bir Unix/Linux kütüphanesidir.
conio.h	DOS ve Windows komut satırında imleç (cursor) denetimini destekleyen kütüphanedir

Tablo 5.1: En Çok Kullanılan Başlık Dosyaları

5.2 Konsola Biçimlendirilmiş Veri Yazma

C programlama dilinde değişkenlerimizde olan veriyi **biçimlendirilmiş** (**formatted**) bir şekilde imlecin bulunduğu noktadan itibaren konsola yazabiliriz (**print**). Bunu sağlayan fonksiyon **stdio.h** başlık dosyasındaki **printf** fonksiyonudur.

Bir fonksiyonun hangi parametreleri aldığı ve neyi geri döndürdüğünü **prototipinden** (**prototype**) anlarız. Aşağıda **printf** fonksiyonunun prototipi verilmiştir;

```
int printf (const char *format, ... );
```

Bu fonksiyon bir **biçimlendirme metnini** (**format string**) ilk argüman olarak alır ve konsola yazdırılan karakter sayısını döndürür.

C dilinde **metinler**, **çift tırnak karakterleri arasında yazılır!**
Örnek olarak **"İlhan"**, **"Merhaba! Mehmet"** verilebilir.

Biçimlendirme metni, aynı zamanda kendisinden sonra gelecek argümanların hangi biçimde konsola yazılacağını belirler. Biçimlendirme metni, içinde kendisinden sonra gelen argümanların hangi biçimde yazılacağını belirten **biçim belirleyicisi** (**format specifier**) karakterleri içerir.

Biçim belirleyicileri iki karakter olup ilk karakteri yüzde (%) karakteridir.

Aşağıda konsola biçimlendirilmemiş ve biçimlendirilmiş metin yazan kod örneği verilmiştir;

```
1  /* Bu program, printf örneğidir. */
2  #include <stdio.h> //Konsola bir şey yazmak için gereklidir!
3  int main() {
4      int yas=50;
5      float kilo=100.0;
6      printf("Merhaba!");
7      printf("Yaşınız:%d",yas);
8      printf("Kilonuz:%f",kilo);
9      return 0;
10 }
11 /*Program Çıktısı:
12 Merhaba!Yaşınız:50Kilonuz:100.000000
13 */
```

Bu kodu aşağıdaki şekilde açıklayabiliriz;

- ◊ İkinci satırdaki yönerge ile `stdio.h` kütüphanesi kaynak kodumuza dahil edilir.
- ◊ Dördüncü ve beşinci satırdaki talimatlarla değişkenler başlatılır.
- ◊ Altıncı satırdaki `printf` fonksiyonu, sadece değişmez metin (**string literal**) argümanı ile çağrılmıştır;
 - İçinde biçim belirleyicisi yoktur!
 - Bu nedenle konsolda imlecin bulunduğu yerden itibaren "Merhaba!" yazar ve imleci (**cursor**) metnin sonuna taşır.
 - **Bu metinden sonra argüman olsa bile konsola bir şey yazılmaz!**
- ◊ Yedinci satırdaki `printf` fonksiyonu, içinde biçim belirleyicisi (`%d`) olan bir değişmez metin (**string literal**) argümanı ile çağrılmıştır;
 - Dolayısıyla **bu metinden sonra ondalık (decimal) olarak konsola yazdırılabilecek bir argüman verilmelidir.**
 - İkinci argüman olarak `yas` değişkeni verilmiştir.
 - Biçimlendirme metnindeki `%d` biçim belirleyicisi yerine `yas` değişkeninin değerini ondalık (**decimal**) olarak konulur.
 - Böylece konsola yazılacak biçimlendirilmiş metin "`Yaşınız:50`" olacaktır.
 - İmlecin bulunduğu yerden itibaren "`Yaşınız:50`" yazar ve imleci (**cursor**) metnin sonuna taşır.
- ◊ Sekizinci satırdaki `printf` fonksiyonu, içinde biçim belirleyicisi (`%f`) olan bir değişmez metin (**string literal**) argümanı ile çağrılmıştır;
 - Dolayısıyla **bu metinden sonra kayan noktalı sayı olarak konsola yazdırılabilecek bir argüman verilmelidir.**
 - İkinci argüman olarak `kilo` değişkeni verilmiştir.
 - Biçimlendirme metnindeki `%f` biçim belirleyicisi yerine `kilo` değişkeninin değerini kayan noktalı (**floating point**) sayı olarak konulur.
 - Böylece konsola yazılacak biçimlendirilmiş metin "`Kilonuz:100.000000`" olacaktır.
 - İmlecin bulunduğu yerden itibaren "`Kilonuz:100.000000`" yazar ve imleci (**cursor**) metnin sonuna taşır.

Biçim belirleyicileri sadece ondalık ve kayan noktalı sayılarla sınırlı değildir. Aşağıda listesi verilmiştir;

Biçim Karakterleri	Argümanın Konsola Yazılma Biçimi
<code>%d</code>	Konsola onluk tabanda (decimal) sayı olarak yazılır.
<code>%c</code>	Konsola karakter olarak yazılır.
<code>%f</code>	Konsola kayan noktalı sayı (float) olarak yazılır.
<code>%x</code>	Konsola onaltılı tabanda (hexadecimal) sayı olarak yazılır.
<code>%s</code>	Konsola metin (string) olarak yazılır.
<code>%%</code>	Konsola <code>%</code> karakteri yazılır.

Tablo 5.2: Biçim Belirleyiciler

Eğer konsola `%` karakteri yazmak istersek biçimlendirme olarak iki yüzde (%) karakteri kullanmalıyız. Buna ilişkin örnek aşağıda verilmiştir;

```
/* Bu program, printf ile konsola % karakteri basma örneğidir. */
#include <stdio.h> //printf için gerekli!
int main() {
```



```

    int doluluk=80;
    printf("Doluluk Oranı: %% %d",doluluk);
    return 0;
}
/*Program Çıktısı:
Doluluk Oranı: % 80
*/

```

Şu ana kadar tek bir argümanı biçimlendirilmiş olarak konsola yazmayı öğrendik. Birden çok argümanı da konsola yazdırabiliriz. **Bunun için biçimlendirme metni içerisinde izleyen argümanları sırasıyla biçimlendirecek biçim belirleyicileri kullanmalıyız.** Buna aşağıdaki örnek verilebilir.

```

1  /* Bu program, çok argümanlı printf örneğidir. */
2  #include <stdio.h>
3  int main() {
4      char yas=45;
5      float kilo=81.0;
6      int doluluk=80;
7      printf("Yaş:%d, Kilo:%f, Doluluk Oranı:%% %d",yas,kilo,doluluk);
8      return 0;
9  }
10 /* Program Çıktısı:
11 Yaş:45, Kilo:81.000000, Doluluk Oranı:% 80
12 */

```

Bu kodu aşağıdaki şekilde açıklayabiliriz;

- ◊ İkinci satırdaki yönerge ile `stdio.h` kütüphanesi kaynak kodumuza dahil edilir.
- ◊ Dördüncü, beşinci ve altıncı satırdaki talimatlarla değişkenler başlatılır.
- ◊ Yedinci satırdaki `printf` fonksiyonu, içinde sırasıyla `%d`, `%f`, `%%` ve `%d` biçim belirleyicileri olan bir değişmez metin (**string literal**) argümanı ile çağırılmıştır. Dolayısıyla bu metinden sonra **sırasıyla** konsola yazdırılabilecek **bir onluk tabanda sayı**, **bir kayan noktalı sayı** ve **yine bir onluk tabanda sayı** argüman olarak verilmelidir. Yüzde karakteri (%) için argüman aranmaz. Konsola yazdırılacak metin aşağıdaki şekilde oluşturulur;
 - Biçimlendirme metnindeki ilk `%d` biçim belirleyicisi yerine ilk argüman olan `yas` değişkeninin değerini onluk tabanda sayı (**decimal**) olarak konulur.
 - Biçimlendirme metnindeki ikinci `%f` biçim belirleyicisi yerine ikinci argüman olan `kilo` değişkeninin değerini kayan noktalı sayı (**float**) olarak konulur.
 - Biçimlendirme metnindeki üçüncü `%%` biçim belirleyicisi yerine sadece `%` karakteri konulur.
 - Biçimlendirme metnindeki son `%d` biçim belirleyicisi yerine son argüman olan `doluluk` değişkeninin değerini onluk tabanda sayı (**decimal**) olarak konulur.
 - Böylece konsola yazılacak biçimlendirilmiş metin `"Yaş:45, Kilo:81.000000, Doluluk Oranı:% 80"` olacaktır.
 - İmlecin bulunduğu yerden itibaren nihai hali almış olan biçimlendirilmiş metin konsola yazılır ve imleç (**cursor**) metnin sonuna taşınır.

Format metninde imleci hareket ettirecek şekilde karakterler kullanabiliriz. Biçimlendirme metni içine imlecin satır başına geçmesini sağlamak için `\n` şeklinde bir dizi karakter kullanarak imleci kaçırabiliriz. Bu tür karakter dizilerine **kaçış dizisi** (**escape sequence**) adı verilir. Bunlar aşağıda verilmiştir;

Kaçış Dizisi	Açıklama
<code>\n</code>	Konsolda imleç yeni satırın (new line) başına hareket eder.
<code>\t</code>	İmlece, metin yazarken sekme (tab) tuşuna basında olan hareketi yapar. Yani imleç, konsolda sekizin katları olan sütuna hareket eder.
<code>\r</code>	Konsolda imleç bulunduğu satırın başına (carriage return) hareket eder.
<code>\v</code>	Konsolda imleç bulunduğu yerden alt satıra dikey olarak (vertical) hareket eder.
<code>\a</code>	Konsol, uyarı (alert) sesi çıkarır.
<code>\\</code>	Konsola ters bölü karakterini yazar.
<code>\"</code>	Konsola çift tırnak karakterini yazar.
<code>\'</code>	Konsola tek tırnak karakterini yazar.

Tablo 5.3: Kaçış Dizileri

Aşağıda kaçış dizilerine ilişkin örnek program verilmiştir.


```

/* Bu program, escape sequence printf örneğidir. */
#include <stdio.h>
int main() {
    printf("ILHAN\n"); // ILHAN yazılarak konsolda satır başı yapılır.
    printf("OZKAN\r"); // OZKAN yazılarak aynı satırın başına dönlür.

    printf("ABC\tDEF\n");
    /* ABC yazılır ve 8. sütuna ilerlenir ve sonra DEF yazılır, sonrasında satır başı
    yapılır. TAB karakteri 8 ve katları olan sütunlara imleci ilerletir.*/

    printf("\\"İyi\\"-\\"Kötü\\""); // Çift tırnak içeren "İyi"- "Kötü" yazılır.
    return 0;
}
/* Program Çıktısı:
ILHAN
ABCAN  DEF
"İyi"- "Kötü"
*/

```

Biçim belirleyiciler (**format specifier**) ile aşağıdaki desen kullanılarak daha fazla biçimlendirme yapılabilir:

%[önek][genişlik][.ondalık]<biçim karakteri>

Burada;

1. **Genişlik**, konsola kaç karakter genişliğinde yazdırılacağını belirler.
2. **Ondalık**, konsola kayan noktalı sayı yazılacak ise noktadan sonra kaç rakam yazdırılacağını belirler.
3. **Önek** içerisinde aşağıdaki karakterler olabilir;
 - (a) **+** karakteri: Sayının işaretini konsola yazılmasını sağlar.
 - (b) **-** karakteri: Yazdırılacak parametrenin belirtilen genişliğin soluna yaslanarak yazdırılmasını sağlar.
 - (c) **0** karakteri: Yazdırılacak sayının soluna belirtilen genişliğe bağlı olarak **0** karakteri konulmasını sağlar.

Aşağıda **10** karakterlik genişlikte **PI** sayısının biçimlendirme örnekleri verilmiştir;

```

/* Bu program, detaylı biçimlendirme printf örneğidir. */
#include <stdio.h>
int main() {
    const float PI=3.141527;
    /*Büyük karakteri biçimlendirmenin nerede başladığını belirtir.*/
    /*Küçük karakteri biçimlendirmenin nerede bittiğini belirtir.*/
    printf(">1234567890<\n"); //Çıktı: >1234567890<
    printf(">%.2f<\n",PI); //Çıktı: >3.14<
    printf(">%3.f<\n",PI); //Çıktı: > 3<
    printf(">%10.2f<\n",PI); //Çıktı: > 3.14<
    printf(">+10.3f<\n",PI); //Çıktı: > +3.142<
    printf(">+10.4f<\n",PI); //Çıktı: > +3.1415<
    printf(">%010.4f<\n",PI); //Çıktı: >00003.1415<
    printf(">%-10.4f<\n",PI); //Çıktı: >3.1415 <
    printf(">%-+10.2f<\n",PI); //Çıktı: >+3.14 <
    printf(">%-+10.2f<\n",-PI); //Çıktı: >-3.14 <
    printf(">%+10.2f<\n",-PI); //Çıktı: > -3.14<
    printf(">%0+10.2f<\n",-PI); //Çıktı: >-00003.14<
    return 0;
}

```

5.3 Klavyeden Biçimlendirilmiş Veri Okuma

C programlama dilinde değişkenlerimizde klavyeden biçimlendirilmiş (**formatted**) olarak girilen metin satırı tarayıp (**scan**), veriyi tutacak değişkenlere koyabiliriz. Bunu sağlayan fonksiyon **stdio.h** başlık dosyasındaki **scanf** fonksiyonudur. Bu fonksiyonunun prototipi aşağıda verilmiştir;

```
int scanf (const char *format, ... );
```

Bu fonksiyon da bir metni ilk olarak argüman olarak alır. İlk argüman olan bu **biçimlendirme metni** (**format string**) aynı zamanda kendisinden sonra gelecek argümanların da hangi biçimde klavyeden okunacağını

belirler. Argümanların hangi biçimde okunacağını `printf` fonksiyonu gibi **biçim belirleyicisi** (**format specifier**) belirler. Sonuç olarak okunan değişken sayısını döndürür.

`scanf` Fonksiyonunda `printf` fonksiyonundan farklı olarak, okunan değerlerin konulacağı **değişkenlerin adresleri argüman olarak verilir**. Bir değişkenin adresi, `&` işlecisi ile elde edilir.

Aşağıda kapasite oranını okuyup % olarak ekrana yazan bir program örneği verilmiştir.

```
/* Bu program, detaylı biçimlendirme scanf örneğidir. */
#include <stdio.h> //printf ve scanf için gereklidir
int main() {
    float kapasiteOrani;
    printf("Kapasite Oranını (0.00-1.00) Giriniz:");
    scanf("%f",&kapasiteOrani); //önündeki adres(&) işlecine dikkat edelim!
    printf("Girilen Kapasite Oranı: %.2f\n",kapasiteOrani);
    printf("%% olarak Girilen Kapasite Oranı: %%2.2f", 100*kapasiteOrani);
    return 0;
}
/*Program Çıktısı:
Kapasite Oranını (0.00-1.00) Giriniz:0.75
Girilen Kapasite Oranı: 0.75
% olarak Girilen Kapasite Oranı: %75.00
*/
```

Benzer şekilde yarıçapı tamsayı olarak klavyeden okuyan ve çemberin çevresini hesaplayan program aşağıda verilmiştir;

```
/* Bu program, detaylı biçimlendirme scanf örneğidir. */
#include <stdio.h>
int main() {
    int yaricap;
    float cevre;
    printf("Çemberin yarıçapını tamsayı olarak giriniz:");
    scanf("%d",&yaricap);
    cevre=2*3.1415*yaricap;
    printf("Yarıçapı %d olan çemberin çevresi: %.2f",yaricap,cevre);
    return 0;
}
/*Program Çıktısı:
Çemberin yarıçapını tamsayı olarak giriniz:4
Yarıçapı 4 olan çemberin çevresi: 25.13
*/
```

5.4 Düşün ve Çöz

- ◊ **Düşün:** `printf` ve `scanf` fonksiyonlarındaki biçim belirleyicilerin (`%d`, `%f` vb.) bellekteki verinin yorumlanmasındaki önemini açıklayınız.
- ◊ **Düşün:** `scanf` fonksiyonu ile boşluk içeren bir metni (örneğin ad-soyad) okumaya çalışırken neden sadece ilk kelime alınır? Çözümü nedir?
- ◊ **Düşün:** Standart hata akışı (`stderr`) ile standart çıktı akışı (`stdout`) arasındaki fark nedir? Hataları neden ayrı bir akıştan göndeririz?
- ◊ **Çöz:** `printf` kullanarak ekrana isim, okul numarası ve not sütunlarından oluşan, düzgün hizalanmış bir tablo bastran kodu yazınız.
- ◊ **Çöz:** Kullanıcıdan bir dairenin yarıçapını alıp, alanı ve çevresini ekrana virgülden sonra iki basamak gösterecek şekilde yazdıran programı kodlayınız.
- ◊ **Çöz:** Kaçış dizileri (**escape sequences**) kullanarak ekrana tam olarak şu metni yazdırınız: `C:\Program Files\Kitap\C_Dili`

6. Bölüm

Kontrol İşlemleri

6.1 Ardışık İşlem ve Kontrol İşlemleri

Yapısal Programlama başlığı altında programın ana fonksiyondan başlayacağı ve programlamanın ise birbirini çağıran fonksiyonlarla yapıldığı anlatılmıştı. Yapısal programlamada, ana fonksiyon dahil tüm fonksiyonlarda ilk önce işlenecek verileri taşıyan veri yapıları tanımlanır ve ardından bu verileri işleyen kontrol yapılarına ilişkin talimatlar yazılır.

Şu ana karar örneğini verdiğimiz kodlarda veri yapısı olarak sadece değişkenler kullanılmıştır. Sonrasında ise giriş çıkış işlemleri talimatlar içeren programlar yazılmıştır. Programın icrası ilk talimattan başlar ve sırasıyla program bitene kadar devam eder. İşte programın icrasını değiştirmeyen bu tür talimatlara **ardışık işlem** (**sequential operation**) adı verilir.

Ardışık işlemler aşağıdaki üç tür talimattan (**statement**) oluşur.

1. Değişkenlerin kimliklendirildiği değişken tanımlamaları (identifier definition):
 - (a) `int yariCap=3;`
 - (b) `const float PI=3.14;`
 - (c) `float daireninAlani,daireninCevresi;`
2. İfadelerden (**expression**): (Sabit, değişken ve operatör içeren söz dizimlerine **ifade** adı verilir)
 - (a) `daireninAlani=PI*yariCap*yariCap;`
 - (b) `float daireninCevresi=2.0*PI*yariCap;`
3. Klavyeden veri okuma ve konsola veri yazma gibi giriş-çıkış işlemleri (**input-output operation**):
 - (a) `printf("Kapasite Oranını (0.00-1.00) Giriniz:");`
 - (b) `scanf("%f",&kapasiteOrani);`

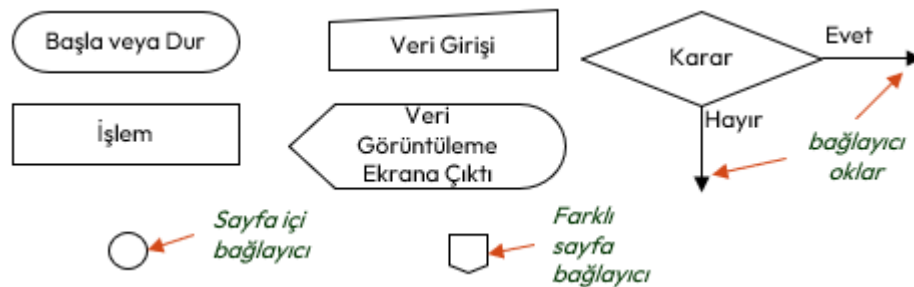
Kontrol işlemleri (**control operation**) ise programın icra sırasını değiştiren kontrol yapıları olup bu talimatlar (**statement**) olup üç türü vardır;

1. **Duruma göre seçimler** (**conditional choice**): `if`, `if-else` ve `switch` talimatları.
2. **İlişkisel döngü** (**relational loop**): `while`, `do-while` ve `for` talimatları.
3. **Dallanmalar** (**jump**): `continue`, `break`, `goto` ve `return` talimatları.

Kontrol işlemlerinde; kodlanan mantıksal satırlar (**logical sequence**) ile fiziksel olarak icra edilen satırlar (**physical sequence**) birbirinden farklıdır.

6.2 Akış Diyagramları

Kontrol işlemi içeren programın icrasını anlamak için akış diyagramlarını da anlamak gerekir. Aşağıda akış diyagramlarındaki temel şekiller verilmiştir.

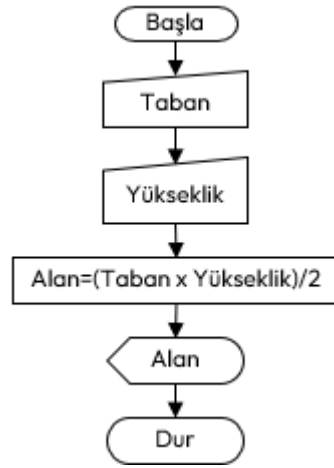


Şekil 6.1: Akış Diyagramları Genel Şekilleri

Akış diyagramları (**flowchart**), adımların grafik gösterimleridir. Algoritmaları ve programlama mantığını temsil etmek için bir araç olarak bilgisayar biliminden ortaya çıkmıştır. Ancak diğer tüm işlem türlerinde kullanılmak üzere genişletilmiştir. Günümüzde akış diyagramları, bilgileri göstermede ve akıl yürütmeye yardımcı olmada son derece önemli bir rol oynamaktadır. Karmaşık süreçleri görselleştirmemize veya sorunların ve görevlerin yapısını açık hale getirmemize yardımcı olurlar. Bir akış şeması, uygulanacak bir süreci

veya projeyi tanımlamak için de kullanılabilir.

Aşağıda taban ve yüksekliği klavyeden girilen bir dik üçgenin alanını hesaplamak için bir akış diyagramı örneği verilmiştir.



Şekil 6.2: Dik Üçgenin Alana Hesabını Gösteren Akış Diyagramı

Akış diyagramı çizilirken aşağıdaki kurallara uyulur;

- ◊ Akış şemalarında tek bir başlangıç simgesi olmalıdır
- ◊ Bitiş simgesi birden çok olabilir.
- ◊ Karar simgesinin haricindeki simgelere her zaman tek giriş ve tek çıkış yolu bulunur.
- ◊ Bağlaç simgesi sayfanın dolmasından ötürü parçalanmış akış şemasının öğelerini birleştirmede kullanılır.
- ◊ Simgeler birbirleri ile tek yönlü okla bağlanırlar.
- ◊ Okların yönü algoritmanın mantıksal işlem akışını tanımlar.

6.3 Duruma Göre Seçimler

Duruma göre seçimler (**conditional choice**) programın akışını bir koşula göre değiştiren talimatlardır.

§6.3.1 If Talimatı

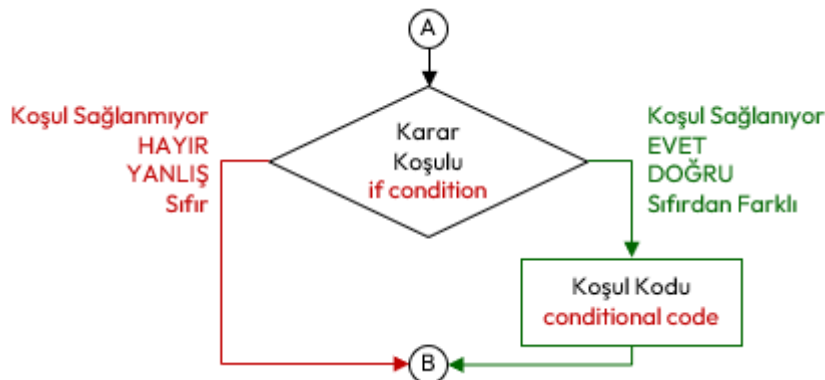
If talimatı, karar vermeye (**decision making**) ilişkin bir talimat olup bir koşula bağlı olarak programın icrasını değiştirir. If talimatının sözde kodu aşağıda verilmiştir;

```
if (karar-kosulu)
    kosul-kodu;
```

Burada ilişkisel işlemler içeren **karar koşulu** (**condition**) ifadesi (**expression**) test edilir. Sonuç DOĞRU ise **koşul kodu** (**conditional code**) icra edilir, değilse icra edilmez.

Karar koşulu test sonucu **sıfırdan farklı ise DOĞRU**, aksi durumda **YANLIŞ** kabul edilir. Bu husus [4.7.3](#) başlığında anlatılmıştı.

Bu talimatın akış diyagramı da aşağıdaki gibidir;



Şekil 6.3: If Talimatı İcra Akışı

Aşağıdaki örnek programı inceleyelim;

```
1 #include <stdio.h>
2 int main() {
3     int yas=18;
4     char cinsiyet='K';
5     if (yas<30)
6         printf("Genç");
7     if (cinsiyet=='E')
8         printf("Erkek");
9     return 0;
10 }
```

Örnek kodumuzu şu şekilde açıklayabiliriz;

1. Birinci Satırda `printf` fonksiyonu için `stdio.h` başlığı koda dahil edilmiştir.
2. İkinci satırda programın başlayacağı ana fonksiyon tanımlanmıştır.
3. Üçüncü ve dördüncü satırda `yas` ve `cinsiyet` değişkenleri başlatılmıştır.
4. Beşinci satırda bir `if` talimatı bulunmaktadır. **Karar koşulu** (**condition**), `yas<30` ifadesiyle test edilecektir. Test sonucu 1 yani DOĞRU olduğundan altıncı satırdaki koşul kodu (**conditional code**) talimatı **icra edilir**. Yani `printf("Genç");` talimatı ile konsola "Genç" yazılır.
5. Yedinci satırda da bir `if` talimatı bulunmaktadır. Karar koşulu, `cinsiyet=='E'` ifadesiyle test edilecektir. Test sonucu 0 yani YANLIŞ olduğundan sekizinci satırdaki koşul kodu talimatı **icra edilmez**.
6. Son olarak dokuzuncu satırdaki `return 0;` ifadesi icra edilerek işletim sistemine dönülür.

If talimatında koşul kodu (**conditional code**) her zaman ardışık işlem (**sequential operation**) olmaz. Bazen `if` gibi bir başka kontrol işlemi (**control operation**) de olabilir.

Aşağıda kademeli (**compound/cascade**) `if` kullanımına ilişkin kod örneğinde ikinci `if`, birinci `if` talimatının koşul kodudur;

```
if (yas<30)
    if (yas<7) // Bu if, üsteki if talimatının koşul kodudur.
        printf("Çocuk");
```

Koşul kodu (**conditional code**) birden fazla talimattan oluşacak ise kod bloğu içine alınır. Aşağıda verilen kod örneğinde ilk `if` talimatında `yas<30` ile test edilen koşul doğrulandığında kod bloğunun içindeki tüm talimatlar icra edilir.

```
if (yas<30) { //koşul-kodu bloğu başlangıcı
    if (yas<7)
        printf("Çocuk");
    if (yas<18)
        printf("Genç");
    if (yas>60)
        printf("Yaşlı");
} //koşul-kodu bloğu bitişi
```

Bazı durumlarda bir `if`, başka bir `if` bloğu içinde yani iç içe (**nested if**) olabilir. Buna ilişkin kod örneği de aşağıda verilmiştir.

```
if (cinsiyet=='E') { // dış if bloğu başlangıcı
    if (yas<7) { // iç if bloğu başlangıcı
        printf("Erkek Çocuk ");
        printf("Yada Erkek Bebek");
    } // iç if bloğu başlangıcı
    if (yas<18)
        printf("Genç Delikanlı");
    if (yas>60)
        printf("Yaşlı Adam");
} // dış if bloğu bitişi
```

If talimatında koşul (**condition**) her zaman tek bir test ifadesinden (**expression**) oluşmayabilir. Bahsedilen duruma ilişkin kod örneği de aşağıda verilmiştir.

```
if ((cinsiyet=='E') && (yas<18)) // hem Erkek, hem de genç
    printf("Genç Delikanlı");
if ((cinsiyet=='K') && (yas>60)) // hem yaşlı, hem de kadın
    printf("Yaşlı Kadın");
```

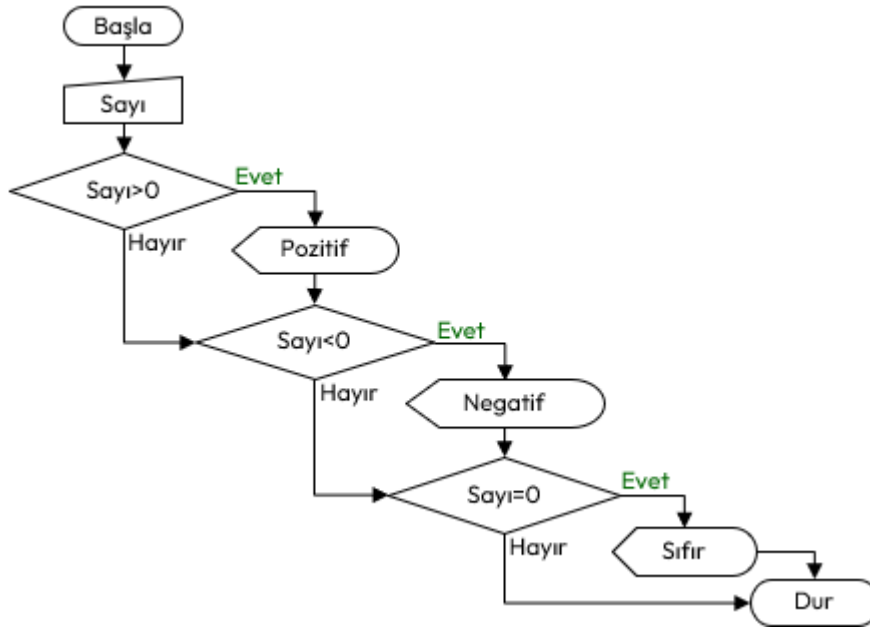
Bazen programcı tarafından aşağıdaki gibi if talimatları yazabilir.

```
if (1)
    printf("Bu metin konsola her zaman yazılır.");
if (0)
    printf("Bu metin konsola hiçbir zaman yazılmaz!");
```

Örnek olarak klavyeden girilen bir sayının işaretini bulan bir C programı yazalım;

```
#include <stdio.h>
int main() {
    int sayi;
    printf("Sayı Giriniz:");
    scanf("%d",&sayi);
    if (sayi>0) // sayının pozitif olup olmadığı test ediliyor.
        printf("Pozitif"); // Burada sayı>0 olduğu biliniyor.
    if (sayi<0) // sayının negatif olup olmadığı test ediliyor.
        printf("Negatif"); // Burada sayı<0 olduğu biliniyor.
    if (sayi==0) // sayının sıfır olup olmadığı test ediliyor.
        printf("Sıfır"); // Burada sayı=0 olduğu biliniyor.
    //Başka ihtimal olsaydı buraya gelecekti.
    return 0;
}
```

Bunun için oluşturacağımız akış diyagramı aşağıdaki şekilde olacaktır.



Şekil 6.4: Bir Sayının İşaretini Bulan Akış Diyagramı

§6.3.2 If-Else Talimatı

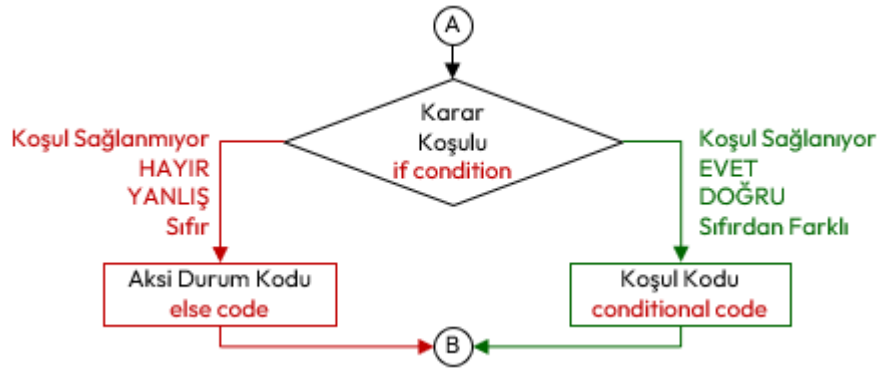
If-Else Talimatı, bir başka karar verme (**decision making**) talimatı olup bir koşula bağlı olarak programın icrasını değiştirir. Sözde kodu aşağıda verilmiştir;

```
if (karar-kosulu)
    kosul-kodu;
else
    aksi-durum-kodu;
```

If-Else talimatında, **karar koşulu** (**condition**) ifadesi DOĞRU ise **koşul kodu** (**conditional code**) icra edilir, karar koşulu YANLIŞ ise **aksi durum kodu** (**else code**) icra edilir.

Aşağıda verilen örnek programı inceleyelim;

```
1 #include <stdio.h>
2 int main() {
3     int yas=65;
4     char cinsiyet='K';
```



Şekil 6.5: If-Else Talimatı İcra Akışı

```

5     if (yas<50)
6         printf("Genç");
7     else
8         printf("Yaşlı");
9     return 0;
10 }

```

Örnek kodumuzu şu şekilde açıklayabiliriz;

1. Birinci Satırda `printf` fonksiyonu için `stdio.h` başlığı koda dahil edilmiştir.
2. İkinci satırda programın başlayacağı ana fonksiyon tanımlanmıştır.
3. Üçüncü ve dördüncü satırda `yas` ve `cinsiyet` değişkenleri başlatılmıştır.
4. Beşinci satırda bir `if` talimatı bulunmaktadır. **Koşul kodu** (`condition`), `yas<50` ifadesiyle test edilecektir. Test sonucu 0 yani YANLIŞ olduğundan altıncı satırdaki koşul kodu (`conditional code`) talimatı **icra edilmez**. Bunun yerine `else` sonrası sekizinci satırdaki aksi durum kodu (`else code`) talimatı **icra edilir**. Yani `printf("Yaşlı");` talimatı ile konsola "Yaşlı" yazılır.
5. Son olarak dokuzuncu satırdaki `return 0;` ifadesi icra edilerek işletim sistemine dönülür.

If talimatında olduğu gibi bu talimatta da **koşul kodu** (`conditional code`) ya da **aksi durum kodu** (`else code`) birden fazla talimattan oluşacak ise kod bloğu içine alınır. Buna ilişkin kod örneği aşağıda verilmiştir.

```

if (cinsiyet=='E') {
    if (yas<7)
        printf("Erkek Çocuk ");
    if (yas<18)
        printf("Genç Delikanlı ");
    if (yas>60)
        printf("Yaşlı Adam ");
} else {
    if (yas<3)
        printf("Kız Bebek ");
    if (yas<7)
        printf("Kız Çocuk ");
    if (yas<18)
        printf("Genç Kız ");
    if (yas>60)
        printf("Yaşlı Kadın ");
}

```

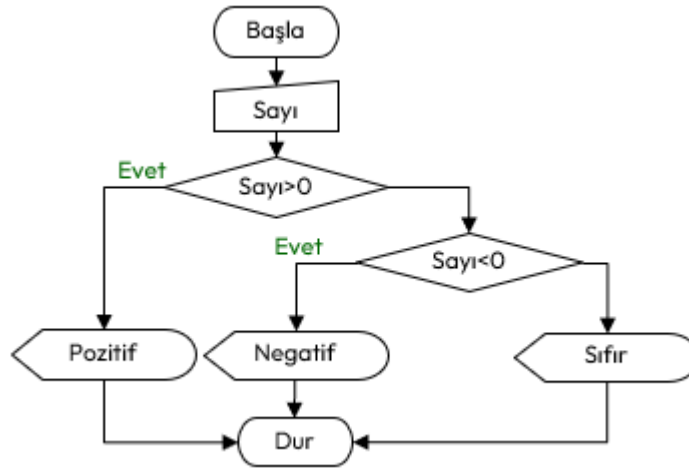
Bir sayının işaretini bulan C programının icra akışı olan Şekil 6.4 incelendiğinde ilk `if` talimatında yapılan test doğru çıkarsa geri kalan testlerin yapılmasına gerek kalmadığı anlaşılabacaktır. Benzer şekilde ilk iki test geçilirse üçüncü `if` talimatındaki teste gerek yoktur. Bu durumu aşağıdaki akış diyagramında daha net görebiliriz;

Bu durumda aynı program `if-else` talimatlarıyla aşağıdaki şekilde yeniden yazabiliriz;

```

1  #include <stdio.h>
2  int main() {
3      int sayi;
4      printf("Sayı Giriniz:");
5      scanf("%d",&sayi);
6      if (sayi>0) // sayı için pozitif testi yapıldı

```

Şekil 6.6: Bir Sayının İşaretini Bulan If-Else Akış Diyagramı

```

7     printf("Pozitif"); // Burada sayı>0 olduğu biliniyor
8   else { // Burada sayı>0 olmadığı biliniyor
9       if (sayı<0) // sayı için negatif testi yapıldı
10          printf("Negatif"); // Burada sayı<0 olduğu biliniyor
11      else // Bu noktada hem sayı>0 olmadığı ve hem de sayı <0 olmadığı biliniyor
12          printf("Sıfır"); // Bu noktada sayı=0 olduğu biliniyor
13  }
14  return 0;
15 }

```

Örnek kodumuzu şu şekilde açıklayabiliriz;

- ◊ Eğer `sayı` pozitif girilirse;
 - Altıncı satırdaki if karar koşulunda `sayı>0` test edilip DOĞRU çıkacak ve sadece yedinci satırdaki koşul kodu olan `printf("Pozitif");` talimatı icra edilecektir. Sekizinci satırdan başlayan ve on üçüncü satırda biten aksi durum kodu icra edilmeyecektir.
 - Daha sonra on dördüncü satırdaki `return 0;` talimatı icra edilerek işletim sistemine dönülecektir.
- ◊ Eğer `sayı` negatif girilirse;
 - Altıncı satırdaki if karar koşulunda `sayı>0` test edilip YANLIŞ çıkacak ve sekizinci satırdan başlayan ve on üçüncü satırda biten aksi durum kodu icra edilecektir.
 - Bu blok içinde dokuzuncu satırdaki if karar koşulu `sayı<0` test edilip DOĞRU çıkacak ve onuncu satırdaki koşul kodu olan `printf("Negatif");` talimatı icra edilecektir. On ikinci satırlardaki aksi durum kodu icra edilmeyecektir.
 - Daha sonra on dördüncü satırdaki `return 0;` talimatı icra edilerek işletim sistemine dönülecektir.
- ◊ Eğer `sayı` sıfır girilirse;
 - Altıncı satırdaki if karar koşulunda `sayı>0` test edilip YANLIŞ çıkacak ve sekizinci satırdan başlayan ve on üçüncü satırda biten aksi durum kodu icra edilecektir.
 - Bu blok içinde dokuzuncu satırdaki if karar koşulu `sayı<0` test edilip YANLIŞ çıkacak ve onuncu satırdaki koşul kodu talimatı icra edilemeyecektir. Onun yerine on ikinci satırlardaki aksi durum kodu olan `printf("Sıfır");` icra edilecektir.
 - Daha sonra on dördüncü satırdaki `return 0;` talimatı icra edilerek işletim sistemine dönülecektir.

Ayrıca kod incelendiğinde her bir `if` ve `else` sonrasında tek talimat bulunmaktadır. Dolayısıyla blok kullanmaya gerek de yoktur. Bu durumda kodun nihai hali aşağıda verilmiştir.

```

#include <stdio.h>
int main() {
    int sayi;
    printf("Sayı Giriniz:");
    scanf("%d",&sayi);
    if (sayi>0)
        printf("Pozitif");
    else
        if (sayi<0)
            printf("Negatif");
        else

```

```

        printf("Sıfır");
    return 0;
}

```

§6.3.3 Sarkan Else

Aşağıdaki program örneği incelendiğinde **else**, hangi **if** talimatına aittir? Böyle bir kod programcı tarafından yazılabilir ve derleyici buna hiçbir sıkıntı çıkarmaz. Bu problem **sarkan else** (**dangling else**) problemi olarak bilinir.

```
if (cinsiyet=='E') if (yas<18) printf("Delikanlı"); else printf("Adam");
```

Bu durumda kodu aşağıdaki gibi okunaklı hale getirdiğimizde ilk **if** talimatının **koşul kodunun** (**conditional code**) ikinci **if-else** talimatı olduğu görülmektedir.

```

if (cinsiyet=='E')
    if (yas<18)
        printf("Delikanlı");
    else
        printf("Adam");

```

Kod bloğu kullanılmadan yazılan bu tip kodlarda **else** her zaman kendinden bir önceki **if** talimatına aittir.

Sarkan else durumundaki mantıksal hataların önüne geçmek için
acemi kullanıcılar her zaman blok kullanmalıdır.

```

if (cinsiyet=='E') {
    if (yas<18) {
        printf("Delikanlı");
    } else {
        printf("Adam");
    }
}

```

§6.3.4 Switch Talimatı

Switch talimatı, bir kontrol ifadesi (**control expression**) sonucunda birden fazla alternatif arasında seçim yapılmasını sağlar. Sözde kodu aşağıda verilmiştir;

```

switch (kontrolifadesi) { //switch bloğu başlangıcı
    case alternatif-1:
        alternatif-1-kodu;
        break; //break talimat zorunlu değil
    case alternatif-2:
        alternatif-2-kodu;
        break; //break talimat zorunlu değil
    // ...
    default:
        varsayılan-alternatif-kodu;
} //switch bloğu bitişi

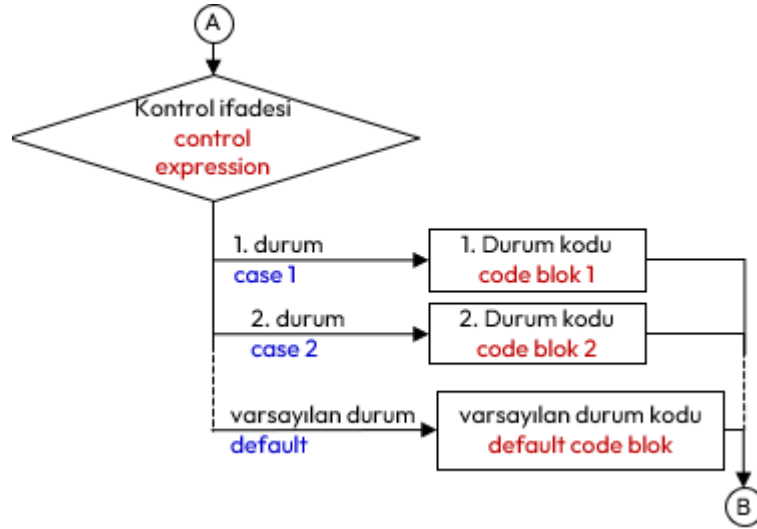
```

Switch talimatına ilişkin kurallar;

1. Kontrol ifadesi (**control expression**), sonucu tamsayı olan ifadedir ve bir kez değerlendirilir.
2. Bloğu olmayan bir switch talimatı yazılamaz!
3. Her bir alternatif bir tamsayı değişmezi (**integer literal**) izleyen ve iki nokta ile biten bir etiket (**label**) olarak tanımlanır.
4. Alternatif hiçbir zaman değişken veya ifade (**expression**) olamaz!
5. Blok için yazılan kod sonuna **break**; talimatı yazılarak **switch** bloğu dışına çıkılır. Zorunlu değildir, yazılmaz ise bir sonraki alternatif için yazılan kod icra edilir.
6. Tüm tam sayıları içerecek şekilde alternatiflerin tümü yazılmayabilir. Blok sonunda yer alacak şekilde üzerinde yer alan alternatifler dışında kalan tüm alternatifler için **default**: etiketli alternatif yazılabilir. Zorunlu değildir.

Aşağıda örnek olarak menü seçimi için yazılmış bir program örneği verilmiştir;

```
1 #include <stdio.h>
```



Şekil 6.7: Switch Talimatı İcra Akışı

```

2  int main() {
3      int menu;
4      printf("Menü İçin Rakam Giriniz:");
5      scanf("%d",&menu);
6      switch(menu) {
7          case 1:
8              printf("1 Numaralı Menü:\n");
9              printf("Hamburger ve Ayrar Hazırlanacak.\n");
10             break;
11         case 2:
12             printf("2 Numaralı Menü:\n");
13             printf("Patates ve Kola Hazırlanacak.\n");
14             break;
15         case 3:
16         case 4:
17             printf("3 veya 4 Numaralı Menü:\n");
18             break;
19         default:
20             printf("1,2, 3 veya 4 Dışında Menü:\n");
21             printf("Böyle bir Menüümüz Yok!\n");
22     }
23     return 0;
24 }

```

Örnek kodumuzu şu şekilde açıklayabiliriz;

- ◊ Eğer `menu`, 1 girilirse;
 - 6.Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu 1 olduğuna karar verilir.
 - Bu durumda 7.Satırdaki `case 1:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
 - İlk önce 8.Satırdaki `printf("1 Numaralı Menü:\n");` icra edilir ve bir sonraki talimata geçilir.
 - Daha sonra 9.Satırdaki `printf("Hamburger ve Ayrar Hazırlanacak.\n");` icra edilir ve bir sonraki talimata geçilir.
 - Ondandan sonra 10.Satırdaki `break;` talimatı bizi `switch` bloğunun sonundan dışına çıkarır.
 - Son olarak 23.Satırdaki `return 0;` talimatı icra edilerek işletim sistemine döndülür.
- ◊ Eğer `menu`, 2 girilirse;
 - 6.Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu 2 olduğuna karar verilir.
 - Bu durumda 11.satırdaki `case 2:` etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
 - İlk önce 12.Satırdaki `printf("2 Numaralı Menü:\n");` icra edilir ve bir sonraki talimata geçilir.
 - Daha sonra 13.Satırdaki `printf("Patates ve Kola Hazırlanacak.\n");` icra edilir ve bir

- sonraki talimata geçilir.
 - Ondan sonra 14.Satırdaki `break`; talimatı bizi `switch` bloğunun sonundan dışına çıkarır.
- Son olarak 23.Satırdaki `return 0`; talimatı icra edilerek işletim sistemine dönülür.
- ◇ Eğer `menu`, 3 girilirse;
 - 6.Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu 3 olduğuna karar verilir.
 - Bu durumda 15.Satırdaki `case 3`: etiketine gidilir ve bu alternatife ilişkin yazılmamıştır. Ancak ilk icra edilebilir koda kadar ilerlenir ve oradaki kodlar sırasıyla icra edilir;
 - İlk önce 17.Satırdaki `printf("3 veya 4 Numaralı Menü:\n");` icra edilir ve bir sonraki talimata geçilir.
 - Daha sonra 18.satırdaki `break`; talimatı bizi `switch` bloğunun sonundan dışına çıkarır.
 - Son olarak 23.Satırdaki `return 0`; talimatı icra edilerek işletim sistemine dönülür.
- ◇ Eğer `menu`, 4 girilirse;
 - 6.Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu 4 olduğuna karar verilir.
 - Bu durumda 16.Satırdaki `case 4`: etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
 - İlk önce 17.Satırdaki `printf("3 veya 4 Numaralı Menü:\n");` icra edilir ve bir sonraki talimata geçilir.
 - Daha sonra 18.Satırdaki `break`; talimatı bizi `switch` bloğunun sonundan dışına çıkarır.
 - Son olarak 23.Satırdaki `return 0`; talimatı icra edilerek işletim sistemine dönülür.
- ◇ Eğer `menu`, -1 , 0, 12 ve 300 gibi yukarıdaki alternatiflerden farklı girilirse;
 - 6.Satırdaki `switch` talimatında kontrol ifadesi test edilir ve sonucu düzenlenememiş bir durum olduğuna karar verilir.
 - Bu durumda 19.Satırdaki `default`: etiketine gidilir ve bu alternatife ilişkin kodlar sırasıyla icra edilir;
 - İlk önce 20.Satırdaki `printf("1,2, 3 veya 4 Dışında Menü:\n");` icra edilir ve bir sonraki talimata geçilir.
 - Daha sonra 21.Satırdaki `printf("Böyle bir Menüümüz Yok!\n");` talimatı bizi `switch` icra edilir.
 - artık `switch` bloğunun sonuna ulaşıldığından bloktan çıkılır.
 - Son olarak 23.Satırdaki `return 0`; talimatı icra edilerek işletim sistemine dönülür.

Aşağıda klavyeden girilen bir sayının 4 rakamına bölünmesinde kalan üzerinden işlem yapan `switch` talimatı bir başka örnek program verilmiştir;

```
#include <stdio.h>
int main() {
    int sayi;
    printf("Bir Sayı Giriniz:");
    scanf("%d",&sayi);
    switch(sayi%4) { //kontrol ifadesi bir işlem içerir
        case 3:
            printf("Sayı 4 rakamına bölündüğünde kalan 3'tür.\n");
            break;
        case 2:
            printf("Sayı 4 rakamına bölündüğünde kalan 2'dir.\n");
            break;
        case 1:
            printf("Sayı 4 rakamına bölündüğünde kalan 1'dir.\n");
            break;
        default: // Başka alternatif yoktur.
            printf("Sayı 4 Rakamına TAM Bölünür.\n");
    }
    return 0;
}
```

§6.3.5 Üçlü Koşul İşleci

Daha önce İşleçler başlığı altında işlenenlere ek olarak, ardışık işlemlerde (**sequential operation**) if talimatlarına gerek kalmadan bir koşula göre ifadeler (**expression**) işlenecek ise **üçlü koşul işleci** (**conditional tenary operator**) kullanılır. Aşağıda üçlü işlecin iki sözdizimi verilmiştir;

koşul ? doğruifadesi : yanlışifadesi;
 değişken = koşul ? doğruifadesi : yanlışifadesi;

Aşağıda oy kullanma durumu örneği bu işleçle işlenmiştir;

```
#include <stdio.h>
int main() {
    int yas;
    printf("Yaşınız?:");
    scanf("%d", &yas);
    (yas >= 18) ? printf("Oy Kullanabilirsin.") : printf("Oy Kullanamazsın!");
    return 0;
}
```

Aşağıda başka kullanımlarına ilişkin farklı kod örneği verilmiştir;

```
#include <stdio.h>
int main() {
    int sayi, tek, cift;
    printf("Bir Sayı Giriniz: ");
    scanf("%d", &sayi);
    tek= (sayi%2) ? 1 : 0;
    cift= tek ? 0 : 1;
    int sonuc1=tek ? sayi+30 : sayi+40;
    int sonuc2=cift ? sayi*30 : sayi*40;
    //...
    return 0;
}
```

6.4 İlişkisel Döngüler

İşlemciler iyi bir sayııcıdırlar. CPU içindeki kaydediciler (**registers**) sayma işlevini ve birçok matematiksel işlemi yapmak için de kullanılır.

Birçok problemlerin çözümüne, belli adımların tekrarlanması ile gidilir. Bu tip problemler şimdiye kadar olan yöntemlerle yapılırsa, tekrar tekrar aynı kodu yazmak zorunda kalırız. Konsola 10 defa ismimizi yazdıran bir program örneğini ele alalım.

```
#include <stdio.h>
int main() {
    printf("ILHAN\n"); //1
    printf("ILHAN\n"); //2
    printf("ILHAN\n"); //3
    printf("ILHAN\n"); //4
    printf("ILHAN\n"); //5
    printf("ILHAN\n"); //6
    printf("ILHAN\n"); //7
    printf("ILHAN\n"); //8
    printf("ILHAN\n"); //9
    printf("ILHAN\n"); //10
    return 0;
}
```

Program incelendiğinde aynı **printf** talimatının tekrar tekrar yazıldığını görürüz. **Bu istenmeyen bir durumdur.**

Yapısal programlamada ise yazdığımız bir kodu tekrar tekrar icra etmenin yolu **ilişkisel döngü** (**relational loop**) talimatlarını kullanırız.

§6.4.1 Sayaç Kontrollü Döngüler

Belirlenen bir sayaç değişkeninin istenilen bir değere ulaşp ulaşmadığı kontrol ederek kodun tekrar tekrar icra edilmesi sağlanır. Yapısal programlama öncesinde tekrar edilecek kodun başına dönüş **goto** talimatıyla sağlanır. **Bu talimat icra sırasını etkileyen bir talimattır.**

Kodlama sırasında tekrar edilecek kodun başına iki nokta ile biten bir **etiket** (**label**) konulur. Etiketlere kimlik verilirken yine değişken kimliklendirme (**identifier definition**) kuralları uygulanır. Tekrar edilecek kodun sonunda ise **goto** talimatı etiketle birlikte kullanılır.

```
1 #include <stdio.h>
2 int main() {
3     int sayac=0;
```

```

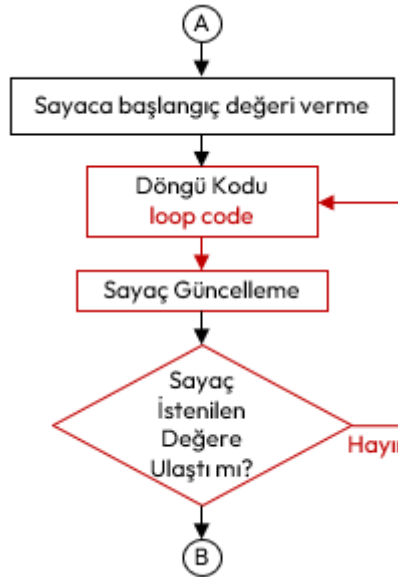
4      Etiket: //Döngü Kodunun Başı ETİKETLENİR!
5      printf("ILHAN %d\n",sayac);
6      sayac++; //Sayaç Güncelleme
7      if (sayac<10) //Sayaç Kontrolü
8          goto Etiket; // İstenilen değere ulaşılmamış ise Etiket'e git.
9      return 0;
10 }

```

Örneği şu şekilde açıklayabiliriz;

- ◇ 3.Satırda tekrar edilecek koda girmeden sayacımıza ilk değer verdik.
- ◇ 4.Satıra geri dönülecek kodun başını işaretleyen bir etiket konulmuştur.
- ◇ 5.Satırda tekrarlanacak talimat yazılmıştır. Birden fazla talimat da yazabiliriz.
- ◇ 6.Satırda sayacımızı güncelledik.
- ◇ 7.Satırda sayacın istenilen değere ulaşp ulaşmadığını kontrol ettik. Eğer ulaşmamış ise 8.Satırdaki **goto Etiket;** talimatıyla 4.Satıra döndük.

Sonuç olarak beşinci ile sekizinci satır arasında dört talimat içeren **mantıksal satır (logical sequence)** olmasına karşın 40 adet **fiziksel satır (physical sequence)** icra edilmiştir



Şekil 6.8: Sayaç Kontrollü Döngü İcra Akışı

C programlama dilinde ara seviye bir dil olduğundan **goto** talimatı desteklenir. 1968 Yılında Edsger W. Dijkstra tarafından **goto** ifadesi zararlı olarak ilan edilmiştir.

Yukarıda verilen döngü kodu örneğindeki gibi arapsaçı kodlamadaki **goto** talimatı yerine, **while**, **do-while** ve **for** kontrol talimatları yapısal programlamaya eklenmiştir.

§6.4.2 While Talimatı

While döngüsü, koşul (**condition**) DOĞRU olduğu sürece döngü kodu (**loop code**) icra edilir. Tekrarlanacak kod, tek bir talimat (**statement**) olabileceği gibi bir kod bloğu da olabilir.

While döngüsünün sözde kodu aşağıda verilmiştir;

```

while (koşul)
    döngü-kodu;

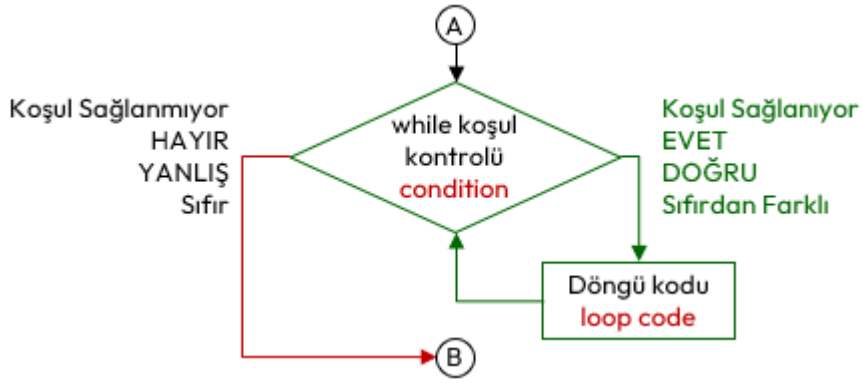
```

Bu durumda 6.4.1 başlığında verilen döngü örneği, **while** talimatı ile aşağıdaki şekilde yazılabilir;

```

#include <stdio.h>
int main() {
    int sayac=0; //sayaca ilk değer verme
    while (sayac<10) { // döngü bloğu başlangıcı
        printf("ILHAN\n");
        sayac=sayac+1; //Sayaç Güncelleme
    }
}

```



Şekil 6.9: While Talimatı İcra Akışı

```

    } //döngü bloğu bitişi
    return 0;
}

```

Sayaç güncelleme ifadesi koşul ifadesine eklenerek program daha da kısaltılabilir. Her koşul kontrolü sonrası sayaç artırılacaktır;

```

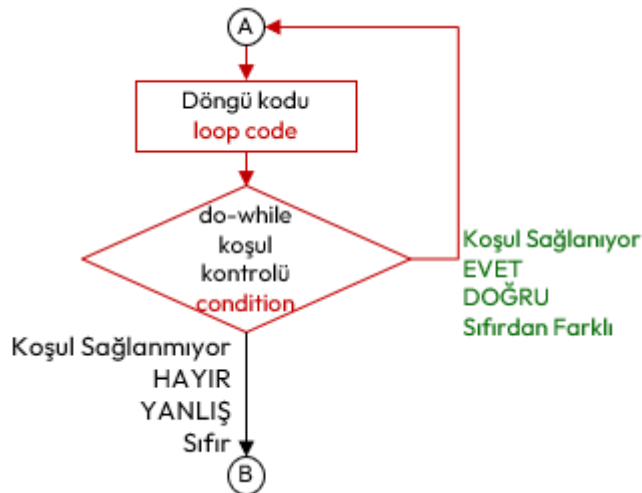
#include <stdio.h>
int main() {
    int sayac=0; //sayaca ilk değer verme
    while (sayac++<10) // hem sayaç güncelleme hem de koşul kontrolü
        printf("ILHAN\n"); // tek talimat olduğundan blok kullanılmadı
    return 0;
}

```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu bir adet talimat içeren mantıksal satır (**logical sequence**) içermesine rağmen 10 adet fiziksel satır (**physical sequence**) icra edilmiştir.

§6.4.3 Do-While Talimatı

Do-While döngüsü, do ile while saklı kelimeleri arasındaki döngü kodu (**loop code**) koşul (**condition**) DOĞRU olduğu sürece tekrarlayarak icra eder. Tekrarlanacak kod tek bir talimat (**statement**) olabileceği gibi bir kod bloğu da olabilir. While döngüsünde koşul kontrolü en başta yapılır, bu döngüde ise döngü bloğu en az bir kez çalıştırıldıktan sonra koşul kontrolü yapılır.



Şekil 6.10: Do-While Talimatı İcra Akışı

Do-While talimatının sözde kodu aşağıda verilmiştir;

```

do // "do" kelimesi goto talimatındaki etiket gibi davranır.
    döngü-kodu;
while (koşul);

```

Bu durumda 6.4.1 başlığında verilen döngü örneği, Do-While talimatı ile aşağıdaki şekilde yazılabilir;

```

#include <stdio.h>

```



```
int main() {
    int sayac=0; //sayaca ilk değer verme
    do { // döngü bloğu
        printf("ILHAN\n");
        sayac=sayac+1; //Sayaç Güncelleme
    } while (sayac<10); // Koşul kontrolü
    return 0;
}
```

Sayaç güncelleme ifadesi koşul ifadesine eklenerek program aşağıdaki şekilde daha da kısaltılabilir.

```
#include <stdio.h>
int main() {
    int sayac=0; //sayaca ilk değer verme
    do
        printf("ILHAN\n"); // tek talimat olduğundan blok kullanıl
    while (sayac++<10); // hem sayaç güncelleme hem de koşul kontrolü
    return 0;
}
```

Yukarıdaki son örnek incelendiğinde ibir adet talimat içeren mantıksal satır (**logical sequence**) içermesine rağmen 10 adet fiziksel satır (**physical sequence**) icra edilmiştir.

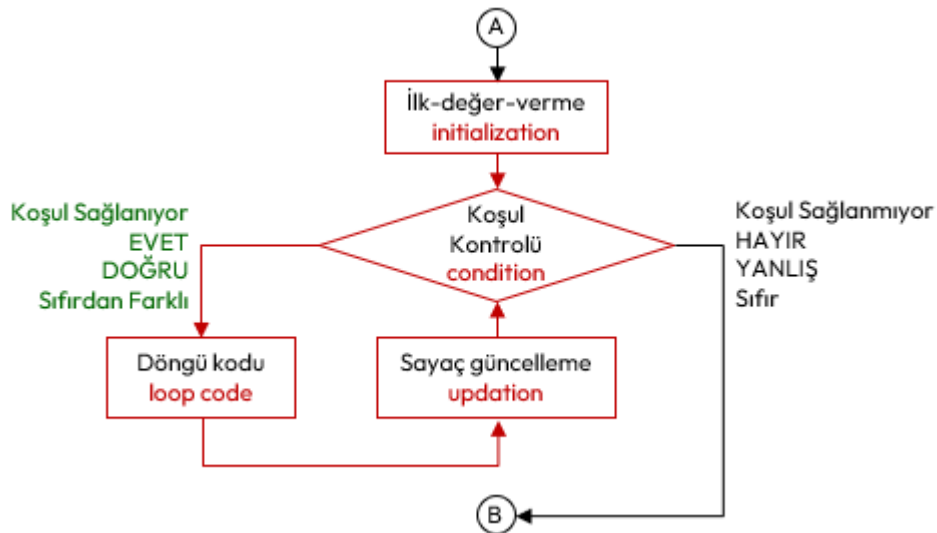
§6.4.4 For Talimatı

For döngüsü özellikle tekrar edilen işlemlerin sayısı belli olduğunda kullanılan bir döngü yapısıdır. Sayaç kontrollü döngüler için en iyi seçimdir. Sözde kodu aşağıda verilmiştir.

```
for (ilk-değer-verme; koşul-kontrolü; sayaç-güncelleme)
    döngü-kodu;
```

For talimatı aşağıdaki şekilde icra edilir;

1. **İlk değer verme ifadesi bir kez icra edilir.**
2. Sonra koşul kontrolü yapılır. Eğer test sonucu **DOĞRU** ise **döngü kodu icrasına geçilir**. Aksi halde döngüden çıkılır.
3. Döngü kodu icra edilir.
4. **Döngü kodu icrası bitince sayaç güncellemeleri yapılır ve tekrar ikinci adımdaki koşul kontrolüne dönülür.**



Şekil 6.11: For Talimatı İcra Akışı

Bu durumda 6.4.1 başlığında verilen döngü örneği, **for** talimatı ile aşağıdaki şekilde yazılabilir.

```
#include <stdio.h>
int main() {
    int sayac;
    for (sayac=1; sayac<10; sayac++)
        printf("ILHAN\n");
}
```

```
    return 0;
}
```

Buradaki döngü mantıksal olarak iki satır talimattan oluşmaktadır. Ancak fiziksel olarak 10 satır icra edilmiştir. Aşağıda konsola yazdığımız her bir satırın başına satır numarası koyan bir program gösterilmektedir.

```
#include <stdio.h>
int main() {
    int sayac;
    for (sayac=0; sayac<10; sayac++)
        printf("%02d-ILHAN\n", sayac+1);
    return 0;
}
/*Program Çıktısı:
01-ILHAN
02-ILHAN
03-ILHAN
04-ILHAN
05-ILHAN
06-ILHAN
07-ILHAN
08-ILHAN
09-ILHAN
10-ILHAN
*/
```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu bir adet talimat içeren mantıksal satır (**logical sequence**) içermesine rağmen 10 adet fiziksel satır (**physical sequence**) icra edilmiştir.

For döngüsü bir sayaç ile çalışılabileceği gibi birden fazla sayaç ile de çalışılabilir. Hem ilk değer verme hem de sayaç güncellemede birden fazla sayaca ilişkin işlem yapılabilir. Bunun için **virgül işlecisi (comma operator)** kullanılır.

```
#include <stdio.h>
int main() {
    int sayac1,sayac2;
    for (sayac1=0,sayac2=10; sayac1<10; sayac1++,sayac2--)
        printf("%02d-%02d-ILHAN\n", sayac1,sayac2);
    return 0;
}
/*Program Çıktısı:
00-10-ILHAN
01-09-ILHAN
02-08-ILHAN
03-07-ILHAN
04-06-ILHAN
05-05-ILHAN
06-04-ILHAN
07-03-ILHAN
08-02-ILHAN
09-01-ILHAN
*/
```

Aşağıda klavyeden girilen iki sayı aralığında tek ve çift sayıların toplamını bulan ve ekrana yazan programı verilmiştir.

```
#include <stdio.h>
int main() {
    int sayac, sayi1,sayi2;
    int tekToplam=0,ciftToplam=0;
    printf("İki Sayı Giriniz (10-25) gibi: ");
    scanf("%d-%d",&sayi1,&sayi2);
    if (sayi2<sayi1) { //sayi2, sayi1 den büyük olmalı.
        int temp=sayi1; //küçük ise yer değiştiriyoruz.
        sayi1=sayi2;
```

```

        sayi2=temp;
    }
    for (sayac=sayi1; sayac<=sayi2; sayac=sayac+1) {
        if (sayac%2==1) //sayac tek mi?
            tekToplam+=sayac;
        else
            ciftToplam+=sayac;
    }
    printf("Tektoplam: %d\nCiftToplam: %d\n", tekToplam, ciftToplam);
    return 0;
}
/*Program Çıktısı:
İki Sayı Giriniz (10-25) gibi: 9-17
Tektoplam: 65
CiftToplam: 52
*/

```

Bir ve kendisinden başka tam böleni olmayan sayıya **asal sayı** denir. Klavyeden girilen pozitif bir tam sayının asal olup olmadığını ekrana yazdıran C programı aşağıdaki şekilde kodlanabilir;

```

#include <stdio.h>
int main()
{
    int sayi, asal=1;
    printf("Bir Sayı Giriniz:");
    scanf("%d",&sayi);
    for (int sayac=sayi-1; (sayac>1)&&(asal==1); sayac--)
        if (sayi%sayac==0)
            asal=0;
    if(asal)
        printf("Girilen %d sayısı asaldır.",sayi);
    else
        printf("Girilen %d sayısı asal değildir.",sayi);
    return 0;
}

```

Klavyeden girilen pozitif bir tam sayının asal çarpanlarını ekrana yazdıran C programı aşağıdaki şekilde kodlanabilir;

```

#include <stdio.h>
int main()
{
    int sayi;
    printf("Bir Sayı Giriniz:");
    scanf("%d",&sayi);
    for (int sayac=sayi; sayac>0; sayac--)
        if (sayi%sayac==0)
            printf("Asal çarpan:%d\n",sayac);
    return 0;
}

```

§6.4.5 İç İç Döngü Kodu

Döngüler içinde yer alan döngü kodu bir başka döngüyü içerebilir. Örneğin ekrana satır ve sütun içeren bir şekil oluşturmak istediğimizde **iç içe döngü (nested loop)** kullanırız. Aşağıda buna ilişkin bir örnek verilmiştir. İlk **for** döngüsünün tekrarlanan kodu ikinci bir **for** döngüsüdür.

```

#include <stdio.h>
int main() {
    int satir,sutun;
    for (satir=0; satir<5; satir++) { //Dış Döngü Satırları Yazar
        for (sutun=0; sutun<4; sutun++) { //İç Döngü Sütunları Yazar
            printf("*"); //İmleç, yazdırılan metnin sonunda olacaktır.
        }
        printf("\n"); //Satır işi bitince imleci yeni satıra geçir!
    }
}

```

```

        return 0;
    }
    /*Program Çıktısı:
    ****
    ****
    ****
    ****
    ****
    */

```

Bir başka örnek olarak elemanları, satır ve sütun çarpımları olan 4x5 boyutlarındaki matrisi ekrana yazdıran program aşağıda verilmiştir;

```

#include <stdio.h>
int main() {
    int satir,sutun;
    //Başlığı Yazdıran Kısım
    printf("   Sütun:\t");
    for(sutun=0; sutun<3;sutun++)
        printf ("%3d ",sutun+1);
    printf("\n");
    //Matrisi Yazdıran Kısım
    for (satir=0; satir<5; satir++) { //Satır için
        printf("%2d.Satir:\t",satir+1); //İmleç, yazdırılan metnin sonunda olacaktır.
        for(sutun=0; sutun<3;sutun++) //Sütun için
            printf ("%03d ",sutun); //İmleç, yazdırılan metnin sonunda olacaktır.
        printf("\n"); //Satır işi bitince imleci yeni satıra geçir!
    }
    return 0;
}
/*Program Çıktısı:
Sütun:      1   2   3
1.Satir:    000 001 002
2.Satir:    000 001 002
3.Satir:    000 001 002
4.Satir:    000 001 002
5.Satir:    000 001 002
*/

```

§6.4.6 Gözcü Kontrollü Döngüler

Döngüler her zaman bir sayaca bağlı çalıştırılmaz. Bir döngüden çıkış koşulu döngü kodu içerisinde üretilen bir değere bağlı ise bu değere **gözcü değeri** (**sentinel value**) adı verilir. Buradaki gözcü değeri beklenen bir değer dışındaki bir değerdir. Buna örnek olarak klavyeden 0 girilene kadar girilen rakamları toplayan bir program verilmiştir.

```

#include <stdio.h>
int main()
{
    int sayi; /* okunan tamsayı */
    int toplam=0; /* girilen sayıların toplamı. başlangıçta 0 olmalı!*/
    do
    {
        printf("Bir sayı giriniz (Bitirmek için 0): ");
        scanf("%d",&sayi);
        toplam+=sayi; // sayi 0 girildiğinde toplam etkilenmez!
    } while (sayi != 0); //sentinel value 0
    printf("Girilen Sayıların Toplamı: %d",toplam);
    return 0;
}
/*Program Çıktısı:
Bir sayı giriniz (Bitirmek için 0): 10
Bir sayı giriniz (Bitirmek için 0): 20
Bir sayı giriniz (Bitirmek için 0): 30
Bir sayı giriniz (Bitirmek için 0): 0

```

```
Girilen Sayıların Toplamı: 60
*/
```

Aşağıda pozitif sayı girilmesini sağlayan bir başka kod örneği verilmiştir.

```
#include <stdio.h>
int main() {
    int sayi;
    do /* Pozitif girilmesini zorluyoruz */
    {
        printf("Lütfen pozitif sayı giriniz: ");
        scanf("%d",&sayi);
        if (sayi<=0)
            printf("HATA: Pozitif Sayı GİRMEDİNİZ!\n");
    } while (sayi <= 0);
    printf("Girilen pozitif tamsayı: %d \n",sayi);
    return 0;
}
/*Program Çıktısı:
Lütfen pozitif sayı giriniz: -1
HATA: Pozitif Sayı GİRMEDİNİZ!
Lütfen pozitif sayı giriniz: 10
Girilen pozitif tamsayı: 10
*/
```

Aynı örnek bir başka şekilde aşağıdaki gibi kodlanabilir. Ancak burada 4. ve 5. satırlar ile 9. ve 10. satırların aynı olduğuna ve kod tekrarı yapıldığına dikkat ediniz!

```
1  #include <stdio.h>
2  int main() {
3      int sayi;
4      printf("Lütfen pozitif sayı giriniz: ");
5      scanf("%d",&sayi);
6      while (sayi <= 0) /* Pozitif girilmesini sağlıyoruz */
7      {
8          printf("HATA: Pozitif Sayı GİRMEDİNİZ!\n");
9          printf("Lütfen pozitif sayı giriniz: ");
10         scanf("%d",&sayi);
11     }
12     printf("Girilen pozitif tamsayı: %d \n",sayi);
13     return 0;
14 }
```

6.5 Dallanmalar

Dallanmalar (**jump**), bir döngünün yada fonksiyonun icrasını kesen ve bir başka talimata yönlendiren talimatlardır.

§6.5.1 Continue ve Break Talimatları

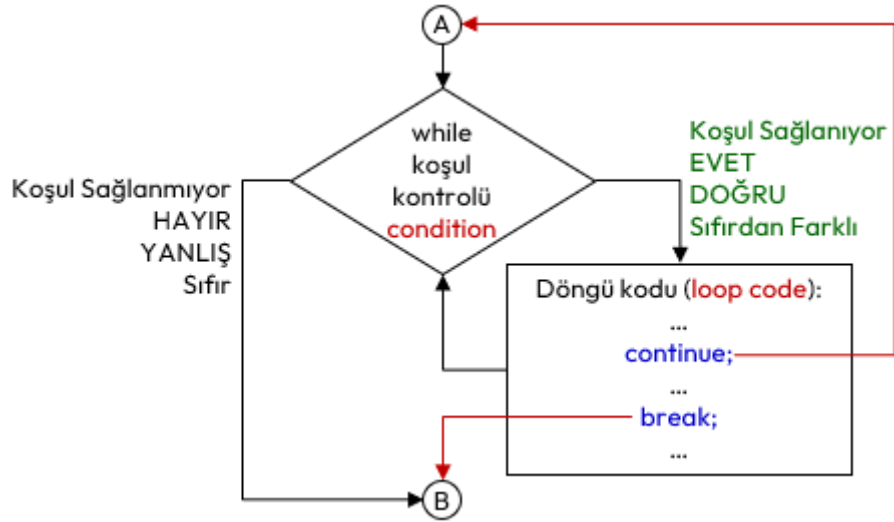
Continue talimatı , yalnızca geçerli yinelemeyi (**iteration**) sonlandırır ve sonraki yinelemelerle devam eder. Yani bu talimat, bir sonraki yinelemeyi yapmak için kullanılır. Dolayısıyla sadece **while**, **do-while** ve **for** döngü kodları (**loop code**) içinde kullanılabilir.

Break talimatı , döngüyü sonlandırır ve program icrası döngü sonrası ilk talimattan devam eder. Bunu daha önce **switch** talimatında görmüştük. Bu talimat, aynı şekilde **while**, **do-while** ve **for** döngüleri ile kullanılabilir.

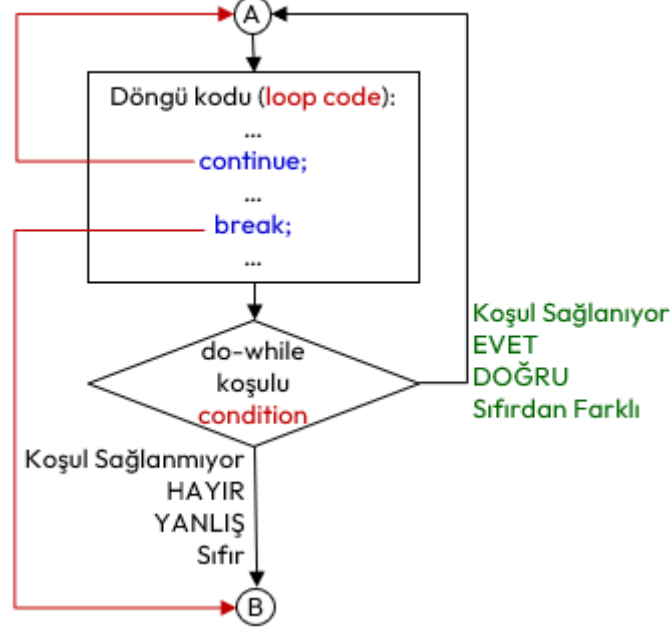
Bu talimatlarının döngü kodlarında kullanılması halinde icra akışı Şekil 6.12’de verilmiştir.

Bu talimatlar döngü kodu içinde amacımıza uygun olarak birden çok kullanabiliriz. Aşağıda 0 ile 15 arasındaki sayıların beşe bölünenleri ve 10 dışındaki sayılar ekrana yazdıran bir program örneği verilmiştir.

```
#include <stdio.h>
int main() {
    int i;
    for (i=0; i<=15; i=i+1) {
        if (i<7) //i'nin değeri 7 den küçük ise
```



(a) While



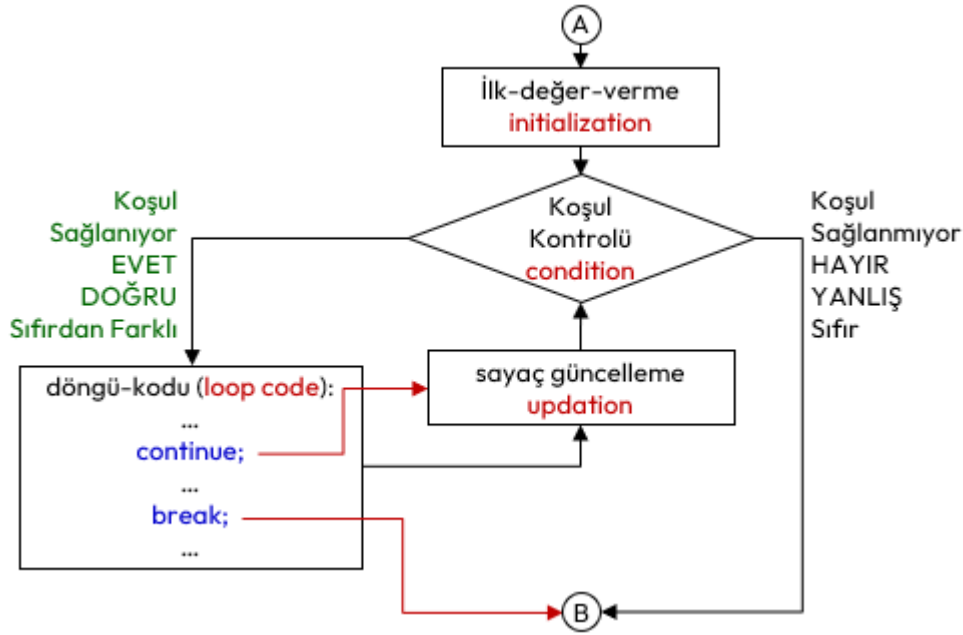
(b) Do-While

Şekil 6.12: While ve Do-While Döngülerinde Break ve Continue Talimatları İcra Akışı

```

        continue; //bir sonraki yinelemeye (iteration) geç
    if (i==10) //i'nin değeri 10 ise
        continue; //bir sonraki yinelemeye (iteration) geç
    if (i%5==0) // i'nin değeri 5 in katı ise
        continue; //bir sonraki yinelemeye (iteration) geç
    printf ("i = %d \n",i);
}
return 0;
}
/*Program Çıktısı:
i = 7
i = 8
i = 9
i = 11
i = 12
i = 13
i = 14
*/

```



Şekil 6.13: For Döngüsünde Break ve Continue Talimatları İcra Akışı

Yukarıdaki örnek incelendiğinde döngü bloğu üç adet talimat içeren mantıksal satır (**logical sequence**) içermesine rağmen 42 adet fiziksel satır (**physical sequence**) icra edilmiştir. Bazı `continue` talimatları `printf` talimatının icrasını engellemiştir.

Aşağıda pozitif sayı girilmesini zorlayan **sonsuz döngü** (**infinite loop**) içeren program verilmiştir. Programda döngü, pozitif sayı girildiğinde `break;` talimatı ile kırılmaktadır.

```
#include <stdio.h>
int main() {
    int sayi;
    int pozitifSayi=0;
    while (1) { /* Sonsuz döngü */
        printf("Lütfen pozitif sayı giriniz: ");
        scanf("%d",&sayi);
        if (sayi<=0)
            printf("HATA: Pozitif Sayı GİRMEDİNİZ!\n");
        else
            break; //döngüden çık
    }
    pozitifSayi=sayi;
    printf("Girilen pozitif tamsayı: %d \n",pozitifSayi);
    return 0;
}
```

Bu talimatları dikkatli kullanmalıyız. Bazı durumlarda sonsuz döngüye girme (**infinite loop**) durumu tüm döngü talimatlarında yaşanabilir. `for` Döngüsünde bu durum farklıdır. `continue;` talimatı sonrasında bir sonraki yineleme için önce sayaç güncelleme ifadeleri icra edilir ve sonra koşul testi yapılır.

§6.5.2 Return Talimatı

Yapısal programlamada ana fonksiyonun (**main function**) sonunda çokça kullandığımız bu talimat, bir fonksiyonun üreteceği ya da belirlenen değeri geri döndürür. Kullanıldığı yerden fonksiyon bloğu dışına çıkarılır. Aşağıda üç defa negatif sayı girilmesi halinde programı sonlandıran bir program verilmiştir.

```
#include <stdio.h>
int main() {
    int sayi;
    int pozitifSayi=0;
    int sayac=0; //kaç defa pozitif olmayan sayı girildi.
    do // Pozitif girilmesini zorluyoruz
    {
        printf("Lütfen pozitif sayı giriniz: ");
```



```

scanf("%d",&sayi);
if (sayi<=0) {
    printf("HATA: Pozitif Sayı GİRMEDİNİZ!\n");
    sayac++;
    if (sayac==3) {
        printf("Üç defa negatif sayı girdiniz. ");
        printf("Programdan Çıkılıyor.\n");
        return 1; // ana fonksiyondan çıkılarak işletim sistemine 1
        ↪ gönderiliyor.
    }
}
} while (sayi <= 0);
pozitifSayi=sayi;
printf("Girilen pozitif tamsayı: %d \n",pozitifSayi);
return 0;
}
/*Program Çıktısı:
Lütfen pozitif sayı giriniz: -1
HATA: Pozitif Sayı GİRMEDİNİZ!
Lütfen pozitif sayı giriniz: -2
HATA: Pozitif Sayı GİRMEDİNİZ!
Lütfen pozitif sayı giriniz: -3
HATA: Pozitif Sayı GİRMEDİNİZ!
Üç defa negatif sayı girdiniz. Programdan Çıkılıyor.

...Program finished with exit code 1
*/

```

§6.5.3 Goto Talimatı

Tanımlı bir etikete program akışını yönlendiren `goto` talimatıdır. 6.4.1 başlığında örneği verilmiştir.

6.6 Düşün ve Çöz

- ◊ **Düşün:** Sarkan else (**dangling else**) problemi nedir? Bu problemi önlemek için neden her zaman süslü parantez kullanılması önerilir?
- ◊ **Düşün:** while ve do-while döngüleri arasındaki farkı, bir kullanıcıdan şifre isteme senaryosu üzerinden karşılaştırınız.
- ◊ **Düşün:** **switch-case** yapısında **break;** kullanılmadığında oluşan başarısız olma (**fall-through**) durumunu bir hata değil, bir özellik olarak nasıl kullanabiliriz?
- ◊ **Çöz:** Kullanıcıdan sürekli sayı alan ve 0 girildiğinde o ana kadar girilen en büyük sayıyı ve sayıların ortalamasını ekrana basan bir gözcü kontrollü döngü (**sentinel value**) tasarımı yapınız.
- ◊ **Çöz:** İç içe döngüler kullanarak ekrana 1'den 10'a kadar olan sayıların çarpım tablosunu düzgün bir formatta yazdırınız.
- ◊ **Çöz:** Kullanıcı negatif bir sayı girene kadar sayı alan ve girilen tek sayıların toplamını, çift sayıların ise adedini ekrana basan programı yazınız.

7. Bölüm

Fonksiyonlar

7.1 Fonksiyon Nedir?

Yapısal Programlama mantığında çözülecek problem genel olarak fonksiyonların birbirini çağırmasıyla yapıldığı 2.7 başlığında anlatılmıştı. Yani kodlaması yapılacak işi birbirini çağırarak fonksiyonlar yazarak ana fonksiyondan bu fonksiyonları çağırarak yaparız. Ayrıca yazılacak programda birbiriyle ilgili fonksiyonlar bir araya toplanarak ayrı bir dosyada tutulur. Bu durum 5.1 başlığı altında anlatılmıştı. Bu bölüme kadar gördüğümüz kodlarda çözümü oluşturan tüm tasarım parçaları ana fonksiyon olan `main()` fonksiyonu içine çözülmüştür. Bir bilgisayar programı, genel olarak, tek başına tüm görevleri yerine getiren tek bir main fonksiyonundan oluşmaz. Bu durumda ana problemin büyüklüğüne göre ana fonksiyon bloğumuz uzar, okunabilirliği azalır, müdahale etmek zorlaşır. Bu tip büyük programları kendi içerisinde, her biri özel bir işi çözmek için tasarlanmış parçacıklarından oluşturmak daha mantıklı olacaktır. Yani böl ve yönet (**divide and conquer**) tekniği uygulanır. Çünkü büyük bir problemin toptan çözülmesi, problemin parçalar halinde çözülmesinden her zaman daha yorucu ve zordur. Yazılım geliştirmede problemi büyük ana parçalara ayırırız. Daha sonra bu büyük parçaları daha küçük parçalara ayırarak çözeriz. Buna yukarıdan aşağıya (**top down**) yaklaşım adı verilir. C dilinde bunu fonksiyonlar ile yaparız;

- ♦ Fonksiyonlar, belirli bir görevi gerçekleştiren bir kod bloğudur.
- ♦ Parametrelili alabilir, girdilerle ya da parametresiz bir şeyler yapar ve ardından cevabı verebilir. Kısaca bilgiyi fonksiyona argümanlar ile aktarırız.
- ♦ Her fonksiyonun bir kimliği vardır. (Değişken Kimliklendirme Kuralları burada da geçerlidir.)
- ♦ Bir fonksiyon program içerisinde istenildiği kadar farklı yerlerden ulaşılarak çalıştırılabilir ya da çağrılabilir. Bir başka deyişle tekrar kullanılabilir.
- ♦ Bir fonksiyon, çağırılan koda bir değer döndürebilir.
- ♦ Bir fonksiyon, bir başka fonksiyondan çağrılabilir.

C dilinde iki tip fonksiyon bulunmaktadır; başlık dosyalarında bize sunulan **hazır fonksiyonlar** ve programcının tanımlayacağı **kullanıcı tanımlı fonksiyonlar** (**user defined function**).

7.2 Hazır Fonksiyonlar

C geliştiricileri, programcılarının kullanmaları için, önceden yazılmış olan hazır fonksiyonlar hazırlamışlardır. Hazır fonksiyonlar teknik olarak C dilinin parçası değildir. Yalnızca standart hale getirilmişlerdir. Programcı, kullanmak istediği hazır fonksiyon hangi modülde ise o modüle ilişkin **başlık** (**header**) dosyasını **#include** ön işlemci yönergesi (**preprocessor directive**) ile kendi projesine dahil eder. Bunu 5.1 başlığı altında incelemiştik. Aslında bu başlık dosyalarında;

- ♦ Hazır fonksiyonların sadece **prototipleri** (**prototype**) başlık dosyalarında (**header file**) bulunur. Prototip derleyiciye, parametreleri ve geri dönüş değeri şu olan bir fonksiyon var demenin yoludur.
- ♦ Fonksiyonların kendileri yani derlenmiş halleri her işletim sistemi için ayrı derlenmiş olan kütüphane (**library**) dosyaları içerisinde bulunur.
- ♦ Derleme işlemi sırasında **bağlayıcı** (**linker**) program tarafından bu fonksiyonlar icra edilebilir dosyaya (**executable file**) dahil edilir.

§7.2.1 Math.h Başlık Dosyası

Matematik işlemleri gerçekleştirmek için standart hale getirilmiş bir başlık dosyasıdır. Tablo 7.1’de fonksiyon listesi verilmiştir. Örnek olarak kenarları verilen bir üçgenin alanını hesaplayan bir program aşağıda verilmiştir.

```
#include <stdio.h> //printf ve scanf için dahil ettik.
#include <math.h> //sqrt için dahil ettik.
int main()
{
    int kenar1, kenar2, kenar3;
    double alan, kenarlarToplamininYarisi;
    printf("Üçgenin Kenarlarını Giriniz:");
    scanf("%d%d%d",&kenar1,&kenar2,&kenar3);
    if ((kenar1+kenar2 <= kenar3) ||
```

```

(kenar2+kenar3 <= kenar1) ||
(kenar1+kenar3 <= kenar2) ) {
    printf("Böyle bir üçgen olamaz!\n");
    return 1;
}
/*AREA OF A TRIANGLE - HERON'S FORMULA */
kenarlarToplamininYarisi=(kenar1+kenar2+kenar3)/2.0;
alan= sqrt( kenarlarToplamininYarisi*
(kenarlarToplamininYarisi-kenar1)*
(kenarlarToplamininYarisi-kenar2)*
(kenarlarToplamininYarisi-kenar3) );
printf("Ucgenin alani: %.2f",alan );
return 0;
}
/*Program Çıktısı:
Üçgenin Kenarlarını Giriniz:3 10 12
Ucgenin alani: 12.18
*/

```

Fonksiyon prototipi	Açıklama	Örnek
<code>double sqrt(double);</code>	$\sqrt{x}$	<code>sqrt(900.0)=30.0</code>
<code>double exp(double);</code>	$\exp(x)$	<code>exp(1.0)=2.718282</code>
<code>double log(double);</code>	$\ln(x)$	<code>log(2.718282)=1.0</code>
<code>double log10(double);</code>	$\log(x)$	<code>log10(1.0)=0.0</code>
<code>double fabs(double);</code>	$ x $	<code>fabs(-5.0)=5.0</code>
<code>double ceil(double);</code>	x'i kendisinden büyük en küçük kesirsiz sayıya yuvarlar.	<code>ceil(-9.8)=-9.0</code>
<code>double floor(double);</code>	x'i kendisinden küçük en büyük kesirsiz sayıya yuvarlar.	<code>floor(9.8)=9.0</code>
<code>double pow(double x,double y);</code>	x^y	<code>pow(2.0,3.0)=8.0</code>
<code>double fmod(double x,double y);</code>	Kayan noktalı sayı olarak x'in y'ye bölümünden kalanı verir.	<code>fmod(13.657,2.333)=1.1992</code>
<code>double sin(double);</code>	Radyan cinsinden x'in sinüsü	<code>sin(0.0)=0.0</code>
<code>double cos(double);</code>	Radyan cinsinden x'in kosinüsü	<code>cos(0.0)=1.0</code>
<code>double tan(double);</code>	Radyan cinsinden x'in tanjantı	<code>tan(0.0)=0.0</code>

Tablo 7.1: Math.h Başlık Dosyası Fonksiyonları

§7.2.2 Stdlib.h Başlık Dosyası

Bu kütüphane içerisinde; Alfa sayısal metinleri ilkel veri tiplerine veya bunun tersini yapan tür dönüşüm fonksiyonları (`atoi()`, `itoa()`, `atof()`, `strtod()`, ... ileride anlatılacaktır.), Çalıştırma anında (run time) bellek tahsisi ve tahsisli belleğin serbest bırakılmasını sağlayan hafıza yerleşim fonksiyonları (`malloc()`, `free()` ... ileride anlatılacaktır.) Rastgele sayı üretme fonksiyonları (`rand()`, `srand()`) bulunur. Şimdilik rastgele sayı üretme üzerinde durulacaktır. Aşağıda bir zar için rastgele sayı üreten bir program verilmiştir.

Fonksiyon	Açıklama
<code>int rand();</code>	0 ile <code>RAND_MAX</code> yani 32767 arasında rastgele bir sayı üretir.
<code>int srand(unsigned int);</code>	<code>rand()</code> fonksiyonu belli bir başlangıç değerinden itibaren bir dizi matematiksel işlem sonucu rastgele bir değer üretir. <code>srand()</code> fonksiyonu, <code>rand()</code> fonksiyonuna başlangıç değerini farklı vermek için kullanılır.

Tablo 7.2: Stdlib.h Başlık Dosyasının Bazı Fonksiyonları

```

#include <stdio.h> // printf için
#include <stdlib.h> //rand için
int main() {

```

```

    int rastgeleZar;
    rastgeleZar=1 + rand() %6;
    /* rand() fonksiyonunun üreteceği sayıyı kalan işleci ile 6 sayısına böleriz ki 0 ile 5
       → arasında bir sayı elde edelim. Buna da 1 ekler isek zarın rakamları ortaya çıkar.*/
    printf("Atılan Zar:%d",rastgeleZar);
    return 0;
}

```

Fakat programın her çalışmasında `rand()` aynı başlangıç değerini kullandığından aynı rastgele değerleri üretir. Bu nedenle aynı zar rakamı gelmiş olur. Bunu değiştirmek için `srand()` fonksiyonu kullanılır.

```

#include <stdio.h> //printf için
#include <stdlib.h> //rand ve srand için
int main() {
    int rastgeleZar;
    int randBaslangicDegeri;
    printf("rand() için başlangıç değeri olabilecek bir sayı giriniz:");
    scanf("%d",&randBaslangicDegeri);
    srand(randBaslangicDegeri);
    rastgeleZar=1 + rand() %6;
    printf("Atılan Zar:%d",rastgeleZar);
    return 0;
}

```

Rastgele sayı her seferinde kullanıcıdan alınan sayıya göre hesaplandığından her seferinde farklı bir zar rakamı üretilmiş olacaktır. Ancak başlangıç değerinin her seferinde kullanıcıdan alınması bir başka problemdir. Bunun üstesinden gelmek için bilgisayarın saatini sayı olarak veren `time.h` başlık dosyasındaki `time()` fonksiyonu `NULL` parametresiyle kullanılabilir.

```

#include <stdio.h> // printf için
#include <stdlib.h> // rand ve srand için
#include <time.h> // time fonksiyonu için
int main() {
    int rastgeleZar;
    int randBaslangicDegeri=time(NULL);
    srand(randBaslangicDegeri);
    // üst iki satır yerine "srand(time(NULL));" yazılabilir!
    rastgeleZar=1 + rand() %6;
    printf("Atılan Zar:%d",rastgeleZar);
    return 0;
}

```

7.3 Kullanıcı Tanımlı Fonksiyonlar

Programcılar kendi fonksiyonlarını oluşturarak kodlarını daha kullanışlı hale getirirler. Bu tür fonksiyonlara **kullanıcı tanımlı fonksiyonlar** (**user defined function**) denilir.

Şimdiye kadar yazdığımız kodlarda ana fonksiyonumuz olan `main()` hep `sqrt()`, `rand()` gibi fonksiyonları çağırmıştır. Çağırın program, kullanılacak fonksiyonu bilmelidir. Kullandığımız başlık dosyalarında aşağıda açıklayacağımız **fonksiyon bildirimleri** (**function declaration**) bulunur.

Bir fonksiyonu çalıştıran koda, **çağırın program** adı verilir. 7.2 Başlığındaki örneklerde `main()` ana fonksiyonumuz çağırın programdır.

Kullanıcı tanımlı fonksiyonu oluşturmak ve kullanmak için bu dört unsuru bilmemiz gerekir;

1. **Fonksiyon Tanımı** (**function definition**): Bir fonksiyonun girdileri, yaptığı işlem ve döndüreceği değeri içerecek şekilde **başlık ve gövdesinin kodlanmasından oluşur**. Fonksiyon tanımı, fonksiyonu derleme yapılabilmesi için eksiksiz bir şekilde tamamlar. **Başlık**, fonksiyonun döndüreceği veri tipi, fonksiyon kimliği ve aç kapa parantez içindeki parametre listesinden oluşur. **Gövde** ise süslü parantezler ve içindeki fonksiyonun yaptığı işi içeren kodlardır.
2. **Fonksiyon Bildirimi** (**function declaration**): Derleyiciye, programın ilerleyen kısımlarında (veya belki başka bir dosyada) verilen kimlikte ve verilen parametrelere sahip bir fonksiyonun var olduğunu söyler. Derleyici bu sayede, fonksiyonun gövdesini görmese bile onun nasıl çağrılacağını, dönüş tipi ve argümanlarını bilir. Bildirimin amacı, derleyiciye "Böyle bir fonksiyon var" bilgisini vermektir. Bildirim yapılıp, tanımı yapılmayan bir fonksiyona sahip program derlen-

meye çalışıldığında, bağlama zamanında (**link-time**) hata alırız.

3. **Fonksiyon Prototipi** (**function prototype**): Aslında fonksiyon bildirimi ile aynı şeydir. Ancak prototip terimi, genellikle fonksiyonun kodun en tepesinde, **main()** fonksiyonundan önce yapılan bildirimini ifade etmek için kullanılır.
4. **Fonksiyon Çağırma**(**function call/invoke function**): Çağırın program içinde, bildirimi daha önce yapılmış bir fonksiyon, kimliğini ve ardından parantez içindeki argüman listesini yazarak çağırabiliriz.

§7.3.1 Fonksiyon Tanımı Nasıl Yapılır?

Fonksiyon tanımı (**function definition**), fonksiyonun işleyip **döndüreceği veri tipini, kimliği, işleyeceği** bağımsız değişkenleri temsil eden **parametre listesini**, içerecek şekilde **başlığının ve gövdesinin** kodlanmasından oluşur. Aşağıda bir fonksiyon tanımının sözde kodu verilmiştir;

```
geri-dönüş-tipi fonksiyon-kimliği(parametre-listesi) //BAŞLIK
{ //GÖVDE BAŞLANGICI
  //...
  // Algoritma
  //...
  return geri-dönüş-değeri;
  //...
} //GÖVDE BİTİŞİ
```

Sözde kodu şu şekilde açıklayabiliriz;

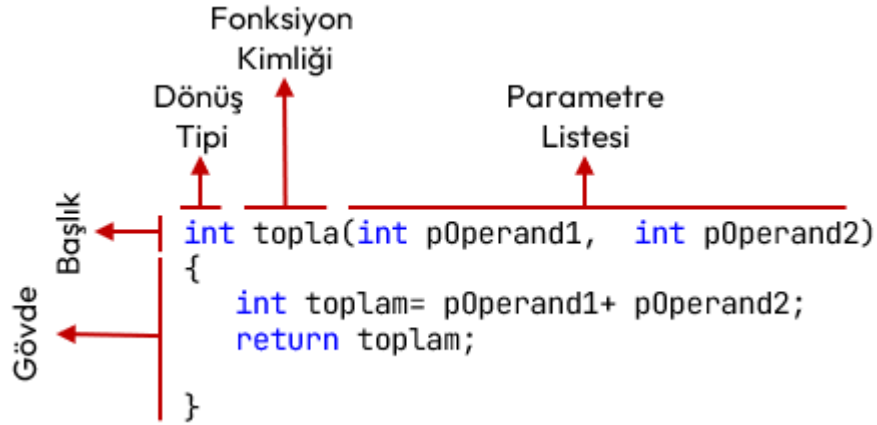
- ◊ **Fonksiyonun kimliği benzersizdir.** Aynı zamanda **fonksiyon adı** (**function name**) olarak adlandırılır. Fonksiyon kimliklendirmede, ?? başlığındaki değişken kimliklendirme kuralları geçerlidir.
- ◊ Fonksiyon kimliğinden sonra **parametreleri barındıracak açık kapatılan parantez** olmalıdır. Bu durum değişken tanımından fonksiyon tanımlarını ayırmak içindir.
- ◊ Bağımsız değişkenler, parametre olarak adlandırılır;
 - **Parametre listesi**, bir fonksiyonun parametrelerinin veri tipini, sırasını ve sayısını belirtir.
 - Parametreler isteğe bağlıdır, yani bir fonksiyon hiçbir parametre içermeyebilir.
 - **Ber bir parametreye kimlik verilmelidir.** Çünkü fonksiyon gövdesinde bu değişkenler işlenir.
 - Parametreler birbirinden **virgül işleci** (**comma operator**) ile ayrılır.
- ◊ Parametreler fonksiyon **gövdesinde** yapılacak işlere temel oluşturur;
 - Gövde, fonksiyonun ne yaptığını tanımlayan ve süslü parantezler arasında yer alan talimatları içerir.
 - Gövdede, **yapısal programlama yapıldığından**, ilk önce olacaksa değişkenler tanımlanmalı daha sonra parametrelerle birlikte bu değişkenleri işleyecek talimatlar yazılmalıdır.
 - Gövdede, fonksiyonda istenilen sonuca ulaşmak için kullanacağı adımlara karar verilir yani **algoritması belirlenir**.
- ◊ İşlenen parametrelerden ve gövde içerisinde tanımlanan değişkenlerden bir **geri dönüş değeri** hesaplanabilir. Değer hesaplanmış ise **return geri-dönüş-değeri**; talimatıyla fonksiyonun icrası sonlandırılır. `textttreturn` Talimatı çoğunlukla fonksiyonundaki son talimat olarak görünür ama fonksiyon gövdesinde herhangi bir yerden geri dönülebilir.
- ◊ **return** Talimatıyla geri döndürülen değerin tipi, **fonksiyonun geri dönüş tipidir**.
- ◊ Bazı durumlarda fonksiyon, geri değer döndürmeyebilir;
 - Bu durumda fonksiyonun **geri dönüş tipi yoktur** ve bu nedenle geri dönüş tipi **void** olarak belirtilir.
 - Geri dönüş tipi **void** olan fonksiyonun icrası **return**; talimatıyla sonlandırılır.
 - Eğer **return**; talimatı hiç yazılmaz ise süslü parantez kapatılıncaya kadar fonksiyon icra edilir ve fonksiyondan geri dönülür.

Bir de fonksiyonun çağrıldığı yerde, **fonksiyonun döndürdüğü değerin tipi ile eşleşen bir değişkene dönüş değeri atanabilir.** Zorunlu değildir.

Ayrıca C++ dilinde **bir fonksiyon gövdesinde bir başka fonksiyon tanımlanamaz!**

Şekil 7.1’de; iki parametresini toplayıp, toplamı geri döndüren bir **topla** fonksiyonunun tanımlaması verilmiştir. Aşağıdaki şekilde açıklayabiliriz;

- ◊ Fonksiyonun kimliği **topla** olarak belirlenmiştir;
- ◊ **topla** Fonksiyonunun parametre listesinde iki adet **int** veri tipinde parametre (**p0operand1, p0operand2**)



Şekil 7.1: Fonksiyon Tanımı Örneği

tanımlanmıştır.

- ◊ `toplama` Fonksiyonu gövdesinde yapısal programlama yapıldığından ilk önce `toplam` yerel değişkeni tanımlandı. Daha sonra parametrelerin toplamı `toplam` değişkenine atandı.
- ◊ Son olarak `return toplam;` talimatı ile hesaplan değer geri döndürülmüştür.

Aşağıda `toplama` adlı bir kullanıcı tanımlı fonksiyona ilişkin program verilmiştir.

```
#include <stdio.h>
// Kullanıcı tanımlı "toplama" fonksiyonu burada tanımlanıyor (definition):
int toplama(int p0perand1,int p0perand2) //Fonksiyon başlığı
{ //Fonksiyon bloğu başlangıcı
    //Fonksiyon Gövdesi:
    int toplam=p0perand1+ p0perand2;
    return p0perand1+ p0perand2; // Toplanan tamsayılardan ortaya çıkan tamsayı geri
    ↪ döndürülüyor!
} //Fonksiyon bloğu bitişi
//Bu noktadan sonra toplama kimlikli fonksiyon çağrılabilir.
int main () {
    // ÇAĞRI ORTAMI:
    /* Ana fonksiyon içinde "toplama" fonksiyonu çeşitli şekillerde ÇAĞRILACAK! (CALL). */
    int sonuc;
    toplama(1,2); /* toplama fonksiyonu çağrılıyor ama geri döndürdüğü değer kullanılmıyor. */
    printf("toplama: %d\n", toplama(2,3)); /* toplama fonksiyonu çağrılıyor. Geri döndürdüğü değer
    ↪ printf fonksiyonuna argüman olarak giriyor. */
    sonuc= toplama(3,4); /* toplama fonksiyonu çağrılıyor. Geri döndürdüğü değer sonuc
    ↪ değişkenine atanıyor. */
    printf("toplama: %d\n", sonuc);
    sonuc= toplama(1,2)+toplama(3,4); /* toplama fonksiyonu iki kez çağrılıyor. Her bir çağrıdaki
    ↪ geri dönüş değeri geçici bir yerde saklanıp toplamı sonuc değişkenine atanıyor. */
    printf("toplama: %d\n", sonuc);
    sonuc= toplama(toplama(1,2),3)+toplama(4,5); // Bir fonksiyon birden fazla çağrılabilir!
    printf("toplama: %d\n",sonuc);
    return 0;
}
/*Program Çıktısı:
toplama: 5
toplama: 7
toplama: 10
toplama: 15
*/
```

Örnek program, aslında 2.7 başlığında bahsedilen yapısal programlamanın çerçevesidir. Ana program içinde problemin çözümü, çeşitli fonksiyonların birbirini çağırması ile yapılmıştır. Fonksiyon içinde işlenecek veriye ilişkin değişken tanımlanmış ve bu veriler ile veri yapılarını işleyecek kontrol işlemleri (bizim örneğimizde kontrol işlemi yok) birbirinden ayrılmıştır. **Okunaklılık çok yüksek bir kod elde edilmiştir.**

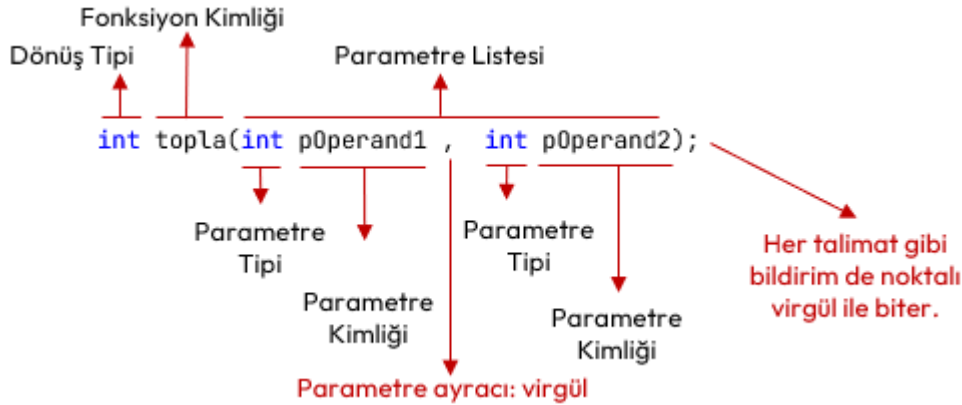
§7.3.2 Fonksiyon Bildirimi Nasıl Yapılır?

Fonksiyon bildirimi (**function declaration**), derleyiciye, programın ilerleyen kısımlarında (belki başka bir dosyada) belirtilen kimlikte ve verilen parametrelere sahip bir fonksiyonun var olduğunu söyler. Aşağıda bir fonksiyon bildiriminin sözde kodu verilmiştir;

```
geri-dönüş-tipi fonksiyon-kimliği(parametre-listesi);
```

Fonksiyon bildirimi, fonksiyon tanımındaki **başlık kısmı gibidir**;

- ◊ Bildirimin, parametre listesinde, **parametre kimlikleri zorunlu değildir**.
- ◊ Bildirim, talimatlar gibi **noktalı virgül ile biter**.
- ◊ Bildirimi yapılan fonksiyonun, fonksiyon tanımı (**function definition**) bir yerde mutlaka yapılmalıdır!
- ◊ Bildirimi yapılmış fonksiyonun tanımında aynı kimlik kullanılır.



Şekil 7.2: Fonksiyon Bildirimi Örneği

Şekil 7.2’de iki adet `int` veritipinde parametresi olan ve `int` veri tipinde geri değer döndüren `topla` kimlikli bir fonksiyon bildirimi örneği verilmiştir. Bildirim örneğini aşağıdaki şekilde açıklayabiliriz;

- ◊ Fonksiyonun kimliği `topla` olarak belirlenmiştir;
- ◊ `topla` Fonksiyonunun parametre listesinde iki adet `int` veri tipinde parametre (`p0operand1`, `p0operand2`) tanımlanmıştır. Parametre kimlikleri verilmeyebilir. Yani `int toplar(int,int);` aynı bildirimdir.
- ◊ Bildirimde `topla` fonksiyonunun geri dönüş veri tipi, `int` olarak belirlenmiştir.

Şekil 7.2’de iki `int` parametresi olan ve `int` veri tipinde geri değer döndüren `topla` kimlikli bir fonksiyon bildirimi örneği verilmiştir. Bildirim örneğini aşağıdaki şekilde açıklayabiliriz;

- ◊ **Dönüş tipi**, fonksiyonun döndüreceği değerın veri tipidir.
- ◊ **Fonksiyonun kimliği** benzersizdir.
- ◊ Fonksiyon kimliğinden sonra parametreleri barındıracak **açıp kapatılan parantez** olmalıdır. Bu durum fonksiyon bildirimini, değişken tanımından ayırmak içindir.
- ◊ **Parametre tipi**, fonksiyon girdisi olan parametrelerin yani bağımsız değişkenlerin veri tipidir. **Bildirimdeki parametrelere kimlik vermek zorunlu değildir!**
- ◊ Parametreler birden çok olabilir. Bu durumda aralarına **virgül işleci** (**comma operator**) konulur.
- ◊ Her talimat gibi fonksiyon bildirimi de **noktalı virgül** ile biter!

7.3.1 Başlığındaki `topla` örneğimizi bildirim yapılarak aşağıdaki şekilde yazılabilir;

```
#include <stdio.h>
int toplar(int,int); /* kullanıcı tanımlı toplar fonksiyonu bildirimi (declaration): Bu bildirimde
→ int geri döndüren ve int tipinde iki farklı parametresi olan "topla" kimlikli fonksiyon
→ derleyiciye bildirildi.*/
int main () {
    // ÇAĞRI ORTAMI:
    /* Ana fonksiyon içinde "topla" fonksiyonu çeşitli şekillerde ÇAĞRILACAK! (CALL). */
    int sonuc;
    toplar(1,2); /* toplar fonksiyonu çağrılıyor ama geri döndürdüğü değer kullanılmıyor. */
    printf("toplam: %d\n", toplar(2,3)); /* toplar fonksiyonu çağrılıyor. Geri döndürdüğü değer
→ printf fonksiyonuna argüman olarak giriyor. */
    sonuc= toplar(3,4); /* toplar fonksiyonu çağrılıyor. Geri döndürdüğü değer sonuc
→ değişkenine atanıyor. */
    printf("toplam: %d\n", sonuc);
```



```

sonuc= toplam(1,2)+toplama(3,4); /* toplama fonksiyonu iki kez çağrılıyor. Her bir çağrıdaki
↳ geri dönüş değeri geçici bir yerde saklanıp toplamı sonuc değişkenine atanıyor. */
printf("toplama: %d\n", sonuc);
sonuc= toplama(toplama(1,2),3)+toplama(4,5); // Bir fonksiyon birden fazla çağrılabilir!
printf("toplama: %d\n",sonuc);
return 0;
}
// Yukarıda Bildirimi yapılmış kullanıcı tanımlı "toplama" fonksiyonu burada tanımlanıyor
↳ (definition):
int toplama(int pOperand1,int pOperand2) //Fonksiyon başlığı
{ //Fonksiyon bloğu başlangıcı
    int toplam=pOperand1+ pOperand2;
    return pOperand1+ pOperand2; // Toplanan tamsayılardan ortaya çıkan tamsayı geri
    ↳ döndürülüyor!
} //Fonksiyon bloğu bitişi

/*Program Çıktısı:
toplama: 5
toplama: 7
toplama: 10
toplama: 15
*/

```

Daha önce fonksiyonun bildirimi (**function declaration**) yapılmış ise fonksiyon tanımı (**function definition**) aynı kimlikle tanımlanmalıdır. Ayrıca 7.3.1 başlığında verilen örnekteki gibi **hiç bildirim yapılmadan da fonksiyonlar tanımlanabilir**. Ancak fonksiyon tanımlandığı yerden sonra çağrılabilir.

Aşağıdaki fonksiyon bildirimi içeren bir başka programı inceleyelim. Bu program derleyici tarafından derlenir ancak **bağlama (linker)** aşamasında hata verir. Çünkü **fonksiyon tanımlamaları (function definition)** yapılmamıştır.

```

int ikiSayiyiTopla(int,int); /* "ikiSayiyiTopla" kimlikli bir fonksiyon olduğu ve iki tamsayı
↳ parametre aldığı ve ayrıca bir tamsayı geri döndürdüğü derleyiciye bildiriliyor. */

void sayiyiKonsolaYaz(int); /* "sayiyiKonsolaYaz" kimlikli bir fonksiyon olduğu ve bu fonksiyonun
↳ bir tamsayı parametre aldığı ve geri değer döndürmediği derleyiciye bildiriliyor. */

int konsoldanTamsayiOku(); /* "konsoldanTamsayiOku" kimlikli bir fonksiyon olduğu ve bu
↳ fonksiyonun bir parametre almadığı ve bir tamsayı geri döndürdüğü derleyiciye bildiriliyor. */

int main() {
    int sayi;
    sayi=konsoldanTamsayiOku();
    sayiyiKonsolaYaz(13);
    sayiyiKonsolaYaz(sayi);
    int toplam= ikiSayiyiTopla (sayi,20);
    //...
    return 0;
}

```

Fonksiyona ilişkin bir bildirim yapıldığında bir yerlerde böyle bir fonksiyon olduğu derleyiciye iletilmiş olur. Bildirim sonrasında artık onu çağırabiliriz. Ancak fonksiyon tanımı yapılmamış ise bağlayıcı (**linker**) program derlemeyi tamamlayamaz.

Program ana fonksiyondan başladığından her programcı ana fonksiyona odaklanır. Bu nedenle **ilk önce fonksiyon bildirimleri yapıp fonksiyon tanımlarının ana fonksiyon sonrasına bırakılması etik bir davranıştır**.

Aşağıda pozitif sayı girilmesini garanti ettikten sonra sayının 10 sayısına bölünebilirliğini bulan program verilmiştir.

```
#include <stdio.h>
```

```

int pozitifSayiOku(); /* parametre almayan bir fonksiyon bildirimi (declaration). Bu fonksiyon
→ geri int değeri döndürüyor. */
int onaBolunurMu(int); /* int veri tipinde parametre alan bir başka fonksiyon bildirimi
→ (declaration): bu fonksiyonda int tipinde veri geri döndürüyor. */
int main () {
    int birPozitifSayi=pozitifSayiOku();
    if (onaBolunurMu(birPozitifSayi))
        printf("Girilen Pozitif Sayı 10 ile bölünebilir.");
    else
        printf("Girilen Pozitif Sayı 10 ile tam bölünmez.");
    return 0;
}
/* Bildirimi yapılan "pozitifSayiOku" fonksiyonunun tanımı (definition): */
int pozitifSayiOku() {
    int okunan;
    do {
        printf("Lütfen pozitif sayı giriniz: ");
        scanf("%d",&okunan);
        if (okunan<=0)
            printf("HATA:Pozitif Sayı GİRMEDİNİZ!\n");
    } while (okunan <= 0);
    return okunan;
}
/* Bildirimi yapılan "onaBolunurMu" fonksiyonunun tanımı (definition): */
int onaBolunurMu(int p){
    return (p%10)==0;
}
/*Program Çıktısı:
Lütfen pozitif sayı giriniz: 30
Girilen Pozitif Sayı 10 ile bölünebilir.
*/

```

Bunların dışında hiçbir değeri geri döndürmeyen fonksiyonlar da tanımlayabiliriz. Bunlara diğer dillerde prosedür (**procedure**) adı da verilir. Bu durumda değeri olmayan veri tipi olan **void** anahtar kelimesi kullanılır. Bu tür fonksiyonların da geri dönüş talimatı yalnızca **return;** şekilde yazılır.

```

#include <stdio.h>
void printMenu(); /* değeri geri döndürmeyen bir fonksiyon bildirimi (declaration) aynı zamanda
→ parametre de almıyor. */
int main() {
    int secim;
    do {
        printMenu();
        scanf("%d",&secim);
        switch(secim) {
            case 1:
                printf("Verileri Sakladım.\n");
                break;
            case 2:
                printf("Verileri Okudum.\n");
                break;
            case 0:
                printf("Programdan Çıkılıyor...\n");
                break;
            default:
                printf("Tekrar Seçim Yapınız!\n");
        }
    } while (secim!=0);
    return 0;
}
/* Bildirimi yapılan "printMenu" fonksiyonunun tanımı (definition): */
void printMenu() {
    printf("\n");
}

```

```

printf("1-Verileri Yaz.\n");
printf("2-Verileri Oku.\n");
printf("0-ÇIKIŞ.\n");
return;
}

```

Bir başlık(**header**) dosyasında bildirimi yapılan bir fonksiyon, C derleyicisi tarafından dahil edileceği kaynak dosyaların tamamında yalnızca bir kez tanımlanabilir. Buna **tek tanım kuralı** (**one definition rule**) adı verilir. Kısaca birden çok dosya için derleme söz konusu olduğunda birden çok bildirim olabilir ama bir adet fonksiyon tanımı olmalıdır. İleride detayı verilecektir.

7.4 Çağrı Kuralı

Çağrı kuralı (**calling convention**), bir fonksiyon çağrısıyla karşılaşıldığında argümanların fonksiyona hangi sıraya göre iletileceğini belirtir. İki olasılık vardır; Birincisi argümanlar soldan başlanarak sağa doğru fonksiyona geçirilir. İkincisi ise C dilinde kullanılır ve **argümanlar sağdan başlanarak sola doğru fonksiyona geçirilir**.

```
fonksiyon(a,b,c,d,e);
```

Yukarıdaki fonksiyon çağrısında argümanların hangi sırada fonksiyona geçirildiğinin bir önemi yoktur. Ancak aşağıdaki verilen kod örneğindeki durumu inceleyelim;

```
int a = 1; printf("%d %d %d", a, ++a, ++a);
```

Bu kodun çıktısı **1 2 3** olarak beklenir ancak çağrı kuralından ötürü çıktı **3 3 1** olur. **Bunun nedeni C dilinin çağrı kuralının sağdan sola olmasıdır**. Bunu şöyle açıklayabiliriz;

1. Fonksiyona sağdan ilk argüman olarak **a** değişkeninin değeri 1 geçirilir.
2. Ardından **++a** ifadesi ile **a** değişkeni 2'ye yükseltilir.
3. Sonra **++a** ifadesi ile **a** değişkeni 3'e yükseltilir.
4. Fonksiyona ikinci argüman olarak 3 geçirilir.
5. Fonksiyona son olarak **a** değişkeninin son değeri olan 3 geçirilir.

7.5 Depolama Sınıfları

Depolama sınıfları (**storage classes**) bir değişkenin ömrünü (**lifetime**), görünürlüğünü (**visibility**), bellek konumunu (**memory location**) ve başlangıç değerini (initial value) belirlemek için kullanılır. Bu özellikler temel olarak bir programın çalışma süresi boyunca belirli bir değişkenin varlığını izlememize yardımcı olan **kapsam** (**scope**), görünürlüğü ve yaşam süresini içerir.

Yapısal programlama ve fonksiyon kavramının bilgisayarlarda kullanılmasıyla birlikte işlemcilerde de değişiklikler olmuştur. Şekil 1.3'de anlatılan işlemciye yeni kaydediciler de eklenmiştir;

- ◊ Verilerle icra edilen kod farklı bellek bölgesinde bulunur. Bu nedenle verileri işaret eden **veri adresleri kaydedicisi** (**data memory register**).
- ◊ **Yığın bellek kaydedicisi** (**stack register**), fonksiyon çağrıları sırasında mevcut işlemci durumu ve yerel değişkenler gibi bazı işlemleri depolamak için kullanılan belleği gösteren (stack pointer) adresi tutan kaydedicidir.
- ◊ **Bayrak kaydedicisi** (**flag register**), işlemcinin durumunu veya sıfır (**zero**) bayrağı, taşıma (**carry**) bayrağı, işaret (**sign**) bayrağı, taşma (**overflow**) bayrağı, eşlik (**parity**) bayrağı, yardımcı taşıma (**auxiliary carry**) bayrağı ve kesme etkinleştirme (**interrupt enable**) bayrağı gibi çeşitli işlemlerin sonucunu tutmak için kullanılan özel bir kayıt türüdür.
- ◊ **Genel amaçla kullanılan kaydediciler** (**general purpose registers**).

Eklenen bu kaydediciler ile derlenen her bir program, dört bellek bölgesinden (**segment**) oluşur;

1. **Kod bellek** ya da kaynak koda ithaf **metin bellek** (**code segment/text segment**) icra edilecek emirleri içeren bellek.
2. Programın işleyeceği verileri tutan **veri belleği** (**data segment**).
3. Fonksiyonları için kullanılan yığın bellek (**stack segment**). Bu bellek bölgesi, son giren veri ilk çıkar LIFO (**last in first out**) mantığıyla çalışır. Çünkü;
 - ◊ **Çağrıdan** (**call**) önce mevcut işlemci durumu ve yerel değişkenler gibi bazı işlemleri depolamak ve
 - ◊ Fonksiyondan dönülünce (**return**) işlemciyi ve çağrı ortamını eski hale getirmek için kullanılır.
4. Programın icrası sırasında dinamik olarak eklenip çıkarılan veriler için hurdalık gibi kullanılan öbek bellek (**heap segment**). İleride dinamik bellek yönetiminde anlatılacaktır.

Depolama sınıfları, tanımlanan değişkenlerin hangi bellek bölgesinde yer alacağını belirler. Dört tür depolama sınıfı vardır. Aşağıda başlıklar halinde verilmiştir.

§7.5.1 auto Depolama Sınıfı

Otomatik depolama sınıfı (**auto storage class**), bir fonksiyon veya blok içinde kimliklendirilmiş tüm yerel değişkenler için varsayılan (**default**) depolama sınıfıdır. Bu nedenle, C dilinde program yazarken **auto** anahtar sözcüğü nadiren kullanılır. Otomatik değişkenlere;

- ◊ **Yalnızca tanımlı olduğu blok içinden erişilebilir.** Yani değişkenin kapsamı, içinde bulunduğu blok ile sınırlıdır.
- ◊ Yığın bellek bölgesinde (**stack segment**) tutulur.

```
#include <stdio.h>
int main() {
    auto int a = 32;
    int b=20;
    /* "int a=32;" talimatı ile eşdeğerdir. a ve b değişkenleri bu fonksiyon bloğu ve iç
    ↪ bloklarda geçerlidir. Yani faaliyet alanları (scope) bu fonksiyon ile sınırlıdır. */
    if (a==32){
        int b=10; /* Ana fonksiyon bloğunda tanımlı b değişkenine artık ulaşılamaz. */
        int c=50;
        /* "auto int b;" talimatı ile eşdeğerdir. if bloğunda ve tanımlanabilecek iç
        ↪ bloklarda geçerlidir. */
        a=16; // Bu blokta a da geçerlidir.
        printf("a:%d, b:%d c:%d\n",a,b,c); // Çıktı: a:16, b:10 c:50
    }
    /* if bloğu dışında sadece ana fonksiyon bloğunda tanımlı değişkenlere ulaşılabilir. */
    printf("a:%d, b:%d\n", a,b); // Çıktı: a:16, b:20
    return 0;
}
```

§7.5.2 static Depolama Sınıfı

Statik depolama sınıfı (**static storage class**) yaygın olarak kullanılan statik değişkenleri (**static variable**) saklamak için kullanılır. Statik değişkenler;

- ◊ **Yalnızca tanımlı olduğu blok içinden erişilebilir.** Yani değişkenin kapsamı, içinde bulunduğu blok ile sınırlıdır.
- ◊ Kodun yazılı olduğu dosyanın başında tanımlı static değişkenlere dosya içerisinde her yerden erişilebilir.
- ◊ **Statik değişkenler, kapsam (scope) dışına çıktıktan sonra bile değerlerini koruma özelliğine sahiptir.**
- ◊ Evrensel (**global**) tanımlanması halinde, programın herhangi bir yerinden erişilebilir.
- ◊ Veri bellek bölgesinde (**data segment**) yer tahsis edilir. Derleme sırasında bu bellek hep sıfırla doldurulduğundan ilk değerler (**initial value**) hep sıfırdır.

Statik olarak tanımlanan değişken (static variable), program başladığında veri bellek bölümünde oluşturulup program bitene kadar aynı bellek bölgesini işgal eden değişkendir.

```
#include <stdio.h>
static int durum=10; /* Bu static değişkene bu kodun bulunduğu dosyada her yerden erişilebilir. */
void func() {
    int sayac = 0;
    durum++;
    for (sayac = 1; sayac < 5; sayac++)
    {
        static int statikOlan = 5; /* Buraya her erişildiğinde son değeri ile işlem
        ↪ görür. */
        int statikOlmayan = 10;
        statikOlan++;
        statikOlmayan++;
        durum++;
        printf("sayac: %d, statikOlan: %d, durum: %d\n", sayac,statikOlan,durum);
        printf("sayac: %d, statikOlmayan: %d, durum: %d\n", sayac, statikOlmayan,durum);
    }
    //Burada "statikOlan" değişkene erişilemez.
}
```

```

        durum++;
        printf("durum: %d\n", durum);
        return;
    }
    int main() {
        func();
        return 0;
    }
/*Program Çıktısı:
sayaç: 1, staticOlan: 6, durum: 12
sayaç: 1, statikOlmayan: 11, durum 12
sayaç: 2, staticOlan: 7, durum: 13
sayaç: 2, statikOlmayan: 11, durum 13
sayaç: 3, staticOlan: 8, durum: 14
sayaç: 3, statikOlmayan: 11, durum 14
sayaç: 4, staticOlan: 9, durum: 15
sayaç: 4, statikOlmayan: 11, durum 15
durum: 16
*/

```

§7.5.3 extern Depolama Sınıfı

Harici depolama sınıfı (**extern storage class**) bize basitçe değişkenin kullanıldığı blokta değil başka bir yerde tanımlandığını söyler. Harici tanımlanan değişkenler;

- ◊ Değerler tanımlandığı yerde değil farklı bir yerde atanır ve üzerinde değişiklik yapılabilir.
- ◊ Bir değişkene **extern** anahtar sözcüğü sıfat olarak verildiğinde evrensel (**global**) değişken olur. Böyle tanımlanan değişkene programın her yerinden erişilebilir.
- ◊ Bu değişkenler de veri bellekte (**data segment**) yer alır. Derleme sırasında bu bellek hep sıfırla doldurulduğundan ilk değerler hep sıfırdır.

Bir harici değişkene **extern** sıfatı verildiğinde **harici değişken** (**extern variable**) adını alır ve bir başka dosyada tanımlanmış olduğu kabul edilir. Buna örnek olarak aşağıdaki iki farklı kaynak kod dosyası gereklidir. Birinci kaynak kod dosyası (**xtrn.c**) aşağıda verilmiştir.

```

//XTRN.C:
int externDegisken; //Diğer dosyalarda harici kullanılacak değişken tanımlandı.
void funcExtern() //Diğer dosyalarda harici kullanılacak fonksiyon tanımlandı.
{
    externDegisken++;
}

```

Bu dosyadaki değişken ve fonksiyonları kullanan bir başka kaynak kod **main.c** aşağıda verilmiştir;

```

//MAIN.C:
#include <stdio.h>
extern void funcExtern(); /* bir başka dosyada tanımlandı. Burada sadece bildirimi yapıldı.
    ↳ Böylece evrensel bir "funcExtern" fonksiyonumuz oldu */
int main()
{
    extern int externDegisken; /* bir başka dosyada tanımlandı. Burada sadece bildirimi
    ↳ yapıldı. Böylece evrensel bir "externDegisken" değişkenine erişim sağlamış olduk */
    funcExtern();
    printf("extern: %d\n", externDegisken);
    externDegisken = 2;
    printf("extern: %d\n", externDegisken);
    return 0;
}

```

Bu iki dosyayı birlikte derlemek için `gcc xtrn.c main.c -o main.exe` komutu ile derleyiciye iki farklı dosya parametre olarak verilir.

§7.5.4 register Depolama Sınıfı

Kaydedici depolama sınıfı (**register storage class**), otomatik değişkenlerle aynı işlevselliğe sahiptir. Tek fark, derleyicinin, eğer boş bir işlemci kaydedicisi (**register**) mevcutsa değişken olarak o kaydedicinin kullanılmasıdır. Genel amaçlı kaydediciler bu amaçla kullanılır. Bu da bu değişkenlerin, bellekte saklanan değişken-

lerden çok daha hızlı olmasını sağlar. Bu değişkenler `register` anahtar sözcüğüyle tanımlanırlar;

- ◊ Eğer boş bir CPU kaydedicisi mevcut değilse, değişken yalnızca bellekte saklanır.
- ◊ Sadece bloklar içinde tanımlanan yerel (**local**) değişkenler **register** sıfatı ile tanımlanırlar. Evrensel (**global**) değişkenler bu anahtar sözcük ile kullanılamaz.

Aşağıda kaydedici bu depolama sınıfına ilişkin örnek verilmiştir. Buradaki sayma ve toplama işlemleri uygun olan işlemci kaydedicilerinde yapılır.

```
#include <stdio.h>
int main() {
    register int k, sum;
    //döngü bloğu olmayan for talimatı örneği sonunda noktalı virgül var!
    for(k = 1, sum = 0; k < 1000; sum += k, k++);
    printf("%d\n",sum); //Çıktı:499500
}
```

Öncelikle C standardı tek bir değişkenle birden fazla depolama sınıfının kullanılmasını yasaklar.
Aynı değişkene hem **static** hem **extern** niteleyicisi kullanılamaz.

Depolama sınıfları aşağıdaki tabloda özetlenmiştir; Tablodaki **çöp** (**garbage**) kavramı tahsis edilen bellek içeriği o anda ne ise değişkenin o değeri alması anlamındadır.

Depolama sıfatı	Bellek bölgesi	Başlangıç Değeri	Değişken Kap- samı (scope)	Değişken Ömrü (lifetime)
auto	Yığın bellek (stack segment)	Çöp	Bulunduğu blok	Bulunduğu blok süresince
static	Veri bellek (data segment)	Sıfır	Bulunduğu Blok veya Bulunduğu Dosya	Program süresi
extern	Veri bellek (data segment)	Sıfır	Evrensel (global)	Program süresi
register	İşlemci Kaydedicileri	Çöp	Bulunduğu blok	Bulunduğu blok süresince

Tablo 7.3: Depolama Sınıfları

7.6 Evrensel ve Yerel Değişken

C dilinde bir değişkene her yerden erişilmek istenilen değişkenlere **evrensel değişken** (**global variable**) olarak tanımlanırlar. Bu değişkenler herhangi bir kod bloğu içinde tanımlanmazlar. Genel olarak kaynak kodun başında tanımlanırlar. Sonra kodun her yerinde kullanılabilirler. Eğer kodumuz birden çok kaynak kod dosyasından oluşacak ise diğer tüm dosyalarda bu değişken **extern** olarak tanımlanırlar.

Blok içinde tanımlanmış tüm değişkenler, o blok içinden erişilebilen **yerel değişkenlerdir** (**local variable**). Yerel değişkenler otomatik depolama sınıfında (**auto storage class**) yer alır ve hepsi yığın bellekte (**stack segment**) yer alır.

```
1 #include <stdio.h>
2 int evrenselDegisken=10; /* evrenselDeğişken bu noktadan sonra her yerde kullanılabilir. */
3 void fonksiyon();
4 int main() {
5     int yerelDegisken=20; /* Bu yerel değişken sadece main bloğu içinde kullanılabilir. */
6     printf("evrensel:%d, yerel:%d\n",evrenselDegisken,yerelDegisken);
7     evrenselDegisken++; // evrensel değişken değiştiriliyor.
8     yerelDegisken--; // yerel değişken değiştiriliyor.
9     printf("evrensel:%d, yerel:%d\n",evrenselDegisken,yerelDegisken);
10    fonksiyon();
11    return 0;
12 }
13 void fonksiyon() {
14     int fonksiyonYerelDegiskeni=30;
15     evrenselDegisken++;
16     printf("evrensel:%d, fonksiyonYerel:%d\n",evrenselDegisken,fonksiyonYerelDegiskeni);
17     if (fonksiyonYerelDegiskeni==30) {
```



```

18         int evrenselDegisken=100; /* Artık 2. satırdaki evrensel değişkene bu blokta
           ↳ ulaşamaz. */
19         printf("evrensel:%d, fonsiyonYerel:%d\n",
           ↳ evrenselDegisken,fonsiyonYerelDegiskeni);
20     }
21 }
22 /*Program Çıktısı:
23 evrensel:10, yerel:20
24 evrensel:11, yerel:19
25 evrensel:12, fonsiyonYerel:30
26 evrensel:100, fonsiyonYerel:30
27 */

```

Burada fonksiyon içinde `if` bloğunda 18. satırda yer alan değişken her ne kadar `evrenselDeğişken` olarak kimliklendirilse de yerel değişkendir ve sadece bu `if` bloğu içinde geçerlidir. Bu yerel değişken 2. satırdaki değişkeni gölgelemiştir. Yani tanımlamada mantıksal hata (**logical error**) yapılmıştır.

7.7 Değerle Çağırma

Yerel değişkenlerin yığın bellekte tutulduğu bir üst başlıkta anlatılmıştı. Fonksiyon blokları içindeki değişkenler de yerel değişkenlerdir (**local variable**). Bir fonksiyon çağrıldığında değişkenlerin bellekteki durumu ve değişimi aşağıdaki programdan anlaşılabilir;

```

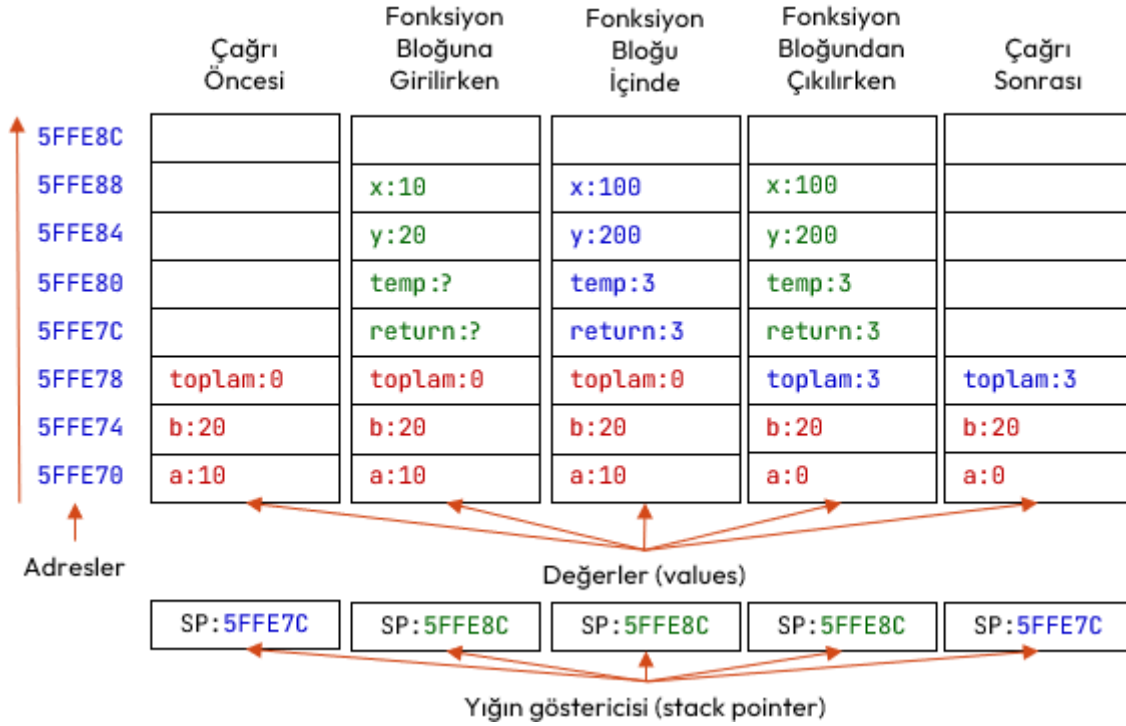
#include <stdio.h>
int toplam(int, int); /* İki tamsayıyı toplayan fonksiyon bildirimi/prototipi */
int main () {
    /* main() fonksiyonu içinde geçerli yerel (local) değişkenler*/
    int a = 10;
    int b = 20;
    int toplam = 0;
    printf("çağrı öncesi a: %d, b:%d, toplam:%d\n",a,b,toplam);
    toplam = toplama(a, b);
    printf("çağrı sonrası a: %d, b:%d, toplam:%d\n",a,b,toplam);
}
int toplama(int x, int y) { /* iki tamsayıyı toplayan fonksiyon tanımı*/
    /* Topla fonksiyonu yerel değişkenler */
    int temp= x+y;
    x=100; y=200;
    /* Burada değiştirilenler parametre değişkenleridir. main() içindeki yerel değişkenler
       ↳ değildir. */
    printf("çağrı içinde x: %d, y:%d\n",x,y);
    return temp;
}
/* Program çalıştırıldığında:
çağrı öncesi a:10, b:20 toplam:0
çağrı içinde x:100, y:200
çağrı sonrası a:10, b:20 toplam:30
*/

```

Yukarıda örneği verilen programda `toplama` fonksiyonunun çağırma sürecindeki yığın bellek bölgesindeki (**stack segment**) değişimi Şekil 7.3'de gösterilmektedir.

Yukarıdaki kod şu şekilde açıklanabilir;

- ♦ İcra sırası `toplama(a,b)` fonksiyonuna geldiğinde fonksiyon çağrılmadan önce `main()` fonksiyonu yerel değişkenleri olan `a`, `b` ve `toplam` değişkenleri yığın belleğe (**stack segment**) itilmiştir (**push**).
- ♦ `toplama(a,b)` fonksiyonu çağrıldığı anda `a` ve `b` değişkenlerinin değerleri, `x` ve `y` parametrelerine kopyalanarak fonksiyona geçirilir. Değerler parametrelere kopyalandığından buna **değerle çağırma** (**call by value**) adı verilir.
- ♦ `toplama(a,b)` fonksiyonu icra edilirken fonksiyon yerelindeki `temp` ve parametre değişkenleri `x`, ve `y` istenildiği gibi değiştirilebilir. Geri dönüş değeri de belirlenir.
- ♦ `toplama(a,b)` fonksiyonu içine bir başka fonksiyon da çağrılabilirdi. Böyle bir durumda `x`, `y` ve `temp` değerleri yığın belleğe itilir ve çağrılan fonksiyon icra edilirdi. Böylece iç içe çağrılan her fonksiyonda yığın bellek büyüyeceği görülmektedir. Program derlendiğinde yığın bellek (**stack segment**) için sınırlı bir bellek bölgesi ayrılır. İç içe çok fonksiyon çağrıldığında bu bellek sınırı aşılabılır. Bu durumda



Şekil 7.3: Fonksiyon Çağırma Sürecinde Yığın Bellek

program **yığın taşması** (**stack overflow**) hatası verecektir.

- ◊ `toplam=topla(a,b)` fonksiyon bloğundan çıkılırken geri dönüş değeri çağrı ortamındaki `toplam` değişkine kopyalanır ve fonksiyon için yığın belleğe itilen değerler geri çekilir (**pop**).

7.8 Özyinelemeli Fonksiyonlar

Özyineleme (**recursion**), bir fonksiyonun kendi çağrısını yapma tekniğidir. Bu teknik, karmaşık sorunları çözülmesi daha kolay basit sorunlara ayırmanın bir yolunu sağlar. Özyineleme, yineleme (**iteration**), döngüler (**loop**) veya tekrar yerine geçebilecek çok güçlü bir tekniktir. Özyinelemeli algoritmalarda, tekrarlar fonksiyonun kendi kendisini kopyalayarak çağırması ile elde edilir. Bu kopyalar işlerini bitirdikçe kaybolur.

Bir problemi özyineleme ile çözmek için problem iki ana parçaya ayrılır;

1. Cevabı kesin olarak bildiğimiz temel durum (**base case**)
2. Cevabı bilinmeyen ancak cevabı yine problemin kendisi kullanılarak bulunabilecek durum.

Faktöriyel örneğini inceleyelim; Matematikte, negatif olmayan bir tam sayı olan n sayısının faktöriyeli, $n!$ ile gösterilir ve n sayısına eşit ve küçük olan tüm pozitif tam sayıların çarpımıdır.

$$n! = n \times (n-1) \times (n-2) \times \dots \times 3 \times 2 \times 1$$

Sıfır sayısı da pozitif bir sayıdır ama sonucun sıfır çıkmaması için **0 sayısının faktöriyeli 1 kabul edilir**. Bu aslında faktöriyel problemi için **temel durumdur** (**base case**) ve **boş ürün** (**empty product**) kuralı olarak bilinir. Aşağıda faktöriyel için girilen sayıdan sonra tüm sayıların çarpımıyla hesaplayan bir fonksiyon yazılmıştır.

```
#include <stdio.h>
unsigned int faktoriyel(unsigned int n)
{
    int sonuc=1, indis;
    for (indis=n; indis>0; indis--)
        sonuc=sonuc*indis;
    return sonuc;
}
int main() {
    unsigned int sayi, sonuc; //Pozitif tamsayı yanımı yapılmış değişkenler.
    printf("Pozitif Bir Sayı Giriniz:");
    scanf("%d", &sayi);
    sonuc=faktoriyel(sayi);
```

```

    printf("%d!=%d\n",sayi,sonuc);
    return 0;
}

```

Aslında negatif olmayan bir tam sayı olan sayının faktöriyeli, kendisi ile kendisinden bir küçük olan sayının faktöriyeli ile çarpımı şeklinde yazılabilir. Yani problemin tanımı yine kendisini içerir:

$$n! = n \times (n - 1)!$$

Bu durumda faktöriyel fonksiyonu aşağıdaki şekilde **özyinelemeli (recursive)** tanımlanabilir;

$$n! = \begin{cases} 1 & \text{eğer } n = 0 \\ n \cdot (n - 1)! & \text{eğer } n > 0 \end{cases}$$

Aynı faktöriyel fonksiyonu **parametrelili olarak** aşağıdaki şekilde yine özyinelemeli olarak tanımlanabilir;

$$f(n) = \begin{cases} 1 & \text{eğer } n = 0 \\ n \cdot f(n - 1) & \text{eğer } n > 0 \end{cases}$$

Buna örnek olarak aşağıdaki hesabı verebiliriz;

$$5! = 5 \times 4! = 5 \times 4 \times 3! = 5 \times 4 \times 3 \times 2! = 5 \times 4 \times 3 \times 2 \times 1! = 5 \times 4 \times 3 \times 2 \times 1 \times 0! = 120$$

Özyinelemeli (**recursive**) olarak ise faktöriyel fonksiyonu aşağıdaki şekilde kodlanabilir;

```

#include <stdio.h>
unsigned int faktoriyel(unsigned int n)
{
    if (n==0)
        return 1;
    else
        return n*faktoriyel(n-1); //Kendi kendini çağırma!
}
int main() {
    unsigned int sayi,sonuc; //Pozitif tamsayı yanımı yapılmış değişkenler.
    printf("Pozitif Bir Sayı Giriniz:");
    scanf("%d",&sayi);
    sonuc=faktoriyel(sayi);
    printf("%d!=%d\n",sayi,sonuc);
    return 0;
}

```

Bir başka örnek olarak, girilen taban ve üs ile kuvvet hesaplama fonksiyonu verilebilir. Bu fonksiyonun tanımı aşağıdaki şekilde yazılabilir;

$$f(\text{taban}, us) = \begin{cases} 0 & \text{eğer } \text{taban} = 0 \\ 1 & \text{eğer } us = 0 \\ \text{taban} \cdot f(\text{taban}, us - 1) & \text{eğer } \text{taban} > 0 \text{ ve } us > 0 \end{cases}$$

Buna ilişkin kod aşağıda verilmiştir;

```

#include <stdio.h>
int kuvvet(int taban, int us) {
    if (taban == 0)
        return 0; //Taban 0 ise 0 dön
    if (us == 0)
        return 1; // Üs 0 ise 1 dön
    return taban*kuvvet(taban, us - 1); //Kendi kendini çağırma!
}
int main(){
    int taban,us;
    printf("Tabanı Giriniz:");
    scanf("%d",&taban);
    printf("Üssü Giriniz:");
    scanf("%d",&us);
    printf("%d üzeri %d = %d", taban,us, kuvvet(taban,us) );
    return 0;
}
/*

```

```

Tabanı Giriniz:3
Üssü Giriniz:6
3 üzeri 6 = 729
*/

```

7.9 Satır İçi Fonksiyonlar

Fonksiyon Çağırma Süreci başlığında anlatıldığı üzere fonksiyonlar çağrılırken sürekli yığın belleğe (**stack segment**) it-çek (**push-pop**) yapılır. Bazı durumlarda bu işlem performans gereği istenmez. Bu durumda fonksiyonlar, **satır içi fonksiyon** (**inline function**) olarak tanımlanır. 2017 C sürümü ile gelen bu özellik, fonksiyona **inline** sıfatı (**inline specifier**) eklenerek kullanılır.

```

#include <stdio.h>
inline int topla (int op1, int op2) {
    int toplam= op1+op2;
    return toplam;
}
int main() {
    int x=10, y=15, sonuc;
    sonuc = topla(x,y);
    printf("%d+%d=%d", x,y, sonuc);
    return 0;
}

```

Bu kod aşağıdaki şekle çevrilmiş gibi derlenir;

```

#include <stdio.h>
int main() {
    int x=10, y=15, sonuc;
    {
        int toplam= op1+op2;
        sonuc = toplam;
    }
    printf("%d+%d=%d", x,y, sonuc);
    return 0;
}

```

Bir fonksiyon aşağıda belirtilen durumlarda satır içi fonksiyon olarak derlenmez;

1. Bir döngü içeriyorsa!
2. Statik değişkenler içeriyorsa!
3. Özyinelemeli ise!
4. Dönüş türü **void** haricinde olup **return** ifadesi işlev gövdesinde mevcut değilse!
5. **switch** veya **goto** talimatı içeriyorsa!

Bu tip fonksiyon tanımlamadaki amacımız, **fonksiyon çağırma yükünün azaltılması için işlemci kaydedicilerinin daha çok kullanılmasıdır. Bu da icra hızını yükseltir.**

7.10 Düşün ve Çöz

- ◊ **Düşün:** **static** bir yerel değişken ile **auto** bir yerel değişkenin bellek ömrü ve kapsamı arasındaki farkları karşılaştırınız.
- ◊ **Düşün:** Fonksiyonlara parametre aktarırken değerle çağırma (**call by value**) yönteminin yığın bellek kullanımına etkisini açıklayınız.
- ◊ **Düşün:** Özyinelemeli fonksiyonlarda temel durum (**base case**) unutulursa bellek seviyesinde ne gerçekleşir? Yığın taşması (**stack overflow**) nedir?
- ◊ **Çöz:** Bir tam sayının basamak değerleri toplamını özyinelemeli olarak hesaplayan fonksiyonu yazınız.
- ◊ **Çöz:** **extern** anahtar kelimesini kullanarak, iki farklı kaynak dosyasında (.c) ortak kullanılan bir evrensel değişken örneği kurgulayınız.

8. Bölüm

Diziler

8.1 Dizi Nedir

Şu ana kadar en basit **veri yapıları** (**data structure**) olan değişkenlerle karşılaştık. Programlamada kullanılan bir başka veri yapısı da **dizilerdir** (**array**). Matematikte diziler; bir sıralı elemanlardan oluşan bir listedir. Sıralı elemanların sayısına **dizinin uzunluğu** (**array size**) denir. Doğal olarak eleman sayısı tam sayıdır (**integer**). Elemanların sırasını tam sayı (**integer**) olan **indis** (**index**) belirtir. Kümelerin (**set**) aksine, aynı elemanlar dizide farklı konumlarda bulunabilir. Elemanlara terim adı da verilir. Örnekler;

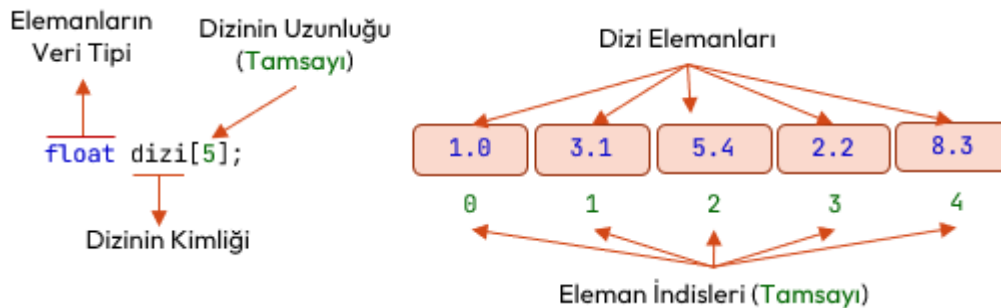
- Örnek olarak elemanları 1'den 10'a kadar tam sayıların karesinden oluşan bir dizi, {1,4,9,...,100} olarak gösterilir.
- Bir başka örnek olarak {K,İ,T,A,P} dizisi verilebilir. Bu dizi ile {P,A,T,İ,K} dizisi birbirinden farklıdır. Aynı elemanlardan oluşmasına rağmen elemanların sıraları yani indisleri farklıdır.
- {1,3,5,1,2,1,7} dizisinde birden farklı sırada yani indiste aynı 1 elemanı vardır. Bu dizinin geçersiz olduğu anlamına gelmez.

Matematikte sonsuz elemanı olan sonsuz diziler tanımlanabilir. Ama programlamada hep sonlu elemanlı diziler kullanılır. Yani **uzunluğu** (**size**) tanımlanırken belirlenir.

8.2 Tek boyutlu Diziler

C Dilinde sonlu elemanlı diziler Şekil 8.1'deki gibi tanımlanır;

- Diziler, benzer veri tipindeki** (**data type**) **öğeleri barındırır ve bu öğeler bitişik bellek bölgesini paylaşırlar.**
- Dizi elemanları, ilkel veri tipleri (**char, short, int, long, float, double**) veya daha sonra göreceğimiz kullanıcı tanımlı tip olan yapı **struct** veya **gösterici** (**pointer**) olabilir.
- Aynı değer birkaç farklı yerde dizi içinde bulunabilir!
- Dizinin veri tipi, dizideki elemanlarının veri tipini belirler.** Bir başka deyişle elemanların veri tipi dizinin veri tipiyle aynı olmalıdır.
- Dizinin uzunluğu** (**size**), **tamsayı değişmez** (**integer literal**) olup dizi kimliklendirmesi sırasında **belirtilmelidir.** Derleyici buna göre bellekte yer ayırır. Dolayısıyla dizi, bir kez kimliklendirildiğinde dizinin uzunluğu değiştirilemez.
- Dizi uzunluğu, sabit olmayan bir değişkenle belirlenemez!**
- Elemanların indisi** (**index**) **sıra belirttiğinden her zaman tamsayıdır.**
- İlk elemanın indisi 0, son elemanın indisi dizi uzunluğundan** (**size**) **bir küçük olan tamsayıdır.** Bunların dışında bir indis girildiğinde **sınır dışı** (**out-of-bounds**) hatası yaşanır.



Şekil 8.1: Tek Boyutlu Dizi Tanımlama

C dilinde beş tamsayı elemanı olan bir dizi aşağıdaki şekilde tanımlanır;

```
int tamsayiDizisi[5];
```

Benzer şekilde beş gerçekte sayı elemanı olan bir dizi ise aşağıdaki şekilde tanımlanır;

```
float reelsayiDizisi[5];
```

Dizinin elemanlarına tanımlama sırasında **başlangıç değeri** (**initial value**) süslü parantezler arasında verilebilir yani dizi bazı değerlerle **başlatılabilir** (**initialize**). Bunun için süslü parantez kullanılır;

```
int tamsayiDizisi1[5]= {1, 2, 3, 4, 5}; //Elemanları 1,2,3,4,5 olacak şekilde başlatıldı.
int tamsayiDizisi2[5]= {0}; //Elemanların hepsi 0 olacak şekilde başlatılıyor.
```

Dizinin elemanlarına hepsine başlangıç değeri verilmeyebilir;

```
float a[5] = {1.0, 2.0, [4] = 12.0}; // 1.Eleman 1.0, 2.Elaman 2.0 ve 5.eleman 12.0
```

Dizinin elemanlarına ayrı ayrı erişilip değerleri değiştirilebilir ya da yeni değer ataması yapılabilir;

```
int a[5]={0}; // a dizisi, tüm elemanları 0 olacak şekilde başlatılıyor
a[0]=1; //Birinci elemana 1 değeri atandı
a[1]++; //İkinci elemanın değeri artırıldı
a[2]--; //Üçüncü elemanın değeri azaltıldı
a[4]=12; //Beşinci elemana 12 değeri atandı.
```

Diziler belleği verimli kullanan ve tek bir değişken ile elemanlara erişim sağlayan bir çözüm sunar. Bir dizideki elemanlar, bellekte bitişik konumda yer aldığından herhangi bir öğeye kolaylıkla erişebiliriz. Dizilerin başlıca üstünlükleri şunlardır:

- ◊ İndisleri kullanarak dizi öğelere rastgele ve hızlı erişim sağlanır. Her öğenin bir indisi (**index**) olduğundan doğrudan erişilebilir ve değiştirilebilir.
- ◊ Birden fazla öğeden oluşan tek bir dizi oluşturduğundan daha az kod satırı yazılır.
- ◊ Daha az kod satırı yazılarak sıralama yapılabilir.

Dizi kullanmanın zayıf yönleri ise;

- ◊ Kimliklendirme sırasında dizi uzunluğuna bir tamsayı değişmez (**integer literal**) ile karar verildiğinden belli sayıda elemanın üzerinde işlem yapılır.
- ◊ Dizi dinamik değildir araya eleman ekleme veya çıkarma yapılamaz.

Örnek olarak klavyeden girilen 5 adet tamsayıyı, giriş sırasının tersinden ekrana yazan C programını verebiliriz;

```
#include<stdio.h>
int main() {
    int dizi[5]; /* 5 elemanı tamsayı olan bir dizi tanımlandı*/
    int indis; /* dizi elemanlarını gezecek ve indis olarak kullanılacak
    tamsayı bir değişken tanımlandı */
    for (indis =0; indis <5; indis++){
        printf("%d. sayiyi girin:", indis);
        scanf("%d",&dizi[ indis ]); //indis ile belirtilen elemana klavyeden olunan değer
        ↳ atanıyor
    }
    printf("Tersten Sayılar:\n");
    for (indis=4; indis >=0; indis --)
        printf("%d\n", dizi[ indis ]);
    return 0;
}
/*Program Çıktısı:
0. sayiyi girin:10
1. sayiyi girin:20
2. sayiyi girin:40
3. sayiyi girin:50
4. sayiyi girin:35
Tersten Sayılar:
dizi[4]=35
dizi[3]=50
dizi[2]=40
dizi[1]=20
dizi[0]=10
*/
```

Bir başka örnek; Klavyeden girilen 10 adet sınav notuna göre, ortalamasının üstünde olan notları ekrana yazan C programı;

```
#include<stdio.h>
#define BOYUT 10
int main() {
    unsigned notlar[BOYUT]; /* Her terimi pozitif olan 10 elemanlı dizi */
```

```

float ortalama, toplam=0;
int indis; //indis için tamsayı değişken
for (indis=0; indis<BOYUT; indis++){
    printf("%d. notu girin:",indis);
    scanf("%d",&notlar[indis]);
    toplam+=notlar[indis]; // Her eleman girildiğinde toplanıyor
}
ortalama=toplam/BOYUT;
printf("Ortalamanın (%.2f) Üzerindeki Notlar:\n",ortalama);
for (indis=0; indis<BOYUT; indis++)
if (notlar[indis]>ortalama)
printf("%d,",notlar[indis]);
return 0;
}
/*Program Çıktısı:
0. notu girin:45
1. notu girin:65
2. notu girin:55
3. notu girin:50
4. notu girin:30
5. notu girin:75
6. notu girin:85
7. notu girin:90
8. notu girin:70
9. notu girin:100
Ortalamanın (66.50) Üzerindeki Notlar:
75,85,90,70,100,
*/

```

Bir başka örnek olarak 10 adet satış miktarlarının, ortalamaya olan uzaklıkları yani sapma (**deviation**) ve karesi alınmış sapmalar yani varyans (**variance**) ile standart sapmayı yazdıran aşağıdaki C programı verilebilir. Ortalamaya olan uzaklıklar yani sapma, verilerin ortalamaya ne kadar yakın olduğunu gösteren dağılımdır. Sapmaların toplamı sıfır olabileceğinden karelerinin toplamı yani varyans hesaplanır. Varyansın eleman sayısına bağlı karekökü de standart sapmayı verir. Standart sapmanın büyük olması verilerin ortalamadan daha uzak yayıldıklarını; küçük bir standart sapma ise verilerin ortalama etrafında daha yakın gruplaştıklarını gösterir.

```

#include<stdio.h>
#include<math.h>
#define BOYUT 10
int main() {
    float satislar[BOYUT]={100.0,850.0,500.0,600.0,750.0,650.0,450.0,800.0,900.0,110.0};
    float ortalama, toplam=0, varyansToplam=0, standartSapma;
    int indis;
    for (indis=0; indis<BOYUT; indis++)
        toplam+= satislar[indis];
    ortalama=toplam/BOYUT;
    printf("Ortalama: %.2f\n",ortalama);
    printf("Sapma\tVaryans: \n");
    for (indis=0; indis<BOYUT; indis++) {
        float sapma= satislar[indis]-ortalama;
        varyansToplam+=sapma*sapma;
        printf("Sapma: %.2f\tVaryans: %.2f\n", sapma,sapma*sapma);
    }
    standartSapma=sqrt(varyansToplam/BOYUT);
    printf("Standart Sapma: %.2f\n",standartSapma);
    return 0;
}
/*Program Çıktısı:
Ortalama:571.00
Sapma   Varyans:
Sapma: -471.00   Varyans: 221841.00
Sapma: 279.00   Varyans: 77841.00

```

```

Sapma: -71.00   Varyans: 5041.00
Sapma: 29.00   Varyans: 841.00
Sapma: 179.00  Varyans: 32041.00
Sapma: 79.00   Varyans: 6241.00
Sapma: -121.00 Varyans: 14641.00
Sapma: 229.00  Varyans: 52441.00
Sapma: 329.00  Varyans: 108241.00
Sapma: -461.00 Varyans: 212521.00
Standart Sapma:270.50
*/

```

Bir başka örnek olarak 5 elemanlı diziye girilen elemanlardan tekil (**unique**) olanlarını bulan program verilebilir.

```

#include <stdio.h>
#define BOYUT 5
int main()
{
    int dizi[BOYUT];
    int i;
    for (i=0; i<BOYUT; i++){
        printf("%d. sayıyı girin:",i);
        scanf("%d",&dizi[i]);
    }
    printf("\nDizi içinde tekil (unique) olan rakamlar: \n");
    for (i=0; i<BOYUT; i++)
    {
        int j, counter = 0; // Her elemanda sayacı sıfırla
        for (j = 0; j < BOYUT; j++)
            if (i != j) //elemanın kendisini kontrol etmiyoruz
                if (dizi[i] == dizi[j])
                    counter++;
        if (counter == 0)
            printf("%d tekil olarak dizide bulundu.\n", dizi[i]);
    }
    return 0;
}
/*Program Çıktısı:
0. sayıyı girin:12
1. sayıyı girin:15
2. sayıyı girin:12
3. sayıyı girin:3
4. sayıyı girin:15

Dizi içinde tekil (unique) olan rakamlar:
3 tekil olarak dizide bulundu.
*/

```

Bir başka örnek olarak 5 elemanlı diziye girilen en büyük elemanına en küçük elemanını ekleyen program verilebilir;

```

#include <stdio.h>
#define BOYUT 5
int main(){
    int dizi[BOYUT]={10,12,3,4,6};
    int enBuyuk, enKucuk, enKucukIndex, enBuyukIndex, i ;
    printf("ÖNCE:"); //Dizinin İlk hali konsola yazdırılıyor.
    for (i = 0; i < BOYUT; i++)
        printf("%02d ",dizi[i]);
    printf("\n");
    enKucuk=dizi[0]; enBuyuk=dizi[0]; /* İlk elemanlar hem en küçük
    hem de en büyük olsun. */
    enKucukIndex=0,enBuyukIndex=0; /* En küçük ve en büyük elemanların
    indisi 0 olsun. */

```



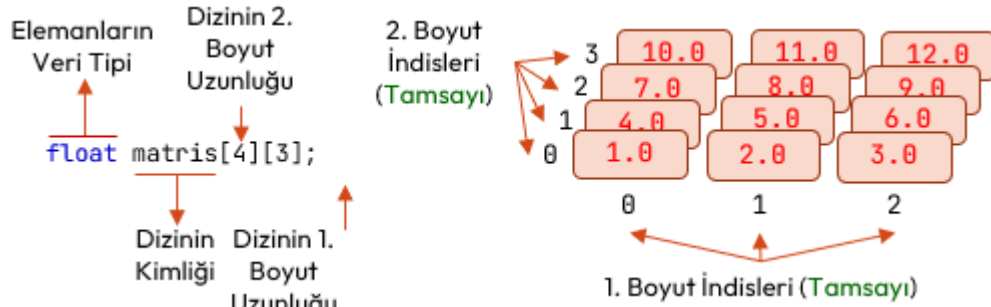
```

for (i=0; i<BOYUT; i++){ /* En küçük ve en büyük eleman ile
    indislerini bulan kısım: */
    if (dizi[i]<enKucuk) {
        enKucuk=dizi[i];
        enKucukIndex=i;
    }
    if (dizi[i]>enBuyuk) {
        enBuyuk=dizi[i];
        enBuyukIndex=i;
    }
}
printf("En Büyük: dizi[%d]=%02d, En Küçük: dizi[%d]=%02d\n",
    enBuyukIndex,dizi[enBuyukIndex],
    enKucukIndex,dizi[enKucukIndex]);
dizi[enBuyukIndex]+=dizi[enKucukIndex]; /* En büyük elemana
en küçüğünü ekleme: */
printf("SONRA:"); //Dizinin son hali konsola yazdırılıyor.
for (i = 0; i < BOYUT; i++)
    printf("%02d ",dizi[i]);
return 0;
}
/*Program Çıktısı:
ÖNCE:10 12 03 04 06
En Büyük: dizi[1]=12, En Küçük: dizi[2]=03
SONRA:10 15 03 04 06
*/

```

8.3 İki Boyutlu Diziler

Şu ana kadar gördüğümüz tek boyutlu dizilerdi. **İki boyutlu diziler** (**two-dimentional array**) matematikten de bildiğimiz üzere matris (**matrix**) olarak adlandırılır. İki boyutlu dediğimizde en-boy ve satır-sütun gibi kavramlar akla gelmelidir. Matris işlemleri gibi bazı problemlerde; bir dizinin her bir elemanının da dizi olması istenir. Bu tür iki boyutlu dizilerde en içteki dizinin boyutu kimliklendirmede sağda yer alır.



Şekil 8.2: İki Boyutlu Dizi Tanımlama

Örneği verilen iki boyutlu dizinin ikinci boyutundan bakacak olursak her biri ayrı bir tek boyutlu dizidir. Bellekte ikici boyutu oluşturan bu diziler de sırasıyla birbirine bitişik (**contiguous**) olarak yer alır. Aslında `float dizi[3][4];` ile `float dizi[12];` aynı bellek yerleşimine sahiptir. Aşağıda iki boyutlu matris tanımlamalarına örnek verilmiştir;

```

float matris[2][3]; //Her bir elemanı 3 elemanlı bir dizi olan 2 satır üç sütunlu bir matris
//Aşağıda 2x3 matrisi başlatma:
float matris2[2][3]= {
    {1.0,2.0,3.0}, //Birinci Satır: 3 Elemanlı bir dizi ile başlatılıyor
    {2.0,4.0,6.0} //İkinci Satır: 3 Elemanlı bir başka dizi ile başlatılıyor.
};
int karematris[2][2];
//2x2 matrise ilk değer verme:
int karematris2[2][2]= {
    {1,2}, //Birinci Satır: 2 elemanlı bir dizi
    {5,6} //Birinci Satır: 2 elemanlı bir dizi
};

```

```
int karematris3[3][3];
```

Örnek olarak 3x4'lük matris elemanlarını klavyeden girip, tablo halinde ekrana yazdıran programı verebiliriz;

```
#include<stdio.h>
#define SATIR 3
#define SUTUN 4
int main() {
    int matris[SATIR][SUTUN];
    int i,j;
    /*
    //İç içe döngü kullanmasaydık:
    for (j=0; j<SUTUN; j++) //Birinci Satır Okuyalım
        scanf("%d",&matris[0][j]);
    for (j=0; j<SUTUN; j++) //İkinci Satır Okuyalım
        scanf("%d",&matris[1][j]);
    for (j=0; j<SUTUN; j++) //Üçüncü Satır Okuyalım
        scanf("%d",&matris[2][j]);
    //Bunun yerine aşağıdaki gibi iç içe iki for tanımlanır;
    */
    for (i=0; i<SATIR; i++) //İkinci Boyut İçin
        for (j=0; j<SUTUN; j++) //Birinci Boyut İçin
            scanf("%d",&matris[i][j]);
    printf("TABLO\n");
    for (i=0; i<SATIR; i++) { //İkinci Boyut İçin
        for (j=0; j<SUTUN; j++) //Birinci Boyut İçin
            printf("%d\t",matris[i][j]);
        printf("\n");
    }
    return 0;
}
/*Program Çıktısı:
3 4 5 6 7 8 4 4 5 8 7 9
TABLO
3      4      5      6
7      8      4      4
5      8      7      9
*/
```

Bir başka örnek olarak elemanları klavyeden girilen 3x2'lik matrisin satır ve sütun toplamalarını ekrana yazan programı verilebilir;

```
#include<stdio.h>
#define SATIR 3
#define SUTUN 2
int main() {
    int matris[SATIR][SUTUN]; //Matris Tanımı
    int i,j; //i sutun için, j satır için tanımlandı
    for (i=0; i<SATIR; i++) { //İkinci Boyut (SATIR) İçin
        printf("%d. satır:",i);
        for (j=0; j<SUTUN; j++) //Birinci Boyut (SUTUN) İçin
            scanf("%d",&matris[i][j]);
    }
    int satirToplamlar[SATIR]={0}; //Bütün elemanları 0 olan dizi
    for (i=0; i<SATIR; i++) { // İkinci Boyut İçin
        for (j=0; j<SUTUN; j++) // Birinci Boyut İçin
            satirToplamlar[i]+=matris[i][j];
        /* İçteki for bitince tüm satırdaki elemanlar toplanmış oldu */
    } /* Dışdaki for ile de her bir satır için toplam tekrarlanıyor */
    printf("\nSatır Toplamları:");
    for (i=0; i<SATIR; i++)
        printf("%d\t",satirToplamlar[i]);
    int sutunToplamlar[SUTUN]={0}; //Bütün elemanları 0 olan dizi
```

```

    for (j=0; j<SUTUN; j++) // Birinci Boyut İçin
        for (i=0; i<SATIR; i++) { // İkinci Boyut İçin
            sutunToplamlar[j]+=matris[i][j];
            /* İçdeki for bitince tüm sutundaki elemanlar toplanmış oldu*/
        }/* Dışdaki for ile de her bir sutun için toplam tekrarlanıyor */
    printf("\nSütun Toplamları:");
    for (j=0; j<SUTUN; j++)
        printf("%d\t",sutunToplamlar[j]);
    return 0;
}
/*Program Çıktısı:
0. satir:12 18
1. satir:22 28
2. satir:2 8

Satır Toplamları:30      50      10
Sütun Toplamları:36      54
*/

```

8.4 Çok Boyutlu Diziler

Şu ana kadar gördüğümüz tek boyutlu diziler veya iki boyutlu diziler olan matrislerdir. Çok boyutlu dizilere örnek olarak En-Boy-Yükseklik içeren 3 boyutlu diziler verilebilir. Üç boyutlu dizilerde dizinin her bir elemanı bir matristir. Çok boyutlu dizilerde de en içteki dizinin boyutu kimliklendirmede sağda yer alır. Aşağıdaki örnekte; çeşitli boyutlarda diziler tanımlanmıştır;

```

int ucbuyutludizi1[3][2][2]; // ucbuyutludizi1: elemanları 3 adet 2x2 matris olan üç boyutlu bir
→ dizi
int ucbuyutludizi1[2][3][3]; // ucbuyutludizi2: elemanları 2 adet 3x3 matris olan üç boyutlu bir
→ dizi
int ucbuyutludizi1[4][3][2]; // ucbuyutludizi3: elemanları 4 adet 3x2 matris olan üç boyutlu bir
→ dizi
int dortbuyutludizi1[5][2][3][2]; /* dortbuyutludizi1: elemanları 5 adet olan ve her bir elemanı
→ 2 adet 3x2 matris olan dört boyutlu bir dizi */

```

Aşağıda üç boyutlu dizilerde bir ilk değer verme kod örneği verilmiştir;

```

#define DERINLIK 3
#define SATIR 3
#define SUTUN 3
int ucBoyutluDizi[DERINLIK][SATIR][SUTUN]={
    { {1,2,3},{4,5,6},{7,8,9} },
    { {11,12,13},{14,15,16},{17,18,19} },
    { {21,22,23},{24,25,26},{27,28,29} }
};

```

Çok boyutlu dizi örneği olarak şöyle bir örnek verilebilir; 3 farklı ders alan 10 öğrenci, her bir dersten 4 değişik not almaktadır. Bu notların ağırlıkları sırasıyla %10, %30, %20 ve %40'tır. Notlar rastgele 0 ile 100 arasında verilecektir. Her bir sınavın ortalaması ile her bir öğrencinin ağırlıklı not ortalamalarını bulan C programı aşağıdaki gibidir;

```

#include<stdio.h>
#include<stdlib.h>
#include<time.h>
#define KACDERS 3
#define KACOGRENCI 10
#define KACDEGISIKNOT 4
int main() {
    int notlar[KACDERS][KACDEGISIKNOT][KACOGRENCI];
    int agirlik[KACDEGISIKNOT]={10,30,20,40};
    //Ödevler:%10, Vize:%30, Hızlı Sınav:%20, Final:%40
    int i,j,k; //indis değişkenleri
    srand(time(NULL)); // rastgele için
    for (k=0; k< KACDERS; k++)
        for (i=0; i< KACOGRENCI; i++)

```

```

    for (j=0; j< KACDEGISIKNOT; j++)
    notlar[k][j][i]=rand()%101;
    //Her bir sınavın Ortalaması hesaplanacak:
    for (k=0; k< KACDERS; k++) {
        printf("%d.Ders İçin:\n",k);
        for (j=0; j<KACDEGISIKNOT; j++) {
            float sinavToplam=0.0;
            for (i=0; i<KACOGRENCI; i++) {
                sinavToplam+=notlar[k][j][i];
            }
            printf("%d. Sınav ortalaması: %.2f\n",j,sinavToplam/KACOGRENCI);
        }
    }
    //Her bir Öğrencinin Ağırlıklı Notu hesaplanacak:
    for (k=0; k< KACDERS; k++) {
        printf("%d.Ders İçin:\n",k);
        for (i=0; i<KACOGRENCI; i++) {
            float agirlikliNot=0.0;
            for (j=0; j<KACDEGISIKNOT; j++) {
                agirlikliNot+=notlar[k][j][i]*agirlik[j]/100.0;
            }
            printf("%d. Öğrenci ortalaması: %.2f\n",i,agirlikliNot);
        }
    }
    return 0;
}

/*Program Çıktısı:
0.Ders İçin:
0. Sınav ortalaması:48.40
1. Sınav ortalaması:51.40
2. Sınav ortalaması:57.10
3. Sınav ortalaması:54.60
1.Ders İçin:
0. Sınav ortalaması:58.60
1. Sınav ortalaması:40.30
2. Sınav ortalaması:74.00
3. Sınav ortalaması:46.60
2.Ders İçin:
0. Sınav ortalaması:59.40
1. Sınav ortalaması:54.20
2. Sınav ortalaması:49.20
3. Sınav ortalaması:36.00
0.Ders İçin:
0. Öğrenci ortalaması:52.90
1. Öğrenci ortalaması:30.10
2. Öğrenci ortalaması:66.40
3. Öğrenci ortalaması:79.70
4. Öğrenci ortalaması:57.80
5. Öğrenci ortalaması:46.10
6. Öğrenci ortalaması:27.70
7. Öğrenci ortalaması:70.20
8. Öğrenci ortalaması:53.10
9. Öğrenci ortalaması:51.20
1.Ders İçin:
0. Öğrenci ortalaması:33.50
1. Öğrenci ortalaması:37.30
2. Öğrenci ortalaması:74.90
3. Öğrenci ortalaması:57.30
4. Öğrenci ortalaması:46.40
5. Öğrenci ortalaması:48.10
6. Öğrenci ortalaması:32.40
7. Öğrenci ortalaması:71.90

```

```

8. Öğrenci ortalaması:59.00
9. Öğrenci ortalaması:53.10
2.Ders İçin:
0. Öğrenci ortalaması:47.10
1. Öğrenci ortalaması:39.90
2. Öğrenci ortalaması:50.50
3. Öğrenci ortalaması:40.70
4. Öğrenci ortalaması:67.10
5. Öğrenci ortalaması:14.90
6. Öğrenci ortalaması:36.00
7. Öğrenci ortalaması:36.90
8. Öğrenci ortalaması:45.10
9. Öğrenci ortalaması:86.20
*/

```

8.5 Parametre Olarak Diziler

Parametre olarak gönderilen dizinin boyutu kadar köşeli parantez açılır ve kapatılır, ilk köşeli parantez içerisine eleman sayısını ifade eden değer yazılmaz, fakat diğerlerine eleman sayıları verilmek zorundadır. Bu durumda tek boyutlu diziler parametre olacak ise dizinin boyutu yazılmayabilir. Aşağıda dizileri parametre olarak alan fonksiyon bildirim örnekleri bulunmaktadır;

```

void diziIsleyenFonksiyon1(int tekBoyutluDizi[],int boyut);
void diziIsleyenFonksiyon2(int pDizi[][2],int ikinciBoyut,int birinciBoyut);
void diziIsleyenFonksiyon3(int pDizi[][3][2],int ucuncuBoyut,int ikinciBoyut,int birinciBoyut);

```

```

// Bu fonksiyonları aşağıdaki şekilde çağırabiliriz;
unsigned notlar[10]; //10 öğrenci notu barındırır.
unsigned dersNotlari[2][10]; //2 dersi alan 10 öğrenci için notlar;
unsigned bolumDersNotlari[4][2][10]; /* 4 bölümde 2 dersi alan 10 öğrenci için notlar;*/
//...
diziIsleyenFonksiyon1(notlar,10);
diziIsleyenFonksiyon2(dersNotlari,2,10);
diziIsleyenFonksiyon3(bolumDersNotlari,4,2,10);
//...

```

Aşağıda bir diziyi okuyan, ekrana yazan ve eleman ortalamalarını hesaplayan fonksiyonlara sahip bir program verilmiştir;

```

#include<stdio.h>
void diziOku(int pDizi[],int pUzunluk);
void diziYaz(int pDizi[],int pUzunluk);
float diziOrtalama(int pDizi[],int pUzunluk);
int main() {
    int dizi[5];
    float ortalama;
    diziOku(dizi,5); /* fonsiyon geri dönüş değeri olmadığından bir değişkene atanmıyor! */
    diziYaz(dizi,5); /* fonsiyon geri dönüş değeri olmadığından bir değişkene atanmıyor! */
    ortalama=diziOrtalama(dizi,5); /* fonsiyon geri dönüş değeri bir değişkene atanıyor.
    ↳ Atandığı değişkenin tipi ile fonksiyonun geri dönüş tipi aynı olmalı! */
    printf("Dizi Ortalaması: %.2f",ortalama);
    return 0;
}
void diziOku(int pDizi[],int pUzunluk){
    int sayac;
    printf("%d Elemanlı Dizi Okunacaktır:",pUzunluk);
    for (sayac=0; sayac < pUzunluk; sayac++)
        scanf("%d",&pDizi[sayac]);
}
void diziYaz(int pDizi[],int pUzunluk){
    int sayac;
    printf("%d Elemanlı Dizi:\n",pUzunluk);
    for (sayac=0; sayac < pUzunluk; sayac++)
        printf("%4d",pDizi[sayac]);
}

```

```

    printf("\n");
}
float diziOrtalama(int pDizi[], int pUzunluk){
    int sayac;
    float toplam=0;
    for (sayac=0; sayac < pUzunluk; sayac++)
        toplam+=pDizi[sayac];
    return toplam/pUzunluk;
}
/*Program Çıktısı:
5 Elemanlı Dizi Okunacaktır:10 20 30 40 50
5 Elemanlı Dizi:
10 20 30 40 50
Dizi Ortalaması: 30.00
*/

```

8.6 Dizi Elemanlarını Sıralama

Dizilerle ilgili bazı durumlarda elemanları sıralamak isteriz. Birden çok sıralama yöntemi vardır; Kabarcık sıralama (**bubble sort**), Seçme sıralama (**selection sort**), Araya sokma sıralama (**insertion sort**), Shell sıralaması (**Shell sort**) ve Çabuk sıralama (**quick sort**) gibi.

§8.6.1 Kabarcık Sıralama

Kabarcık sıralama (bubble sort) algoritmasında;

1. Her bir eleman kendisinden sonra gelen eleman ile karşılaştırılır.
2. Büyük ya da küçük sorgulaması yapılır. Büyük sorgulaması yapıldığında dizi küçükten büyüğe, küçük sorgulaması yapıldığında dizi büyükten küçüğe sıralanır.
3. Eğer şarta uyuyor ise elemanların yerleri değiştirilir (**swap**).
4. Bir sonraki elemana geçilerek birinci adımdan bu adıma kadar olan adımlar son dizi elemanına kadar devam edilir.
5. Yukarıdaki dört adım bitirildiğinde yalnızca birinci eleman doğru yerde olur. Böylece algoritmanın birinci aşaması tamamlanmıştır. Dolayısıyla bir sonraki elemana geçilerek dizinin eleman sayısı kadar bu işlemin tekrar edilmesi gerekir. Son eleman için aşamanın tekrar edilmesine gerek yoktur. Dolayısıyla aşama sayısı dizi uzunluğundan bir eksik yapılır.

Bunu bir örnekle açıklayalım;

```

int dizi[5]={14,11,0,3,7}; //Dizisini Küçükten Büyüğe sıralayalım:

if (a[0]<a[1]) { int gecici=a[0]; a[0]=a[1]; a[1]=gecici; } // 14>11 ise yer değiştir
// Dizinin yeni hali: {11,14,0,3,7}
if (a[1]<a[2]) { int gecici=a[1]; a[1]=a[2]; a[2]=gecici; } // 14>0 ise yer değiştir
// Dizinin yeni hali: {11,0,14,3,7}
if (a[2]<a[3]) { int gecici=a[2]; a[2]=a[3]; a[3]=gecici; } // 14>3 ise yer değiştir
// Dizinin yeni hali: {11,0,3,14,7}
if (a[3]<a[4]) { int gecici=a[3]; a[3]=a[4]; a[4]=gecici; } // 14>7 ise yer değiştir
// Dizinin yeni hali: {11,0,3,7,14}
//Buraya kadar birinci aşama tamamlanmıştır. En son eleman belirlenmiştir.

if (a[0]<a[1]) { int gecici=a[0]; a[0]=a[1]; a[1]=gecici; } // 11>0 ise yer değiştir
// Dizinin yeni hali: {0,11,3,7,14}
if (a[1]<a[2]) { int gecici=a[1]; a[1]=a[2]; a[2]=gecici; } // 11>3 ise yer değiştir
// Dizinin yeni hali: {0,3,11,7,14}
if (a[2]<a[3]) { int gecici=a[2]; a[2]=a[3]; a[3]=gecici; } // 11>7 ise yer değiştir
// Dizinin yeni hali: {0,3,7,11,14}
// Son eleman bir önceki aşamada belirlendiğinden karşılaştırma sayımız eksildi.
//Buraya kadar ikinci aşama tamamlanmıştır. Sondan ikinci eleman belirlenmiştir.

if (a[0]<a[1]) { int gecici=a[0]; a[0]=a[1]; a[1]=gecici; } // 0>3 ise yer değiştir
// Dizinin yeni hali: {0,3,7,11,14}
if (a[1]<a[2]) { int gecici=a[1]; a[1]=a[2]; a[2]=gecici; } // 3>7 ise yer değiştir
// Dizinin yeni hali: {0,3,7,11,14}

```

```
// Son eleman bir önceki aşamada belirlendiğinden karşılaştırma sayımız eksildi.
//Buraya kadar üçüncü aşama tamamlanmıştır. Sondan üçüncü eleman belirlenmiştir.

if (a[0]<a[1]) { int gecici=a[0]; a[0]=a[1]; a[1]=gecici; } // 0>3 ise yer değiştir
// Dizinin yeni hali: {0,3,7,11,14}
// Son eleman bir önceki aşamada belirlendiğinden karşılaştırma sayımız eksildi.
//Buraya kadar dördüncü aşama tamamlanmıştır. Sondan dördüncü eleman belirlenmiştir.
```

// Bir aşamaya daha gerek yoktur. Çünkü kalan 1 elemanın yeri değişmeyecektir.

Şimdi de sıralamayı döngü ile yapalım;

```
#include <stdio.h>
#define UZUNLUK 5
int main() {
    int dizi[UZUNLUK]={14,11,0,3,7};
    int indis;
    printf("Dizinin SıralanMAMış hali:\n");
    for (indis=0; indis<UZUNLUK; indis++)
        printf("%d, ",dizi[indis]);
    printf("\n");
    int asama;
    for (asama=0; asama<UZUNLUK-1; asama++) { //asama için
        for (indis=0; indis<UZUNLUK-asama-1;indis++) { /* karşılaştırma
            asama kadar eksik
            yapılır */
            if (dizi[indis] > dizi[indis+1]) { /* birinci eleman ile
                ikincisini karşılaştır */
                int yedek=dizi[indis]; // birinci elemanı yedekle
                dizi[indis]=dizi[indis+1]; /* birinci elemana
                ikicici elemanı ata */
                dizi[indis+1]=yedek; // yedeği ikinci elemana ata
            }
        }
    }
    printf("Dizinin SıralanMIŞ hali:\n");
    for (indis=0; indis<UZUNLUK; indis++)
        printf("%d, ",dizi[indis]);
    return 0;
}
/*Programın Çıktısı:
Dizinin SıralanMAMış hali:
14, 11, 0, 3, 7,
Dizinin SıralanMIŞ hali:
0, 3, 7, 11, 14,
*/
```

Bu sıralama türü küçük veri setleri veya eğitim amaçlı olarak kullanılır.

§8.6.2 Seçme Sıralama

Seçme sıralama (selection sort) algoritmasında;

1. Küçükten büyüğe seçme sıralama aşağıdaki adımlar izlenerek yapılır;
 - (a) Dizi üzerinde ilk elemandan başlanarak dizi sonuna kadar ilerlenir.
 - (b) İlerleme sırasındaki elemanın sonrasındaki elemanlar arasında en küçük eleman bulunur.
 - (c) Bulunan en küçük eleman ile ilerleme sırasındaki eleman yer değiştirilir.
2. Büyükten küçüğe seçme sıralama aşağıdaki adımlar izlenerek yapılır;
 - (a) Dizi üzerinde ilk elemandan başlanarak dizi sonuna kadar ilerlenir.
 - (b) İlerlemeye sırasındaki elemanın sonrasındaki elemanlar arasında en büyük eleman bulunur.
 - (c) Bulunan en küçük eleman ile ilerleme sırasındaki eleman yer değiştirilir.

Aşağıda buna ilişkin örnek program verilmiştir;

```
#include <stdio.h>
#define UZUNLUK 5
void diziYaz(int pDizi[], int pBoyut);
```



```

void selectionSortKucuktenBuyuge(int pDizi[], int pBoyut);
void selectionSortBuyuktenKucuge(int pDizi[], int pBoyut);
int main() {
    int dizi[UZUNLUK]={14,11,0,3,7};
    printf("Dizinin SıralanMAMIŞ hali:\n");
    diziYaz(dizi, UZUNLUK);
    selectionSortKucuktenBuyuge(dizi,UZUNLUK);
    printf("Dizinin Küçükten Büyüğe SıralanMIŞ hali:\n");
    diziYaz(dizi, UZUNLUK);
    selectionSortBuyuktenKucuge(dizi,UZUNLUK);
    printf("Dizinin Büyüktem Küçüğe SıralanMIŞ hali:\n");
    diziYaz(dizi, UZUNLUK);
    return 0;
}
void diziYaz(int pDizi[], int pBoyut){
    int sayac;
    for (sayac=0; sayac<pBoyut; sayac++)
        printf("%d,",pDizi[sayac]);
    printf("\n");
}
void selectionSortKucuktenBuyuge(int pDizi[], int pBoyut){
    int ilerleme;
    for (ilerleme=0; ilerleme<pBoyut-1; ilerleme++) {
        int sayac;
        int enKucukIndis=ilerleme;
        for (sayac=ilerleme+1; sayac<pBoyut; sayac++)/* Geri kalanındaki
        en Küçüğün İndisi */
            if (pDizi[sayac] < pDizi[enKucukIndis])
                enKucukIndis =sayac;
        /* Bulunan en küçükkindisli eleman ile
        ilerlemedeki eleman yer değiştirilir */
        int yedek=pDizi[ilerleme]; /* yer değiştirme:
        yedek, dizi veri tipinde olmalı */
        pDizi[ilerleme]=pDizi[enKucukIndis];
        pDizi[enKucukIndis]=yedek;
    }
}
void selectionSortBuyuktenKucuge(int pDizi[], int pBoyut){
    int ilerleme;
    for (ilerleme=0; ilerleme<pBoyut-1; ilerleme++) {
        int sayac;
        int enBuyukIndis=ilerleme;
        for (sayac=ilerleme+1; sayac<pBoyut; sayac++) /*Geri kalanındaki
        en Küçüğün İndisi */
            if (pDizi[sayac] > pDizi[enBuyukIndis])
                enBuyukIndis =sayac;
        /* Bulunan en küçükkindisli eleman ile
        ilerlemedeki eleman yer değiştirilir */
        int yedek=pDizi[ilerleme]; //yer değiştirme:
        pDizi[ilerleme]=pDizi[enBuyukIndis];
        pDizi[enBuyukIndis]=yedek;
    }
}
/*Program Çıktısı:
Dizinin SıralanMAMIŞ hali:
14,11,0,3,7,
Dizinin Küçükten Büyüğe SıralanMIŞ hali:
0,3,7,11,14,
Dizinin Büyüktem Küçüğe SıralanMIŞ hali:
14,11,7,3,0,
*/

```

Bu sıralama türü, yer değiştirme sayısı az olduğu için bellek yazma işleminin maliyetli olduğu durumlarda

kullanılır.

§8.6.3 Araya Sokma Sıralaması

Araya sokma sıralamasında (**insertion sort**), diziyi hayali olarak **sıralı** ve **sıralanmamış** olarak ikiye böler. Sıralanmamış kısımdan her seferinde bir eleman alır ve sıralı kısımdaki doğru yerine yerleştirir.

```
#include <stdio.h>
#define UZUNLUK 5
int main() {
    int dizi[UZUNLUK]={14,11,0,3,7};
    int indis;
    printf("Dizinin SıralanMAMış hali:\n");
    for (indis=0; indis<UZUNLUK; indis++)
        printf("%d, ",dizi[indis]);
    printf("\n");

    //insertionSort
    int i, key, j;
    for (i = 1; i < UZUNLUK; i++) {
        key = dizi[i]; // Sıralanacak eleman
        j = i - 1;
        // Key'den büyük olan elemanları bir sağa kaydır
        while (j >= 0 && dizi[j] > key) {
            dizi[j + 1] = dizi[j];
            j = j - 1;
        }
        dizi[j + 1] = key; // Boşalan yere key'i yerleştir
    }

    printf("\nDizinin SıralanMIŞ hali:\n");
    for (indis=0; indis<UZUNLUK; indis++)
        printf("%d, ",dizi[indis]);
    return 0;
}
```

Bu sıralama türü, veriler **neredeyse sıralıysa** çok hızlı çalışır.

§8.6.4 Shell Sıralaması

Shell sıralaması (**Shell sort**), aslında araya sokma algoritmasının çok daha zekice ve hızlandırılmış bir versiyonudur. 1959 yılında *Donald Shell* tarafından geliştirilmiştir.

Araya sokma sıralamasının en büyük zayıflığı, elemanları sadece bir yanındakiyle kıyaslamasıdır. Eğer dizinin en sonundaki küçük bir elemanı en başa taşımak isterseniz, algoritma onu tek tek tüm elemanlarla kaydırarak götürür. Shell sıralaması bu sorunu şöyle çözer:

- ◊ Diziyi belirli bir aralık (**gap**) değeriyle parçalara böler (örneğin dizinin yarısı kadar).
- ◊ Birbirinden uzak olan bu elemanları kendi aralarında sıralar.
- ◊ Aralığı kademeli olarak azaltır (genelde her adımda yarıya düşürür).
- ◊ Aralık 1 olduğunda, dizi zaten **neredeyse sıralı** hale gelmiş olur. Son adımda standart bir araya sokma sıralaması yapar ve işlem biter.

```
#include <stdio.h>
void shellSort(int dizi[], int n) {
    // Aralığı (gap) dizinin yarısından başlat, her seferinde yarıya düşür
    for (int gap = n / 2; gap > 0; gap /= 2) {
        // Bu aralık değeri için araya sokma sıralaması yap
        for (int i = gap; i < n; i++) {
            int temp = dizi[i];
            int j;
            // Elemanları gap mesafesiyle kıyasla ve kaydır
            for (j = i; j >= gap && dizi[j - gap] > temp; j -= gap) {
                dizi[j] = dizi[j - gap];
            }
            // temp (başta saklanan eleman) doğru yere yerleştirilir
            dizi[j] = temp;
        }
    }
}
```

```

    }
}
int main() {
    int dizi[] = {23, 12, 1, 8, 34, 54, 2, 3};
    int n = sizeof(dizi) / sizeof(dizi[0]);

    shellSort(dizi, n);

    printf("Shell Sort ile sıralanmış dizi: ");
    for (int i = 0; i < n; i++)
        printf("%d ", dizi[i]);

    return 0;
}

```

§8.6.5 Çabuk Sıralama

Çabuk sıralama (quick sort), bir pivot eleman seçer. Pivotun soluna ondan küçükleri, sağına ise ondan büyükleri toplar. Ardından aynı işlemi oluşturan iki küçük grup için özyinelemeli (**recursive**) olarak tekrarlar.

```

#include <stdio.h>
// Bu örnek için Göstericiler bilinmelidir!
void takas(int* a, int* b) { // İki elemanın yerini değiştiren yardımcı fonksiyon
    int t = *a;
    *a = *b;
    *b = t;
}
int partition(int dizi[], int low, int high) { // Diziye parçalara ayıran fonksiyon
    int pivot = dizi[high]; // Son elemanı pivot seçtik
    int i = (low - 1); // Küçük elemanların indisi

    for (int j = low; j <= high - 1; j++) {
        if (dizi[j] < pivot) {
            i++;
            takas(&dizi[i], &dizi[j]);
        }
    }
    takas(&dizi[i + 1], &dizi[high]);
    return (i + 1);
}
void quickSort(int dizi[], int low, int high) {
    if (low < high) {
        int pi = partition(dizi, low, high); // Pivotun doğru yerini bul
        quickSort(dizi, low, pi - 1); // Pivotun solunu sırala
        quickSort(dizi, pi + 1, high); // Pivotun sağını sırala
    }
}
#define UZUNLUK 5
int main() {
    int dizi[UZUNLUK]={14,11,0,3,7};
    int indis;
    printf("Dizinin SıralanMAmış hali:\n");
    for (indis=0; indis<UZUNLUK; indis++)
        printf("%d, ",dizi[indis]);
    printf("\n");

    quickSort(dizi,0, UZUNLUK-1);

    printf("\nDizinin SıralanMIŞ hali:\n");
    for (indis=0; indis<UZUNLUK; indis++)
        printf("%d, ",dizi[indis]);
    return 0;
}

```

```
}
```

Bu sıralama türü genel amaçlı en hızlı algoritmalarından biridir. Binlerce veya milyonlarca veriyi sıralayacaksan vazgeçilmezdir. Ortalama çalışma süresi $O(n \log n)$ olduğu için devasa fark yaratır.

§8.6.6 Hazır Sıralama Fonksiyonları

C dilinde bu algoritmaları elle yazmak yerine, standart kütüphanede (`stdlib.h`) bulunan çok hızlı ve optimize edilmiş `qsort()` fonksiyonunu kullanabilirsiniz;

```
#include <stdio.h>
#include <stdlib.h> //qsort() için

int karsilastir(const void *a, const void *b) { // Karşılaştırma fonksiyonu (qsort için
    ↪ gereklidir)
    return (*(int*)a - *(int*)b);
}

int main() {
    int dizi[] = {42, 15, 8, 99, 23};
    int n = sizeof(dizi) / sizeof(dizi[0]);
    // qsort(dizi_adresi, eleman_sayisi, eleman_boyutu, karsilastirma_fonksiyonu)
    qsort(dizi, n, sizeof(int), karsilastir);
    printf("Sıralı dizi: ");
    for(int i = 0; i < n; i++) printf("%d ", dizi[i]);
    return 0;
}
```

8.7 Düşün ve Çöz

- ◊ **Düşün:** Çok boyutlu dizilerin (matrislerin) bellekte aslında tek boyutlu ve ardışık olarak tutulmasının, dizi elemanlarına erişim hızına etkisini tartışınız.
- ◊ **Düşün:** C dilinde dizi sınırlarının (**out of bounds**) kontrol edilmemesinin, bellek güvenliği ve "buffer overflow" saldırıları açısından riskini tartışınız.
- ◊ **Düşün:** İkili Arama (**binary search**) algoritmasının çalışabilmesi için dizinin neden mutlaka sıralı olması gerektiğini açıklayınız.
- ◊ **Çöz:** 10 elemanlı rastgele sayılardan oluşan bir diziyi kabarcık sıralama algoritması ile küçükten büyüğe sıralayan fonksiyonu yazınız.
- ◊ **Çöz:** 20 elemanlı bir dizideki elemanların frekansını (hangi sayıdan kaç tane olduğunu) bulan ve ekrana listeleyen programı yazınız.
- ◊ **Çöz:** Kullanıcıdan alınan iki adet 3x3'lük matrisin toplamını bulan ve sonucu ekrana matris formunda basan fonksiyonu kodlayınız.

9. Bölüm

Göstericiler

9.1 Gösterici nedir? Niçin İhtiyaç Duyarız?

Şu ana kadar gördüğümüz `char`, `int`, `long`, `float`, `double` tipli değişkenler ve bunlara ait diziler, değer tipler (**value type**) olarak adlandırılır. Çünkü kimliklendirilen değişken, değeri doğrudan ifade eder. Değer tiplerin yanı sıra değişkenlerin adreslerini tutan değişkenlere de ihtiyaç duyarız. İşte adres tutan bu değişkenlere **gösterici tipler** (**pointer type**) adını veririz. Göstericileri işaret ettikleri veri tipine (**data type**) göre tanımlarız. Bu nedenle göstericiler, **türetilmiş tipler**(**derived type**) olarak adlandırılır.

Gösterici tipler, gösterdiği yerdeki verileri değiştirmek için de kullanılır. **Gösterici tipler işaret ettiği verinin tipine göre tanımlanırlar. Bütün gösterici tipler bellekte aynı miktarda yer kaplar. Çünkü hepsi adres tutar.** Bütün bu tanımlamaların ışığında göstericileri, vatandaşların ikametlerini gösteren adres numaraları olarak düşünebiliriz.

Gösterici tiplere ihtiyaç duyulmasının sebebi hız ve esnekliktir. Genel olarak birkaç madde ile sıralayacak olursak;

- ♦ Belleğin istenilen bölgesine erişim sağlamak için göstericiler kullanılır. Günümüz modern dillerinde **referans tipler** (**referans type**) kullanılır. Göstericilerden farklı olarak referans tipler, gösterilen adreste bir verinin/nesnenin bulunduğunu garanti eder. Göstericiler her bellek bölgesine erişebilir.
- ♦ Bazen çalışma anında yeni bellek bölgelerine ihtiyaç duyarız bu durumda göstericiler gereklidir. **Dinamik veri yapıları** (dynamic data structure) göstericiler yardımıyla oluşturulup yönetilir.
- ♦ Fonksiyonlara parametre geçirilirken argüman olarak kullanılan yerel değişkenlerin kopyaları kullanılır. Bu durum Şekil 7.3'de açıklanmıştı. Fonksiyondan geri döndüğünde ise yerel değişkenler eski haline yığın bellekten geri çekilerek çevrilir. Fonksiyonun yerel değişken ya da argümanlardan birini değiştirmesi istendiğinde fonksiyona argüman olarak değişkenin adresi verilir. Böylece fonksiyona parametre olarak geçirilen yerel değişkenler göstericiler üzerinden fonksiyon içinde değiştirilebilir hale gelir.

Göstericilere olan ihtiyacı bir başka örnekle de açıklayabiliriz; Bir ormandaki ağaçları boylarına göre sıralamaya kalkarsak her birini yer değiştirme yöntemiyle sıralamak oldukça külfetli ve zaman alan bir işlemdir. Halbuki ağaçlara numara vererek ve bu numaraların karşısına uzunluklarını yazacağımız bir liste oluşturduğunda sadece numaraları sıralamak oldukça kolay ve zahmetsiz bir işlemdir. İşte buradaki numaralara karşılık gelen değişkenler göstericilerdir.

9.2 Gösterici Tanımlama

Göstericiler, işaret ettiği verinin tipine göre tanımlanırlar. Gösterici tanımlanırken veri tipi izleyen **başvuru kaldırma işleci** (**dereference operator**) olan yıldız kullanılır. Başvuru kaldırma deyimi, gösterici tarafından tutulan bellek adresinde saklanan değere erişmeyi ifade eder.

```
//GÖSTERİCİ TANIMLAMA:
```

```
veritipi *gostericiKimligi1;  
veritipi* gostericiKimligi2;
```

```
//AYNI VERİ TİPİNDE TEK SATIRDA BİRDEN ÇOK GÖSTERİCİ TANIMLAMA:
```

```
veritipi* birinciGosterici, ikinci gosterici;
```

```
//AYNI VERİ TİPİNDE TEK SATIRDA HEM DEĞİŞKEN HEM DE GÖSTERİCİ TANIMLAMA:
```

```
veritipi değişkenKimliği, *gostericiKimliği3;
```

Göstericinin işaret ettiği değişkene değer atama aşağıdaki şekilde yapılır. Burada göstericinin veri tipi ile atanan değer aynı veri tipinde olmalıdır;

```
gostericikimligi = &degiskenkimligi;
```

Bir değişkenin adresinin **adres işleci** ile elde edilebileceği 4.7.2 başlığında anlatılmıştı. Aşağıda göstericilere ilişkin kod örnekleri verilmiştir;

```
char *ptrToChar; /*göstericinin işaret ettiği adreste "char" tipinde veri olan "ptrToChar"  
→ kimlikli göstericisi tanımlandı.*/
```

```
int i=10;
int *ptrToInt=&i; /*göstericinin işaret ettiği adreste "int" tipinde veri olan "ptrToInt"
→ kimlikli göstericisi tanımlandı. Ancak göstericinin işaret ettiği adres olarak i değişkeninin
→ adresi atanıyor.*/

*ptrToInt =20; /* "ptrToInt" göstericisinin işaret ettiği adresteki tamsayı değeri 20 yaptık.
→ "ptrToInt" göstericisi, i değişkeninin adresini işaret ettiği i değişkeninin değeri de 20
→ olmuştur */
```

Veri tiplerinin işlendiği 4.3 başlığında, **hiçbir değeri olmayan ve bellekte yer kaplamayan void** veri tipi anlatılmıştı. Veri tipi **void** olan **void*** göstericisine **jenerik gösterici (generic pointer)** adı verilir.

Bazı durumlarda göstericilerin veya gösterdiği değerlerin sabit olması gerekebilir. Bunun için üç durum ortaya çıkabilir;

1. **İşaret edilen değerin sabit olması durumu:** Bu durumda gösterici tanımlaması aşağıdaki gibi yapılır;

```
int main() {
    int i=10;

    const int *ptrToi=&i; // değerin sabit olması durumunda gösterici tanımı

    *ptrToi=20; /* HATA: error: assignment of read-only location ‘*ptrToi’
pi göstericisinin işaret ettiği adresteki tamsayı değeri 20 yapılamaz. */

    return 0;
}
```

2. **Göstericinin işaret ettiği adresin sabit olması durumu:** Kısaca göstericinin sabit olması durumunda tanımlama aşağıdaki gibi yapılır;

```
int main() {
    int i=10;
    int *const ptrToi=&i; // göstericinin sabit olması durumunda gösterici tanımı
/* Bu tür tanımlamada bir adres ilk değer (initialize) olarak mutlaka
→ verilmelidir. */

    int j=30;
    *ptrToi=20; // pi göstericisinin işaret ettiği adresteki tamsayı değeri 20 olur.
ptrToi =&j; /*HATA:error: assignment of read-only variable ‘ptrToi’
pi göstericisinin işaret ettiği adres değiştirilemez!*/

    return 0;
}
```

3. **Hem göstericinin hem de değerin sabit olması durumu:**

```
int main() {
    int j=10, k=100;
    const int * const ptr=&j; /* Hem gösterici, hem de gösterilen veri sabit */
/* Bu tür tanımlamada da bir adres ilk değer (initialize) olarak mutlaka
→ verilmelidir. */

    *ptr=20; /* HATA: error: assignment of read-only location ‘*(const int *)ptr’*/
ptr=&k; /*HATA: error: assignment of read-only variable ‘ptr’ */

    return 0;
}
```

Gösterici tipe atamaya uygun kesin bir adresiniz yoksa, **NULL** değeri atamak her zaman iyi bir uygulamadır. İlk değeri olmayan gösterici tanımlamak yerine, ilk değeri **sıfır** olan göstericiler tanımlanmak doğru bir yöntemdir. Bu durumda gösterici, **boş gösterici (NULL Pointer)** adı verilir. Aşağıdaki gibi kullanılır;

```
veritipi *gostericikimligi = NULL;
gostericikimligi = NULL;
```

Aşağıda boş gösterici kullanımına ilişkin örnek bir uygulama verilmiştir;

```
#include <stdio.h>
```

```

int main(){
    int *ptr = NULL;
    printf("ptr göstericisinin işaret ettiği adres: %p\n", ptr);
    int agirlik=80;
    ptr=&agirlik;
    printf("ptr göstericisinin işaret ettiği adres: %p ve değeri: %d\n", ptr, *ptr);
    return 0;
}
/* Program Çıktısı:
ptr göstericisinin işaret ettiği adres: (nil)
ptr göstericisinin işaret ettiği adres: 0x7ffdedc33dec ve değeri: 80
*/

```

Bir göstericinin işaret ettiği yerde bir başka gösterici olabilir. Buna **çifte gösterici (double pointer)** adı verilir. Aşağıda buna ilişkin bir örnek verilmiştir;

```

#include <stdio.h>
int main(){
    int var = 10;
    int *intPtr = &var;
    int **ptrToPtr = &intPtr; //double pointer
    printf("var:%d &var:%p \n",var, &var);
    printf("inttPtr: %p *inttPtr: %p \n\n", intPtr, &intPtr);
    printf("var: %d *intPtr: %d \n", var, *intPtr);
    printf("ptrToPtr: %p &ptrtPtr: %p \n", ptrToPtr, &ptrToPtr);
    printf("&intPtr: %p *ptrToPtr : %p \n\n", &intPtr, *ptrToPtr);
    printf("var: %d *intPtr: %d **ptrToPtr: %d", var, *intPtr, **ptrToPtr);
    return 0;
}
/*Program Çıktısı:
var:10 &var:0x7fff67f1b2bc
inttPtr: 0x7fff67f1b2bc *inttPtr: 0x7fff67f1b2b0

var: 10 *intPtr: 10
ptrToPtr: 0x7fff67f1b2b0 &ptrtPtr: 0x7fff67f1b2a8
&intPtr: 0x7fff67f1b2b0 *ptrToPtr : 0x7fff67f1b2bc

var: 10 *intPtr: 10 **ptrToPtr: 10
*/

```

Bir tam sayı bellekte 4 bayt yer işgal eder. Bu baytların bellekte nasıl yerleştiğini gösteren program verilmiştir.

```

#include <stdio.h>
int main() {
    int i=0x10203040;
    char *p= (char*) &i;
    /* int işaret eden adres, char içeriyor diye (char*) ifadesiyle tip dönüşümü (cast)
    ↪ yapılıyor. */
    printf("Sayı :%10d-%x\n",i,i);
    printf("1.bayt:%10d-%8x\n",*p,*p);
    printf("2.bayt:%10d-%6x\n",*(p+1),*(p+1));
    printf("3.bayt:%10d-%4x\n",*(p+2),*(p+2));
    printf("4.bayt:%10d-%x\n",*(p+3),*(p+3));
    return 0;
}
/*Program Çıktısı:
Sayı : 270544960-10203040
1.bayt: 64- 40
2.bayt: 48- 30
3.bayt: 32- 20
4.bayt: 16-10
*/

```


9.3 Gösterici Aritmetiği

Göstericilerde toplama ve çıkarma tam sayılardan farklı şekilde çalışır. **Bir gösterici artırıldığında veya azaltıldığında, işaret ettiği adres, işaret ettiği veri tipinin bellek boyutu kadar artırılır veya azaltılır.** Bunun nedeni bilgisayarların tasarımından ötürü işlemciye bağlı belleklerde her bir bayt ayrı adreslenmesidir.

Bir `char*` göstericisi, işaret ettiği yerdeki `char` verisinden bir sonraki `char` verisini göstermesi istendiğinde **gösterici adresi 1 artar**. Çünkü `char` veri tipi bellekte 1 bayt yer kaplar. Aşağıdaki program çıktısındaki adres değişimlerini inceleyiniz;

```
#include <stdio.h>
#define BOYUT 5
void printDizi(char *d) {
    printf("DİZİ:");
    for (int i=0; i<BOYUT; i++)
        printf("\'%c\'", d[i]);
    printf("\n");
}
int main() {
    char dizi[BOYUT]={'I','L','H','A','N'}; /* Bellekte art arda 5 char'dan oluşan dizi. */

    char *ptrToChar=&dizi[0]; /* ptrToChar göstericisi dizinin ilk elemanını işaret ediyor.*/
    printf("İşaret Edilen Adres: %p\n", ptrToChar);
    *ptrToChar='X'; /* Göstericinin işaret ettiği yerdeki char değeri 'X' olarak
        ↳ değiştirildi. */
    printDizi(dizi);

    ptrToChar++; /* ptrToChar göstericisi bir sonraki baytı gösterecek şekilde artırıldı. */
    printf("İşaret Edilen Adres: %p\n", ptrToChar);
    *ptrToChar='Y'; /* Göstericinin işaret ettiği yerdeki char değeri 'X' olarak
        ↳ değiştirildi.*/
    printDizi(dizi);

    ptrToChar+=3; /* ptrToChar göstericisi üç sonraki baytı gösterecek şekilde artırıldı. */
    printf("İşaret Edilen Adres: %p\n", ptrToChar);
    *ptrToChar='Z'; /* Göstericinin işaret ettiği yerdeki char değeri 'X' olarak
        ↳ değiştirildi.*/
    printDizi(dizi);
    return 0;
}
/*Program Çıktısı:
İşaret Edilen Adres: 0x7ffd59be4313
DİZİ: {'X','L','H','A','N'},
İşaret Edilen Adres: 0x7ffd59be4314
DİZİ: {'X','Y','H','A','N'},
İşaret Edilen Adres: 0x7ffd59be4317
DİZİ: {'X','Y','H','A','Z'},
*/
```

Benzer şekilde `int*` göstericisi, işaret ettiği yerdeki `int` verisinden bir sonraki `int` verisini göstermesi istendiğinde **gösterici adresi 4 artar**. Çünkü `int` veri tipi bellekte 4 bayt yer kaplar. Aşağıdaki program çıktısındaki adres değişimlerini inceleyiniz;

```
#include <stdio.h>
#define BOYUT 5
void printDizi(int *d) {
    printf("DİZİ:");
    for (int i=0; i<BOYUT; i++)
        printf("%d", d[i]);
    printf("\n");
}
int main() {
    int dizi[BOYUT]={10,20,30,40,50}; /* Bellekte art arda 5 int'den oluşan dizi. */

    int *ptrToInt=&dizi[0]; /* ptrToInt göstericisi dizinin ilk elemanını işaret ediyor.*/
```

```

printf("İşaret Edilen Adres: %p\n", ptrToInt);
*ptrToInt=-1; /* Göstericinin işaret ettiği yerdeki int değeri -1 olarak değiştirildi. */
printDizi(dizi);

ptrToInt++; /* ptrToInt göstericisi bir sonraki int'i gösterecek şekilde artırıldı. */
printf("İşaret Edilen Adres: %p\n", ptrToInt);
*ptrToInt=-2; /* Göstericinin işaret ettiği yerdeki int değeri -2 olarak değiştirildi.*/
printDizi(dizi);

ptrToInt+=3; /* ptrToInt göstericisi üç sonraki int'i gösterecek şekilde artırıldı. */
printf("İşaret Edilen Adres: %p\n", ptrToInt);
*ptrToInt=-3; /* Göstericinin işaret ettiği yerdeki int değeri -3 olarak değiştirildi.*/
printDizi(dizi);
return 0;
}
/*Program Çıktısı:
İşaret Edilen Adres: 0x7ffec841be70
DİZİ:{-1,20,30,40,50,}
İşaret Edilen Adres: 0x7ffec841be74
DİZİ:{-1,-2,30,40,50,}
İşaret Edilen Adres: 0x7ffec841be80
DİZİ:{-1,-2,30,40,-3,}
*/

```

Bunun gibi;

- ◊ **short*** göstericisi, işaret ettiği yerdeki **short** verisinden bir sonraki **short** verisini göstermesi istendiğinde gösterici adresi 2 artar. Çünkü **short** veri tipi bellekte 2 bayt yer kaplar.
- ◊ **long*** göstericisi, işaret ettiği yerdeki **long** verisinden bir sonraki **long** verisini göstermesi istendiğinde gösterici adresi 4 artar. Çünkü **long** veri tipi bellekte 8 bayt yer kaplar;
- ◊ **float*** göstericisi, işaret ettiği yerdeki **float** verisinden bir sonraki **float** verisini göstermesi istendiğinde gösterici adresi 4 artar. Çünkü **float** veri tipi bellekte 4 bayt yer kaplar.
- ◊ **double*** göstericisi, işaret ettiği yerdeki **double** verisinden bir sonraki **double** verisini göstermesi istendiğinde gösterici adresi 8 artar. Çünkü **double** veri tipi bellekte 8 bayt yer kaplar;

Ayrıca göstericinin artırılması yanında eksiltilmesi de mümkündür. Bu durumda **-**, **--** ve **--** işlemleri kullanılır.

- ◊ Görüldüğü üzere **göstericilerin üzerinde işlemler (operator) işlem yaparken göstericinin işaret ettiği verinin tipine göre farklı işlem yaparlar.**
- ◊ Gösterici aritmetikinde (**++**, **--**, **+**, **-**, **+=** ve **-=**) ile (**<**, **>** ve **==**) çalışır. Diğer işlemler çalışmaz.

9.4 Gösterici Dizileri

Göstericiler de dizi olarak tanımlanabilir. Aşağıda birbirinden bağımsız tanımlanmış yerel değişken adreslerinden bir gösterici dizisi tanımlanmıştır. Bu dizi üzerinden değişkenler daha kolay işlenebilir;

```

#include <stdio.h>
int main() {
    int i = 10, j=20;
    float f=1.0;
    int k=30, l=40;

    int *ptr[4]; /* int gösteren 4 gösteri içeren bir dizi tanımlandı */

    /* Göstericilere ilk değer veriliyor*/
    ptr[0] = &i; // Birinci eleman i değişkeninin adresini
    ptr[1] = &j; // İkinci eleman j değişkeninin adresini
    ptr[2] = &l; // Üçüncü eleman l değişkeninin adresini
    ptr[3] = &k; // Dördüncü eleman k değişkeninin adresini

    *ptr[2] = -1; // Üçüncü elemanın işaret ettiği adresteki değer -1 olarak değiştirildi. (l
    → değişkeni)

```

```

// Değerlere Ulaşma
int indis;
for (indis = 0; indis < 4; indis++)
    printf("ptr[%d] ile gösterilen değer: %d\n", indis, *ptr[indis]);

return 0;
}
/*Program Çıktısı:
ptr[0] ile gösterilen değer: 10
ptr[1] ile gösterilen değer: 20
ptr[2] ile gösterilen değer: -1
ptr[3] ile gösterilen değer: 30
*/

```

9.5 Referansla Çağırma

Fonksiyonlara parametre geçirilirken argüman olarak kullanılan yerel değişkenlerin kopyaları kullanılır. Bu durum Şekil 7.3’de açıklanmıştı. Fonksiyondan geri döndüğünde ise yerel değişkenler eski haline yığın bellekten geri çekilerek çevrilir. Buna ilişkin örnek aşağıda verilmiştir;

```

#include <stdio.h>
void degistir(int,int);
int main(){
    int a = 10, b = 20;
    degistir(a, b);
    // çağrı sonrası "yerel değişkenler" yığın bellekten geri çekilerek eski haline
    ↪ gelmiştir.
    printf("1- a:%d, b:%d\n", a, b); //Çıktı: a:10, b:20
}
void degistir(int pX, int pY) {
    int z = pX;
    pX=pY;
    pY=z;
}

```

Fonksiyona geçirilen yerel değişken ya da argümanlardan birinin değiştirilmesi istendiğinde bu fonksiyona argüman olarak değişkenin adresi verilir. Böylece fonksiyona parametre olarak geçirilen yerel değişkenler dolaylı olarak göstericiler üzerinden fonksiyon içinde değiştirilebilir hale gelir. Fonksiyona geçirilecek argümanlar gösterici olarak tanımlanır. Çağrı ortamına geri döndüğünde yerel değişkenler de değişmiş olur. Bu işleme **referansla çağırma** (**call by reference**) adı verilir. Çünkü fonksiyona değişken adresleri değer olarak geçirilmiştir. Buna ilişkin örnek aşağıda verilmiştir;

```

#include <stdio.h>
void degistir(int *pX, int *pY);
int main(){
    int a = 10, b = 20;
    degistir(&a, &b);
    // çağrı sonrası "yerel değişkenlerin adresleri" yığın bellekten geri çekilerek eski
    ↪ haline gelmiştir.
    printf("2- a:%d, b:%d\n", a, b); //Çıktı: a:20, b:10
}
void degistir(int* pX, int* pY){
    int z = *pX;
    *pX=*pY;
    *pY=z;
}

```

9.6 Göstericilerin Dizileri İşaret Etmesi

Çoğu durumda, bir dizi yardımıyla gerçekleştirdiğimiz işleri gösterici ile de gerçekleştirilebiliriz. Dizi değişkeni, dizinin ilk öğesinin adresi işaret eder. Kısaca dizi değişkenleri gösterici gibi davranırlar.

```

#include <stdio.h>

```

```
int main () {
    int dizi[5] = {10, 20, 30, 40, 50};
    int *ptr = dizi; // int *ptr=& dizi[0]; olarak da kodlanabilir.

    int indis;
    printf("Dizi elemanlarına gösterici ile erişim: \n");

    for(i = 0; i < 5; i++) {
        printf("dizi[%d]: %d\n", indis, *(ptr+indis));
    }

    return 0;
}
```

Fonksiyonlara dizileri işaret eden göstericileri parametre olarak verebiliriz. Dizi değişkeni, dizinin ilk öğesinin adresi işaret ettiğinden dolayı **dizi değişkenleri (array) parametre olarak kullanılırken adres işleci (address operator) kullanılmaz**. Bu durumda parametre olarak dizi elemanının veri tipinde bir göstericiyi parametre olarak tanımlarız.

```
float notlar[10];
float notlarOrtalamasi(float*); //prototype
//...
float ort=notlarOrtalamasi(notlar); // yada notlarOrtalamasi(&notlar[0]);
//...
float matris[4][3];
float defetminant(float**,int,int); //prototype
//...
float det=defetminant(matris,4,3);
//...
```

Aşağıda öğrencilerin notlarına ilişkin işlemler yapılan bir program verilmiştir.

```
#include <stdio.h>
#include <math.h>
#define OGRENCISAYISI 10

float notlarOrtalamasi(float*); // parametre olarak dizi kabul eder
void notlariArtir(float*); // parametre olarak dizi kabul eder

int main(){
    float notlar[OGRENCISAYISI]= { 55.0,60.0,70.0,35.0,30.0,50.0,65.0,90.0,95.0,100.0 };
    float toplam=notlarOrtalamasi(notlar);
    printf("Notlar Ortalaması: %f\n", toplam);
    notlariArtir(notlar); // Bu fonksiyonla dizi elemanları değiştiriliyor
    int indis;
    for (indis=0; indis<OGRENCISAYISI; indis++)
        printf("Not[%d]=%f\n",indis,notlar[indis]);
    return 0;
}

float notlarOrtalamasi(float* pNotlar){
    int indis;
    float top=0.0;
    for (indis=0; indis<OGRENCISAYISI; indis++)
        top+=pNotlar[indis]; // Gösterici dizi gibi kullanılıyor
    return top/OGRENCISAYISI;
}

void notlariArtir(float* pNotlar) {
    int indis;
    for (indis=0; indis<OGRENCISAYISI; indis++)
        if (pNotlar[indis]<=90) //Göstericinin gösterdiği değerler değiştiriliyor
            pNotlar[indis]+=10.0;
};

/*Program Çıktısı:
```

```

Notlar Ortalaması: 65.0
Not[0]=65.0
Not[1]=70.0
Not[2]=80.0
Not[3]=45.0
Not[4]=40.0
Not[5]=60.0
Not[6]=75.0
Not[7]=100.0
Not[8]=95.0
Not[9]=100.0
*/

```

9.7 Fonksiyonlardan Bir Diziyi Geri Döndürme

C dilinde bir fonksiyonun geri dönüş değeri olarak tüm bir dizi verilemez! Ancak, bir dizinin göstericisini fonksiyondan geri dönüş değeri olarak döndürebilirsiniz. **Burada dikkat edilmesi gereken fonksiyondan çıkılınca dizinin bellekten kaldırılmamasıdır.** Çünkü yerel değişkenler yığın bellekte tutulduğundan fonksiyondan çıkılınca bellekten silinirler. Aşağıda bir tamsayı dizi göstericisini döndüren bir fonksiyon örnekleri verilmiştir;

```

int* tekBoyutluDiziDondur() {
    /*... */
}
int** ikiBoyutluDiziDondur() {
    /*... */
}

```

Yerel değişkenlerin yığın bellekte (**stack segment**) tutulduğu ve fonksiyondan geri dönünce fonksiyon yerelindeki değişkenlerin kaybolduğu 7.5.1 başlığında ve Şekil 7.3’de anlatılmıştı. Bu nedenle bir fonksiyon içinde tanımlanan bir dizinin göstericisi geri döndürülecek ise veri bellekte (**data segment**) yer almasını sağlamak için **static** olarak tanımlanmak zorundadır.

```

#include <stdio.h> // printf() ve scanf()
#include <stdlib.h> // rand() ve srand()
#include <time.h> // time()
#define BOYUT 10
int* rastgeleOgrenciNotlari( ) {
    static int notlar[BOYUT]; // Fonksiyondan çıkıldığında bellekten kaldırılmaması için
    ↪ "static" olarak tanımlanmıştır!

    for (int i = 0; i < BOYUT; ++i)
        notlar[i] = rand()%101;
    return notlar;
}
void notlariYaz(int* pNotlar,int pUzunluk) {
    printf("Öğrenci Notları:\n");
    for (int i = 0; i < pUzunluk; ++i)
        printf("%4d", pNotlar[i]);
    return;
}
int main () {
    int *notlar;
    srand(time(NULL));
    notlar = rastgeleOgrenciNotlari();
    notlariYaz(notlar,BOYUT);
    return 0;
}
/*Program Çıktısı:
Öğrenci Notları:
30 31 13 40 41 37 34 83 93 76
*/

```

9.8 Fonksiyon Göstericisi

Geri çağırma (**callback**) fonksiyonları, özellikle **olay güdümlü programlamada** (**event-driven programming**) çok kullanışlıdır. Bir olay tetiklendiğinde, bu olaylara yanıt olarak ona eşlenen bir geri çağırma fonksiyonu çalıştırılır. Genellikle grafik ara yüzü işletim sistemlerinde kullanıcı ara yüzünde bir düğmeye tıklanması gibi bir olay, önceden tanımlanmış bir dizi işlemi başlatabilir. Bu durum, geri çağırma işlevleriyle gerçekleştirilir.

Geri çağırma fonksiyonu, temel olarak, başka bir koda argüman olarak geçirilen ve belirli bir zamanda argümanı geri çağırması beklenen bir icra edilebilir koddur. Geri çağırma mekanizması fonksiyon göstericisine bağlıdır. **Fonksiyon göstericisi** (**function pointer**), bir fonksiyonun bellek adresini depolayan bir değişkendir. Aşağıda buna ilişkin basit bir örnek bulunmaktadır;

```
#include <stdio.h>
void merhaba() {
    printf("Merhaba!\n");
}
void callback(void (*ptr)()) {
    printf("Geri çağırma fonksiyonu çağrılacak!\n");
    (*ptr)(); // Geri çağırma fonksiyonu çağrılıyor
}
main() {
    void (*ptr)() = merhaba;
    callback(ptr);
}
```

9.9 Düşün ve Çöz

- ◇ **Düşün:** "Bir gösterici aslında sadece bir tam sayıdır (adres)" ifadesi doğru mudur? Gösterici tipi (**int***, **char*** vb.) gösterici aritmetiğini nasıl etkiler?
- ◇ **Düşün:** **const int *p;** ile **int * const p;** tanımları arasındaki farkı bellek ve erişim yetkisi açısından açıklayınız.
- ◇ **Düşün:** Göstericiler ile diziler arasındaki adres-offset ilişkisini, dizi ismiyle gösterici aritmetiği yaparak açıklayınız.
- ◇ **Çöz:** Bir fonksiyon yazınız; bu fonksiyon iki tam sayının adresini parametre olarak alsın ve bu sayıların değerlerini bellekte yer değiştirsin.
- ◇ **Çöz:** Bir karakter dizisini (**string**) gösterici aritmetiği kullanarak tersten ekrana yazdıran bir fonksiyon yazınız.
- ◇ **Çöz:** Bir fonksiyona parametre olarak başka bir fonksiyonun adresini gönderen (**function pointer**) basit bir matematiksel işlem seçici (topla/çıkar) tasarlayın.

10. Bölüm

Tip Dönüşümleri

10.1 Üstü Kapalı Tip Dönüşümleri

C dilinde üstü kapalı veri tipi dönüşümü (**implicit type casting**), standart tip dönüşümüdür (**standart type casting**). Otomatik tip dönüşümü olarak da adlandırılır ve programcının açık talimatlar vermesine gerek kalmadan bir veri tipini otomatik olarak başka bir uyumlu tipe dönüştürmesi işlemidir. Şu durumlarda gerçekleşir;

- ◊ Dönüşüm farklı veri tiplerinin değerleri üzerinde gerçekleştirilir.
- ◊ Farklı bir veri tipi bekleyen bir fonksiyona bir argüman geçirmeniz halinde gerçekleşir.
- ◊ Bir veri tipinin değerini başka bir veri tipinin değişkenine atama durumunda gerçekleşir.

Otomatik tip dönüşümde, bellekte az yer kaplayan veri tipi ile tanımlanmış değişkenler, bellekte çok yer kaplayan değişkenlere atanmaya (**widening casting**) çalışıldığında otomatik olarak çok yer kaplayan değişkenin veri tipine dönüştürülür.

```
//...
char c;    //1 bayt
int i;     //4 bayt
long l;    //8 bayt
float f;   //4 bayt
double d;  //8 bayt
c=65;
i=c; //int veri tipine atanmadan önce; c değişkeninin değeri int veri tipine dönüştürülür.
l=i; //long veri tipine atanmadan önce; i değişkeninin değeri long veri tipine dönüştürülür.
f=12.50;
d=f; //double veri tipine atanmadan önce; f değişkeninin değeri double veri tipine dönüştürülür.
//...
```

Benzer şekilde otomatik tip dönüşümde, bellekte çok yer kaplayan veri tipinden tanımlanmış değişkenler, bellekte az yer kaplayan değişkenlere atanmaya (**narrowing casting**) çalışıldığında, tip dönüşümü otomatik olarak az yer kaplayan veri tipine dönüştürülür. **Ancak veri kaybı söz konusu olur.**

```
//...
char c;
int i;
float f;
i=4095; //i=0x00000FFF -> 0x00,0x00,0x0F,0xFF olarak 4 bayt
c=i;    /*c=0xFF -> c değişkenine int veri tipinden char veri tipine en anlamsız baytı atanır.*/

f=12.50; /* Kayan noktalı bir sayıdan tamsayıya atama yapılıyor ise tam kısmı tamsayıya çevrilir
↳ ve sonrasında atama yapılır. */
i=f;    //i=12
//...
```

10.2 Bilinçli Tip Dönüşümü

Bazen belli işlemleri daha kontrollü yapmak amacıyla programcı, işlenen veri tipini bir başka veri tipine kasıtlı olarak değiştirir. Bu tür dönüşümüne **bilinçli tür dönüşümü** (**explicit type casting**) denir ve **tekli tip dönüştürme işleci** (**unary cast operator**) ile yapılır. Bu işleç, diğer tekli işleçler gibi ifadenin (**expression**) önünde kullanılır ve parantez içerisinde dönüştürülmek istenen veri tipi yazılarak **int j=(int) 12.8/4;** şeklinde kodlanır. Bu işlecin önceliği, diğer +, -, !, --(**pre-decrement**), ++(**pre-increment**) gibi işleçlerle aynı önceliklidir. Aşağıda buna ilişkin bir örnek program verilmiştir;

```
#include<stdio.h>
int main() {
```



```

float x = 9.9;
int y = x+3.3;
int z = (int)(x)+3.3; // x değişkeni, int veri tipine dönüştürülmüş ve ardından toplama
→ yapılmıştır.

printf("x: %f\n",x); // x: 9.900000
printf("y: %d\n",y); // y: 13
printf("z: %d\n",z); // z: 12
return 0;
}

```

Jenerik gösterici (generic pointer) 9.2 başlığı altında berilmiştir. `void` veri tipi bellekte yer işgal etmemesine rağmen, `void*` jenerik göstericisi bellekte bir adresi tutacak kadar yer kaplar. Bir başka deyişle **`void*` bir adresi işaret eder ama işaret ettiği yerde hangi tipte veri olduğunu söylemez!** Bunun için bilinçli tip dönüşümü yapılması gerekir.

Aşağıdaki şekilde tanımlanır;

```

void* ptr;
void* genericPtr=NULL;
//...

```

```

int* intPtr=(int*) genericPtr; //void* göstericisi int* gösterisine dönüştürüldü.

```

10.3 Üstünlük ve zayıflıklar

Tür dönüşümünün üstünlükleri;

- ◊ **İşlemlerde Esneklik:** Farklı veri tiplerini içeren işlemleri gerçekleştirmede esneklik sağlar. Programcının verileri bir tipten diğerine açıkça dönüştürmesini sağlayarak karışık tip aritmetiğini ve diğer işlemleri kolaylaştırır.
- ◊ **Uyumluluk:** Farklı veri tipleri arasında uyumluluğun sağlanmasına yardımcı olur. Programcının verileri belirli bir bağlamda kullanmadan önce uyumlu bir tipe dönüştürmesini sağlayarak veri uyumsuzluğu hatalarını önler.
- ◊ **Hassasiyet Kontrolü:** Hassasiyet kontrolünün kritik olduğu durumlarda, programcının özellikle sayısal işlemlerde veri tipleri arasında dönüşüm yaparak istenen hassasiyeti açıkça belirtmesini sağlar.
- ◊ **Açıklık:** Tip dönüşümü, programcının veri tipini değiştirme niyetini belirterek kodu daha açık hale getirir. Bu, kod okunabilirliğini artırabilir ve işlenen veri tipiyle ilgili kafa karışıklığını azaltabilir.

Tür dönüşümünün zayıf yönleri;

- ◊ **Hassasiyet Kaybı:** Tip dönüştürmenin en büyük dezavantajlarından biri hassasiyet kaybı potansiyelidir. Örneğin, kayan noktalı bir sayıyı tam sayıya dönüştürürken kesirli kısım kesilir ve bu da bilgi kaybına yol açar.
- ◊ **Çalışma Zamanı Yükü:** Bilinçli tip dönüştürme genellikle program yürütme sırasında dönüştürmenin gerçekleştirilmesi gerektiğinden çalışma zamanı yüküne neden olur. Bu ek işlem, özellikle tip dönüştürmenin sık olduğu durumlarda performansı etkileyebilir.
- ◊ **Derleyici Uyarıları ve Hataları:** Yanlış veya güvenli olmayan tip dönüştürme, derleyici uyarılarına veya hatalarına yol açabilir. Örneğin, uyumsuz tipler arasında dönüştürme yapmaya çalışmak veya geçersiz dönüştürme sözdizimi kullanmak, hata ayıklaması zor olabilecek sorunlara yol açabilir.
- ◊ **Tanımsız Davranış Potansiyeli:** Bazı durumlarda, tip dönüştürme, özellikle uyumsuz tipler arasında dönüştürme yaparken veya dönüştürülen değer hedef tipin aralığının dışında olduğunda tanımsız davranışa yol açabilir. Bu, programda öngörülemez duruma neden olabilir.
- ◊ **Kod Bakım Zorlukları:** Bilinçli tip dönüşümüne kodun bakımı ve anlaşılması, özellikle karmaşık veya iç içe ifadelerde gerçekleştirildiğinde, daha zor hale gelebilir. Bu, artan kod karmaşıklığına ve azalan okunabilirliğe yol açabilir.
- ◊ **Taşınabilirlik Endişeleri:** Bilinçli tip dönüşümüne farklı platformlar veya derleyiciler arasında daha az taşınabilir olabilir. C'de tip dönüşümünün davranışı değişebilir ve belirli veri tipi boyutları hakkındaki varsayımlar tüm ortamlarda geçerli olmayabilir.

10.4 Düşün ve Çöz

- ◊ **Düşün:** Bilinçli tip dönüşümü yaparken veri kaybı yaşanabileceği durumları örneklendiriniz.
- ◊ **Düşün:** Jenerik gösterici (**generic pointer**) olan `void*` tipi neden tehlikelidir? Bu tipi kullanırken neden

"type casting" (tip dönüşümü) zorunludur?

- ◊ **Çöz:** Bir `double` sayıyı `int` tipine dönüştürdüğünüzde, ondalık kısmın nasıl kırıldığını ve veri kaybını gösteren bir test programı yazınız.

11. Bölüm

Numaralandırma ve Takma İsimler

11.1 Numaralandırma

C numaralandırması (**enumeration**), bir grup tam sayılardan oluşan (**integral**) değişmezden (**literal**) oluşan numaralandırılmış bir veri türüdür. Bir bütünün parçalarını tamsayı olarak ifade eden değişmezlere kullanıcı tanımlı adlar atamak istediğinizde kullanışlıdır. C dilinde numaralandırılmış sabitler **integral type** olarak adlandırılır ve **enum** anahtar kelimesiyle tanımlanırlar.

```
enum numaralandirmakimligi { DEĞER1, DEĞER2, ...};
```

Yukarıdaki sözde kodda **DEĞER1**, **DEĞER2** olarak belirtilenler aslında tam sayılara verilen isimlerdir. Yani tamsayı sabitleridir. Sabit olduklarından büyük harfle yazılırlar ve belirtilmedikçe ilkinin değeri 0, ikincisinin değeri 1, ... şeklinde ilerler. Örnek;

```
#include <stdio.h>
```

```
enum cinsiyetKodlari {ERKEK = 1, KADIN = 2, BELIRSIZ = 0};
```

```
/*Burada tanımlanan tamsayı sabitler aynı konuya (burada cinsiyet) ilişkin bir bütünün parçaları  
→ olan (integral) sabitlerdir.*/
```

```
enum renkKodlari {  
    SIYAH=0x000000, // Hiç belirtilmez ise 0  
    KIRMIZI=0xFF0000, // Hiç belirtilmez ise 1  
    YESIL=0x00FF00, // Hiç belirtilmez ise 2  
    MAVI=0x0000FF, // Hiç belirtilmez ise 3  
    BEYAZ=0xFFFFFF // Hiç belirtilmez ise 4  
};  
int main() {  
    // Sabitleri konsola yazıyoruz  
    printf("ERKEK = %d\n", ERKEK);  
    printf("KADIN = %d\n", KADIN);  
    printf("BELIRSIZ = %d\n", BELIRSIZ);  
    int cinsiyet;  
    puts("Cinsiyet Giriniz (0-1-2):");  
    scanf("%d",&cinsiyet);  
    switch (cinsiyet) {  
        case ERKEK: puts("ERKEK Girdiniz"); break;  
        case KADIN: puts("KADIN Girdiniz"); break;  
        case BELIRSIZ: puts("Cinsiyet Bilinmiyor"); break;  
    }  
    return 0;  
}
```

11.2 Takma İsimler

C Dilinde halihazırda mevcut veri tiplerinin adını yeniden tanımlamak ya da **takma isim** (**typedef**) verilebilir;

```
typedef mevcutisim takmaisim;
```

Aşağıda takma isim verilmiş yeni veri tipleriyle yazılmış bir kod örneği verilmiştir;

```
#include <stdio.h>  
typedef char karakter;  
typedef int tamsayi;  
typedef long long buyukolceklitamsayi;  
typedef unsigned int pozitifamsayi;  
int main() {  
    karakter k='A';  
    tamsayi t=12;
```

```

        buyukolceklitamsayi bot=0xFFFFFFFF;
        pozitiftamsayi pt=1;
        printf("k:%d veya %c\n",k,k);
        printf("t:%d\n", t);
        printf("bot:%ld\n",bot);
        printf("pt:%u\n",pt);
        return 0;
}

```

Tip tanımlama, daha sonra göreceğimiz **yapı** (**struct**) ve **birlik** (**union**) veri tiplerinin çokça kullanıldığı kodlarda kodun boyunu oldukça kısaltır. Bunların dışında fonksiyonlara da takma isim verilebilir;

```

typedef int Fonk(int); /* Fonk bir fonksiyona verilen bir takma isimdir; Fonk, int tipinde
→ parametre alan ve int geri döndüren her fonksiyon olabilir */

```

```

int faktoriyel(int n) { //faktoriyel fonksiyonu da Fonk tipindedir.
    return (n==1)? 1: n*faktoriyel(n-1);
}

```

```

int onKat(int i) {
    return i*10;
}

```

```

int main() {
    Fonk *fn; // fn bir Fonk tipinde olan fonksiyonlara olan göstericidir.
    fn = &faktoriyel; // faktoriyel fonksiyonunu göster
    fn(3); // 3! Hesaplanır
    fn = &onKat; // onKat fonksiyonunu göster
    fn(4); // 4*10 hesaplanır
}

```

11.3 Düşün ve Çöz

- ◇ **Düşün:** Numaralandırma kullanmanın, çok sayıda **#define** ön işlemcisine göre hata ayıklama (**debugging**) sürecindeki avantajı nedir?
- ◇ **Çöz:** Haftanın günlerini temsil eden bir **enum** yapısı ve bir **struct** yapısını daha kısa ifade etmek için bir **typedef** tanımı oluşturunuz.
- ◇ **Çöz:** **typedef** kullanarak karmaşık bir fonksiyon göstericisi (örneğin iki **int** alıp **int** dönen) için anlamlı bir takma isim oluşturunuz.

12. Bölüm

Dizgiler

12.1 Dizgi Tanımlama

C dilinde **dizgi** (**string**), dizideki son karakteri `'\0'` yani **boş karakter** (NULL character) olan, **char** veri tipinde tek boyutlu bir dizidir. Çoğunlukla metin içerirler;

```
//Karakter Dizisi olarak dizgi tanımı:
char metin1[]={ 'C','L','a','n','g','\0' };
char metin2[]={ 'C','L','a','n','g', 0 };
// metin1 ve metin2 dizgileri 6 karakterden oluşan dizi ile başlatılmıştır.
```

Konsola metin yazmak 5.2 başlığında metinlerin çift tırnak karakteri arasında harflerin yazılmasıyla oluştuğu anlatılmıştı. Yukarıdaki metni ifade eden dizgileri uzun uzun dizi şeklinde tanımlamak yerine çift tırnak karakteri kullanılır;

```
char metin[] = "CLang"; //Kısaca Dizgi Tanımı:
```

Bu tanımda çift tırnak karakterleri kullanılarak ilk değer olarak metin kimlikli dizgiye **"Clang"** **metin değişmezi** (**string literal**) atanmıştır. Aslında karakter dizisi olan bu metin aşağıdaki iki şekilde de tanımlanabilir; Dizgiler karakter göstericisi olarak da tanımlanır. Çünkü diziler göstericileri ifade etmesi 9.6 başlığında anlatılmıştı. Bu durumda gösterici dizinin ilk elemanını gösterir;

```
char* metin3="CLang";
```

12.2 Konsola Metin Yazma

Dizgiler, karakter dizileri olduğundan döngü kullanarak ekrana aşağıdaki şekilde yazabiliriz;

```
#include <stdio.h>
int main()
{
    char metin[] = "CLang";
    //For döngüsü ile
    for (int i = 0; i < 5; i++) {
        printf("%c", metin[i]);
    }
    printf("\n");
    //while döngüsü ile boş karakteri görene kadar
    char* ptrc=&metin[0]; //char *ptr3=metin3;
    while (*ptrc!=0)
        printf("%c",*ptrc++);
    printf("\n");
    return 0;
}
```

Konsola biçimlendirilmiş veri yazmak 5.2 başlığında anlatılmıştı. Dizgileri yukarıdaki gibi uzun uzadıya döngü yazarak konsola yazma yerine döngü kullanmadan **printf** fonksiyonunun biçimlendirme metni içinde **%s** **biçim belirleyicisi** (**format specifier**) kullanarak da yazdırabiliriz.

```
#include <stdio.h>
int main() {
    char metin[]="CLang";
    printf("%s\n", metin); //Clang
    char metin1[]={ 'C','L','a','n','g','\0' };
    printf("%s\n", metin1); //Clang
    char metin2[]={ 'C','L','a','n','g', 0 };
    printf("%s\n", metin2); //Clang
    char *metin3="CLang";
    printf("%s\n", metin3); //Clang
    return 0;
}
```

12.3 Klavyeden Metin Okuma

Klavyeden metin okunacak ise ilk önce okunacak karakter sayısına bağlı olarak bir dizgi tanımlanmalıdır; `char string[20];` //20 karakterlik "dizgi" kimlikli değişken tanımı.

Dizgileri klavyeden okumak için, `printf` fonksiyonundaki gibi, `scanf` fonksiyonlarında da `%s` biçim belirleyicisi kullanılır. Dizgi için belirlenen karakter sayısından fazlası okunduğunda belleğin istenmeyen bölgelerine ulaşılacağından okunacak metnin koyulacağı dizginin boyutuna dikkat edilmelidir. Okunan değerlerin konulacağı değişkenin adresi `scanf` fonksiyonunda kullanılır. Ancak dizgiler de karakter dizileri olduğundan ve diziler de gösterici gibi kullanıldıklarından, dizgi değişkeninin önünde `&` adres işleci (address operator) kullanılmaz.

```

1  #include <stdio.h>
2  #include <string.h>
3  int main (){
4      char adi[20];
5      char soyadi[20];
6      printf("Adınızı Giriniz:");
7      scanf("%s", adi);
8      printf("Soyadınızı Giriniz:");
9      scanf("%s", soyadi);
10     printf("Girdiğiniz değerler:");
11     printf("%s-%s\n", adi,soyadi);
12     printf("Adınızı ve Soyadınızı Giriniz:");
13     scanf("%s %s", adi, soyadi);
14     printf("Girdiğiniz değerler:");
15     printf("%s-%s\n", adi,soyadi);
16     char tamAdi[40];
17     printf("Adınızı ve Soyadınızı Tam Giriniz:");
18     scanf("%s", tamAdi);
19     printf("Girdiğiniz değer:");
20     printf("%s\n", tamAdi);
21     return 0;
22 }
23 /* Program Çıktısı
24 Adınızı Giriniz:Ilhan
25 Soyadınızı Giriniz:Ozkan
26 Girdiğiniz değerler:Ilhan-Ozkan
27 Adınızı ve Soyadınızı Giriniz:Ilhan Ozkan
28 Girdiğiniz değerler:Ilhan-Ozkan
29 Adınızı ve Soyadınızı Tam Giriniz:Ilhan OZKAN
30 Girdiğiniz değer:Ilhan
31 */

```

Yukarıdaki kodu incelediğimizde 18. satırda girilen değere ilişkin çıktı 29. satırda gösterilmiştir. Burada girilen "Ilhan Ozkan" metnine karşılık `tamAdi` değişkenine konulan değer sadece "Ilhan" olmuştur. Bu da 30. satırda görülmektedir. Bunu düzeltmenin yolu `fgets()` fonksiyonunu kullanmaktır. Bu fonksiyona dizginin boyutunu parametre olarak verirsiniz. Belirtilen boyuttan fazla karakter okumaz. Ayrıca sadece klavyeden (`stdin`) değil, dosyalardan da veri okuyabilir. İleride anlatılacaktır

```

#include <stdio.h>
int main() {
    char isim[20];
    // gets(isim);
    // gets TEHLİKELİ. Artık C standartlarında yok. Kullanıcı 20'den fazla karakter girerse
    //   ↪ program çöker/hacklenebilir.
    // GÜVENLİ YÖNTEM: fgets
    printf("Adınızı giriniz: ");
    // fgets(dizi_adı, maksimum_boyut, giriş_kaynağı)
    if (fgets(isim, sizeof(isim), stdin) != NULL) {
        printf("Merhaba, %s", isim);
    }
    return 0;
}

```

12.4 Çok Kullanılan Dizgi Fonksiyonları

C dili, dizgiler üzerinde çeşitli işlemler gerçekleştirmek için standart `string.h` başlığına sahiptir. Bu kütüphanedeki fonksiyonları kullanırken göstericilerin bir karakter dizinini işaret ettiği unutulmamalıdır.

Fonksiyon	Açıklama
<code>strcat()</code>	Bir dizgiyi diğerinin sonuna ekler
<code>strchr()</code>	Dizgi içinde aranan bir karakterin ilk bulunanına ait göstericiyi döndürür.
<code>strcmp()</code>	ASCII olarak iki dizgiyi karşılaştırır.
<code>strcpy()</code>	Hafızada yer alan iki dizgiden birinin içeriğini diğerine kopyalar.
<code>strlen()</code>	Dizginin karakter uzunluğunu verir.
<code>strrchr()</code>	Dizgi içinde aranan bir karakterin son bulunanına ait göstericiyi döndürür.
<code>strstr()</code>	Dizgi içinde aranan ikinci bir dizginin ilk bulunanına ait göstericiyi döndürür.
<code>strtok()</code>	Dizgiyi verilen ayraçlara bağlı olarak alt dizgilere böler

Tablo 12.1: Çok Kullanılan Dizgi Fonksiyonları

§12.4.1 strlen() fonksiyonu

Bu fonksiyon dizgideki karakter sayısını verir.

```
#include <stdio.h>
#include <string.h>
int main(){
    char *metin = "Merhaba";
    printf("Dizgi: %s\n", metin);
    printf("Dizgi Uzunluğu: %ld", strlen(metin));
    return 0;
}
```

§12.4.2 strcpy() fonksiyonu

Bu fonksiyon, ikinci parametredeki dizgiyi, birinci parametreye kopyalar. Burada dikkat edilmesi gereken husus; hedef gösterilen dizginin karakter sayısının veya göstericinin işaret ettiği bellek bölgesinin büyüklüğünün en az kaynak kadar olması gerektiridir.

```
#include <stdio.h>
#include <string.h>
int main(){
    char* ptrToDegimmezDizgi = "Ne Haber?";
    char bosDizgi[20]; //Üzerine kopyalanacak metnin karakter sayısı 20 kabul edilmiş!
    char* ptr;
    strcpy(bosDizgi, ptrToDegimmezDizgi);
    printf("%s\n", bosDizgi);
    //strcpy(ptr, ptrToDegimmezDizgi);
    /* HATA: ptr sadece adres tutan değişkendir.
    Şu an için hiçbir dizgiyi göstermiyor! */
    ptr = bosDizgi;
    strcpy(ptr, ptrToDegimmezDizgi);
    printf("%s", ptr);
    return 0;
}
```

Bu fonksiyonu kullanmak yerine kendimiz de dizgi kopyalayan fonksiyon yazabiliriz;

```
#include <stdio.h>
void copyString(char*, char*);
int main(){
    char str1[100] , str2[100]="Deneme Metni";
    printf("str1: %s \nstr2: %s\n", str1, str2);
    copyString(str1, str2);
    printf("str1: %s \nstr2: %s\n", str1, str2);
    return 0;
}
void copyString(char* hedef, char* kaynak){
    int i = 0;
    for(i = 0; kaynak[i]!='\0'; i++)
```



```

        hedef[i] = kaynak[i];
        hedef[i] = '\0';
    }

```

§12.4.3 strcat() fonksiyonu

Bu fonksiyon, parametre olarak girilen iki dizgiyi, birinci parametreye birleştirir.

```

#include <stdio.h>
#include <string.h>
int main() {
    char adi[] = "Ilhan";
    char* soyadi = "OZKAN";
    char adiSoyadi[50]; /* Birleştirme sonrası oluşacak metnin
    karakter sayısı 50 kabul edilmiş! */
    strcat(adiSoyadi, adi);
    strcat(adiSoyadi, " ");
    strcat(adiSoyadi, soyadi);
    printf("%s", adiSoyadi); // Çıktı: Ilhan OZKAN
    return 0;
}

```

§12.4.4 strchr() fonksiyonu

Bu fonksiyon, birinci parametre olarak verilen dizgide, ikinci parametrede verilen karakteri arar. Bulunamaz ise bu fonksiyon boş gösterici (NULL Pointer) döndürür.

```

#include <stdio.h>
#include <string.h>
int main() {
    char dizgi[] = "Merhaba!";
    char aranan='h';
    char* ptr = strchr(dizgi, aranan);
    if (ptr != NULL) {
        printf("'%' karakteri \"%s\" dizgisinde %ld indistedir.\n",
        *ptr, dizgi, ptr - dizgi);
    } else {
        printf("'%' karakteri \"%s\" dizgisinde bulunamadı.\n",
        aranan, dizgi);
    }
    return 0;
}

```

§12.4.5 strcmp() fonksiyonu

Bu fonksiyon, parametre olarak girilen iki dizginin, karakterlerini karşılıklı olarak karşılaştırır. Karşılaştırılan iki karakter birbirinden farklı ise ASCII kodu farklarını döndürür. Tüm karakterler aynı ise sıfır döndürür.

```

#include <stdio.h>
#include <string.h>
int main() {
    char str1[] = "ARMUT";
    char str2[] = "armut";
    int result = strcmp(str1, str2);

    if (result == 0) {
        puts("İki metin de eşit.");
    } else if (result < 0) {
        puts("Birinci metin ikincisinden küçük");
    } else {
        puts("İkinci metin birincisinden küçük");
    }
    return 0;
}
// Çıktı: Birinci metin ikincisinden küçük

```

Alternatif olarak karşılaştırma fonksiyonunu aşağıdaki gibi yazabiliriz;

```
#include <stdio.h>
int compareString(const char*, const char*);
int main() {
    char str1[] = "Elma";
    char str2[] = "Elma2";
    int result = compareString(str1, str2);
    if (result == 0)
        printf("İki Metin Aynıdır.\n");
    else if (result < 0)
        printf("Birinci Metin İkincisinden Küçüktür:%d\n", result);
    else
        printf("Birinci Metin İkincisinden Büyüktür:%d\n", result);
    return 0;
}
int compareString(const char* metin1, const char* metin2) {
    while (*metin1 == *metin2) {
        if (*metin1 == '\0') return (0);
        metin1++;
        metin2++;
    }
    return (*metin1 - *metin2);
}
// Birinci Metin İkincisinden Küçüktür:-50
```

12.5 Karakter Fonksiyonları

C dilinde, standart olan bir başka başlık dosyası olan `ctype.h` içinde karakter fonksiyonları bulunur. Bu başlıkta dizgileri oluşturan karakterler üzerinde işlem yapmak için kullanılır. Karakterler aşağıdaki şekilde gruplandırılmıştır;

- ♦ **Alfasayısal (alphanumeric)**: Hem rakam, hem de harf barındıran karakter kümesidir.
- ♦ **Beyaz boşluk**: boşluk (' '), yeni satır ('\n'), satır başı ('\r'), alt satır ('\v'), ve tab ('\t') karakterlerinin oluşturduğu kümedir.
- ♦ **Onaltılık Sayı Rakamları (hexadecimal digit)**: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f

Aşağıda bu başlık dosyasında yer alan birçok fonksiyon yer almaktadır;

Fonksiyon	Açıklama
<code>int isalnum(int c)</code>	Verilen karakter kodunun alfa sayısal olup olmadığını kontrol eder.
<code>int isalpha(int c)</code>	Verilen karakter kodunun harf olup olmadığını kontrol eder.
<code>int iscntrl(int c)</code>	Verilen karakter kodunun kontrol karakteri olup olmadığını kontrol eder.
<code>int isdigit(int c)</code>	Verilen karakter kodunun rakam olup olmadığını kontrol eder.
<code>int isgraph(int c)</code>	Verilen karakter kodunun konsola yazdırılabilir karakteri olup olmadığını kontrol eder.
<code>int islower(int c)</code>	Verilen karakter kodunun küçük harf olup olmadığını kontrol eder.
<code>int isprint(int c)</code>	Verilen karakter kodunun boşluk dahil konsola yazılabilir olup olmadığını kontrol eder.
<code>int ispunct(int c)</code>	Verilen karakter kodunun noktalama işareti olan karakterler olup olmadığını kontrol eder.
<code>int isspace(int c)</code>	Verilen karakter kodunun beyaz boşluk olup olmadığını kontrol eder.
<code>int isupper(int c)</code>	Verilen karakter kodunun büyük harf olup olmadığını kontrol eder.
<code>int isxdigit(int c)</code>	Verilen karakter kodunun onaltılık sayı rakamı olup olmadığını kontrol eder.
<code>int isblank(int c)</code>	Verilen karakter kodunun boşluk karakteri olup olmadığını kontrol eder.
<code>int toupper(int c)</code>	Verilen karakterin büyük harfinin kodunu döner.
<code>int tolower(int c)</code>	Verilen karakterin küçük harfinin kodunu döner.

Tablo 12.2: Karakter Fonksiyonları

Bu fonksiyonlar kullanılarak küçük-büyük harf ayrımı yapmadan metinleri karşılaştıran bir program örneği verilmiştir;

```
#include <stdio.h>
```

```

#include <string.h>
#include <ctype.h>
int compareString(char*, char*);
int main() {
    char str1[] = "Elma";
    char str2[] = "elma";
    int result = compareString(str1, str2);
    if (result == 0)
        printf("İki Metin Aynıdır.\n");
    else if (result < 0)
        printf("Birinci Metin İkincisinden Küçüktür.\n");
    else
        printf("İkinci Metin Birincisinden Küçüktür.\n");
    return 0;
}
int compareString(char* hedef, char* kaynak){
    int i = 0;
    for (i = 0; hedef[i]; i++) //hedefi küçük harfe çevir.
        hedef[i] = tolower(hedef[i]);
    for (i = 0; kaynak[i]; i++) //kaynağı küçük harfe çevir.
        kaynak[i] = tolower(kaynak[i]);
    return strcmp(hedef, kaynak);
}

```

12.6 Metinden İlkel Veri Tiplerine Dönüşüm

Standart başlık dosyalarından olan `stdlib.h` başlık dosyasında daha önce işlediğimiz rastgele fonksiyonların yanında metinden ilkel veri tiplerine dönüşüm sağlayan bazı fonksiyonlar da bulunur.

Fonksiyon	Açıklama
<code>double atof(const char*)</code>	Dizgi olarak verilen metni double tipine dönüştürür ve geri döndürür.
<code>int atoi(const char*)</code>	Dizgi olarak verilen metni int tipine dönüştürür ve geri döndürür.
<code>long atol(const char*)</code>	Dizgi olarak verilen metni long tipine dönüştürür ve geri döndürür.
<code>double strtod(const char*, char**)</code>	Dizgi olarak verilen metni double tipine dönüştürür ve geri döndürür. İkinci parametre ise metinde kayan noktalı sayıya çevrilmeyen ilk karakterin göstericisidir.
<code>long strtol(const char*, char**, int)</code>	Dizgi olarak verilen metni long tipine dönüştürür ve geri döndürür. İkinci parametre ise metinde kayan noktalı sayıya çevrilmeyen ilk karakterin göstericisidir. Üçüncü parametre ise metnin hangi sayı tabanında sayı içerdiğidir. Bu 2, 8, 10, 16 olabilir.

Tablo 12.3: Metinden İlkel Veri Tiplerine Dönüşüm Fonksiyonları

Aşağıda bu fonksiyonları içeren bir program örneği verilmiştir.

```

#include <stdio.h>
#include <stdlib.h>
int main() {
    char tamsayiMetin[]="-23234";
    char kayannoktalisayiMetin[]="+2.3234e-1";
    char metin[]="-12,3e-2'nin karesi";
    int i = atoi(tamsayiMetin);
    printf("Tamsayı: %d\n",i); // Tamsayı: -23234
    double d= atof(kayannoktalisayiMetin);
    printf("Kayan Noktalı Sayı: %lf\n",d);
    // Kayan Noktalı Sayı: 0.232340
    double dm;
    char* gerikalanmetin;
    dm= strtod(metin,&gerikalanmetin);
    printf("Metinden Alınan Sayı: %lf\n",dm);
}

```

```

// Metinden Alınan Sayı: -12.000000
printf("Geri Kalan Metin: %s\n",gerikalanmetin);
// Geri Kalan Metin: ,3e-2'nin karesi
return 0;
}

```

12.7 Arama Tabloları

Arama tabloları LUT (**lookup tables**), önceden hesaplanmış belirli değerlerle doldurulmuş dizilerdir. Uzun iç içe **if**, **if-else** veya **switch** ifadeleri yerine, bir programının verimliliğini artırmak için bu tablolar kullanılabilir.

```

int kareler[10] = {1, 4, 9, 16, 25, 36, 49, 64, 81, 100};
float karekokler[5] = {1.0, 1.41421, 1.7320, 2.0, 2.2360};
char* renkler[] = {"Beyaz", "Mavi", "Yeşil", "Kırmızı", "Siyah"};

```

Örnek olarak verilen bu tablolarda her seferinde hesaplama yapmak yerine önceden hesaplanmış değerleri tutarız. Arama tabloları genellikle dizgilerden oluşur. Aşağıda örnek bir program verilmiştir;

```

#include <stdio.h>
enum renlerenum {BEYAZ,MAVI,YESIL,KIRMIZI,SIYAH};
int main() {
    int kareler[10] = {1, 4, 9, 16, 25, 36, 49, 64, 81, 100};
    for (int i = 0; i < 10; i++){
        printf("%d \tKaresi(%d): %d\n", i+1, i+1, kareler[i]);
    }
    float karekokler[5] = {1.0, 1.41421, 1.7320, 2.0, 2.2360};
    for (int i = 0; i < 5; i++){
        printf("%d \tKarekökü(%d): %0.4f\n", i+1, i+1, karekokler[i]);
    }
    char* renkler[] = {"Beyaz", "Mavi", "Yeşil", "Kırmızı", "Siyah"};
    int secim=YESIL;
    printf("Seçtiğiniz renk: %s",renkler[YESIL]);
    return 0;
}
/*Program Çıktısı:
1      Karesi(1): 1
2      Karesi(2): 4
3      Karesi(3): 9
4      Karesi(4): 16
5      Karesi(5): 25
6      Karesi(6): 36
7      Karesi(7): 49
8      Karesi(8): 64
9      Karesi(9): 81
10     Karesi(10): 100
1      Karekökü(1): 1.0000
2      Karekökü(2): 1.4142
3      Karekökü(3): 1.7320
4      Karekökü(4): 2.0000
5      Karekökü(5): 2.2360
Seçtiğiniz renk: Yeşil
*/

```

12.8 Düşün ve Çöz

- ◊ **Düşün:** C dilinde dizgilerin sonunda bulunan boş karakter (**NULL Character**) karakterinin fonksiyonlar (**strlen**, **strcpy** vb.) için önemi nedir?
- ◊ **Düşün:** **gets()** fonksiyonunun C standartlarından (C11) neden kaldırıldığını ve güvenli alternatifinin ne olduğunu araştırınız.
- ◊ **Düşün:** **char s[] = "kitap";** ile **char *s = "kitap";** tanımlarının bellekte (stack vs read-only data) tutulma farkını açıklayınız.
- ◊ **Çöz:** **strcat** fonksiyonunu kullanmadan, iki ayrı dizgiyi üçüncü bir dizgiye birleştiren kendi fonksiyonunuzu yazınız.

- ◊ **Çöz:** Kullanıcının girdiği bir cümlenin içindeki kelime sayısını (boşlukları sayarak) bulan bir fonksiyon yazınız.
- ◊ **Çöz:** Standart kütüphanedeki `strcmp` fonksiyonunu kullanmadan, iki dizgiyi karşılaştırıp eşit olup olmadıklarını dönen kendi fonksiyonunuzu yazınız.

13. Bölüm

Yapılar ve Birlikler

13.1 Yapılar

Yapısal programlamaya adını veren yapı (**structure**), türetilmiş (**derived**) veya kullanıcı tanımlı (**user defined**) bir veri tipidir. Farklı tiplerdeki elemanları bir arada gruplandıran özel bir veri tipi tanımlamak için **struct** anahtar kelimesini kullanırız. Yapı bir veya birden fazla ilkel veri tipinin (**char**, **int**, **long**, **float**, **double**, ...) ve bu tiplere ait dizilerin bir araya gelmesiyle oluşturulan yeni veri tipleridir.

Diziler, aynı tipte elemanlardan oluşmasına rağmen, yapılar farklı dipte elemanların bir araya gelmesiyle oluşabilir. Aşağıda sözde kodu verilmiştir;

```
struct yapı-kimliği {
    veri-tipi yapı-elemanı-kimliği1;
    veri-tipi yapı-elemanı-kimliği1;
    ...
} [değişken-kimliği1][, değişken-kimliği2];
```

Örnek olarak **adı**, **soyadı**, **numarası**, **yaşı**, **cinsiyeti** gibi farklı öğeler ile bir **ogrenci** yapısı tanımlanabilir;

```
struct ogrenci {
    char adi[50];
    char soyadi[0];
    int yas;
    int cinsiyet;
} ogrenci1;
```

Yukarıda yapılan bu tanımlama ile **struct ogrenci** veri tipine **ogrenci1** kimliğine sahip bir değişken tanımlanmıştır. Yapı tanımlandıktan sonra aşağıdaki sözde kodda verildiği gibi yeni değişkenler tanımlanabilir;

```
struct yapı-kimliği değişken-kimliği;
```

Yukarıda tanımlanan **ogrenci** yapısından **ogrenci2** ve **ogrenci3** kimlikli birçok değişken tanımlanabilir;

```
struct ogrenci ogrenci2,ogrenci3;
```

Yapının elemanlarına tanımlama sırasında **başlangıç değeri** (**initial value**) verilebilir yani yapı değişkeni bazı değerlerle **başlatılabilir** (**initialize**). Bunun için diziler gibi süslü parantez kullanılır;

```
struct ogrenci ogrenci4={"Deniz", "AK", 35, 2};
struct ogrenci ogrenci5={"Damla", "YILDIZ", 28, 2};
```

Bir yapının içinde bir başka yapı da tanımlanabilir. Yani iç içe yapılar (**nested structs** tanımlanabilir.

13.2 Yapı Elemanlarına Erişim

Tanımlanan yapı değişkenleri üzerinden her bir alamana **nokta işleci** (**dot operator**) erişilir. Bu işlecin önceliği parantez ile aynıdır. Atama işleci (=) bir yapıyı doğrudan kopyalamak için kullanılabilir. Ayrıca, bir yapının üyesinin değerini başka birine atamak için atama işlecini de kullanabiliriz.

```
#include <stdio.h>
#include <string.h>
struct ogrenci {
    char adi[50];
    char soyadi[50];
    int yas;
    int cinsiyet;
} ogrenci1;
int main() {
    /* ogrenci1 yapısı başlatılmadığından aşağıdaki gibi değerleri güncelleniyor: */
    strcpy(ogrenci1.adi, "Ilhan");
    strcpy(ogrenci1.soyadi, "OZKAN");
```

```

ogrenci1.yas=50;
ogrenci1.cinsiyet=1;
printf("Adı:%s\nSoyad:%s\nYaşı:%d\nCinsiyeti:%d\n", ogrenci1.adi, ogrenci1.soyadi,
    ↪ ogrenci1.yas, ogrenci1.cinsiyet);
struct ogrenci ogrenci2={"Deniz","AK",35,2}; /* yapı bazı değerlerle başlatıldı. */
printf("Adı:%s\nSoyad:%s\nYaşı:%d\nCinsiyeti:%d\n",ogrenci2.adi, ogrenci2.soyadi,
    ↪ ogrenci2.yas, ogrenci2.cinsiyet);
struct ogrenci ogrenci3=ogrenci2; // yapı kopyalama
printf("Adı:%s\nSoyad:%s\nYaşı:%d\nCinsiyeti:%d\n", ogrenci3.adi, ogrenci3.soyadi,
    ↪ ogrenci3.yas, ogrenci3.cinsiyet);
return 0;
}

```

13.3 Yapı Göstericileri

Yapı göstericileri, tıpkı diğer değişkenlere gösterici tanımladığımız gibi tanımlayabiliriz. Gösterici üzerinden yapı değişkenlerine **dolaylı işleç (indirection operator)** yani (->) ile erişilir. Bu nokta işleci ile aynı önceliklidir.

Yapılar ve göstericileri; veri tabanları, dosya yönetim uygulamaları ve ağaç ve bağlı listeler gibi karmaşık veri yapılarını işlemek gibi farklı uygulamalarda kullanılır.

```

#include <stdio.h>
#include <string.h>
struct ogrenci {
    char adi[50];
    char soyadi[50];
    int yas;
    int cinsiyet;
};
int main() {
    struct ogrenci ogrenci1;
    strcpy(ogrenci1.adi, "Ilhan");
    strcpy(ogrenci1.soyadi, "OZKAN");
    ogrenci1.yas=50;
    ogrenci1.cinsiyet=1;
    struct ogrenci* ogrenciGosterici;
    ogrenciGosterici=&ogrenci1;
    printf("Adı:%s\nSoyad:%s\nYaşı:%d\nCinsiyeti:%d", ogrenciGosterici->adi,
        ↪ ogrenciGosterici->soyadi, ogrenciGosterici->yas,ogrenciGosterici->cinsiyet);
    return 0;
}

```

13.4 Yapılar ve Fonksiyonlar

Sıradan dizilere benzer şekilde bir yapı dizisi tanımlayabiliriz. Ayrıca yapı değişkenini bir fonksiyona parametre olarak gönderebilir ve bir fonksiyondan bir yapı döndürebilirsiniz;

```

#include <stdio.h>
#include <string.h>
enum cinsiyet {BELIRTILMEMIS,KADIN,ERKEK};
struct ogrenci {
    char adi[50];
    char soyadi[50];
    int yas;
    int cinsiyet;
};
struct ogrenci yeniOgrenci();
void ogreciYaz(struct ogrenci);
int main() {
    struct ogrenci ogrenciler[30];
    struct ogrenci o=yeniOgrenci();
    ogreciYaz(o);
    return 0;
}

```



```

struct ogrenci yeniOgrenci() {
    struct ogrenci yeni={"Ilhan","Ozkan",50,ERKEK};
    return yeni;
}
void ogrenciYaz(struct ogrenci pOgrenci) {
    printf("Adı:%s\nSoyad:%s\nYaşı:%d\nCinsiyeti:%d\n", pOgrenci.adi, pOgrenci.soyadi,
        ↪ pOgrenci.yas, pOgrenci.cinsiyet);
}

```

Yapı elemanları değer tipler olduğundan, yapılara ait göstericileri parametre olarak kullanmak, olabilecek değişiklikleri aktarmak için üstünlük sağlar;

```

#include <stdio.h>
#include <string.h>
enum cinsiyet {BELIRTILMEMIS,KADIN,ERKEK};
struct ogrenci {
    char adi[50];
    char soyadi[50];
    int yas;
    int cinsiyet; //1:erkek,2:Kadın,0:Belirtmiyor
};
struct ogrenci yeniOgrenci();
void ogrenciOku(struct ogrenci*);
void ogrenciYaz(struct ogrenci*);
int main() {
    struct ogrenci o=yeniOgrenci();
    ogrenciYaz(&o);
    ogrenciOku(&o);
    ogrenciYaz(&o);
    return 0;
}
struct ogrenci yeniOgrenci() {
    struct ogrenci yeni={"Ilhan","Ozkan",50,ERKEK};
    return yeni;
}
void ogrenciOku(struct ogrenci* pOgrenci) {
    printf("Adı Giriniz:");
    scanf("%s",pOgrenci->adi);
    printf("Soyadı Giriniz:");
    scanf("%s",pOgrenci->soyadi);
    printf("Yaş Giriniz:");
    scanf("%d",&(pOgrenci->yas));
    printf("Cinsiyet Giriniz (0-1-2):");
    scanf("%d",&(pOgrenci->cinsiyet));
}
void ogrenciYaz(struct ogrenci* pOgrenci) {
    printf("Adı:%s\nSoyad:%s\nYaşı:%d\nCinsiyeti:%d\n", pOgrenci->adi, pOgrenci->soyadi,
        ↪ pOgrenci->yas, pOgrenci->cinsiyet);
}

```

13.5 Anonim Yapılar

Anonim yapı, kimlik veya **takma isim** (**typedef**) ile tanımlanmayan bir yapı tanımıdır. Genellikle başka bir yapının içine yerleştirilir. C11 sürümü ile kullanılmaya başlanan bu özelliğin aşağıda sıralanan üstünlükleri vardır;

- ◊ **Esneklik (flexibility)**: Anonim yapılar, verilerin nasıl temsil edildiği ve erişildiği konusunda esneklik sağlayarak daha dinamik ve çok yönlü veri yapılarına olanak tanır.
- ◊ **Kolaylık (convenience)**: Bu özellik, farklı veri tiplerini tutabilen bir değişkenin kompakt bir şekilde temsil edilmesine olanak tanır.
- ◊ **Başlatma Kolaylığı (easy of initialization)**: Yapı değişkenine ilişkin ek kimliklendirme yapılmadan ilk değer verilmeleri ve kullanılmaları daha kolay olabilir.

İç içe yapıların (**nested structs**) bir örneği olan anonim yapılara, **ogrenci** yapımıza doğum tarihini içerecek anonim yapı eklenen örnek, aşağıda verilmiştir;

```

#include <stdio.h>
#include <string.h>
struct ogrenci {
    char adi[50];
    char soyadi[50];
    int yas;
    int cinsiyet; //1:erkek,2:Kadın,0:Belirtmiyor
    struct { //anonim yapı: yapı kimliği yoktur!
        int gun;
        int ay;
        int yil;
    };
};
struct ogrenci yeniOgrenci();
void ogrenciYaz(struct ogrenci);
int main() {
    struct ogrenci o=yeniOgrenci();
    ogrenciYaz(o);
    return 0;
}
struct ogrenci yeniOgrenci() {
    struct ogrenci yeni={"Ilhan","Ozkan",50,1, {1,1,1970}};
    return yeni;
}
void ogrenciYaz(struct ogrenci pOgrenci) {
    puts(pOgrenci.adi);
    puts(pOgrenci.soyadi);
    printf("Yas:%d\n",pOgrenci.yas);
    printf("Cinsiyet:%d\n",pOgrenci.cinsiyet);
    printf("Dogum Tarihi:%d-%d-%d\n",pOgrenci.gun,pOgrenci.ay,pOgrenci.yil);
}

```

13.6 Öz Referanslı Yapılar

Öz referanslı yapı (**self-referential struct**), öğelerinden bir veya daha fazlası kendi tipindeki göstericilerden oluşan bir yapıdır. Öz referanslı yapılar, bağlantılı listeler ve ağaçlar gibi karmaşık olan **dinamik veri yapıları** (**dynamic data structure**) içinde yaygın olarak kullanılırlar.

```

#include <stdio.h>
struct ogrenci {
    char adi[10]; char soyadi[10]; int yas;
    struct ogrenci* sonrakiOgrenci; // Kendi veri tipinden bir yapı göstericisi!
};
void ogrecileriYaz(struct ogrenci*);
int main() {
    struct ogrenci ilk={"Ilhan","OZKAN",50,NULL};
    struct ogrenci sonraki={"Ayse","YILMAZ",40,NULL};
    ilk.sonrakiOgrenci=&sonraki;
    struct ogrenci birsonraki={"Tahsin","BULUT",35,NULL};
    sonraki.sonrakiOgrenci=&birsonraki;
    ogrecileriYaz(&ilk);
    return 0;
}
void ogrecileriYaz(struct ogrenci* pIlk) {
    int sayac=0;
    while (pIlk!=NULL) {
        printf("OGRENCI %d: Adı:%s Soyad:%s Yaş:%d\n", ++sayac, pIlk->adi, pIlk->soyadi,
            ↪ pIlk->yas);
        pIlk=pIlk->sonrakiOgrenci;
    }
}
/*Program Çıktısı:
OGRENCI 1: Adı:Ilhan Soyad:OZKAN Yaş:50

```

```
OGRENCI 2: Adı:Ayşe Soyad:YILMAZ Yaş:40
OGRENCI 3: Adı:Tahsin Soyad:BULUT Yaş:35
*/
```

13.7 Yapı Dolgusu

C dilinde **yapı dolgusu** (**structure padding**), işlemci (CPU) mimarisi ile belirlenir. Dolgu işlemi ile üyelerinin bellekte doğal olarak hizalanması için belirli sayıda boş bayt eklenir. Bunun nedeni 32 veya 64 bitlik bir bilgisayarda işlemcinin tek seferde bellekten 4 bayt okumasından kaynaklanmaktadır.

```
#include <stdio.h>
struct yap1 {
    char a;
    char b;
    int c;
};
struct yap2 {
    char d;
    int e;
    char f;
};
int main() {
    printf("char bellek miktarı: %d\n", sizeof(char));
    // char bellek miktarı: 1
    printf("int bellek miktarı: %d\n", sizeof(int));
    // int bellek miktarı: 4
    printf("-----\n");
    printf("Yapılardaki 2 adet char ve 1 adet int olması gereken bellek miktarı : %d\n",
        ↪ 2*sizeof(char)+sizeof(int));
    // yapılar için olması gereken bellek miktarı: 6
    printf("yap1 için bellekte ayrılan miktar: %d\n", sizeof(struct yap1));
    // yap1 için bellekte ayrılan miktar: 8
    printf("yap2 için bellekte ayrılan miktar: %d\n", sizeof(struct yap2));
    // yap2 için bellekte ayrılan miktar: 12
    return 0;
}
/* Program Çıktısı:
char bellek miktarı: 1
int bellek miktarı: 4
-----
Yapılardaki 2 adet char ve 1 adet int olması gereken bellek miktarı : 6
yap1 için bellekte ayrılan miktar: 8
yap2 için bellekte ayrılan miktar: 12
*/
```

Yukarıda verilen örnekte aynı bellek miktarına sahip elemanlar için yapının bütününe bakıldığında farklı miktarda bellek ayrıldığı anlaşılmaktadır. Dolgu (**padding**) işlemi ile üyelerinin bellekte doğal olarak hizalanması için belirli sayıda boş bayt eklenir. **Kod içerisindeki tanımlanacak tüm yapılar için bunu engellemenin yolu** aşağıdaki ön işlemci yönergesini (**preprocessor directive**) kaynak koda eklemektir;

```
#pragma pack(1)
```

Eğer yalnızca belirlenen bir yapı için bunun yapılması isteniyorsa yapı tanımına `__attribute__((packed))` özelliği eklenir. Yapı dolgusuna ilişkin program örneği aşağıda verilmiştir;

```
#include <stdio.h>
#pragma pack(1) // Tüm yapılarda dolgu yapılsın!
struct yap1 {
    char a;
    char b;
    int c;
};
struct __attribute__((packed)) yap2 { //Sadece Bu yapıda dolgu yapılsın!
    char a;
    int b;
};
```

```

    char c;
};
int main() {
    printf("char bellek miktarı: %d\n", sizeof(char));
    // char bellek miktarı: 1
    printf("int bellek miktarı: %d\n", sizeof(int));
    // int bellek miktarı: 4
    printf("-----\n");
    printf("Yapılardaki 2 adet char ve 1 adet int olması gereken bellek miktarı: %d\n",
        ↪ 2*sizeof(char)+sizeof(int));
    // yapılar için olması gereken bellek miktarı: 6
    printf("yapi1 için bellekte ayrılan miktar: %d\n", sizeof(struct yapil));
    // yapil için bellekte ayrılan miktar: 6
    printf("yapi2 için bellekte ayrılan miktar: %d\n", sizeof(struct yapil2));
    // yapil için bellekte ayrılan miktar: 6
    return 0;
}
/* Program Çıktısı:
char bellek miktarı: 1
int bellek miktarı: 4
-----
Yapılardaki 2 adet char ve 1 adet int olması gereken bellek miktarı: 6
yapi1 için bellekte ayrılan miktar: 6
yapi2 için bellekte ayrılan miktar: 6
*/

```

13.8 Birlikler

Birlikler (union), Pascal dilindeki **record-case** talimatına benzer. Birlik, yapı (**struct**) gibi tanımlanır. Aralarındaki fark **yapı elemanlarının her birine ayrı bellek bölgesi ayrılırken, birlik üyelerinin her biri aynı bellek bölgesini paylaşırlar**. Birlik tanımına ilişkin sözde kod aşağıda verilmiştir;

```

union yapı-kimliği {
    veri-tipi yapı-elemanı-kimliği1;
    veri-tipi yapı-elemanı-kimliği1;
    ...
} [değişken-kimliği1][, değişken-kimliği2];

```

Birliğin bellek boyutu, elemanlarından en fazla bellek kaplayanı kadardır.

Aşağıda üyelerinin aynı bellek bölgesini paylaştığını gösteren program örneği bulunmaktadır;

```

#include <stdio.h>
union tamsayiBirliigi {
    char baytlar[4];
    int tamsayi;
    char bayt;
    short int kisatamsayi;
} birlik1,birlik2;
int main() {
    union tamsayiBirliigi birlik3,birlik4;
    birlik1.tamsayi=0x1B2C3D4F; //16lık sayılarda her çift rakam bir bayt olur
    printf("sizeof tamsayiBirliigi.baytlar: %d\n", sizeof birlik1.baytlar); //4
    printf("sizeof tamsayiBirliigi.bayt: %d\n", sizeof birlik1.bayt); //1
    printf("sizeof tamsayiBirliigi.tamsayi: %d\n", sizeof birlik1.tamsayi); //4
    printf("sizeof tamsayiBirliigi.kisatamsayi: %d\n", sizeof birlik1.kisatamsayi); //2
    printf("sizeof tamsayiBirliigi: %d\n", sizeof birlik1); //4
    printf("birlik1.tamsayi:%x\n",birlik1.tamsayi); //1b2c3d4f
    printf("birlik1.baytlar:%x-%x-%x-%x\n", birlik1.baytlar[0], birlik1.baytlar[1],
        ↪ birlik1.baytlar[2],birlik1.baytlar[3]); //4f-3d-2c-1b
    printf("birlik1.bayte:%x\n",birlik1.bayt); //4f
    printf("birlik1.kisatamsayi:%x\n",birlik1.kisatamsayi); //3d4f
}

```

```
    birlik1.bayt=0x00; //bayt değişkeni değiştirildiğinde diğer tüm üyelerin içeriği değişir!  
    ↪ Tamsayının son iki hanesine dikkat ediniz!  
    printf("birlik1.tamsayi:%x\n",birlik1.tamsayi); //1b2c3d00  
    return 0;  
}
```

13.9 Düşün ve Çöz

- ◇ **Düşün:** Yapı Dolgusu kavramını açıklayınız. İşlemcinin veriye erişim hızı ile bellek tasarrufu arasındaki denge bu noktada nasıl kurulur?
- ◇ **Düşün:** Birlik (**union**), neden aynı anda sadece bir üyenin değerini tutabilir? Bellek paylaşımı mantığını bir şema ile açıklayınız.
- ◇ **Düşün:** Bit alanları (**bit-fields**) kullanımı, düşük seviyeli donanım kontrolünde (**register**) neden vazgeçilmezdir?
- ◇ **Çöz:** Bir öğrencinin adını, numarasını ve 3 dersinin notunu tutan bir **struct** tanımlayınız. Bu yapıdan bir dizi oluşturup sınıf ortalamasını hesaplayınız.
- ◇ **Çöz:** İç içe yapılar (**nested structs**) kullanarak bir "Şirket" yapısı ve bu yapının içinde "Departman" ve "Çalışan" bilgilerini tutan bir model oluşturunuz.
- ◇ **Çöz:** Bir **union** kullanarak, 4 baytlık bir **int** sayının bellekteki her bir baytına karakter göstericisiyle erişen bir program yazınız.

14. Bölüm

Dinamik Bellek Yönetimi

14.1 Bellek Tahsisi ve Serbest Bırakma

C yapısal bir dil olduğundan, programlama için bazı sabit kurallara sahiptir. Örneğin bir dizinin boyutunu tanımlandıktan sonra değiştirmesiniz.

Dinamik bellek yönetimi ile çalışma anında istenilen boyuta göre dizi oluşturulabilir. Çalıştırma anında (**run time**) istenilen boyutta oluşturulabilen bu dizileri de göstericiler sayesinde işleriz. Göstericilerin (**pointer**) bellek adreslerini tutan değişkenler olduğu 9 başlığında açıklanmıştır.

Dinamik olarak tahsis edilen bellekteki veriler **öbek bellekte** (**heap segment**) tutulur. Yerel değişkenlerin (**local variable**) yığın bellekte (stack segment) tutulduğu, statik ve evrensel (**global variable**) değişkenlerin de veri bellekte (**data segment**) tutulduğu 7.5 başlığında anlatılmıştı.

Çalıştırma anında öbek bellekten (**heap segment**) yapılan bellek tahsisine, **dinamik bellek tahsisi** (**dynamic memory allocation**) denir. Bellek tahsisine ilişkin fonksiyonlar **stdlib.h** başlık dosyasında bulunmaktadır. Bu başlıktaki **malloc()**, **calloc()** ve **realloc()** fonksiyonları kullanılarak dinamik bellek tahsisi yapılır. **free()** Fonksiyonu ile tahsis edilen bellek bölgesi serbest bırakılır.

§14.1.1 malloc() fonksiyonu

Bu fonksiyon, **belirtilen boyutta tek bir büyük bellek bloğunu** bayt cinsinden dinamik olarak tahsis etmek için kullanılır. Bu fonksiyon herhangi bir biçimdeki bir göstericiye dönüştürülebilen **void*** jenerik gösterici (**generic pointer**) döndürür. İstenilen veri tipine uygun gösterici olarak kullanılması için bilinçli tip dönüşümü (**explicit type casting**) yapılması gerekir. Aşağıda fonksiyonun prototipi verilmiştir;

```
void* malloc(size_t size);
```

Bu fonksiyon, aşağıdaki şekilde öbek bellekten yer ayrılır;

```
ptr = (dönüştürülecek-veri-tipi*) malloc(byte-size);
```

Buna ilişkin bir örnek aşağıda verilmiştir;

```
int* ptrToInt;
ptrToInt = (int*) malloc(50 * sizeof(int)); /* 50 adet int içerecek bellek bloğu tahsis edilerek
→ göstericisi ptrToInt değişkenine atandı. Bellek ayrılmasaydı boş gösterici (NULL pointer)
→ geri döndürecekti.*/
```

```
float* ptrToFloat= (float*) malloc(10 * sizeof(float)); /* 10 adet float içerecek bellek bloğu
→ tahsis edilerek göstericisi ptrToFloat değişkenine atandı. */
```

§14.1.2 calloc() fonksiyonu

Bu fonksiyon, **belli bir veri tipi için belirtilen sayıda bellek bloğunu birbirine bitişik olarak** (**contiguous**) tahsis etmek için kullanılır. Bu fonksiyon da **void*** jenerik gösterici (**generic pointer**) döndürür. Aşağıda fonksiyonun prototipi verilmiştir;

```
void* calloc(size_t num, size_t size);
```

Bu fonksiyon da, aşağıdaki şekilde öbek bellekten yer ayrılır;

```
ptr = (dönüştürülecek-veri-tipi*) calloc(kaç-adet,bellek-uzunluğu);
```

Buna ilişkin bir örnek aşağıda verilmiştir;

```
float* ptr;
ptr = (float*) calloc(20, sizeof(float)); /* 20 adet float içerecek bellek bloğu tahsis edilerek
→ göstericisi ptr değişkenine atandı. Bellek ayrılmasaydı boş gösterici (NULL pointer) geri
→ döndürecekti.*/
```

§14.1.3 realloc() fonksiyonu

Bu fonksiyon, daha önce tahsis edilmiş bir belleğin tahsis miktarını değiştirmek için kullanılır. Bu fonksiyon da **void*** jenerik gösterici (**generic pointer**) döndürür. Aşağıda fonksiyonun prototipi verilmiştir;

```
void* realloc(void *ptr, size_t size);
```

Bu fonksiyon, aşağıdaki şekilde öbek bellekten yer ayrılır;

```
ptr = (dönüştürülecek-veri-tipi*) realloc(daha-önce-bellek-tahsis-edilmiş-gösterici,bellek-uzunluğu);
```

Buna ilişkin bir örnek aşağıda verilmiştir;

```
int* ptr1= (int*) malloc(30 * sizeof(int)); //ilk tahsis
float* ptr1= (float*) malloc(10 * sizeof(float)); //ilk tahsis
//...
ptr1 = (int*) realloc(ptr1, 10 * sizeof(int));
ptr2 = (float*) realloc(ptr2, 40 * sizeof(float));
/* Bellek yeniden ayrılmasaydı boş gösterici geri döndürecekti.*/
```

§14.1.4 free() fonksiyonu

Bu fonksiyon, tahsis edilen belleği serbest bırakmak için kullanılır. `malloc()`, `calloc()` ve `realloc()` fonksiyonları kullanılarak tahsis edilen bellek serbest bırakılır. Aşağıda örnek kod bulunmaktadır;

```
int* ptr1= (int*) malloc(30 * sizeof(int)); //ilk tahsis
//...
free(ptr1);
ptr1 = NULL;
```

Bir gösterici, tahsis edilmiş bir bellek bölgesini işaret ediyorken bellek bölgesi serbest bırakılırsa, gösterici serbest bırakılan bölgeyi işaret etmeye devam edebilir. **Bu nedenle serbest bırakma sonrası göstericiye boş gösterici (NULL Pointer) ataması iyi bir uygulamadır.**

14.2 Dinamik Bellek Kullanım Örnekleri

Aşağıda öğrenci sayısını çalıştırma anında alıp, bu öğrencilere ilişkin notları rastgele belirleyen bir program verilmiştir.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int main() {
    int* notlar=NULL;
    int n, i;
    srand(time(NULL)); //Notlar rastgele atanacak.

    printf("Öğrenci Sayısını Giriniz:");
    scanf("%d",&n);
    printf("%d Elemanlı Notlar Dizisi Oluşturulacak\n", n);

    notlar = (int*) malloc(n * sizeof(int)); //Tahsis
    if (notlar == NULL) {
        printf("Hafıza tahsis edilemedi!\n");
        exit(-1); //İşletim sistemine hata ile dönülüyor.
    } else {
        printf("Hafıza başarılı bir şekilde tahsis edildi.\n");
        for (i = 0; i < n; ++i)
            notlar[i] = rand()%101; //Rastgele not veriliyor
        printf("Öğrenci Notları:\n");
        for (i = 0; i < n; ++i)
            printf("%d, ", notlar[i]);
    }
    free(notlar); // Serbest Bırakma
    notlar=NULL; // Zaten programdan çıkılacağı için yazılmayabilir!
    return 0;
}
```

Aşağıda karakter sayısını çalıştırma anında alıp, yığın bellekte bir dizgi (`string`) oluşturan bir program verilmiştir.

```
#include <stdio.h>
```



```
#include <stdlib.h>
int main() {
    char* string;
    int n;
    printf("Girilen Metin En Fazla Kaç Karakter:");
    scanf("%d",&n);
    string = (char*) malloc(n); // Tahsis
    if (string == NULL) {
        printf("Hafıza tahsis edilemedi!\n");
        exit(-1); //İşletim sistemine hata ile dönülüyor.
    } else {
        printf("Hafıza başarılı bir şekilde tahsis edildi.\n");
        puts("Metni Giriniz:");
        scanf("%s",string);
        printf("Girilen Metin:\n%s",string);
    }
    return 0;
}
```

14.3 Dinamik Veri Yapıları

Çalıştırma anında bellek tahsis edilerek kullanılan veri yapılarına **dinamik veri yapıları** (**dynamic data structure**) denir. Bunlar; Bağlı Listeler (**linked list**), Yığınlar (**stack**), Kuyruklar (**queue**), İkili Ağaçlar (**binary tree**) ve Sözlüktür (**dictionary**). Buradaki veri yapılarının çoğu 13.6 başlığında anlatılan **öz referanslı yapılar** (**self-referential structure**) kullanılarak hayata geçirilir.

§14.3.1 Bağlı Liste

Bağlı liste (**linked list**), düğüm olarak bilinen her bir ögenin göstericiler kullanılarak bir sonraki düğüme (**node**) bağlandığı doğrusal bir veri yapısıdır. Diziden farklı olarak, bağlı listenin öğelerinin her biri öbek bellekte (**heap segment**) ayrı ayrı oluşturulduğundan, rastgele bellek konumlarında saklanır. Düğümlerde saklanan veri tek bir değişken olacağı gibi, birden fazla değişken de olabilir. Bağlantılı liste;

- ♦ Aşağıda verilen örnekte görüldüğü gibi tek yönlü olabileceği gibi,
- ♦ Hem sonraki düğümü hem de önceki düğümü gösteren iki yönlü liste,
- ♦ Veya son ve ilk düğümü olmayan halka şeklinde liste tanımlanabilir.

```
#include <stdio.h>
#include <stdlib.h>
struct dugumYapi {
    int veri;
    //char veri2; ...
    struct dugumYapi* sonraki; //sonraki düğümü gösteren öz referanslı yapı!
};
typedef struct dugumYapi Dugum;
void ilkDugumeEkle(Dugum**,int);
void sonaDugumEkle(Dugum*,int);
void listeyiYaz(Dugum*);
void ilkDugumuSil(Dugum**);
void sonDugumuSil(Dugum*);
int main() {
    Dugum* ilk=NULL; //Başlangıçta hiç düğüm yok.
    ilkDugumeEkle(&ilk,10);
    sonaDugumEkle(ilk,20);
    sonaDugumEkle(ilk,30);
    ilkDugumeEkle(&ilk,-10);
    listeyiYaz(ilk);
    return 0;
}
void ilkDugumeEkle(Dugum** ilkDugumGostericisi, int pVeri) {
    Dugum * yeni;
    yeni = (Dugum *) malloc(sizeof(Dugum)); //yeni düğüm için bellek tahsisi
    yeni->veri = pVeri; //tahsis edilen düğüme veri konuluyor
    yeni->sonraki = *ilkDugumGostericisi; // sonraki düğüm ilkDugumGostericisi olsun.
```

```

        *ilkDugumGostericisi = yeni; //İlk düğüm göstericisi yeni düğüm olsun
    }
    void sonaDugumEkle(Dugum* pIlk, int pVeri) {
        Dugum* hangi = pIlk; //hangi ilk düğümü gösterecek.
        if (hangi==NULL) {
            printf("Liste olmadığından eklenemedi!\n");
            return;
        }
        while (hangi->sonraki != NULL) {
            hangi = hangi->sonraki; //listenin sonuna kadar ilerle
        }
        hangi->sonraki = (Dugum*) malloc(sizeof(Dugum)); //sonuna yeni düğüm ekle
        hangi->sonraki->veri = pVeri; //yeni düğümün verisi konuluyor
        hangi->sonraki->sonraki = NULL; //son düğümün sonraki düğüm NULL olsun.
    }
    void listeyiYaz(Dugum* pIlk) {
        Dugum* hangi = pIlk; int sayac=0;
        while (hangi != NULL) {
            printf("Dugum (%d): veri=%d\n",
                ++sayac, hangi->veri);
            hangi = hangi->sonraki;
        }
    }
    void ilkDugumuSil(Dugum** ilkDugumGostericisi) {
        Dugum* sonrakiDugum = NULL;
        if (*ilkDugumGostericisi == NULL) return;
        sonrakiDugum = (*ilkDugumGostericisi)->sonraki;
        free(*ilkDugumGostericisi);
        *ilkDugumGostericisi = sonrakiDugum;
    }
    void sonDugumuSil(Dugum* pIlk) {
        if (pIlk->sonraki == NULL) {
            free(pIlk);
            return;
        }
        Dugum* hangi = pIlk;
        while (hangi->sonraki->sonraki != NULL)
            hangi = hangi->sonraki;
        free(hangi->sonraki);
        hangi->sonraki = NULL;
    }
}

```

§14.3.2 Yığın

Yığınlar (stack), bağlı listenin kısıtlanmış halidir. Haliyle doğrusal bir veri yapısıdır. Yığın veri yapısında;

- ◊ Bağlantılı listeye yeni düğüm, ancak listenin başına eklenir. Bu işlem itme (**push**) olarak adlandırılır.
- ◊ Bağlantılı listede silme işlemi yalnızca listenin başındaki düğümüne uygulanır. Bu işleme çekme (**pop**) adı verilir.
- ◊ Böylece listeye son giren veri ilk alınır LIFO (**last in first out**).

```

#include <stdio.h>
#include <stdlib.h>
struct dugumYapi {
    int veri;
    struct dugumYapi* sonraki; //sonraki düğümü gösteren öz referanslı yapı!
};
typedef struct dugumYapi Dugum;
void it(Dugum**,int);
int cek(Dugum**);
int main() {
    Dugum* istif=NULL;
    it(&istif,10);
    it(&istif,20);
}

```

```

        it(&istif,30);
        int veri=cek(&istif);
        printf("Çekilen Veri:%d\n",veri);
        veri=cek(&istif);
        printf("Çekilen Veri:%d\n",veri);
        return 0;
    }
    void it(Dugum** ilkDugumGostericisi, int pVeri) {
        Dugum* yeni;
        yeni = (Dugum *) malloc(sizeof(Dugum));
        yeni->veri = pVeri;
        yeni->sonraki = *ilkDugumGostericisi;
        *ilkDugumGostericisi = yeni;
    }
    int cek(Dugum** ilkDugumGostericisi) {
        Dugum * sonrakiDugum = NULL;
        if (*ilkDugumGostericisi == NULL) {
            printf("İstifde düğüm yok!");
            exit(-1);
        }
        int veri=(*ilkDugumGostericisi)->veri;
        sonrakiDugum = (*ilkDugumGostericisi)->sonraki;
        free(*ilkDugumGostericisi);
        *ilkDugumGostericisi = sonrakiDugum;
        return veri;
    }
}

```

§14.3.3 Kuyruk

Kuyruk (queue) de, bağlı listenin kısıtlanmış halidir. Haliyle bu veri yapısı da doğrusal bir veri yapısıdır. Kuyruk veri yapısında;

- ◊ Bağlantılı listeye yeni düğüm, ancak listenin başına eklenir. Bu işlem, kuyruğa sokma (**enqueue**) olarak adlandırılır.
- ◊ Bağlantılı listede silme işlemi yalnızca listenin sonundaki düğüme uygulanır. Bu işleme kuyruktan çıkarma (**dequeue**) adı verilir.
- ◊ Böylece listeye ilk giren veri ilk alınır FIFO (**first in first out**).

```

#include <stdio.h>
#include <stdlib.h>
struct dugumYapi {
    int veri;
    struct dugumYapi* sonraki;
};
typedef struct dugumYapi Dugum;
void enqueue(Dugum**,int);
int dequeue(Dugum*);
int main() {
    Dugum* kuyruk=NULL;
    enqueue(&kuyruk,10);
    enqueue(&kuyruk,20);
    enqueue(&kuyruk,30);
    int veri=dequeue(kuyruk);
    printf("Alınan Veri:%d\n", veri);
    veri=dequeue(kuyruk);
    printf("Alınan Veri: %d\n",veri);
    return 0;
}
void enqueue(Dugum** ilkDugumGostericisi, int pVeri) {
    Dugum* yeni;
    yeni = (Dugum*) malloc(sizeof(Dugum));
    yeni->veri = pVeri;
    yeni->sonraki = *ilkDugumGostericisi;
    *ilkDugumGostericisi = yeni;
}

```

```

}
int dequeue(Dugum* pIlk) {
    int veri;
    if (pIlk->sonraki == NULL) {
        veri=pIlk->veri;
        free(pIlk);
        return veri;
    }
    Dugum* hangi = pIlk;
    while (hangi->sonraki->sonraki != NULL)
        hangi = hangi->sonraki;

    veri=hangi->sonraki->veri;
    free(hangi->sonraki);
    hangi->sonraki = NULL;
    return veri;
}

```

§14.3.4 İkili Ağaç

İkili ağaç (**binary tree**) yapısı, bir kök ve dallarından oluşan veri yapısıdır. İkili (**binary**) denmesinin sebebi bir düğümden (**node**), genellikle sağ ve sol olarak adlandırılan, yalnızca iki dal çıkabildiğindendir. Haliyle bu veri yapısı da doğrusal olmayan (**nonlinear**) hiyerarşik bir veri yapısıdır.

Ağacın ilk düğümlüne kök düğüm (**root node**) adı verilir. Kök düğümden dallanan iki düğüm çocuk düğüm (**child node**) olarak adlandırılır. Bu şekilde her dala eklenen düğüm ile ters bir ağaç oluşur. En uçtaki çocuk düğüme yaprak düğüm (**leaf node**) adı verilir. İkili bir ağaçta gerçekleştirilebilecek temel işlemler şunlardır:

- ◊ Ekleme (**insertion**)
- ◊ Silme (**deletion**)
- ◊ Arama (**search**) ve
- ◊ Gezinme (**traversing**). Üç şekilde yapılır;
 - Ön Sıralı (**pre-order traversal**)
 - Son Sıralı (**post-order traversal**)
 - Sıralı (**in-order traversal**)

```

#include <stdio.h>
#include <stdlib.h>
struct agacDugum {
    int veri;
    struct agacDugum* sol;
    struct agacDugum* sag;
};
typedef struct agacDugum Dugum;
Dugum* yeniDugum(int pVeri) {
    Dugum* yeni = (Dugum*)malloc(sizeof(Dugum));
    if (yeni != NULL) {
        yeni->veri = pVeri; // Tahsis yapılan düğüme veri aktarılıyor
        yeni->sol = NULL;
        yeni->sag = NULL;
    }
    return yeni;
}
Dugum* dugumEkle(Dugum* kok, int pVeri) {
    if (kok == NULL)
        return yeniDugum(pVeri); // Ağaç yoksa yeni düğüm ekle
    if (pVeri < kok->veri) // Eklenecek veriyi karşılaştır
        kok->sol = dugumEkle(kok->sol, pVeri);
    else if (pVeri > kok->veri)
        kok->sag = dugumEkle(kok->sag, pVeri);
    return kok; // Değişen kök düğümü döndür.
}
void siraliGazinme(Dugum* kok) {

```

```

// Sıralı Gezinti: sol altagac, kok, sag altagac
if (kok != NULL) {
    siraliGezinme(kok->sol);
    printf("%d ", kok->veri);
    siraliGezinme(kok->sag);
}
}
void onSiraliGezinme(Dugum* kok) {
    // Ön Sıralı Gezinti: kok, sol altagac, sag altagac
    if (kok != NULL) {
        printf("%d ", kok->veri);
        onSiraliGezinme(kok->sol);
        onSiraliGezinme(kok->sag);
    }
}
void sonSiraliGezinme(Dugum* kok) {
    // Ön Sıralı Gezinti: kok, sol altagac, sag altagac
    if (kok != NULL) {
        sonSiraliGezinme(kok->sol);
        sonSiraliGezinme(kok->sag);
        printf("%d ", kok->veri);
    }
}
void agaciBelllektenKaldir(Dugum* kok) {
    if (kok != NULL) {
        agaciBelllektenKaldir(kok->sol);
        agaciBelllektenKaldir(kok->sag);
        free(kok);
    }
}
}
int main() {
    Dugum* kok = NULL;
    int dugumVerisi;
    char secim;
    kok=dugumEkle(kok, 20);
    kok=dugumEkle(kok, 30);
    kok=dugumEkle(kok, 40);
    kok=dugumEkle(kok, 50);
    kok=dugumEkle(kok, 60);
    kok=dugumEkle(kok, 70);
    kok=dugumEkle(kok, 80);
    printf("\nSıralı Gezinti ile Ağaç: ");
    siraliGezinme(kok);
    printf("\n");
    printf("\nOn Sıralı Gezinti ile Ağaç: ");
    onSiraliGezinme(kok);
    printf("\n");
    printf("\nSon Sıralı Gezinti ile Ağaç: ");
    sonSiraliGezinme(kok);
    printf("\n");
    agaciBelllektenKaldir(kok);
    kok=NULL;
    return 0;
}

```

§14.3.5 Sözlük

Sözlük (**dictionary**) veri yapısı, tekil olarak anahtarları (**key**) barındıran ve her anahtara bir değer (**value**) verilebilen bir yapıdır. Tanımından hareketle bu veri yapısına sözlük veya eşleme (**map**) adı verilir.

```

#include <stdio.h>
#include <string.h>
#define MAX_ESLEME 100 //Sözlükte en fazla 100 eşleşme olacak.

```

```

int eslesmeSayisi = 0; // Aktif Eşleşme Sayısı
char keys[MAX_ESLEME][100];
int values[MAX_ESLEME];
int indisiBul(char key[])
{
    int i;
    for (i = 0; i < eslesmeSayisi; i++) {
        if (strcmp(keys[i], key) == 0)
            return i;
    }
    return -1;
}
void sozlugeEkle(char key[], int value)
{
    int index = indisiBul(key);
    if (index == -1) {
        strcpy(keys[eslesmeSayisi], key);
        values[eslesmeSayisi] = value;
        eslesmeSayisi++;
    } else
        values[index] = value;
}
int sozlukteBul(char key[])
{
    int index = indisiBul(key);
    if (index == -1)
        return -1;
    else
        return values[index];
}
void sozluguYazdir()
{
    for (int i = 0; i < eslesmeSayisi; i++)
        printf("%s: %d\n", keys[i], values[i]);
}
int main()
{
    sozlugeEkle("Elma", 100);
    sozlugeEkle("Armut", 200);
    sozlugeEkle("Muz", 300);
    printf("Sozluk İçeriği: \n");
    sozluguYazdir();
    printf("\nElmanın Sözlükteki Değeri: %d\n",
        sozlukteBul("Elma"));
    printf("Muzun İndisi: %d\n",
        indisiBul("Muz"));
    return 0;
}

```

14.4 Düşün ve Çöz

- ◊ **Düşün:** Bellek Sızıntısı (**memory leak**) nedir? **malloc** ile ayrılan bir alanın **free** edilmemesi uzun süre çalışan sistemlerde (örn: işletim sistemi) ne gibi sonuçlar doğurur?
- ◊ **Düşün:** **calloc** ile **malloc** arasındaki temel fark nedir? İlk değer ataması (**initialization**) kritik sistemlerde neden önemlidir?
- ◊ **Düşün:** Bağlı listelerde bir elemana erişim hızı ($O(n)$) ile dizilerdeki erişim hızını ($O(1)$) veri yapısı mimarisi üzerinden karşılaştırınız.
- ◊ **Çöz:** Dinamik bellek kullanarak kullanıcıdan alınan N adet sayıyı tutan bir dizi oluşturunuz, sayıları yazdırınız ve ardından belleği sisteme geri iade ediniz.
- ◊ **Çöz:** Dinamik bir dizinin boyutunu kullanıcı veri girdikçe **realloc** kullanarak kademeli olarak artıran bir fonksiyon yazınız.
- ◊ **Çöz:** Tek yönlü bağlı listede (**singly linked list**) verilen bir değeri arayan ve bulursa düğümün adresini

döner fonksiyonu kodlayınız.

15. Bölüm

Değişken Sayıda Argüman Alan Fonksiyonlar

15.1 Değişken Sayıda Parametre Tanımı

Belirsiz sayıda argüman alabilen fonksiyona **değişken sayıda argüman alan fonksiyon** (**variadic function**) denir. C dilindeki güçlü ancak çok nadiren kullanılan özelliklerden biridir. `printf()` ve `scanf()` fonksiyonları aslında değişken sayıda argümanı biçimlendirme metninden (**format string**) sonra koyabildiğimiz fonksiyonların en bilinen örnekleridir. Aşağıda verilen `printf` fonksiyonunun prototipindeki üç noktaya (**ellipsis**) dikkat ediniz.

```
int printf(const char* format,...);
```

Bu fonksiyonun değişken sayıda argüman aldığı aşağıdaki kod örneklerinden anlaşılmaktadır.

```
printf("%d %c %s %c",i,c,str,c2); //5 argümanı olan printf
printf("%s %d %c",str,i,c); //4 argümanı olan printf
printf("%s",str); //2 argümanı olan printf
```

C Dilinde, belirsiz sayıda bağımsız değişkeni olan bir fonksiyon; **en az bir sabit bağımsız değişkene** sahip olacak şekilde tanımlanır ve ardından derleyicinin değişken sayıda bağımsız değişkeni ayrıştırmasını sağlayan bir üç nokta (...) eklenir. **Birinci parametre kendisinden sonra gelebilecek parametre sayısı hakkında bilgi verir.** Aşağıda buna ilişkin sözde kod verilmiştir;

```
dönüş-tipi fonksiyonkimliği(veri-tipi birinci-parametre, ...);
```

Bunu `printf` fonksiyonunda söyle açıklayabiliriz. İlk parametresi olan biçimlendirme metni (**format string**), kendisinden sonra gelecek parametreler ve sayısını, içindeki biçim belirleyici (**format specifier**) karakterleri belirler.

Değişken sayıda parametreleri işlemek için kodunuza `stdarg.h` başlığını dahil etmeniz gerekir. Bu başlık-taki fonksiyonlar aşağıda verilmiştir;

Fonksiyon	Açıklama
<code>va_start(va_list ap, arg)</code>	Bu fonksiyon, üç nokta ile verilen argümanları <code>va_list</code> değişkenine aktarır.
<code>va_arg(va_list ap, type)</code>	Her seferinde, üç nokta ile temsil edilen değişken listesindeki bir sonraki argümanı <code>va_list</code> üzerinden işler ve listenin sonuna ulaşana kadar onu <code>type</code> ile verilen veri tipine dönüştürür.
<code>va_copy(va_list dest, va_list src)</code>	<code>va_list</code> 'teki argümanların bir kopyasını oluşturur.
<code>va_end(va_list ap)</code>	Bu, <code>va_list</code> değişkenlerine erişimi sonlandırır.

Tablo 15.1: Stdarg.h Fonksiyonları

Aşağıda ilk parametre ile belirlenen parametre sayısı kadar argüman alan bir **değişken sayıda argüman alan fonksiyon** tanımlanmıştır;

```
#include <stdio.h>
#include <stdarg.h>
int argümanlarınHepsiniTopla(int kacAdet,...) {
    va_list argumanlar;
    int sayac, toplam = 0;
    va_start(argumanlar, kacAdet);
    for (sayac = 0; sayac < kacAdet; sayac++)
        toplam += va_arg(argumanlar, int);
    va_end(argumanlar);
    return toplam;
}
int main(){
    printf("3 Argüman Toplamı = %d \n",
        argümanlarınHepsiniTopla(3, 10, 20, 30));
    printf("5 Argüman Toplamı = %d \n",
        argümanlarınHepsiniTopla(5, 10, 20, 30,40,50));
}
```

```

    return 0;
}

```

15.2 Değişken Sayıda Argüman Alan Ana Fonksiyon

Ana fonksiyon (main function) programın icra edilmeye başladığı fonksiyondur. Şu ana kadar argüman-sız ana fonksiyonu gördük. C Dilinde yazdığımız programlar da çalıştırılırken konsoldan argüman alabilir.

Ana fonksiyon;

- ◊ Satır içi (**inline**) fonksiyon olarak bildirilemez!
- ◊ Adresi alınamaz!
- ◊ Programın başka hiçbir yerinden çağrılmaz (**call**)!

Yazdığımız programlar da çalıştırılırken konsoldan argüman alabilir. Argüman alan ana fonksiyon (**main function**) aşağıdaki iki şekilde tanımlanır;

```

int main(int argc, char* argv[]) {
    // Ana fonksiyon Gövdesi
    // ...
}

```

Ya da aşağıdaki şekilde tanımlanır;

```

int main(int argc, char* argv[], char ** envp) {
    // Ana fonksiyon Gövdesi
    // ...
}

```

Aşağıda konsoldan çalıştırılırken alınan argümanları ve işletim sisteminin ortam değişkenleri (**environment variable**) konsola yazan bir program örneği verilmiştir;

```

#include <stdio.h>
int main(int argc, // argüman sayısı
char* argv[], // Metin dizisi olarak konsoldan girilen komut ve argümanlar
char** envp ) // Metin dizisi olarak ortam (environment) değişkenleri
{
    int sayac;
    printf_s( "\nKonsoldan Girilen Argümanlar:\n" );
    for( sayac = 0; sayac < argc; sayac++ )
        printf_s( " argv[%d] %s\n", sayac, argv[sayac] );
    printf_s( "\nOrtam Değişkenleri:\n" );
    while( *envp != NULL )
        printf_s( " %s\n", *(envp++) );
    return 0;
}

```

Programın derlenmesi ve çalıştırılması sonrası çıktı aşağıda verilmiştir;

```

C:\Users\ILHANOZKAN>notepad main.c
C:\Users\ILHANOZKAN>gcc main.c -o main.exe
C:\Users\ILHANOZKAN>main 10 20 otuz elli

```

Konsoldan Girilen Argümanlar:

```

argv[0]   main
argv[1]   10
argv[2]   20
argv[3]   otuz
argv[4]   elli

```

Ortam Değişkenleri:

```

ALLUSERSPROFILE=C:\ProgramData
APPDATA=C:\Users\ILHANOZKAN\AppData\Roaming
CommonProgramFiles=C:\Program Files\Common Files
CommonProgramFiles(x86)=C:\Program Files (x86)\Common Files
CommonProgramW6432=C:\Program Files\Common Files
COMPUTERNAME=DAMLANURO

```

```
ComSpec=C:\WINDOWS\system32\cmd.exe
configsetroot=C:\WINDOWS\ConfigSetRoot
DriverData=C:\Windows\System32\Drivers\DriverData
EFC_8812=1
FPS_BROWSER_APP_PROFILE_STRING=Internet Explorer
FPS_BROWSER_USER_PROFILE_STRING=Default
HOMEDRIVE=C:
HOMEPATH=C:\Users\ILHANOZKAN
LOCALAPPDATA=C:\Users\ILHANOZKAN\AppData\Local
LOGONSERVER=\\DAMLANURO
NUMBER_OF_PROCESSORS=8
OneDrive=C:\Users\ILHANOZKAN\OneDrive
OneDriveConsumer=C:\Users\ILHANOZKAN\OneDrive
OS=Windows_NT Path=C:\WINDOWS\system32;C:\Users\ILHANOZKAN\Downloads\codeblocks\MinGW\bin;
PATHEXT=.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC
PROCESSOR_ARCHITECTURE=AMD64
PROCESSOR_IDENTIFIER=Intel64 Family 6 Model 142 Stepping 12, GenuineIntel
PROCESSOR_LEVEL=6
PROCESSOR_REVISION=8e0c
ProgramData=C:\ProgramData
ProgramFiles=C:\Program Files
ProgramFiles(x86)=C:\Program Files (x86)
ProgramW6432=C:\Program Files
PROMPT=$P$G
PSModulePath=C:\Program Files\WindowsPowerShell\Modules;C:\WINDOWS\system32\WindowsPowerShell\v1.0\Modules
PUBLIC=C:\Users\Public
SESSIONNAME=Console
SystemDrive=C:
SystemRoot=C:\WINDOWS
TEMP=C:\Users\ILHANOZKAN\AppData\Local\Temp
TMP=C:\Users\ILHANOZKAN\AppData\Local\Temp
USERDOMAIN=DAMLANURO
USERDOMAIN_ROAMINGPROFILE=DAMLANURO
USERNAME=ILHANOZKAN
USERPROFILE=C:\Users\ILHANOZKAN
windir=C:\WINDOWS
ZES_ENABLE_SYSMAN=1

C:\Users\ILHANOZKAN>
```

16. Bölüm

Dosyalar

16.1 Dosya Nedir?

Her programın ihtiyaç duyduğu ortak görevler kullanıcıdan girdiyi (**input**) okumak ve konsolda çıktıyı (**output**) göstermektir. Bu şekilde girdi ve çıktı işlemleri yaparsak, program çalıştığı sürece veriler var olur, program sonlandırıldığında o verileri tekrar kullanamayız. Bunun için ve elektrik kesildiğinde bellekteki veriler kaybolacağından harici hafıza ortamında (**external memory**) verileri saklamak gerekir.

Dosya (file), verilerin bir araya geldiği (**collection of data**) ikincil saklama ortamıdır (**seconder storage**). Bu ikincil ortam; Bir bilgisayar hafızası (**memory**) olabileceği gibi, verilerin elektrikler kesildiğinde kaybolmayacağı sabit disk (**hard disc**), ya da bir başka ortama/bilgisayara veri gönderen (**send**) veya alan (**receive**) modem ve benzeri bir cihaz (**device**) olabilir.

16.2 Standart Dosyalar

C dili programcıya, girdi ve çıktı kısaca IO (**input-output**) için çeşitli işlevleri içeren başlık dosyaları sağlar ve tüm aygıtları (**device**) dosyalar olarak ele alınır. Bu nedenle, "ekran" gibi aygıtlar "dosyalar" ile aynı şekilde ele alınır.

Standart Dosya	Dosya Değişkeni	Aygıt
Standart Giriş	stdin	Klavye
Standart Çıktı	stdout	Ekran
Standart Hata	stderr	Kullanıcı Ekranı

Tablo 16.1: Standart Giriş-Çıkış Dosyaları

Dosyalarla işlem yapmak için `stdio.h` başlık dosyası çeşitli fonksiyonlara sahiptir ve aşağıda başlıklar halinde verilmiştir;

§16.2.1 Karakterler İçin Giriş Çıkış Fonksiyonları

```
int getchar(void);
```

Bu fonksiyon yeni satır ('\n') tuşuna basmadan tek bir tuş vuruşunu/karakteri konsoldan okur.

```
int putchar(int c);
```

Bu fonksiyon tek bir karakteri konsola yazar. Yazdırılacak karakterin ASCII kodu parametre olarak gönderilebilir.

```
#include <stdio.h>
int main() {
    char ch;
    printf("Bir karakter Giriniz: ");
    ch = getchar();
    puts("Girdiğiniz karakter: ");
    putchar(ch);
    return 0;
}
```

§16.2.2 Biçimlendirilmemiş Metin İçin Giriş Çıkış Fonksiyonları

Okuma ve yazma amacıyla dosyaya erişmek için önceden tanımlanmış bir **FILE** yapısı (**struct**) kullanır. Standart dosyalar da bu parametreye argüman olarak verilebilir.

```
char* fgets(char* str, int size, FILE* stream);
```

Bu fonksiyonda `stream` dosyasından, `str` ile kimliklendirilmiş parametre olan tampon belleğe (**buffer**), yeni satır veya dosya sonu EOF (**end-of-file**) ile karşılaşılıncaya kadar bir satırı okur. Hata yoksa `str` değişkeni, varsa EOF ya da NULL geri döndürür.

```
int fputs(const char* str, FILE* stream);
```

Bu fonksiyonda ise `str` dizgisini ve sonuna yeni satır karakteri koyarak `stream` dosyasına yazar. Hata

yoksa yazdırılan karakter sayısını içeren pozitif bir değer döner, varsa EOF geri döndürür.

```
#include <stdio.h>
int main() {
    char adi[20];
    printf("Adınız: ");
    fgets(adi, sizeof(adi), stdin);
    fputs("Girdiğiniz Adınız", stdout);
    fputs(adi, stdout);
    return 0;
}
```

§16.2.3 Biçimlendirilmiş Metin İçin Giriş Çıkış Fonksiyonları

Şimdiye kadar çıktığı standart çıktı dosyası `stdout` standart çıkış dosyasına yani konsola gönderen `printf` fonksiyonu ile standart giriş dosyası `stdin` standart dosyasından yani konsoldan veri girişi sağlanan `scanf` fonksiyonlarını çokça gördük.

```
int fscanf(FILE* stream, const char* format, ...);
```

Bu fonksiyon `FILE` yapısı (`struct`) ile belirtilen dosyadan girişi sağlanan biçime göre okur. `stdin` standart giriş dosyası olarak kullanılabilir.

```
int fprintf(FILE* stream, const char* format, ...);
```

Bu fonksiyon ise `FILE` yapısı ile belirtilen dosyaya biçimlendirme metnine (`format`) göre çıktıyı üreterek yazar. `stdout` ve `stderr` standart çıkış dosyalarıdır.

Bu fonksiyonların kullanımı `scanf` ve `printf` fonksiyonlarına benzer. Yalnızca hangi dosyanın kullanılacağı ilk parametre olan `stream` ile belirlenir.

16.3 Standart Olmayan Dosyalar

Standart olan `stdin`, `stdout` ve `stderr` dosyaları her zaman veri alışverişine açıktır. Bu dosyalarla metinler üzerinden veri alışverişi yapılır. Standart olmayan dosyalar ise her zaman veri alışverişine açık değildir. Bu nedenle dosya ile işlem yapmadan önce dosyayı açmalı (`open`) ve işlemiz bittiğinde de kapatmalıyız (`close`).

§16.3.1 Dosyayı Açma ve Kapatma

Standart olmayan dosyaları açıp kapatmayı sağlayan iki fonksiyon vardır;

```
FILE *fopen(const char* file_name, const char* mode_of_operation);
```

Bu fonksiyon dosya sistemindeki adını temsil eden `file_name` ile verilen dosyayı; okumak (`read`), yazmak (`write`) ya da sonuna veri eklemek (append) için ya da bunlardan her ikisini yapmak için açar. Açılan dosyanın `FILE` yapısına (`struct`) ilişkin göstericiyi geri döndürür. Ayrıca `mode_of_operation` parametresine aşağıda verilen argümanlarla dosyayı çeşitli şekillerde işlem yapmak için açabiliriz. Bu argümanlar **dosya açma modları** (**file opening modes**) olarak adlandırılır. Aşağıda metin dosyaları için kullanılan modlar verilmiştir.

Aşağıdaki fonksiyon ise açık olan dosyayı kapatır.

```
int fclose(FILE* stream);
```

Dosyayı sürekli açık tutmak sistem kaynaklarını meşgul etmek anlamına gelir. Dolayısıyla programcı tarafından açılan dosyayı en kısa sürede kapatmak alışkanlık haline getirilmelidir.

Aşağıda verilen örnekte görüldüğü gibi aynen konsola yazar gibi yazılması gerekenleri `fprintf()` fonksiyonu ile aynen dosyaya yazılmıştır. Kullanımı `printf()` ile aynıdır sadece ilk parametresi dosya göstericisidir. Dosya `"w+"` modunda açıldığından, program her çalıştığında dosya içeriği silerek yeniden aynı şeyleri yazacaktır.

```
#include <stdio.h>
int main()
{
    FILE* dosyaGostericisi;
    dosyaGostericisi = fopen("metin.txt", "w+"); /* çalışılan klasörde/dizinde "metin.txt"
    ↳ dosyası yoksa oluşturulur. Varsa üzerine yazılır. Dosya, hem yazma hem okuma modunda
    ↳ açıldı. */
    fprintf(dosyaGostericisi, "Adi Soyadi;Yaşı;Cinsiyeti\n");
    fprintf(dosyaGostericisi, "%s;%d;%c\n", "Ilhan OZKAN", 50, 'E');
    fprintf(dosyaGostericisi, "%s;%d;%c\n", "Yagmur OZKAN", 45, 'K');
```

Modlar	Açıklaması
"r"	Dosyayı, çalışılan klasörde/dizinde arar. Dosyayı yalnızca okumak için açar. Dosya başarıyla açılırsa <code>fopen()</code> onu belleğe yükler ve içindeki ilk karakteri işaret eden bir gösterici ayarlar. Dosya açılmıyorsa <code>NULL</code> değerini döndürür.
"w"	Dosyayı, çalışılan klasörde/dizinde arar. Dosya zaten mevcutsa içeriğinin üzerine yazılır. Dosya mevcut değilse yeni bir dosya oluşturulur. Dosya açılmıyorsa <code>NULL</code> değerini döndürür. Yalnızca yazmak için yeni bir dosya oluşturur.
"a"	Dosyayı çalışılan klasörde/dizinde arar. Dosya başarıyla açılırsa içeriği belleğe yükler ve içindeki son karakteri işaret eden bir gösterici ayarlar. Dosya mevcut değilse yeni bir dosya oluşturulur. Dosya açılmıyorsa <code>NULL</code> değerini döndürür. Dosya yalnızca dosyanın sonuna yazma yani ekleme amacıyla açılır.
"r+"	Dosyayı, çalışılan klasörde/dizinde arar. Dosyayı hem okumak hem de yazmak için açar. Başarıyla açılırsa, içeriği belleğe yükler ve içindeki ilk karakteri işaret eden bir gösterici ayarlar. Dosya açılmıyorsa <code>NULL</code> değerini döndürür.
"w+"	Dosyayı, çalışılan klasörde/dizinde arar. Dosya mevcutsa içeriğinin üzerine yazılır. Dosya mevcut değilse yeni bir dosya oluşturulur. Dosya açılmıyorsa <code>NULL</code> değerini döndürür. "w" ve "w+" arasındaki fark, <code>w+</code> kullanılarak oluşturulan dosyayı da okuyabilmemizdir.
"a+"	Dosyayı, çalışılan klasörde/dizinde arar. Dosya başarılı bir şekilde açılırsa içeriği belleğe yükler ve içindeki son karakteri işaret eden bir gösterici ayarlar. Dosya mevcut değilse yeni bir dosya oluşturulur. Dosya açılmıyorsa <code>NULL</code> değerini döndürür. Dosya okumak ve sonuna yazmak yani eklemek için açılır.

Tablo 16.2: Dosya Açma Modları

```

        fclose(dosyaGostericisi);
        return 0;
    }

```

Oluşan metin dosyası olan `metin.txt` içeriği aşağıda verilmiştir;

```

Adi Soyadi;Yaşı;Cinsiyeti
Ilhan OZKAN;50;E
Yagmur OZKAN;45;K

```

Aynı dosyayı her bir satırı tam olarak okuyan `fgets()` fonksiyonu ile aşağıdaki kodla okuyabiliriz;

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define BIRSATIRDAKIENFAZLAKARAKTER 80
int main()
{
    char *dosyaAdi="metin.txt";
    char satir[BIRSATIRDAKIENFAZLAKARAKTER] = {0};
    unsigned int satirNumarasi = 0;
    FILE *dosyaGostericisi = fopen(dosyaAdi, "r");
    if (!dosyaGostericisi)
    {
        perror(dosyaAdi);
        return EXIT_FAILURE;
    }
    while (fgets(satir, BIRSATIRDAKIENFAZLAKARAKTER, dosyaGostericisi))
    {
        printf("satir[%06d]: %s", ++satirNumarasi, satir);
        if (satir[strlen(satir) - 1] != '\n')
            printf("\n");
    }
    if (fclose(dosyaGostericisi))
    {
        return EXIT_FAILURE;
        perror(dosyaAdi);
    }
}
/* Program Çıktısı:

```

```

satir[000001]: Adi Soyadi;Yasi;Cinsiyeti
satir[000002]: Ilhan OZKAN;50;E
satir[000003]: Yagmur OZKAN;45;K
*/

```

Bunların yanında `perror()` fonksiyonu, programınızda neyin yanlış gittiğini anlamana ve hata ayıklamanıza (`debug`) yardımcı olan standart hata akışına (`stderr`) açıklayıcı bir hata mesajı yazdırmak üzere tasarlanmıştır;

```

#include <stdio.h> /* perror(), fopen(), fputs() and fclose() */
#include <stdlib.h> /* EXIT_* macroları için */
int main(int argc, char **argv){
    int e = EXIT_SUCCESS;
    char *path = "output.txt";
    FILE *file = fopen(path, "w");
    if (!file) {
        perror(path);
        return EXIT_FAILURE;
    }
    if (fputs("Dosyaya Yazılacak Metin...\n", file) == EOF) {
        perror(path);
        e = EXIT_FAILURE;
    }
    if (fclose(file)) {
        perror(path);
        return EXIT_FAILURE;
    }
    return e;
}

```

Ayrıca aşağıda verilen program `popen()` aracılığıyla bir program ya da servisi çalıştırır ve işlem den gelen tüm standart çıktıyı okur ve bunu standart çıktı olan konsola yansıtır;

```

#include <stdio.h>
void print_all(FILE *stream) {
    int c;
    while ((c = getc(stream)) != EOF)
        putchar(c);
}
int main(void){
    FILE *stream;
    if ((stream = popen("dir", "r")) == NULL)
        return 1;
    print_all(stream);
    pclose(stream);
    return 0;
}

```

§16.3.2 İkili Dosyalar

Değişkenlerimizi her zaman insan gözünün okuyacağı metne çevirerek dosyalara kaydetmeyiz. Onun yerine daha az yer kaplaması açısından bellekte saklandığı şekliyle dosyaya hızlıca yazarız. Ancak bu durumda dosyayı, bellekte ikili olarak tutulan veriler gibi yazar veya okuruz. Bu dosyalar **ikili dosya** (**binary file**) olarak adlandırılır.

İkili dosyalar, insan gözüyle okunamazlar. Okunmaları için özel programlar gerekir.

İkili dosyalar da metin dosyaları gibi açılıp kapatılır. Ancak ikili dosyanın, dosya açma modları (**file opening modes**) farklıdır ve 16.3 tablosunda verilmiştir.

İkili dosyalara veri okumak için kullanılan fonksiyon `fread()` fonksiyonudur;

```
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
```

Bu fonksiyon;

- ◊ `stream` dosya göstericisi üzerinden veri okur ve `ptr` ile işaret edilen bellek bölgesine koyar

Modlar	Açıklaması
"rb"	Dosyayı çalışılan klasörde/dizinde arar. İkili dosyayı okuma modunda açar. Dosya mevcut değilse, fopen() işlevi NULL değerini döndürür.
"wb"	İkili dosya yazma modunda açılır. Gösterici dosyanın başlangıcına ayarlanır ve içeriklerin üzerine ikili olarak yazılır. Dosya mevcut değilse yeni bir dosya oluşturulur.
"ab"	İkili dosya, ekleme modunda açılır. Dosya göstericisi, dosyadaki son karakterden sonra ayarlanır. Verilen dosya isminden bir dosya yoksa yeni bir dosya oluşturulur.
"rb+"	İkili dosya okuma ve yazma modunda açılır. Dosya mevcut değilse, open() işlevi NULL değerini döndürür.
"wb+"	İkili dosya okuma ve yazma modunda açılır. Dosya mevcutsa içeriğin üzerine yazılır. Dosya mevcut değilse oluşturulacaktır.
"ab+"	İkili dosya okuma ve ekleme modunda açılır. Dosya mevcut değilse bir dosya oluşturulacaktır.

Tablo 16.3: İkili Dosya Açma Modları

- ◇ `nmemb`, `size` ile verilen uzunluktan kaç adet okunacağını belirtir.
- ◇ Bu fonksiyon, genellikle ikili dosyaları okumak için kullanılır ancak metin dosyaları için de kullanılabilir.
- ◇ Bir okuma hatası veya dosya sonu (EOF) oluşursa `nmemb`'den az olabilen, başarıyla okunan öğelerin sayısını döndürür. `size` veya `nmemb` sıfırsa, fonksiyon sıfır döndürür ve `ptr` tarafından işaret edilen belleğin içerikleri değişmez.

İkili dosyalara veri yazmak için kullanılan fonksiyon biri `fwrite()` fonksiyonudur;

```
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
```

Bu fonksiyon;

- ◇ `ptr` ile işaret edilen bellekteki verileri belirtilen `stream` dosya göstericisi üzerinden dosyaya yazar.
- ◇ `nmemb`, `size` ile verilen uzunluktan kaç adet yazılacağını belirtir.
- ◇ Bu fonksiyon da genellikle ikili dosyalara yazmak için kullanılır ancak metin dosyaları için de kullanılabilir.
- ◇ Bu fonksiyon başarıyla yazılan toplam öğe sayısını döndürür. Bu sayı `nmemb`'den az ise bir hata oluşmuştur veya dosya sonuna ulaşılmıştır.

Aşağıda dosyaya yazılan yapı (`struct`) örneği verilmiştir. Yapılar aynen bellekte olduğu gibi dosyaya yazılır. Dosya, `"wb+"` modunda açıldığından, program her çalıştığında içeriğini silerek yeniden aynı şeyleri yazacaktır.

```
#include <stdio.h>
struct kisi {
    char adiSoyadi[16];
    int yas;
    char cinsiyet;
    float kilo;
};
typedef struct kisi Kisi;
int main() {
    FILE* dosyaGostericisi;
    Kisi kisi1={"Ilhan OZKAN",50,'E',100.0};
    Kisi kisi2={"Yagmur OZKAN",45,'K',60.0};
    int dizi[5]={1,2,3,4,5};
    // "kisi.bin" dosyası yoksa oluşturulur.
    // Varsa üzerine yazılır. Hem yazma hem okuma modunda dosya açılıyor.
    dosyaGostericisi = fopen("kisi.bin", "wb+");
    fwrite(&kisi1, sizeof(Kisi),1,dosyaGostericisi);
    fwrite(&kisi2, sizeof(Kisi),1,dosyaGostericisi);
    fwrite(dizi,sizeof(dizi),1,dosyaGostericisi);
    fwrite(dizi,sizeof(int),5,dosyaGostericisi);
    fclose(dosyaGostericisi);
    return 0;
}
```

Oluşan `kisi.bin` dosyasına ilk önce iki adet kişi yapısı kaydedilmiştir. Sonrasında 5 tamsayıdan oluşan `dizi` kaydedilmiştir. Veriler bellekte saklandığı şekliyle dosyaya kaydedildiğinden içeriği ikili olarak okuna-

bilir. İkili dosyaları açan görüntüleyen programlar ile açılıp görülebilir. Bir ikili dosyayı onaltılık rakamlara çevirip metin olarak görüntülemek için Windows ortamında `certutil` komutu kullanılabilir;

```
C:\Users\ILHANOZKAN>certutil -encodehex kisi.bin kisi.txt
```

```
Input Length = 97
```

```
Output Length = 503
```

```
CertUtil: -encodehex command completed successfully.
```

Daha sonrasında dönüştürülen `kisi.txt` dosyası herhangi bir metin editörüyle açılıp okunabilir;

```
0000 49 6c 68 61 6e 20 4f 5a 4b 41 4e 00 00 00 00 00  Ilhan OZKAN.....
0010 32 00 00 00 00 45 00 00 00 00 00 c3 88 42 59 61 67  2...E.....BYag
0020 6d 75 72 20 4f 5a 4b 41 4e 00 00 00 00 2d 00 00  mur OZKAN....-..
0030 00 4b 00 00 00 00 00 70 42 01 00 00 00 02 00 00  .K.....pB.....
0040 00 03 00 00 00 04 00 00 00 05 00 00 00 01 00 00  .....
0050 00 02 00 00 00 03 00 00 00 04 00 00 00 05 00 00  .....
0060 00
```

Dosyaya hep aynı veri tipinden veriler yazılacağı gibi, birbirinden farklı olabilen yapı (`struct`) ve veri tipleri de yazılabilir. **Burada önemli olan, yazıldığı sırada bu öğelerin okunmasıdır.** Aynı dosyayı okuyan program aşağıda verilmiştir;

```
#include <stdio.h>
struct kisi {
    char adiSoyadi[16];
    int yas;
    char cinsiyet;
    float kilo;
};
typedef struct kisi Kisi;
int main() {
    FILE* dosyaGostericisi;
    Kisi kisi1;
    Kisi kisi2;
    int dizi1[5];
    int dizi2[5];
    dosyaGostericisi = fopen("kisi.bin", "rb");
    fread(&kisi1, sizeof(Kisi), 1, dosyaGostericisi);
    fread(&kisi2, sizeof(Kisi), 1, dosyaGostericisi);
    fread(dizi1, sizeof(dizi1), 1, dosyaGostericisi);
    fread(dizi2, sizeof(int), 5, dosyaGostericisi);
    fclose(dosyaGostericisi);
    // Okunan Değerleri Konsola Yazalım.
    printf("Kişi1: Adi:%s, Yas:%d, Cinsiyet:%c, Kilo:%f\n", kisi1.adiSoyadi, kisi1.yas,
        ↪ kisi1.cinsiyet, kisi1.kilo);
    printf("Kişi2: Adi:%s, Yas:%d, Cinsiyet:%c, Kilo:%f\n", kisi2.adiSoyadi, kisi2.yas,
        ↪ kisi2.cinsiyet, kisi2.kilo);
    printf("{");
        for (int sayac=0; sayac<5; sayac++)
            printf("%d,", dizi1[sayac]);
        printf("}\n");
    printf("{");
        for (int sayac=0; sayac<5; sayac++)
            printf("%d,", dizi2[sayac]);
        printf("}\n");
    return 0;
}
/*Program Çıktısı:
kişi1: Adi:Ilhan OZKAN, Yas:50, Cinsiyet:E, Kilo:100.000000
kişi2: Adi:Yagmur OZKAN, Yas:45, Cinsiyet:K, Kilo:60.000000
{1,2,3,4,5,}
{1,2,3,4,5,}
*/
```

16.4 Düşün ve Çöz

- ◇ **Düşün:** Metin dosyaları (**text files**) ile ikili dosyalar (**binary files**) arasındaki temel farklar nelerdir? Neden veri tabanı gibi yapılar ikili dosyaları tercih eder?
- ◇ **Düşün:** **fseek** ve **ftell** fonksiyonları, rastgele erişimli (**random access**) dosyalarda belirli bir kayda doğrudan ulaşmak için nasıl kullanılır?
- ◇ **Düşün:** Bir dosyayı "**a**" modunda açmak ile "**w**" modunda açmak arasındaki veriyi koruma farkı nedir?
- ◇ **Çöz:** Bir **struct** yapısını (örneğin bir kişi bilgisi) ikili dosya modunda dosyaya yazan ve sonra bu dosyadan geri okuyup ekrana basan bir program yazınız.
- ◇ **Çöz:** Bir metin dosyasındaki tüm karakterleri okuyup, her karakterin ASCII değerini 1 artırarak (basit bir şifreleme) başka bir dosyaya kaydeden programı yazınız.
- ◇ **Çöz:** İkili bir dosyada kayıtlı olan öğrenci yapılarını (**struct**) not ortalamasına göre filtreleyip ekrana basan kodu yazınız.

17. Bölüm

Ön İşlemci Yönergeleri

17.1 Ön İşlemci Yönergesi Nasıl Çalışır?

C dilinde **ön işlemci** (**preprocessor**) yönergeleri, derleyicinin bir parçası değildir, ancak derleme sürecinde ayrı bir adımdır. Ön işlemci, yalnızca bir metin değiştirme aracıdır ve gerçek derlemeden önce gerekli ön işlemleri yapmasını sağlar. Derleme süreci, Şekil 17.1’de gösterilmiştir. Kısaca derleme önce bul-değiştir mantığı ile kodumuzda değişiklikler yapar.

- ◊ Ön işleme bir C kodunun derlenmesi öncesindeki ilk adımdır.
- ◊ Kodu belirteçlere ayırma (**tokenization**) adımından önce gerçekleşir.
- ◊ Ön işlemcinin önemli işlevlerinden biri de programda kullanılan kütüphane işlevlerini içeren başlık dosyalarını koda dahil etmek için kullanılmasıdır.
- ◊ Ön işlemci ayrıca değişmezleri (**literal**) tanımlar ve makro kullanımını sağlar.

C dilindeki ön işlemci ifadelerine, **yönerge** (**directive**) denir. Programda ön işlemci bölümü her zaman C kodunun en üstünde görünür. Her ön işlemci ifadesi, kare (**hash**) (**#**) sembolüyle başlar. Aşağıda ön işlemci yönergelerinin listesi verilmiştir;

Yönerge	Açıklama
#define	Ön işlemci makrosunu değiştirmek ya da ilk defa tanımlamak için kullanılır.
#include	Bir başlık dosyasını diğer ile dahil etmek için kullanılır.
#undef	Tanımlanmış bir makroyu tanımsız hale getirmek için kullanılır.
#ifdef	Bir makro tanımlı ise doğru/true değerini döndürür.
#ifndef	Bir makro tanımlı değilse doğru/true değerini döndürür.
#if	Derleme zamanında bir durumun kontrolü için kullanılır.
#else	#if Yönergesinde alternatif durumu ifade eder.
#elif	#else ve #if yönergelerini tek bir şekilde ifade etmek için kullanılır.
#endif	Şart ön işlemcilerini (#if, #else, #elif) bitirir.
#error	stderr standart dosyasına hata mesajını yazar.
#pragma	Derleyiciye özel komutlar vermek için kullanılır.

Tablo 17.1: Ön İşlemci Yönergeleri

17.2 Ön İşlemci Makroları

#define yönergesi ile yeni bir makro tanımlanabileceği gibi tanımlanmış bir makro **#undef** ile ortadan kaldırabilir veya yenisini tanımlayabiliriz. Aşağıdaki örnekte **PI** adlı bir makro tanımlanmış ve 3.1415 reel sayısını vermektedir. Daha önce **PI** adlı bir makro tanımlanmış ise derleyici hata verir. Aşağıda buna bir örnek verilmiştir;

```
#include <stdio.h>
#define PI 3.1415 // Yeni bir PI makrosu tanımlanıyor. Daha önce PI adlı makro tanımlı ise hata
↪ alırız.
#ifdef ENFAZLAOGRECISAYISI // Eğer ENFAZLAOGRECISAYISI daha önceden makro olarak TANIMSIZ ise
#define ENFAZLAOGRECISAYISI 100 // Yeni bir ENFAZLAOGRECISAYISI makrosu tanımlanıyor
#endif
int main() {
    printf("PI:%f\n",PI); //PI:3.141500
    printf("ENFAZLAOGRECISAYISI:%d\n",ENFAZLAOGRECISAYISI);
    ↪ //ENFAZLAOGRECISAYISI:100
#ifdef ENFAZLAOGRECISAYISI // Eğer ENFAZLAOGRECISAYISI daha önceden makro olarak TANIMLI ise
#undef ENFAZLAOGRECISAYISI // ENFAZLAOGRECISAYISI makrosunu kaldır!
#define ENFAZLAOGRECISAYISI 20 // Yeni bir ENFAZLAOGRECISAYISI makrosu tanımlanıyor
#endif
    printf("ENFAZLAOGRECISAYISI:%d\n",ENFAZLAOGRECISAYISI); //ENFAZLAOGRECISAYISI:20
    return 0;
}
```

17.3 Parametrelİ Ön İşlemci Makroları

Ön işlemci yönergeleriyle tanımlanan makrolar derlemeye geçilmeden önce işlem görür. Dolayısıyla bu aşamada bazen parametrelerle işlem yapılması gerekebilir.

```
#define kare(x) ((x) * (x))
#define kup(x) ((x) * (x) * (x))
#define buyugu(x,y) ((x) > (y) ? (x) : (y))
```

Burada kullanılacak parametrelerde veri tipi tanımlanmaz. Dolayısıyla kullanılacağı yerlere buna dikkat edilerek parametrelİ makro tanımlanmalıdır. Eğer makro birkaç satırdan oluşacak ise ters bölü (back slash) (\) karakteri kullanılır.

```
#include <stdio.h>

#define SAYIYAZ(num, str) { \
    /* 1.Satir */           \
    printf("%d :", num);    \
    /* 2.Satir */           \
    printf(" %s Numara \n", str); \
    /* Son satırda ter bölü olmaz! */ \
}

#define MY_MULTI_LINE_MACRO(arg1, arg2) do { \
    /* Line 1 of the macro */           \
    printf("Arg1: %d\n", arg1);          \
    /* Line 2 of the macro */           \
    printf("Arg2: %d\n", arg2);          \
    /* Last line (no backslash) */      \
} while (0);

int main() {
    SAYIYAZ(4, "Dort")
    MY_MULTI_LINE_MACRO(1,2)
    return 0;
}

/*Program Çıktısı:
4 : Dort Numara
Arg1: 1
Arg2: 2
*/
```

17.4 Ön Tanımlı Ön İşlemci Makroları

Her C derleyicisi için standart olarak tanımlı ön işlemci yönergesi ile makrolar bulunmaktadır;

Makro	Açıklama
__DATE__	Mevcut tarihi "MMM DD YYYY" biçiminde dizgi (string) olarak tanımlıdır.
__TIME__	Mevcut saat "HH:MM:SS" biçiminde dizgi (string) olarak tanımlıdır.
__FILE__	Dosya adı biçiminde dizgi (string) olarak tanımlıdır.
__LINE__	Dosyadaki satır numarası tamsayı (integer) olarak tanımlıdır.
__STDC__	ANSI standardında derleme yapılıyorsa 1 olarak tanımlıdır.

Tablo 17.2: Standart Ön İşlemci Makroları

Standart makroları kullanan örnek program aşağıda verilmiştir;

```
1 #include <stdio.h>
2 int main() {
3     printf("File: %s\n", __FILE__ ); //File: main.c
4     printf("Date: %s\n", __DATE__ ); //Date: Mar 22 2025
5     printf("Time: %s\n", __TIME__ ); //Time: 15:32:08
6     printf("Line: %d\n", __LINE__ ); //Line: 6
7     printf("ANSI: %d\n", __STDC__ ); //ANSI: 1
8 }
```

DEBUG Makrosu da hazır tanımlanmış bir makrodur ve hata ayıklama (debug) modunda kod derlenmesi

halinde `#if` yönergelerinde doğru/true değerini döndürür. Bunun için derleme sırasında derleyiciye `-DDEBUG` argümanı verilir. Buna aşağıda bir örnek verilmiştir;

```
#include <stdio.h>
#define PI 3.1415 // Yeni bir PI makrosu tanımlanıyor. Daha önce PI adlı makro tanımlı ise hata
→ alırız.
#ifndef ENFAZLAOGRECISAYISI // Eğer ENFAZLAOGRECISAYISI daha önceden makro olarak TANIMSIZ ise
#define ENFAZLAOGRECISAYISI 100 // Yeni bir ENFAZLAOGRECISAYISI makrosu tanımlanıyor
#endif
int main() {
    printf("PI:%f\n",PI); //PI:3.141500
#ifdef DEBUG
    /* Hata ayıklama modunda derleme yapıldığında #endif koduna kadar olan kodlar çalışır. */
    printf("ENFAZLAOGRECISAYISI:%d\n",ENFAZLAOGRECISAYISI); //ENFAZLAOGRECISAYISI:100
    printf("ENFAZLAOGRECISAYISI:%d\n",ENFAZLAOGRECISAYISI); //ENFAZLAOGRECISAYISI:20
#endif
    return 0;
}
```

17.5 Kullanıcı Tanımlı Başlık Dosyası

`#include` yönergesi ile; hazır olan başlık dosyalarını küçük (<) ve büyük (>) karakterleri arasında yazarak koda dahil ederiz. Hazır olmayan ve programcı tarafından hazırlanan başlık dosyalarını ise çift tırnak (") karakterleri arasında yazarak koda dahil ederiz. Aşağıda kullanıcı tanımlı bir başlık dosyası `Baslik.h` örneği verilmiştir;

```
#ifndef _BASLIK_H_
#define _BASLIK_H_
// _BASLIK_H_ makrosu sadece tanımlanıyor. Yerine bir metin konulmuyor

#define PI 3.1415
#define ENFAZLAOGRECISAYISI 100
float* ogrenciNotlari() {
    static float notlar[ENFAZLAOGRECISAYISI];
    return notlar;
}
float ogrenciNotlarOrtalamasi(float* pNotlar, int pOgranciSayisi) {
    int sayac;
    float ortalama=0;
    for (sayac=0;sayac<pOgranciSayisi;sayac++) {
        ortalama+=pNotlar[sayac];
#ifdef DEBUG
        printf("%d. Ogrencide Ortalama:%f\n",sayac, ortalama);
        // Buradaki kod hata ayıklama modunda derlendiğinde çalışır.
        // Bunun için derleyiciye -DDEBUG argümanı verilir
#endif
    }
    return ortalama/sayac;
}
#endif
```

Başlık dosyasının bir koda birden fazla dahil (`include`) edilmesini önlemek için `_BASLIK_H_` makrosu, şartlı (`conditional`) olarak tanımlanmıştır. Bu başlık dosyası bir kod projesinde birden fazla dahil olması halinde, `_BASLIK_H_` tanımlanmaz ise, başlık içindeki değişken ve fonksiyonlar birden fazla aynı kimlikle tanımlanacağından derleme hatası alınacaktı.

Kendi hazırlamış olduğumuz `baslik.h` başlık dosyasını kodumuza dahil ettiğimizde artık başlık dosyamızdaki fonksiyon ve değişkenleri kullanabilir hale geliriz. Aynı klasörde/dizinde bulunacak şekilde bu başlık dosyasını kullanan `main.c` örnek programı aşağıda verilmiştir;

```
#include <stdio.h>
#include "baslik.h"
int main() {
    printf("PI=%f\n",PI);
    printf("En fazla öğrenci Sayısı=%d\n",ENFAZLAOGRECISAYISI);
}
```

```

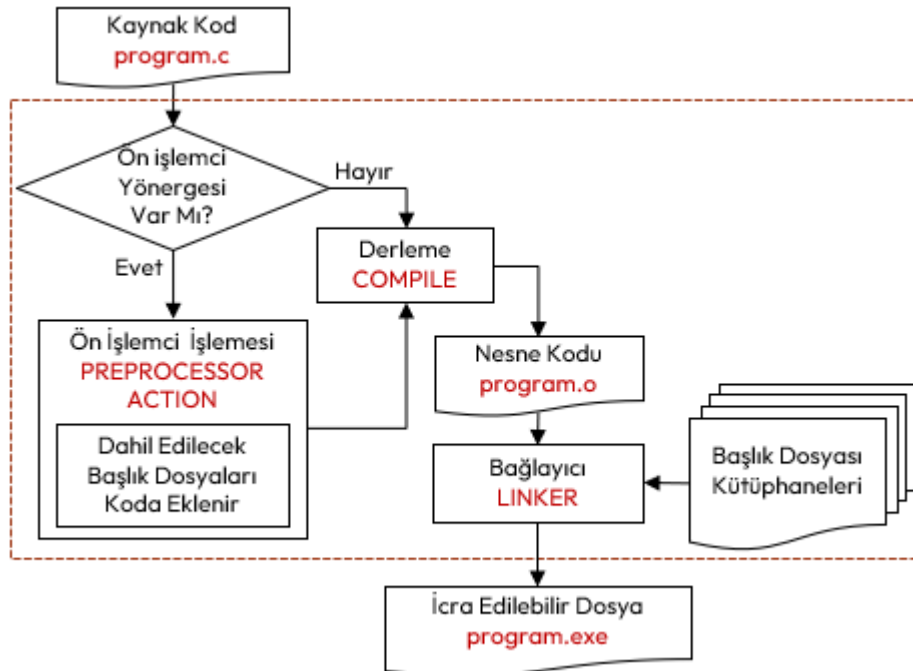
float* notlar=ogrenciNotlari();
notlar[0]=100;
notlar[1]=80;
notlar[2]=60;
float ortalama=ogrenciNotlarOrtalamasi(notlar,3);
printf("%d öğrenci için Ortalama %f",3,ortalama);
}

```

17.6 Derleme Sürecinin Detayı

C dilinde derleme sürecinin iki aşamadan oluştuğu daha önce anlatılmıştı. Aşağıda tüm süreç verilmiştir;

1. Birinci aşama:
 - (a) Kaynak koddaki tüm açıklamalar silinir.
 - (b) `#define`, `#if`, `#include`, gibi tüm yönergeler ile verilen işlemler yapılır.
 - (c) Bunlar arasında başlık dosyalarının koda dahil edilmesi de vardır.
 - (d) Bu duruma gelen kod, `gcc -E main.c` komutu dosya haline getirilebilir.
2. İkinci aşama:
 - (a) Kaynak kod montaj koda (assembly code) çevrilir.
 - (b) Bu duruma gelen kod, `gcc -S main.c` komutu dosya haline getirilebilir.
 - (c) Assembly kod derlenerek makine diline çevrilir ve amaç dosya (object file) oluşur.
 - (d) Bu duruma gelen kod, `gcc -c main.c` komutu ile dosya haline getirilebilir.
 - (e) Amaç dosya kodda belirtilen başlık dosyalarına uygun kütüphaneler (library) ile birbirine bağlayıcı (linker) ile bağlanır ve icra edilebilir dosya (executable file) elde edilir. Burada bahsedilen kütüphaneler her işletim sistemi için ayrı olarak oluşturulur ve derleyici kurulumunda bir klasörde tutulur.
 - (f) Bu aşamaya gelen derleme `gcc main.c -o main.exe` komutuyla icra edilebilir dosya oluşturulmaktadır.



Şekil 17.1: Derleme Süreci

17.7 Düşün ve Çöz

- ◇ **Düşün:** `#define` ile tanımlanan makroların, normal fonksiyonlara göre avantajları ve riskleri (yan etkileri) nelerdir?
- ◇ **Düşün:** Başlık dosyalarında kullanılan `#ifndef`, `#define`, `#endif` başlık güvenliği (header guard) bloklarının mükerrer dahil etmeyi nasıl önlediğini açıklayınız.
- ◇ **Düşün:** Koşullu derleme (`#ifdef DEBUG`) kullanarak, programın sadece geliştirme aşamasında ekrana teknik loglar basmasını nasıl sağlarsınız?
- ◇ **Çöz:** Bir sayının karesini alan bir makro (`#define KARE(x)`) tanımlayınız. `KARE(3+2)` ifadesinin sonucunu makronun nasıl açıldığını düşünerek analiz ediniz.

- ◇ **Çöz:** İki sayının küçüğünü bulan bir fonksiyonel makro (`#define MIN(a,b)`) yazınız ve makro **yan etkilerini** (*side effects*) önlemek için neden parantez kullanılması gerektiğini gösteriniz.
- ◇ **Çöz:** C'nin ön tanımlı makrolarını (`__DATE__`, `__TIME__`, `__FILE__`) kullanarak programın derlendiği anı ve dosya adını ekrana basan bir "Sürüm Bilgisi" fonksiyonu yazınız.

Dizin

- abstraction, 34
- access, 24
- accumulator, 3
- actual parameter, 12
- address operator, 29, 96
- alphanumeric, 108
- argument, 12
- arithmetic logic unit, 3
- arithmetic operator, 28
- array, 13, 75, 96
- array size, 75
- assembly, 18
- assembly code, 6
- assignment, 32
- assignment operator, 31
- auto storage class, 68, 70

- base case, 72
- binary, 23
- binary file, 134
- binary search, 89
- binnary, 11, 12
- BIOS, 6
- bit-fields, 118
- bitwise, 33
- bitwise operator, 30
- break statement, 55
- bubble sort, 84
- bus, 1, 3
- byte, 2

- call, 12
- call by reference, 95
- call by value, 71, 74
- callback, 98
- calling convention, 67
- calling overhead, 34
- camel case, 25
- change management, 34
- character literal, 25
- clock, 2
- code block, 21
- code segment, 67
- comma operator, 52, 62, 64
- comment, 23
- compile, 15, 16
- compile error, 17
- compile time error, 18
- compile warning, 18
- compiler, 6, 15
- computer program, 5
- condition, 40, 41, 49, 50
- conditional choice, 39, 40
- conditional code, 40, 41, 43
- conditional code register, 3
- conditional tenary operator, 47
- const, 26
- const qualifier, 26
- const variable, 25, 26
- constant, 25
- continue statement, 55
- control expression, 45
- control operation, 39, 41
- control structure, 13
- control unit, 3
- current instruction code, 4
- current instruction register, 3
- cursor, 34

- dangling else, 45
- data, 1
- data memory register, 67
- data segment, 67–70, 97, 119
- data structure, 13, 75
- data type, 10, 12, 24, 30, 75, 90
- debug, 139
- debugger, 17
- debugging, 103
- declaration, 24
- decode, 3, 4
- decrement, 29
- default, 68
- definition, 24
- dereference operator, 90
- derived type, 90
- deviation, 77
- directive, 138
- do-while statement, 50
- dot operator, 112
- double pointer, 92
- double precision, 23
- dynamic data structure, 115, 121
- dynamic memory allocation, 119

- elipsis, 128
- else code, 42, 43
- empty product, 72
- engineer, 1
- enumeration, 102
- equality, 32
- error handling, 19
- escape sequence, 36
- event-driven programming, 98
- exception, 19
- executable, 16
- execute, 3, 4, 16
- explicit type casting, 99, 119
- expression, 39, 40, 45, 99

- extern storage class, 69
- extern variable, 69

- fetch, 3
- first in first out, 123
- flag, 3
- flag register, 67
- floating point data type, 23
- floating point number, 11
- flowchart, 39
- for statement, 51
- format specifier, 34, 38, 104, 128
- format string, 34, 37, 128
- formatted, 34
- free format, 22
- function, 12
- function block, 21
- function call, 62
- function declaration, 61, 64, 65
- function definition, 61, 62, 64, 65
- function name, 62
- function pointer, 98
- function prototype, 62

- garbage, 70
- garbage collection, 19
- general purpose registers, 67
- generic pointer, 91, 100, 119
- global, 70
- global variable, 70, 119
- goto, 48

- hash, 138
- header, 34, 59, 67
- header file, 59
- heap segment, 67, 119, 121
- hexadecimal digit, 108
- high level programming language, 9

- identifier, 24
- identifier definition, 24, 48
- if statement, 40
- if-else statement, 42
- imperative programming, 19, 21
- implementation, 16
- implicit type casting, 99
- include, 34
- increment, 29
- index, 75, 76
- indirection operator, 113
- infinite loop, 57
- information, 1
- initialization, 27

- initialize, 25, 75, 112
- inline function, 19, 74
- inline specifier, 74
- input output, 34
- input-output operation, 39
- insertion sort, 84, 87
- instruction, 1, 3
- instruction code, 4, 5, 15, 21
- instruction cycle, 3
- instruction set, 4
- integer, 75
- integer data type, 23
- integer literal, 25, 45, 75, 76
- integral, 102
- integrated circuit, 1
- intellisense, 17
- invoke function, 62
- iteration, 55, 72
- jump, 39, 55
- keywords, 21, 33
- label, 48
- last in first out, 67, 122
- library, 59
- link-time, 62
- linked list, 13, 121
- linker, 59, 65
- literal, 25, 26, 102
- local variable, 70, 71, 119
- logical error, 18, 71
- logical operator, 31
- logical sequence, 9, 21, 39, 49–52, 57
- lookup tables, 110
- loop, 72
- loop code, 49, 50, 55
- low level programming, 8, 13
- machine code, 4
- machine language, 5
- main function, 13, 21, 57, 129
- memory, 1
- memory data register, 4
- mnemonic, 5, 15
- modular programming, 34
- modulus, 28
- narrowing casting, 99
- nested if, 41
- nested loop, 53
- nested structs, 112, 114, 118
- node, 121
- NULL Character, 26, 110
- NULL Pointer, 91, 107, 120
- object oriented programming, 19
- one definition rule, 67
- opcode, 4
- operand, 4, 28, 29, 31
- operating system, 6, 15
- operator, 18, 94
- operator precedence, 32
- out of bounds, 89
- out-of-bounds, 75
- physical sequence, 39, 49–52, 57
- plug-in, 17
- pointer, 75, 119
- pointer type, 90
- pop, 72
- postfix, 32
- pre-decrement, 99
- pre-increment, 99
- preprocessor, 138
- preprocessor directive, 34, 59, 116
- primitive data type, 13, 23
- print, 34
- procedural programing, 19
- procedure, 19, 66
- processor, 1
- program counter, 3
- programming, 5
- prototype, 34, 59
- pseudocode, 14
- punc card, 8
- queue, 123
- quick sort, 84, 88
- recursion, 72
- recursive, 73, 88
- referans type, 90
- register, 3, 69
- register storage class, 69
- relational, 32
- relational loop, 39, 48
- reserved words, 21
- return, 12
- reusing, 34
- run, 16
- run time, 26, 60, 119
- run time error, 18
- scan, 37
- scope, 67
- segment, 67
- selection sort, 84, 85
- self-referetial struct, 115
- sentinel value, 54
- sequential operation, 39, 41, 47
- set, 75
- Shell sort, 84, 87
- short-circuit, 33
- sign, 11
- signed, 24
- single precision, 23
- size, 75
- spaghetti code, 10
- stack, 122
- stack overflow, 72, 74
- stack register, 67
- stack segment, 67, 68, 70, 71, 74, 97
- standart type casting, 99
- statement, 9, 13, 18, 19, 21, 27
- static linking, 20
- static storage class, 68
- static variable, 68
- stderr, 134
- stdin, 105
- storage classes, 67
- string, 98, 104, 120
- string literal, 104
- struct, 103, 117, 135
- structural programming, 13
- structure, 13
- structure padding, 116
- swap, 84
- switch statement, 45
- syntax, 18, 21
- tab, 22
- technician, 1
- text segment, 67
- trace, 16
- two-dimentional array, 79
- typedef, 102, 114
- unary, 32
- unary cast operator, 99
- unary operator, 29
- union, 103, 117
- unique, 78
- unsigned, 24
- user defined function, 61
- user interface, 17
- value type, 90
- variable, 11, 13, 23
- variable declaration, 24
- variable scope, 24
- variance, 77
- version control, 17
- void, 24, 62, 91
- while statement, 49
- white space, 21, 27
- widening casting, 99